

**O‘ZBEKISTON RESPUBLIKASI MUDOFAA VAZIRLIGI
AXBOROT-KOMMUNIKASIYA TEKNOLOGIYALARI VA ALOQA
HARBIY INSTITUTI**

**“KIBERXAVFSIZLIK” FAKULTETI
“AXBOROT TEKNOLOGIYALARI VA DASTURIY INJINIRING”
kafedrası**



**B.K. YUSUPOV, SH.R. SAPAYEV, A.A. ABIDOV
“PYTHON DASTURLASH TILI”
DARSLIK**

Mazkur darslik O‘zbekiston Respublikasi Oliy aviatsiya bilim yurti qoshidagi muvofiqlashtiruvchi kengashning 2024- yil 05- yanvardagi 1- sonli bayonnomasiga asosan nashr etishga ruxsat berilgan.

Toshkent-2023

UO‘K 004.031.6.
KBK 32.973.2-018.01
B9

Python dasturlash tili: Darslik / **B.K.Yusupov, Sh.R.Sapayev, A.A.Abidov.** – Toshkent: “Aloqa nashriyoti”, 2024. - 316 b.

Darslik kursant va tinglovchilar uchun zamonaviy dasturlash tili - pythonning imkoniyatlaridan foydalanib turli operation tizimlarda ishlovchi GUI ilovalari ishlab chiqish va ularni turli sohalarga tadbiq qilishda muhim ahamiyatga ega.

Darslik 61030700 - qo‘shinlarning taktik qo‘mondonlik muhandisligi (Axborot tizimlari va texnologiyalari, aloqa qo‘shinlari) yo‘nalishida tahsil olayotgan kursantlar uchun mo‘ljallangan bo‘lib, boshqa yo‘nalarda tahsil olayotgan kursantlarga ham tavsiya etiladi.

Ushbu darslik “Python dasturlash tili” o‘quv fanining o‘quv dasturiga muvofiq ishlab chiqilgan.

Darslik 10 ta bob, 63 ta mavzu, 93 ta rasm va 11 ta jadvaldan iborat bo‘lib, har bir mavzuga doir turli dasturlar tuzilgan va ularning tuzilishini tahlil qilish bilan tushuntirib ketilgan. Shuningdek har bir mavzu so‘ngida mavzuga doir nazorat savollari berilgan.

UO‘K 004.031.6.
KBK 32.973.2-018.01

Taqrizchilar:

O. Allanov – PhD, Muhammad al-Xorazmiy nomidagi Toshkent Axborot Texnologiyalari universiteti “Kiberxavfsizlik va kriminalistika” kafedrası mudiri

D. Jaksimov – PhD, dotsent, Axborot-kommunikatsiya texnologiyalari va aloqa harbiy instituti “Chegara qo‘shinlari informatsion texnologiya mutaxassislarini tayyorlash bo‘yicha intellektual tizimlar” kafedrası boshlig‘i.

© **Sh.R. Sapayev, B.K. Yusupov, A.A. Abidov, 2024**
© “Aloqa nashriyoti”, 2024

M U N D A R I J A

KIRISH.....	6
I BOB. PYTHON DASTURLASH TILIGA KIRISH	8
1.1.§. Python dasturlash tili va uning rivojlanish tarixi	8
1.2.§. Python va PyCharm dasturini Windowsga oʻrnatish va birinchi dastur	18
II BOB. PYTHON ASOSLARI	26
2.1.§. Pythonda dastur strukturasi	26
2.2.§. Oʻzgaruvchilar va maʼlumotlar turlari	29
2.3.§. Pythonda kiritish va chiqish funksiyalari	33
2.4.§. Arifmetik operatorlar	36
2.5.§. Mantiqiy ifodalar	40
2.6.§. Shart operatori – if, elif	44
2.7.§. Pythonda takrorlanuvchi jarayonlarni dasturlash	49
2.8.§. Funksiyalar	56
2.9.§. Parametrli funktsiya	60
2.10.§. Qiymat qaytaruvchi funktsiya	65
2.11.§. Lambda funktsiya	67
2.12.§. Oʻzgaruvchilarning tiplarini oʻzgartirish	69
III BOB. SATRLAR BILAN ISHLASH	74
3.1.§. Satrlar va ularni eʼlon qilish	74
3.2.§. Satr bilan ishlovchi operatorlar va metodlardan foydalanish	81
3.3.§. Format metodining qoʻllanilishi	87
IV BOB. ROʻYXAT, KORTEJ, LUGʻAT VA TOʻPLAMLAR BILAN ISHLASH	92
4.1.§. Pythonda roʻyxat tushunchasi va u bilan ishlovchi metodlar bilan tanishish	92
4.2.§. Pythonda kortej tushunchasi va u bilan ishlovchi metodlar bilan tanishish	106
4.3.§. Pythonda lugʻat tushunchasi va u bilan ishlovchi metodlar bilan tanishish	110
4.4.§. Toʻplamlar bilan ishlash	119
V BOB. PYTHONDA FAYLLAR	126
5.1.§. Pythonda fayllar bilan ishlashning oddiy usuli	126
5.2.§. Pythonda matnli fayllar bilan ishlash	132
5.3.§. OS modulidan foydalanib fayllar bilan ishlash	136

5.4.§.	Fayllar bilan ishlovchi metodlar	146
5.5.§.	Faylga yo‘l, fayl nomi va fayl kengaytmasi bilan ishlash	150
5.6.§.	Faylni tozalash usullari	153

VI-BOB. PYTHONDA OB’YEKTGA MO’LJALLANGAN DASTURLASH

ASOSLARI	156
6.1.§. Pythonda ob’yektga mo’ljallangan dasturlash	156
6.2.§. Pythonda Sinflar va ob’yektlar	157
6.3.§. Pythonda “Inkapsulyatsiya” tushunchasi	165
6.4.§. Pythonda “Merosxorlik” tushunchasi	169

VII-BOB. PYTHONDA MA’LUMOTLAR BAZASI. SQLITE ASOSLARI

7.1.§.	SQLite ning kelib chiqish tarixi	174
7.2.§.	SQLite moduli bilan tanishish	175
7.3.§.	Ma'lumotlar bazasiga ma'lumotlarni kiritish	188
7.4.§.	SQLite bazasidan ma'lumotlarni olish	195
7.5.§.	Bazaga yozilgan ma'lumotlarni o'zgartirish	202
7.6.§.	Bazadagi ma'lumotni o'chirish	209

VIII-BOB. PYQT5 PAKETI

8.1.§.	Pythonda turli grafik modullar.....	216
8.2.§.	PyQt5 paketi va QtDesigner dasturini o‘rnatish	224
8.3.§.	PyQt5 da dastur tuzilishi	229
8.4.§.	PyQt5 paketining asosiy vidjetlari	323
8.4.1.§.	QLabel vidjeti	323
8.4.2.§.	QPushButton vidjeti	236
8.4.3.§.	QLineEdit vidjeti	241
8.4.4.§.	QComboBox vidjeti	243
8.4.5.§.	QCheckBox vidjeti	245
8.4.6.§.	QRadioButton vidjeti	247
8.4.7.§.	QTextEdit vidjeti	251
8.4.8.§.	QInputDialog vidjeti	254
8.4.9.§.	QMessageBox vidjeti.....	261
8.4.10.§.	QProgressBar vidjeti	268
8.4.11.§.	QSlider (slyayder) vidjeti	270

8.4.12.§.	QFileDialog vidjeti	273
8.4.13.§.	QPixmap vidjeti.....	276
8.4.14.§.	QSplitter vidjeti.....	278
8.4.15.§.	QCalendar vidjeti.....	280
8.5.§.	PyQt da CSS ssenariylarini qo'llanilishi	282
8.6.§.	PyQt5 da oddiy bloknot dasturi	285
IX-BOB. PYTHONDA TARMOQ DASTURLASH		297
9.1.§.	Pythonda Tarmoq dasturlashga kirish	297
9.2.§.	Mijoz-server ilovalari	299
9.3.§.	Server dasturini yaratish	302
X-BOB. PYINSTALLER va AUTO-PY-TO-EXE paketlari bilan ishlash		309
10.1.§.	Pythonning PYINSTALLER paketi va uning imkoniyatlari	309
10.2.§.	Pythonning AUTO-PY-TO-EXE paketi va uning imkoniyatlari	310
10.3.§.	Python paketlarini offline holatda kompyuterga o'rnatish	314
FOYDALANILGAN ADABIYOTLAR RO'YXATI		316

KIRISH

O‘zbekiston Respublikasi Prezidenti – Qurolli Kuchlari Oliy Bosh Qo‘mondoni Sh.M. Mirziyoyev ta’kidlab o‘tganidek, oliy harbiy ta’lim muassasalarining intellektual saviyasi yuksak, dunyoqarashi keng, uzoqni ko‘ra oladigan ofitser kadrlarni tayyorlash borasidagi faoliyatini tubdan qayta ko‘rib chiqish talab etiladi.

2019 yilning 8- oktabrida O‘zbekiston Respublikasi Prezidentining “O‘zbekiston Respublikasi oliy ta’lim tizimini 2030-yilgacha rivojlantirish konsepsiyasini tasdiqlash to‘g‘risida”gi PF -5847-sonli farmonining imzolanishi oliy ta’lim tizimidagi islohotlarni yanada shiddat bilan, sifatli amalga oshirilishiga turtki bo‘ldi.

O‘zbekiston Respublikasi Qurolli Kuchlari tashkil etilganligining 30 yilligi munosabati bilan Vatan himoyachilariga yo‘llagan bayram tabrigida Prezidentimiz quyidagilarni alohida ta’kidlab o‘tdilar: “...harbiy ta’lim va ilm-fan tizimida boshlangan islohotlarni izchil amalga oshirish, soha uchun kadrlar tayyorlashda nazariya va amaliyotni imkon qadar yaqinlashtirishga alohida e’tibor qaratish lozim”.

Davlatimiz rahbari ta’kidlaganidek, “Biz yashayotgan bugungi zamon hech qachon bir joyda to‘xtab turmasligini barchamiz o‘zimizga doimo yaxshi tasavvur qilganmiz va tasavvur qilamiz. Zamon talablariga javob bera oladigan, yangicha fikrlaydigan, zamonaviy bilimlarga ega bo‘lgan, yuksak malakali harbiy mutaxassislarni tayyorlash bugungi kunning dolzarb vazifasidir”.

Bugungi kunda kursant va tinglovchilarga ta’lim sohasida tashkil etilgan zamonaviy o‘quv-moddiy-texnik bazadan samarali foydalanishni ta’minlash, oliy harbiy ta’lim tizimidagi yo‘nalishlar va mutaxassisliklar bo‘yicha davlat ta’lim standartlari, mutaxassislik bo‘yicha malakaviy talablar, o‘quv dasturlari, o‘quv-metodik adabiyotlarni qayta ko‘rib chiqishi va takomillashtirish borasida ko‘plab ishlar amalga oshirilmoqda.

Ushbu “Python dasturlash tili” darsligining asosiy maqsadi – kursant va tinglovchilarga fanning yuqorida ko‘rsatib o‘tilgan talablarda mustaqil o‘qish jarayonida quyidagi bilim va ko‘nikmalarni hosil qilish: python dasturlash tilining kelib chiqishi, python dasturlash tili asoslari, satrlar bilan ishlovchi metodlar, python tilida ro‘yxat, kortej va lug‘atlar bilan ishlash, Python dasturlash tilida fayllar bilan ishlash, OS moduli imkoniyatlaridan foydalanish, pythonda ob‘yektga mo‘ljallangan dasturlash asoslari bilan tanishish, pythonda SQLite ma’lumotlar bazasi bilan ishlash, SQLite jadvallari bilan turli amallar bajarish, pythonda GUI dasturlarini yaratishda PyQt5 paketining asosiy vidjetlari bilan ishlash va ularga doir dasturlar tuzish va QtDesigner dasturidan foydalanishni o‘rganish, turli crossplatformali ilovalarni mustaqil ravishda ishlab chiqish ko‘nikmalarini shakllantirish.

O‘z-o‘zidan ma’lumki, davr talabidan va shiddat bilan rivojlanib borayotgan fan-texnika yutuqlaridan kelib chiqqan holda fanning o‘quv dasturi muntazam ravishda

yangilanib borishi talab etiladi. Shu sababli ushbu darslik tarkibiy tuzilishi, mavzularning yoritilish darajasi, ularni bayon etish usuli bo'yicha muayyan kamchiliklarga ega bo'lishi mumkin. Shunday ekan mualliflar bu haqida o'z fikr-mulohazalarini bildirgan kitobxonlarga o'z minnatdorchiligini bildiradi.

I-BOB. PYTHON DASTURLASH TILIGA KIRISH

1.1.§. Python dasturlash tili va uning rivojlanish tarixi.

Python dasturlash tili. Python yuqori darajadagi dasturlash tili bo'lib, u AT ning turli sohalarida, masalan, mashinani o'rganish, turli ilovalarni ishlab chiqish, web illovalari ishlab chiqish, tahlil qilish va boshqalarda keng qo'llaniladi.

2019-yilda Python Java tilini 10 foizga ortda qoldirib, eng ommabop dasturlash tiliga aylandi. Bu juda ko'p sabablarga bog'liq bo'lib, ulardan biri malakali mutaxassislariga to'lanadigan ish haqining yuqoriligidir (yiliga taxminan 100 ming dollar).

Hozirgi kunda Turli xil dasturlash tillari mavjud bo'lib, odatda ular ma'lum bir sohada (yoki bir nechta) liderlik qilishadi. Python dasturlash tili bo'lsa, turli xil sohalarda ilovalar yaratish imkomiyatiga egaligi, dasturchilarni o'ziga jalb qilmasdan qolmaydi.

Web ilovalar ishlab chiqish bozorining kichik qismini Python tili yaratilgan web illovalari egallagan. Python tilini desktop ilovalarini yozish uchun ham ishlatilsa, mashinani o'rganish (machine learning) sohasida to'liq yetakchilik qiladi.

Python nomining kelib chiqish tarixi.

Gido van Rossum python dasturlash tilini yaratganidan keyin, uni nomlamasdan turib tarqatishni hohlamadi. Ammo mukammal nom tanlash uchun vaqt sarflash Gvido uchun vaqtni behuda o'tkazish hisoblanardi. Shunda uning xayoliga kelgan birinchi fikr "Python" bo'ldi va dasturlash tilining nomini "Python"deb nomlab qo'yaqoldi. Chunki 1970-yillarning boshida mashhur bo'lgan, "Monty Python's Flying Circus" komediya seryali uning sevimli shoularidan biri edi.

Yangi til nomi bilan bir qatorda logotip o'ylab topish kerak edi va Guido tasodifiy shrift tanladi va "Python" so'zini shu shriftida yozdi. Gvido bu nomning ilonlarga aloqasi yo'qligini qayta-qayta ta'kidlagan bo'lsada, ommaning fikriga ta'sir qilishning ilojisi yo'q edi.

Ko'pchilik dasturchilar Python tilini ilonlar bilan bog'lab, uning nomini emas turli turdagi ilonlar rasmlari bilan kitoblar chop qilishar edi. Bu holat 2006 yilgacha davom etdi.

Payton (python) yoki Piton (ПИТОН) – bu qanday talaffuz qilinadi?

Bu ingliz teleko'rsatuvining nomi bo'ladimi yoki "ilon" so'zining inglizcha tarjimasi bo'ladimi, "Python" nomi Pyton kabi to'g'ri talaffuz qilinadi. Biroq, rus dasturchilarining taxminan 80% "Piton" (ПИТОН) deb talaffuz qilishga o'rganib qolishgan.

Ushbu variantlardan birini aniq to'g'ri ikkinchisini xato deb aytish qiyin. Biroq, "Piton" deb talaffuz qilish varianti faqat rus tilida so'zlashuvchi suhbatdoshlar bilan suhbatda foydalanish uchun mos keladi, chunki har qanday xalqaro konferentsiyada "Piton" so'zi tushunarsiz bo'ladi. Chunki ingliz tilida bunday so'z mavjud emas.

Bunday konferensiyalarda faqat "Python (Python)" deb talaffuz qilish kerak bo'ladi.

Logotip. Logotipda ikkita ilon kvadrat shaklda tasvirlangan, bu esa ko'pincha foydalanuvchilarni til nomini sudralib yuruvchi bilan bog'lashga undaydi.



1-rasm. Python dasturlash tili logotipi

Bu logotip muallifning ukasi, dasturchi va tip dizayneri Joost van Rossum tomonidan yaratilgan. U logotip dizaynini ishlab chiqishda kvadrat shaklini egallagan ikkita ilon va Flux Regular matn shriftida yozilgan Python so'zidan foydalandi.

Yaratilish tarixi. Til 1980-yillarning oxirida dasturchi Guido van Rossum tomonidan ishlab chiqilgan. O'sha paytda u Niderlandiyadagi matematika va informatika markazida ishlayotgan edi.

Guido van Rossum maktab yillaridanoq apparat vositalari bilan ishlashni yaxshi ko'rardi va u tengdoshlari tomonidan qo'llab-quvvatlanmasa ham, ma'qullamasa ham, bu uning mustaqil ravishda dasturlash tilini rivojlantirishiga to'sqinlik qilmadi.

Rossum bo'sh vaqtlarida Pythonda ishlagan va u bir paytlar rivojlanishiga yordam bergan ABC dasturlash tiliga asoslanib ishlagan.

Python dasturlash tili tarixidagi bosqichlar :

•**1991-yilning fevralda** python tilining kodlari *alt.sources* saytida chop etildi. Til ob'ektga yo'naltirilgan yondashuvga amal qildi va obyekt, class, metodlar, merosxorlik, funktsiyalar, istisnolarni qayta ishlash va barcha asosiy ma'lumotlar tuzilmalari bilan ishlash imkoniyatiga ega edi.

•**2000 yilda Pythonning ikkinchi versiyasi chiqdi.** Unga ko'plab muhim vositalar, jumladan Unicode yordami va qoldiqlarni yig'uvchi funksiyasi qo'shildi.

•**2008-yil 3-dekabrda Pythonning uchinchi versiyasi ishlab chiqildi.** Bu versiyasi hozircha oxirgi versiya hisoblanadi. Bu versiyada tilning ko'pgina xususiyatlari qayta ishlangan va oldingi versiyalar bilan mos kelmaydigan holga keltirildi. Uchinchi versiyaning funktsionalligi ikkinchisidan kam bo'lmasada, tilning rivojlanishi ikki tarmoqqa bo'lingan. Kimdir eski loyihalarni qo'llab-quvvatlash uchun Pythonning 2-versiyasida foydalanishni davom ettirsa, kimdir butunlay uchinchi versiyada ishlashni tanladi.

Ikkinchi versiyaning tugatilish vaqti 2015 yilga belgilangan edi. Biroq barcha mavjud kodlarni Python 3 ga o'tkazishga ulgurmaslikdan qo'rqib, Python 2 ning ishlash muddati 2020 yilgacha uzaytirilgan.

Pythonning sintaksisi uni har doim boshqa dasturlash tillaridan ajratib turadi. U ortiqcha operatorlardan xalos qilingan. Oddiy ingliz tili bilan sintaksisning o'xshashligi hatto oddiy foydalanuvchiga ham kodni tushunishga imkon beradi. Bundan tashqari, dasturchi kamroq kod yozadi, chunki ortiqcha belgilarni ishlatishning hojati bo'lmaydi. Masalan ; { }....

Pythonning oddiyligi qisman tilning ABC tiliga asoslanganligi bilan bog'liq bo'lib, u dasturlashni va dasturchi bo'lmaganlarning kundalik ishlarini o'rgatish uchun ishlatilgan.

Python kod yozishni soddalashtiradi va rivojlanishni tezlashtiradi, chunki u quyidagi xususiyatlarga ega:

•**Dinamik tip.** Dasturchi o'zgaruvchilar tipini ko'rsatishi shart emas, tilning o'zi o'zgaruvchiga qanday ma'lumot yuklanganiga qarab tipini aniqlab oladi.

•**Funktsiya orqali bir nechta qiymatlarni qaytarish.** Ushbu qiymatlarni vergul bilan ajratilgan holda to'rtburchak qavslar ichiga yoziladi va avtomatik ravishda

ro'yxatga aylantiriladi. Funktsiyadan massivni qaytarish uchun "`return ro'yxat_nomi`" yozish kifoya.

•**Avtomatik xotiradan joy ajratish.** Dasturchi o'zgaruvchilarga xotiradan joy ajratish uchun ortiqcha kod yozishi talab qilinmaydi. Bu, bir tomondan, dasturchining dastur ustidan nazoratini kamaytiradi, ikkinchi tomondan, rivojlanish sezilarli darajada tezlashadi.

•**Chiqindilarni yig'uvchi.** Agar ob'ekt keraksiz bo'lib qolsa, u chiqindi yig'uvchi tomonidan avtomatik ravishda o'chiriladi. Chiqindi yig'uvchi sizga xotiradan foydalanishni optimallashtirish va keraksiz narsalarni qo'lda o'chirmaslik imkonini beradi.

•**a, b = b, a ifodasi.** Bu ifoda o'zgaruvchilarning qiymatlarini o'zaro almashtiradi. Endi **a** o'zgaruvchining qiymati **b** o'zgaruvchisiga yuklandi va aksincha. Shunday qilib, siz nafaqat ikkita o'zgaruvchining, balki uch va undan ortiq ob'yektlarning qiymatlarini mos ravishda bir-biriga almashtirishingiz mumkin. Faqat taraflardagi o'zgaruvchilar soni teng bo'lishi talab qilinadi.

•**O'zgaruvchi tipining qiymatga bog'liqligi.** O'zgaruvchining qiymati - bu uning turini va boshqa xususiyatlarini belgilaydigan atributlarga ega bo'lgan qandaydir ob'ekt. O'zgaruvchi esa bu ob'ektning nomidir. Ushbu yondashuv qat'iy tipdagi ta'riflarga bo'lgan ehtiyojni yo'qqa chiqaradi va o'zgaruvchilarga boshqa turdagi qiymatni qayta tayinlashni sezilarli darajada soddalashtirdi.

•**For tsikli** . Pythonda massivlar, ro'yxatlar va boshqa konteynerlar bilan ishlash oson va qulay. Uning barcha elementlarini takrorlash zarur bo'lganda, konstruktsiya quyidagicha ko'rinadi: `for x in konteyner:` (*iteratsiya 0 dan oxirgi -1 elementgacha elementlarni x ga qaytaradi*). Agar siz tsiklni aniq bir miqdorda takrorlanishini amalga oshirish uchun `range()` funksiyasidan foydalanish kerak bo'ladi: `for x in range(1,9):` (*tsikl 1 dan 8 gacha bo'lgan qiymatlarini x ga uzatadi*).

•**Machine learning sohasidagi yetakchiligi.** Python dasturlash tili "Mashina o'rganish" (Machine learning) sohasida yaqqol yetakchilik qilib kelmoqda. Ilm-fan bilan u yoki bu tarzda shug'ullanadigan odamlar kod yozish kabi narsalarga ko'p vaqt

sarflamaslikni afzal ko'radilar, shuning uchun Python ularga yuklangan vazifalarni amalga oshirish uchun juda mos keladi.

Python ham soddalik, ham kuchli vositalarni o'zaro birlashtiradi. U deyarli har qanday dasturning prototipini yaratish uchun ishlatilishi mumkin.

Kod misoli :

```
def kattasi(a, b):
    if a > b:
        print(a, "katta", b, "dan")
    else:
        print(b, "kichik", a, "dan")
def max_mass (massiv):
    max = 0
    for x in massiv:
        if massiv[x-1] > max:
            max = massiv[x]
    return max

def 2mass(massiv):
    massiv = massiv * 2
    return massiv

print("Oddiy Python dasturi")

a = [1,2,3,6,1,6]
kattasi (1,5)
r1 = max_mass(a)
r2 = 2mass (a)

print("Massiv max elementi -", r1)
print("Ikkilangan massiv - ", r2)
```

Dastur natijasi:

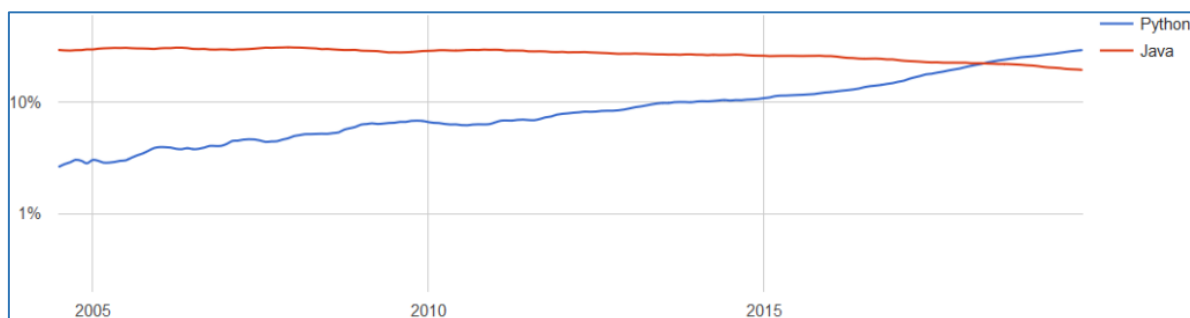
```
Oddiy Python dasturi
5 katta 1 dan
Massiv max elementi - 6
Ikkilangan massiv - [1, 2, 3, 6, 1, 6, 1, 2,3,6,1,6]
```

Ommalashganligi. Til 30 yoshdan oshgan bo'lsada, butun dunyodagi dasturchilar orasida juda keng tarqalgan. Python deyarli har bir o'rta yoki katta loyihada, asosiy ishlab chiqish vositasi sifatida bo'lmasada, prototiplash yoki uning bir qismini yozish uchun vosita sifatida ishlatiladi.

U o'z atrofida juda katta dasturchilar guruhini to'pladi.

Stackoverflow saytida o'tkazilgan so'rov natijalariga ko'ra, Python deyarli 39% ovoz bilan 7-o'rinni egalladi.

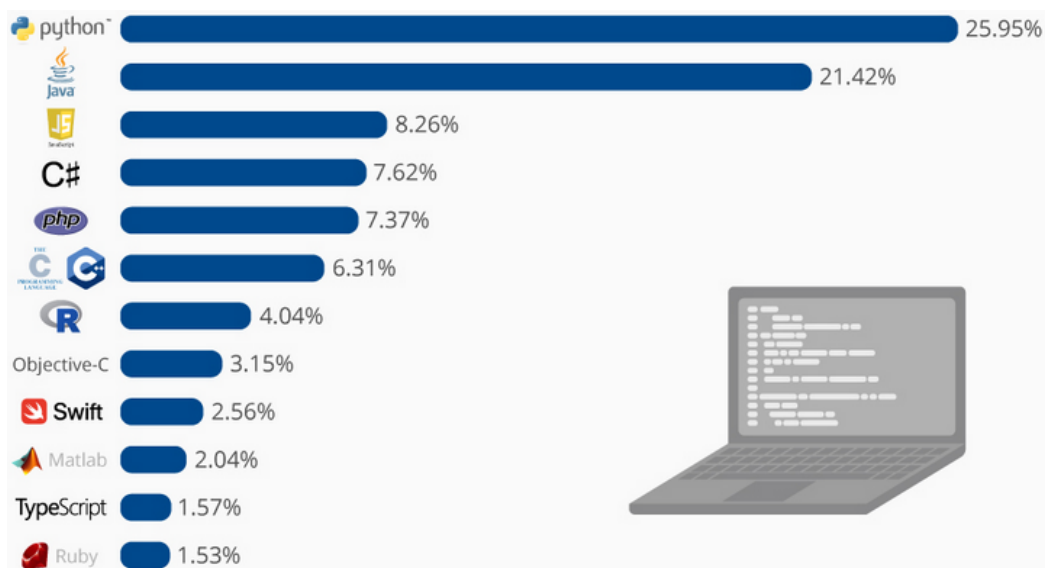
PYPL. Ushbu indeks tilni o'rganish materiallari bilan bog'liq qidiruv so'rovlari soniga asoslanadi.



2-rasm. PYPl indeksi bo'yicha solishtirma sxemasi

PYPL ma'lumotlariga ko'ra, Python mashhurligi bo'yicha Javadan 10% ga oldinda.

statista.com. Xizmat turli xil statistik ma'lumotlarni taqdim etadi, ular orasida dasturlash tillarining mashhurligi bo'yicha statistikalar ham mavjud.



3-rasm. **Statista.com** saytidagi solishtirish diagrammasi

Ish tezligi. Dasturchilar ko'pincha o'zlariga savol berishadi: "Python-dan foydalanish ishlashning pasayishiga olib keladimi?". Batafsil tekshiruvsiz hech qanday xulosa chiqarmang.

Agar biz faqat kodni bajarish tezligini hisobga olsak, Python C kabi boshqa dasturlash tillaridan past ekanligi ayon bo'ladi. Darhaqiqat, dinamik terish, izohlash va

dasturchi ishini osonlashtiradigan boshqa xususiyatlar ishlashning pasayishiga olib keladi. Biroq, zamonaviy IT-da nafaqat dasturlarning tezligi, balki ularni ishlab chiqish tezligi ham muhimdir. Ishlab chiqish, sinovdan o'tkazish, disk raskadrovka va qo'llab-quvvatlash - bularning barchasi juda ko'p pul talab qiladi. Agar Python dasturlarining tezligi bo'yicha past bo'lishi mumkin, ammo rivojlanish tezligida bo'yicha unga teng keladigani yo'q.

Dasturchilar Pythonda yaratilgan dasturlarni bajarilish tezligini oshirish uchun turli yo'llardan foydalanadilar:

- **C kodini o'rnatish** . Ushbu texnikadan foydalanib, siz ishlashni sezilarli darajada yaxshilashingiz mumkin, odatda vaqt birligida ko'p so'rovlarni qayta ishlaydigan kod bo'limlari C tilida yoziladi. Masalan, bitta ma'lumotlar bazasidan ma'lumotlarni qabul qiladigan, uni qayta ishlaydigan va boshqasiga yuboradigan funksiyalar, agar o'tadigan ma'lumotlarning miqdori yetarlicha katta bo'lsa, C tilida yozilgani afzal xisoblanasi.

- **ifodalarni optimallashtirish**. Python dasturlarining tezligi ifodalarning ishlash tezligiga juda bog'liq. Quyidagi jadvalda tezroq va sekinroq ishlovshi ifodalar keltirilgan.

1-jadval

Tezroq	Sekinroq
<code>a, b = c, d</code>	<code>a = c; b = d</code>
<code>a < b < c</code>	<code>a < b and b < c</code>
<code>not not a</code>	<code>bool(a)</code>
<code>a = 5</code>	<code>a = 2 + 3</code>

- **Dasturni testlash uchun tayyor modullar**. Kodning qaysi bo'limlari umumiy ish faoliyatini sezilarli darajada kamaytirishini aniqlash uchun dasturchi testlash uchun maxsus modullardan foydalanishi mumkin. Shunday qilib, bu modullar yordamida qaysi kodni optimallashtirish yoki C kodi bilan almashtirish kerakligini bilib olishi mumkin.

•**Tayyor asboblari.** Aksariyat vazifalar uchun samarali echimlar allaqachon ishlab chiqilgan. O'z yechimingizni noldan yozishdan ko'ra, ba'zi kutubxonaning tayyor, tuzatilgan kodini ishlatish yaxshiroqdir, bu 100% samarali bo'lmaydi.

Pythonda qanday dasturlar yozish mumkin?

Python tilida turli sferalarda dasturlar tuzish mumkin.

Back-end – saytning dasturiy qismi. Saytning server tomonini rivojlantirish uchun turli frameworklar qo'llaniladi: **Django** va **Flask**. Bu frameworklar Pythonni boshqa mashhur vositalar bilan raqobatlashadigan imkoniyatlarga ega server tomonidagi dasturlash tiliga aylantiradilar.

PHP tili server tomonidagi web ishlab chiqish bozorining ko'p qismini nazorat qilsada, tobora ko'proq dasturchilar Pythonda ishlab chiqishni afzal ko'rishmoqda.

Blokcheyn. Blokcheyn - bu ketma-ket bloklar zanjiri bo'lib, unda har bir blok ma'lumotni o'z ichiga oladi va har doim oldingisiga ulanadi. Texnologiya har qanday sohada qo'llanilishi mumkin. Ayniqsa moliya sektorida va bitcoin kriptovalyutasi sohasida mashhurdir.

Blokcheyn axborot xavfsizligi va ochiqligini o'zida mujassamlashtiradi. U foydalanuvchiga dunyoning istalgan nuqtasidan ma'lumotlarga kirish imkonini beradi.

Bot. Bu ma'lum bir vaqtda yoki berilgan signalga javoban ba'zi harakatlarni avtomatik ravishda bajaradigan dasturdir. Botlar inson xatti harakatlarini bevosita taqlid qilishi mumkin. Shuning uchun ular ko'pincha texnik yordam ko'rsatishda (chat botlarida), Internetda ma'lumot qidirishda (qidiruv botlarida), virtual dunyoda odam yoki boshqa mavjudotning harakatlarini taqlid qilishda (kompyuter o'yinlari) zarur bo'ladi.

Python sizga tezkor xususiyatga ega va nisbatan aqlli botlarni yaratish imkonini beradi. Shuni tushunish kerakki, botlar 500 qatorli kodlardan iborat oddiy dastur emas. Biznes uchun bot yaratish buyurtmasi bir necha millionga tushishi mumkin. Narx insondan farqlash qiyin bo'ladigan botni loyihalash juda qiyin ekanligi bilan bog'liq. Muloqotning ko'plab variantlarini ta'minlash, inson xatti-harakatlarining omillarini tahlil qilish va ularni dasturda amalga oshirish kerak. Oddiy

qilib aytganda, faqat nol va birlarni tushunadigan mashinadan siz ibtidoiy "miya" qilishingiz kerak bo'ladi.

Malumotlar bazasi. Ma'lumotlar bazasi umumiy xususiyatlar va maxsus qoidalarga muvofiq tizimlashtirilgan ma'lumotlardir. Har qanday yirik loyihada ma'lumotlar bazalaridan foydalaniladi. Ular foydalanuvchilar haqidagi ma'lumotlarni, dasturdagi o'zgarishlarni va hokazolarni saqlaydi.

Ma'lumotlar bazasini boshqarish tizimi Pythonda yozilishi mumkin.

Kengaytirilgan haqiqat (Дополненная реальность). "To'ldirilgan reallik" virtual texnologiyalar yordamida jismoniy dunyoni to'ldiradi. Ya'ni, virtual ob'ektlar real muhitga proyeksiya qilinadi va oddiy jismoniy ob'ektlarning xususiyatlari va xatti harakatlariga taqlid qiladi.

"Kengaytirilgan haqiqat"ni turli 3D filmlarda guvohi bo'lish mumkin. Haqiqiy dunyoda u, masalan, jangovar jangchilarda (maqsad tizimi) ishlatiladi.

"Kengaytirilgan haqiqat" ishi teglar bilan o'zaro ta'sirga asoslangan. Elektron qurilma ma'lumot oladi va atrofdagi makonni tahlil qiladi, kompyuter ko'rish yordamida u odamning oldida nimani ko'rayotganini "Tushunadi". Keyin qurilma real dunyoda "Virtual qatlam"ni qoplaydi.

"Kengaytirilgan haqiqat" sferasida yaratilgan mukammal dasturlar taxminan 7-10 ming AKSH dollarini tashkil qiladi. Ularni loyihalash va yozish oson emas, 3D-dizaynerlardan dasturchilargacha turli mutaxassislar ishlab chiqish jarayonida tinmasdan mehnat qilishadi.

Python – bunday sferadagi loyihalarni yaratish uchun ajoyib tanlov.

BitTorrent mijozi. BitTorrent - bu Internet orqali katta hajmdagi ma'lumotlarni tezlikda uzatish, qabul qilish imkonini beruvchi noyob texnologiya.

BitTorrentning 6-versiyasidan keyingi versiyalari butunlay Pythonda yozilgan. Keyinchalik u C++ da butunlay qayta yozilgan bo'lsa-da, bu Python tilidan xuddi shunga o'xshash vazifalarni bajarish uchun foydalanish mumkinligini anglatadi.

Neyron tarmoq. "Neyron tarmoq" tushunchasi dasturlashga biologiyadan kirib kelgan. Biologiyada neyron tarmoq bir-biriga bog'langan neyronlar ketma-

ketligidir. Dasturiy ravishda yaratilgan neyron tarmoqlar nafaqat ma'lumotni tahlil qilish va saqlash, balki uni xotira qayta ishlab chiqishga qodir.

Ular inson miyasi tomonidan bajariladigan hisob-kitoblarni talab qiladigan murakkab masalalarni hal qilish uchun ishlatiladi. Odatda, neyron tarmoqlar biror narsani xususiyatlari bo'yicha tasniflash, bashorat qilish, masalan, fotosurat yoki videodan odamni tanib olish kabi funksiyalarni amalga oshirish uchun foydalaniladi.

Python neyron tarmoqlarni rivojlantirishda yaqqol yetakchi hisoblanadi. Standart vositalardan tashqari, u “Mashinani o'rganish” sohasida juda ko'p kutubxonalar mavjud. Buning yordamida hatto katta va murakkab loyihalar nisbatan tez yozilishi mumkin.

Parser - bu ma'lumotlarni yig'ish va qayta ishlash uchun mo'ljallangan dastur. Foydalanuvchi valyuta kursi kabi ma'lumotlarni tahlil qilishi yoki turli kompaniyalar aksiyalaridagi o'zgarishlarni kuzatishi va tahlil qilishi mumkin.

Parser turli xil tillarda yozilishi mumkin. Ya'ni parser dasturlar boshqa dasturlash tillari yordamida ham yaratish mumkin. Lekin parser (ma'lumot yig'uvchi) dasturlarni python tilida boshqa dasturlash tillariga qaraganda tez va samarali yozish mumkin.

O'yin yaratish. Pythonda katta o'yinlar yaratilmaydi. U prototipni (demo versiyasini) ishlab chiqish yoki ba'zi bir qismini amalga oshirish uchun ishlatiladi (masalan, o'yinning server qismidagi logik qismi).

Kichik loyihani yozish uchun siz kichik 2D o'yinni yaratish uchun barcha kerakli vositalarni taqdim etadigan Pygame kutubxonasidan foydalanishingiz mumkin.

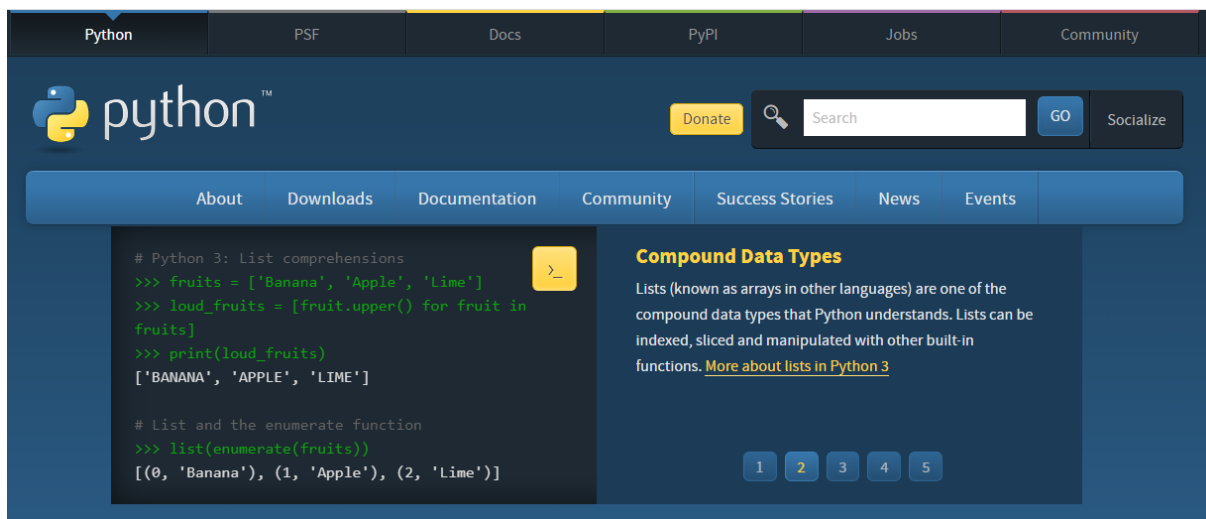
Nazorat savollari:

1. Python qanday dasturlash tillari sirasiga kiradi. Interpretatsiya qilinuvchimi yoki kompilyatsiya qilinuvchi?
2. Python nomi qanday tarixga ega?
3. Python logotipi kelib chiqishi qanday?
4. Python tili bilan qanday dasturlar tuzish mumkin?
5. Pythonning reytingi qanday?
6. Python tilining boshqa tillardan qanday ustunliklari bor?

1.2.8. Python va PyCharm dasturini Windowsga o'rnatish va birinchi dastur

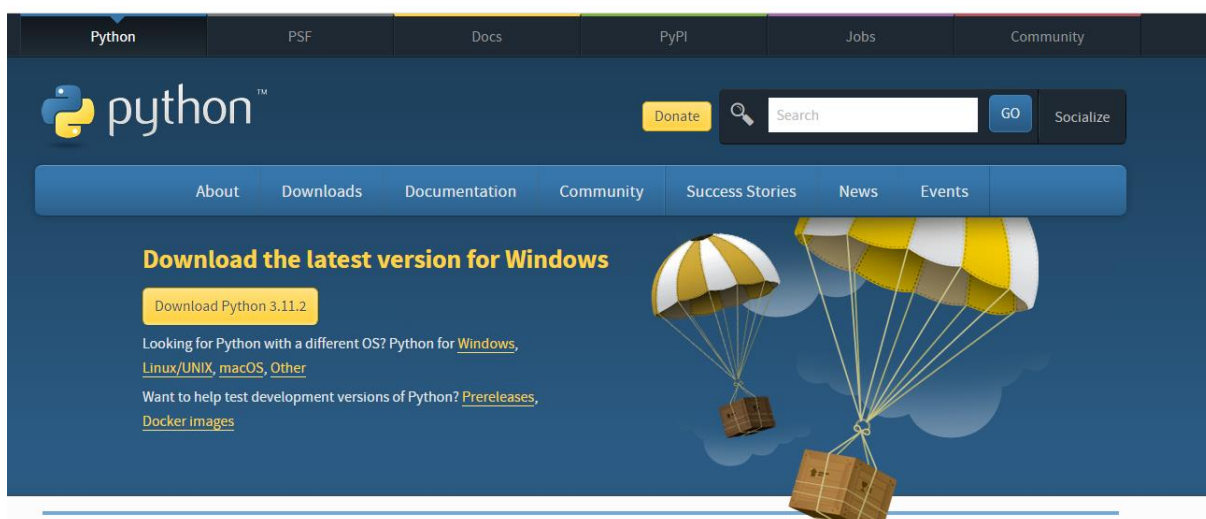
Python dasturini windows operatsion tizimiga o'rnatish.

1-qadam. Python-ni yuklab olish va o'rnatish uchun Pythonning rasmiy veb sayti python.org ga kiriladi (4-rasm) va **Downloads** bo'limidan pythonning oxirgi versiyasini tanlanadi (5-rasm).



4-rasm. Python.org saytining ko'rinishi

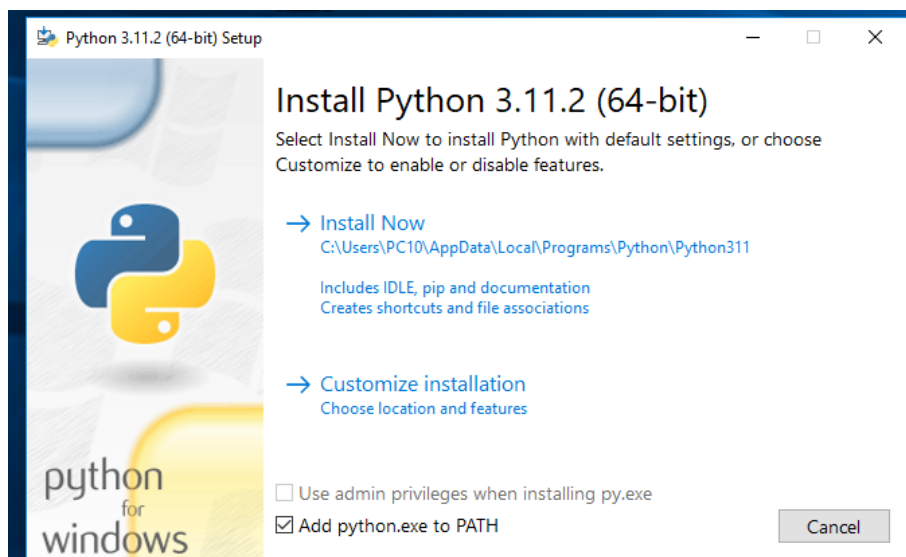
2-qadam. Pythonning tegishli oxirgi versiyasiga doir, o'rnatiluvchi faylni yuklab olish tugallangach, Pythonni o'rnatish uchun yuklab olingan, yuklanuvchi faylni (.exe faylini) sichqoncha tugmasi bilan ikki marta bosish orqali ishga tushiriladi. (5-rasm)



5-rasm. [Python.org](https://python.org) saytidan python dasturini ko'chirib olish oynasi

Keyin paydo bo'ldan oynadan "Install now" tugmasini bosiladi.

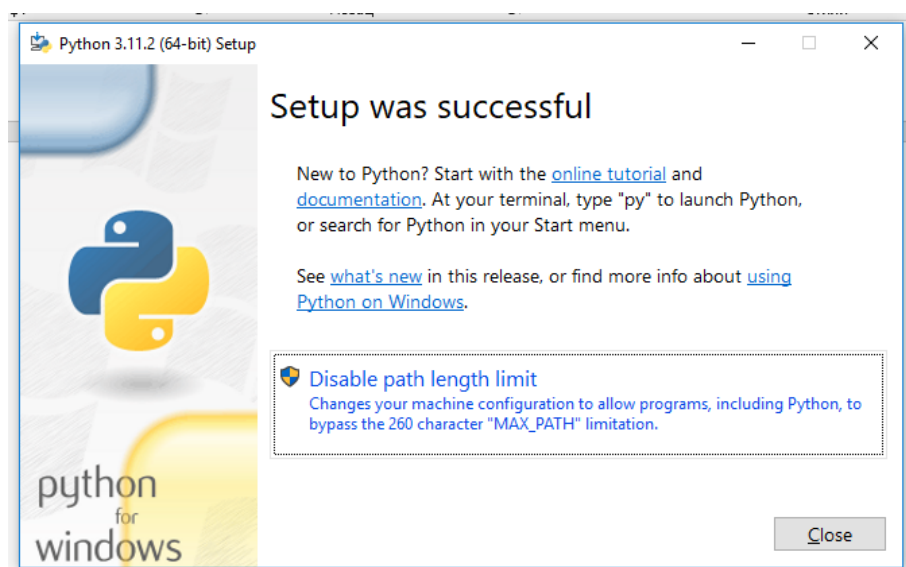
Shundan so'ng Python o'rnatilishi jarayonini ko'rishingiz mumkin.



6-rasm. Python.exe faylini ishga tushirishdan hosil bo‘ladigan o‘rnatish oynasi ko‘rinishi

3-qadam. O‘rnatish tugallangach, o‘rnatish muvaffaqiyatli amalga oshirilgan bo‘lsa, quyidagi panel ekranda paydo bo‘ladi va **“Close”** tugmasini bosilib paneldan chiqiladi.

Bilamizki barcha dasturlash tillari uchun o‘zining kodlarini kiritish uchun qulay kompilyator yoki interpretatori mavjud bo‘ladi. Masalan C++ va C# uchun VisualStudio bo‘lsa, web bilan ishlovchilar uchun VisualStudioCode, SublimeText3 va shu kabi dasturlardan foydalaniladi. (7-rasm)



7-rasm. O‘rnatilish muvoffaqiyatli tugaganligi to‘g‘risidagi oyna

Python tili bilan ishlashda dasturchi dastur kodini interpretatsiya qilib beruvchi, o‘ziga qulay dasturni o‘rnatib olishi kerak bo‘ladi. Bunday dasturlar turi juda ko‘p.

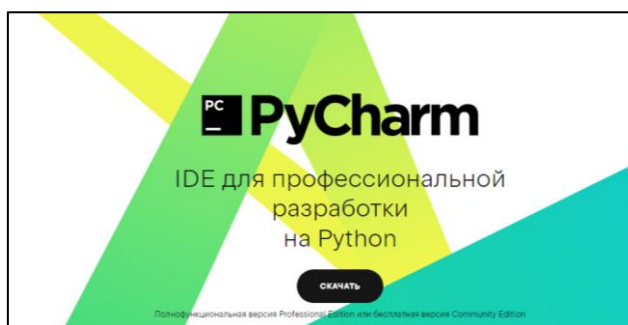
Masalan: VSCode, SublimeText, PyCharm, VisualStudio, Atom bu ro'yxatni yana davom ettirish mumkin. Ammo bulardan bittasini tanlash kerak bo'lganda PyCharm dasturini tanlash maqsadga muvofiq bo'ladi.

PyCharm - JetBrains tomonidan ishlab chiqilgan kross-platformali interpretator dasturi. Pycharm dasturi Python uchun zarur bo'lgan barcha vositalarni taqdim eta oladi. Quyida Windows muhiti uchun Python va PyCharmni o'rnatish bo'yicha batafsil ko'rsatmalar keltirib o'tilgan.

PyCharm dasturini windows operatsion tizimiga o'rnatish

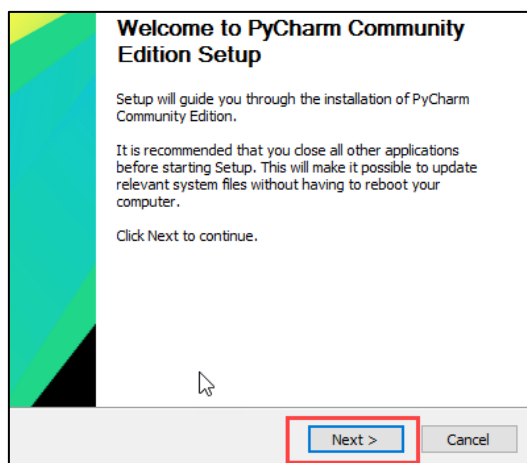
Dasturni o'rnatish judayam oddiy. Boshqa o'rnatiluvchi dasturlar kabi o'rnatiladi. Buni qadamma qadam ko'rib chiqamiz:

PyCharmni yuklab olish uchun jetbrains.com/ru-ru/pycharm (8-rasm) veb saytiga tashrif buyurish kerak va “Скачать” tugmasini bosiladi.



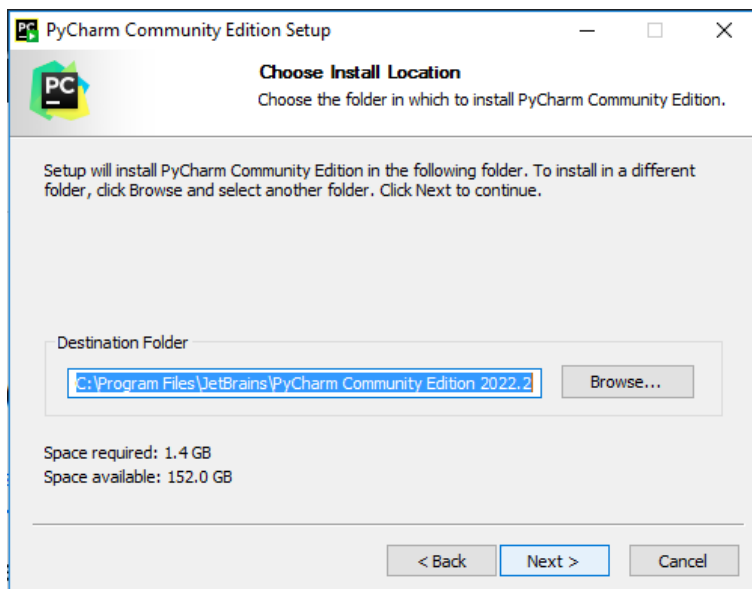
8-rasm. jetbrains.com/ru-ru/pycharm saytining ko'rinishi

Yuklab olish tugallangach, PyCharm-ni o'rnatish uchun ko'chirib olingan, yuklanuvchi faylni (.exe faylini) ishga tushiriladi. Installer ishga tushadi va “Next” tugmasini bosish orqali keyingi oynasiga o'tiladi.



9-rasm. PyCharm dasturini o'rnatish jarayoni

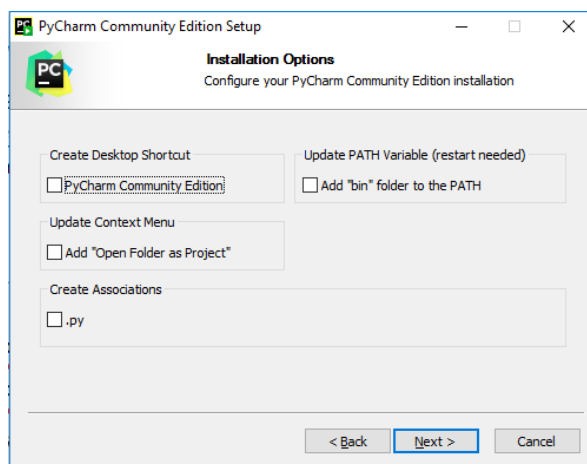
Agar dasturni o'rnatilish yo'lini ko'rsatilmasa, unda dastur kompyuterning operatsion tizimi ko'rsatgan joyga o'rnatiladi. Kimdir dasturning o'rnatilish joyini o'zgartirishni istasa unda, o'rnatilish joyini **“Browse..”** tugmasini bosib ko'rsatib ketish kerak bo'ladi.



10-rasm. PyCharm dasturini qaysi URL ga o'rnatishni tanlash oynasi

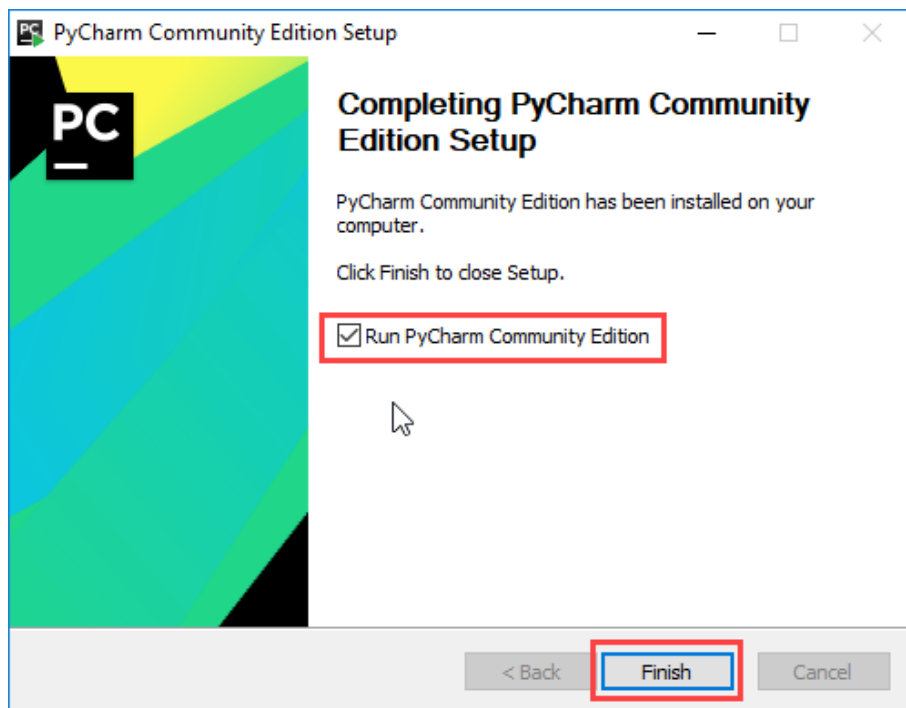
Keyingi bosqichda, ish stolida dastur yorlig'ini yaratish mumkin bo'ladi. Buning uchun **“Create desktop Shortcut”** bo'limini belgilab, so'ngra **“Next”** tugmasini bosiladi.

Add “bin” folder to the PATH bo'limini tanlab qo'yish maqsadga muvofiq bo'ladi (11-rasm). Chunki bu hozir belgilanmasdan o'rnatiladigan bo'lsa, keyinchalik bu Path ni qo'lda ko'rsatib o'rnatish talab qilinadi. Bu esa noqulaylik tug'dirishi mumkin.



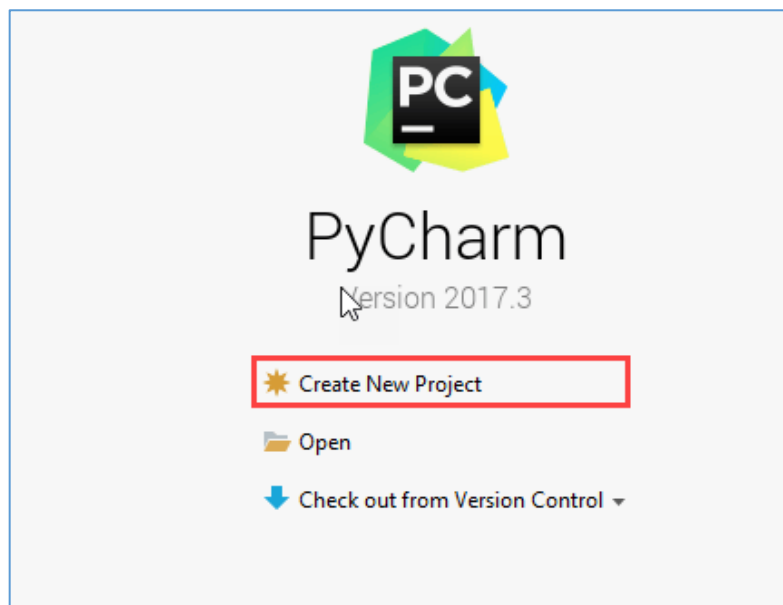
11-rasm. “Installetion option” oynasi

O‘rnatishni tugatish. O‘rnatish tugallangandan so‘ng PyCharm o‘rnatilganligi haqida xabar paydo bo‘ladi. Agar dasturchi uni ishga tushirishni xohlasa, avval "Run PyCharm Community Edition" katagiga belgi qo‘ying va "Finish" tugmasini bosishi kerak.



12-rasm. O‘rnatish yakunlanganligi haqidagi habar oynasi

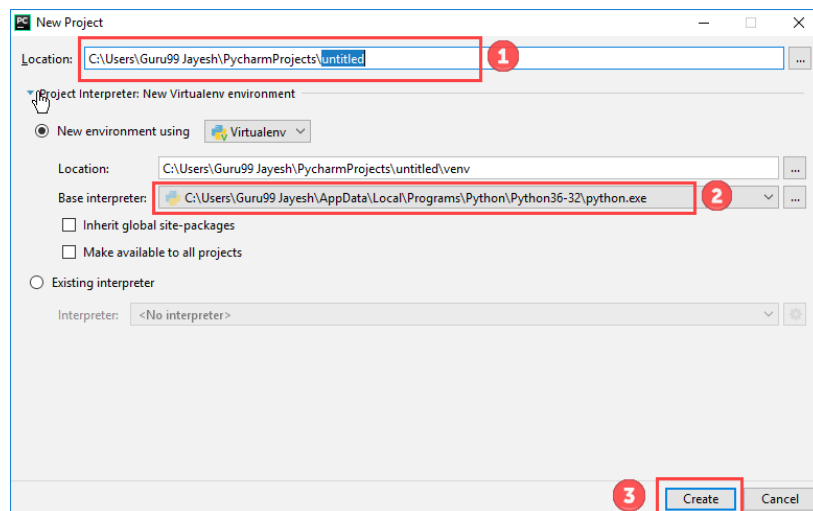
PyCharm dasturini ishga tushirilgandan keyin quyidagi “Welcome to PyCharm” oynasi paydo bo‘ladi. Yangi loyiha yaratish uchun "Create New Project" tugmasini bosiladi.



13-rasm. Yangi loyiha yaratish oynasi

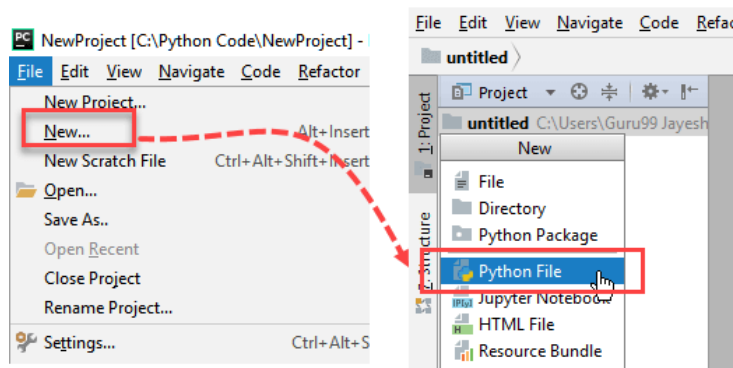
Loyiha yaratilayotganda uning nomi va joyini tanlanadi. Agar joylashish joyini o'zgartirilmasa, unda oparatsion tizim python qayerga o'rnatilgan bo'lsa, o'sha joyda proyekt yaratiladi.

Nom berish va joy tanlangandan keyin "Create" tugmasini bosiladi.



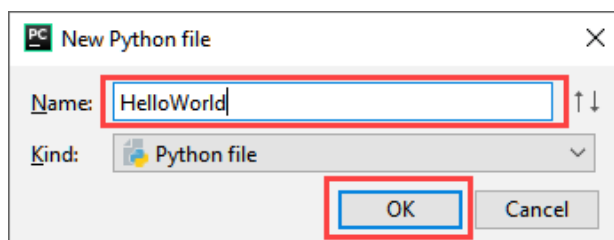
14-rasm. Yangi loyiha yaratish uchun loyihaga nom berish oynasi

Hosil bo'lgan oynadan **File** menyusiga o'tiladi va **New** buyrug'ini tanlangandan keyingi kontekst menyudan **Python file** bo'limi tanlanadi.



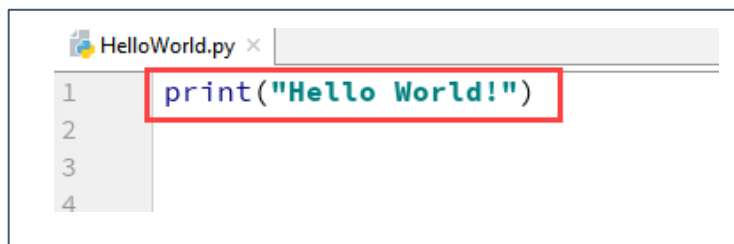
15-rasm. Loyiha ichida python fayl yaratish oynasi

Yangi paydo bo'lgan oynadan fayl nomini kiritiladi va **OK** yoki klaviaturadan **Enter** tugmasini bosish kerak bo'ladi.



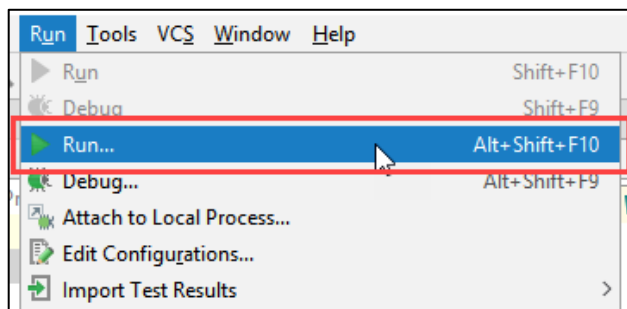
16-rasm. Loyiha ichida yaratilgan python faylga no berish oynasi

Shundan so‘ng Yangi hosil bo‘lgan oynada “Salom dunyo” dastur yozish uchun quyidagi kodni kiritish kifoya: `print("Salom Dunyo!")`



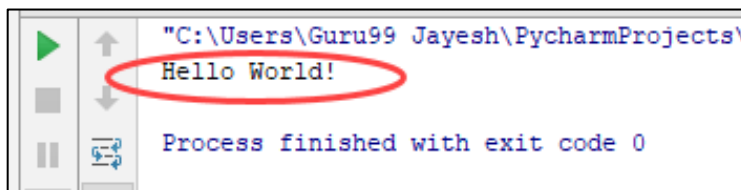
17-rasm. Salom dunyo dasturi

“Salom dunyo” dasturini ishga tushirish uchun menular satridan “RUN” menyusini tanlanadi va u yerdan “Run” buyrug‘i tanlanadi. Yoki oddiygina dastur kodining ustiga yoki bo‘sh joyga sichqoncha o‘ng tugmasini bosiladi va hosil bo‘lgan konteks menudan “Run” buyrug‘ini tanlash orqali ishga tushirish mumkin.



18-rasm. Salom dunyo dasturini ishga tushurish jarayoni

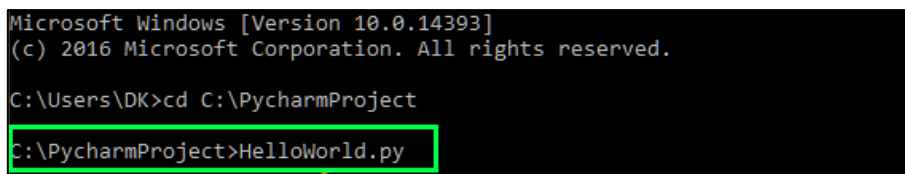
Dastur natijasi ekranning pastki qismida, konsolda paydo bo‘ladi.



19-rasm. Salom dunyo dasturining natijasi

Python faylni konsoldan ishga tushirish

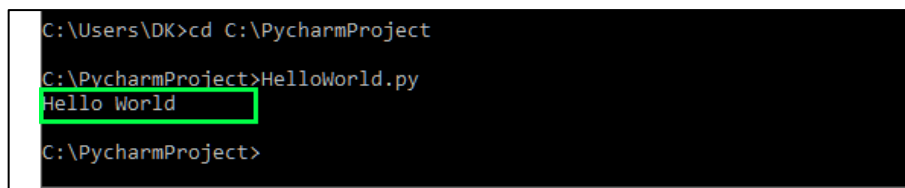
Agar kompyuterda python faylni ochish uchun interpretatorlar o‘rnatilmagan bo‘lsa, python faylni “Buyruq satiri”dan ishga tushirish mumkin.



20-rasm. Salom dunyo dasturini konsoldan ishga tushirish

Dasturni ishga tushirish uchun faylga to'g'ri yo'lini ko'rsatish kerak.

Kodning natijasi quyidagicha bo'ladi:

A screenshot of a Windows command prompt window with a black background and white text. The first line shows the command 'C:\Users\DK>cd C:\PycharmProject'. The second line shows the command 'C:\PycharmProject>HelloWorld.py'. The third line shows the output 'Hello World' which is highlighted with a green rectangular box. The fourth line shows the prompt 'C:\PycharmProject>'.

21-rasm. Salom dunyo dasturini konsoldan ishga tushurishdan olingan natijasi

Nazorat savollari:

1. Python kodlari qaysi dasturlarga yozish mumkin?
2. PyCharm dasturi qaysi kompaniyaning dasturiy maxsuloti hisoblanadi?
3. Krosplatformali dastur deganda nimani tushunasiz?
4. Pythonni o'rnatilish jarayonini tushuntirib bering.
5. PyCharmni o'rnatilish jarayonini tushuntirib bering.
6. Python faylning formati qanday bo'ladi?

II BOB. PYTHON ASOSLARI

2.1.8. Pythonda dastur strukturasi

Python dasturlash tilida yozilgan dastur ifodalar jamlanmasidan iborat. Har bir ifoda yangi qatorga joylashtiriladi. Masalan:

```
print(2 + 3)
print("Hello")
```

Python dasturlash tili kodida ifodalarning oldidan qoldirilgan oraliq masofalar katta ro'l o'ynaydi. Agar dasturchi tomonidan bilmasdan yoki chiroy uchun qoldirilgan har qanday oraliq masofa xatolik kelib chiqishiga sabab bo'lishi mumkin. Masalan, garchi kod yuqoridagi bilan deyarli bir xil bo'lsada, quyidagi holatda dasturda oraliq masofalar bilan bog'liq xatolik sodir bo'ladi:

```
print(2 + 3)
print("Hello")
```

Shuning uchun, python dasturlash tilida dastur kodining boshi satrning boshidan boshlanishi kerak. Bu Python bilan C# yoki Java kabi boshqa dasturlash tillari o'rtasidagi muhim farqlardan biridir.

Ammo shuni yodda tutish kerakki, tilning ba'zi operatorlarining sintaksisi bir necha qatordan iborat bo'ladi. Masalan, shart operatori if:

```
if 1 < 2:
    print("salom")
```

Bu dasturda, agar 1 soni 2 dan kichik bo'lsa, u holda **“salom”** so'zi ekranga chiqariladi. Bu dasturda print(“Salom”) ifodasining oldida bo'sh joy bo'lishi kerak, chunki bu ifoda if shartli konstruktsiyasining bir qismi sifatida ishlatiladi.

Bundan tashqari, kodni yozish mobaynida qoldirib ketiladigan bo'shliqlar 4 ga karrali probellar soniga (ya'ni 4, 8, 16 va boshqalar) teng bo'lishi maqsadga muvofiqdir. Ammo 4 ga karrali bo'lmay qolishi xatolik keltirib chiqarmaydi. Ifoda oldida qoldiriladigan masofalarning 4 ta probelga yoki 4ga karrali probellar soniga teng bo'lishining sababi shundaki, 1 ta tab 4 ta probelga tengligida.

Harflar katta-kichikligini farqlanishi

(Регистрозависимость)

Python katta-kichik harflarni turli belgilan sifatida qabul qiladigan tildir. Shuning uchun **print** va **Print** yoki **PRINT** ifodalari turli ifodalarni anglatadi. Shunday ekan agar konsolga “Salom dunyo” xabarini Print funksiyasi orqali chiqarmoqchi bo‘lsak, hech qanday narsa chiqara olmaymiz, aksincha xatolik sodir bo‘ladi:

```
Print("Hello World")
```

Izohlar

Izohlar – kodning ma’lum bir qismida nima amalga oshirilayotganligini ifodalab foydalanuvchi uchun tushunchalar berib ketish uchun foydalaniladi. Dastur interpretatsiya qilinganda (ishga tushirilganda) interpretator izohlarni e’tiborsiz qoldiradi, shuning uchun ular dasturning ishlashiga ta’sir qilmaydi. Pythondagi izohlar blok va inline izohlarda keladi.

Bitta satrda joylashuvchi izohlardan oldin panjara belgisi- # qo‘yiladi:

```
# Konsolga chiqish
# Salom Dunyo xabarlar
print("Salom Dunyo ")
```

belgisidan keyingi har qanday belgilar to‘plami izohni bildiradi. Ya’ni, yuqoridagi misolda kodning birinchi ikki qatori izohlar hisoblanadi.

Shuningdek izohlar ifodalardan so‘ng, til ifodalari bilan bir qatorda joylashgan bo‘lishi ham mumkin:

```
print("Salom dunyo")    # Xabarni konsolga chop etish
```

Blok izohlarida sharh matnidan oldin va keyin uchta bitta tirnoq qo‘yiladi:

“‘*sharh matni*’”’. Masalan:

```
'''
    Konsol chiqishi
    Salom dunyo xabarlar
'''
print ("Salom dunyo")
```

Asosiy funktsiyalari

Python ko'plab o'rnatilgan, standart funktsiyalarni taqdim etadi. Ulardan ba'zilari, ayniqsa, til o'rganishning dastlabki bosqichlarida juda tez-tez qo'llaniladi, shuning uchun ularni ko'rib chiqaylik.

Konsolga ma'lumot chiqarishning asosiy funktsiyasi - bu `print()` funktsiyasi hisoblanadi. Biz chiqarmoqchi bo'lgan satr ushbu funktsiyaga argument sifatida uzatiladi:

```
print ("Salom Python")
```

Agar konsolga bir nechta qiymatlarni chop etish kerak bo'lsa, ularni vergul bilan ajratib, `print()` funktsiyasiga beriladi:

```
print("To'liq ismi:", "Tom", "Smit")
```

Natijada, vergul bilan ajratilgan joylarga probel bilan ajratilgan holatda, barcha berilgan qiymatlar, bir qatorga joylashtirilib konsolga chiqariladi:

```
To'liq ismi: Tom Smit
```

Agar **`print()`** funktsiyasi chiqarish uchun javobgar bo'lsa, u holda **`input()`** funktsiyasi axborotni kiritish uchun xizmat qiladi. **`input()`** funktsiyasi kiritilgan ma'lumotni satr tipida qaytaradi va bu satrni o'zgaruvchiga saqlanadi va shu qiymatga o'zgaruvchi orqali murojaat qilinadi:

```
name = input("Ismni kiriting:")  
print ("Salom", ism)
```

Konsol chiqishi:

```
Ismni kiriting: Eugene  
Salom Eugene
```

Nazorat savollari:

1. Pythonda dastur strukturasi qanday tuzilgan?
2. Satrning oldida qoldirilgan oraliq masofalar (indents) nimani anglatadi?
3. Pythonning sintaksisi boshqa dasturlash tillarining sintaksisidan eng katta farqli jihati nimada?

4. Pythonda harflarning katta kichikligiga sezgirligi (Регистрозависимость) qanday?
5. Pythonda kommentariyalar qanday amalga oshiriladi?
6. Ko‘p satrlik kommentariyalar qanday bajariladi?
7. Pythonda kiritish, chiqarish funksiyalari qanday?

2.2.§. O‘zgaruvchilar va ma’lumotlar turlari

O‘zgaruvchilar

O‘zgaruvchilar qiymatlarni saqlash uchun mo‘ljallangan dastur elementi. Python tilidagi o‘zgaruvchi nomi alifbo belgisi yoki pastki chiziq bilan boshlanishi kerak va alfanumerik belgilar va pastki chiziqni o‘z ichiga olishi mumkin. Bundan tashqari, o‘zgaruvchining nomi Python tilidagi kalit so‘zlarning nomi bilan bir xil bo‘lmasligi kerak. Kalit so‘zlar ko‘p emas, ularni eslab qolish mumkin:

<i>False</i>	<i>await</i>	<i>else</i>	<i>import</i>	<i>pass</i>
<i>None</i>	<i>break</i>	<i>except</i>	<i>in</i>	<i>raise</i>
<i>True</i>	<i>class</i>	<i>finally</i>	<i>is</i>	<i>return</i>
<i>and</i>	<i>continue</i>	<i>for</i>	<i>lambda</i>	<i>try</i>
<i>as</i>	<i>def</i>	<i>from</i>	<i>nonlocal</i>	<i>while</i>
<i>assert</i>	<i>del</i>	<i>global</i>	<i>not</i>	<i>with</i>
<i>async</i>	<i>elif</i>	<i>if</i>	<i>or</i>	<i>yield</i>

O‘zgaruvchilarni e‘lon qilish quyidagicha amalga oshiriladi:

```
foydalanuvchi = "Tomson"
```

Bu yerda “foydalanuvchi” nomli o‘zgaruvchiga "Tom" satri o‘zlashtirilmoqda.

Dasturlash tillarida o‘zgaruvchilarni nomlashning ikki xil usuli mavjud: **camel case** va **underscore notation**.

Camel case o‘zgaruvchi nomidagi har bir keying so‘z bosh harf bilan boshlanadi va oldingi so‘zga biriktirib yoziladi. Masalan:

```
userName = "Tomson"
```

Underscore notation – bu usulda o‘zgaruvchi nomidagi so‘zlar pastki chiziq bilan ajratiladi. Masalan:

```
user_name = "Tomson"
```

Shuningdek, siz katta-kichik registr sezgirligini hisobga olishingiz kerak, shuning uchun **ism** va **Ism** o‘zgaruvchilari turli ob’ektlarni ifodalaydi.

```
# ikki xil o‘zgaruvchi  
ism = "Jenifer"  
Ism = "Loktfud"
```

O‘zgaruvchilarning o‘ziga xos xususiyati shundagi dastur davomida uning qiymati va tipi o‘zgartirishi mumkin.

Quyida o‘zgaruvchini aniqlab, uni dastur davomida qiymatidan foydalanishni ko‘rib chiqamiz:

```
name = "Tomson"    # o‘zgaruvchi nomi "Tomson" ga teng  
print(name)        # print: Tomson  
name = "Bobbi"     # qiymatni "Bobbi" ga o‘zgartiring  
print(name)        # chiqish: Bobbi
```

Ma’lumotlar turlari

O‘zgaruvchi ma’lumotlar turlaridan birining ma’lumotlarini saqlaydi. Pythonda juda ko‘p turli xil ma’lumotlar turlari mavjud. Shulardan eng asosiy turlarni ko‘rib chiqamiz: **bool**, **int**, **float** va **str**.

bool (Mantiqiy qiymatlar). bool turli o‘zgaruvchilar ikkita mantiqiy qiymatni qabul qiladi: True (rost yoki 1) yoki False (yolg‘on yoki 0). True qiymati biror narsaning haqiqat ekanligini ko‘rsatish uchun ishlatiladi. False qiymati esa, aksincha biror fikrning yoki narsa noto‘g‘ri ekanligini ko‘rsatadi. Ushbu turdagi o‘zgaruvchilardan foydalanishga misol:

```
isMarried = False  
print(isMarried)    # False  
isAlive = True  
print(isAlive)      # True
```

int (Butun sonlar). int turi 1, 4, 8, 50 kabi butun sonlar to‘plamidagi sonlarni ifodalaydi. Ya’ni minus cheksizlikdan plus cheksizlikkacha bo‘lgan barcha butun sonlar to‘plamiga tushuvchi biror bir sonni qabul qiladi. dasturda foydalanishga misol:

```
age = 21
print("Yosh:", age)      # Yosh: 21
count = 15
print("Hisob:", count) # Hisob: 15
```

Odatiy bo‘lib, standart raqamlar o‘nlik sonlar sifatida qabul qilinadi. Ammo Python ikkilik, sakkizlik va o‘n oltilik tizimlardagi raqamlarni ham qo‘llab-quvvatlaydi.

Raqam **ikkilik sanoq sistemasida** ekanligini ko‘rsatish uchun raqam oldiga **0b** prefiksi qo‘yiladi:

```
a = 0b11
b = 0b1011
c = 0b100001
print(a)      # 3 o'nlik sanoq sistemasida
print(b)      # 11 o'nlik sanoq sistemasida
print(c)      # 33 o'nlik sanoq sistemasida
```

Raqam **sakkizlik sanoq sistemasini** ifodalashini ko‘rsatish uchun raqamga **00** prefiksi qo‘yiladi:

```
a = 007
b = 0011
c = 0017
print(a)      # 7 o'nlik sanoq sistemasida
print(b)      # 9 o'nlik sanoq sistemasida
print (c)     # 15 o'nlik sanoq sistemasida
```

Raqam **o‘n oltilik sanoq sistemasini** ifodalashini ko‘rsatish uchun raqamga **0x** prefiksi qo‘yiladi:

```
a = 0x0A
b = 0xFF
c = 0xA1
print(a)      # 10 o'nlik sanoq sistemasida
print(b)      # 255 o'nlik sanoq sistemasida
print(c)      # 161 o'nlik sanoq sistemasida
```

Shuni ta'kidlash kerakki, qaysi tizimda biz konsolga chiqarish uchun bosib chiqarish funksiyasiga raqam o'tkazsak, u sukut bo'yicha o'nli kasrda chiqariladi.

float (Kasr sonlar). float turi vergulli sonini ifodalaydi va haqiqiy sonlar to'pamiga kiruvchi biror-bir sonni qabul qiladi. masalan: 1,2 yoki 34,76 va hokazo. Butun va kasr qismlar orasidagi ajratuvchi sifatida nuqta ishlatiladi.

```
uzunligi = 1.68
p = 3.14
vazn = 68.
print (uzunligi)      # 1.68
print(p)              # 3.14
print(vazn)           # 68.0
```

kasr sonini eksponensial belgida aniqlash mumkin:

```
x = 3.9e3
print(x)      # 3900.0

x = 3.9e-3
print(x)      # 0.0039
```

Bu yerda $x = 3.9e3$ ifodadagi $e3 = 10^3$ ni anglatadi va $3.9 \cdot 10^3$ ko'payma x o'zgaruvchisiga yuklanmoqda. O'z navbatida $e-3 = 10^{-3}$

str (satrli tip). str turi satrlarni ifodalaydi. Satr "salom" va 'dunyo' kabi bitta yoki qo'sh tirnoq ichiga olingan belgilar ketma-ketligini ifodalaydi. Python 3.x da satrlar Unicode belgilar to'plamini ifodalaydi

```
hi = "Salom dunyo!"
print (hi )      # Salom dunyo!
ism = "Tomson"
print (ism)      # Tomson
```

O'zgaruvchan tip. Python dasturlash tilida o'zgaruvchilarning tipi dinamik tip hisoblanadi. Bu o'zgaruvchining tiplari dastur davomida qiymatning tipiga bog'liq tarzda o'zgarishi mumkinligini anglatadi.

Shunday qilib, ikki yoki bitta tirnoq ichida satr tayinlanganda, o'zgaruvchi str turida bo'ladi. Butun sonni o'zlashtirganda Python avtomatik ravishda o'zgaruvchining turini int sifatida aniqlaydi. O'zgaruvchini float ob'ekti sifatida

aniqlash uchun unga kasr son beriladi, bu yerda kasr va butun qismlarni ajratuvchi sifatida nuqtadan foydalaniladi. Quyida bunga misol keltirilgan:

```
f_Id = "abc"    # satr turiga mansub
print (f_Id)

f_Id = 234      # butun sonlar turiga mansub
print (f_Id)
```

Standart **type()** funksiyasidan foydalanib, o‘zgaruvchining joriy tipini bilib olishingiz mumkin:

```
f_Id = "abc"      # tip str
print(type(f_Id)) # <class 'str'>
f_Id = 234        # tip int
print(type(f_Id)) # <class 'int'>
```

Nazorat savollari:

1. Pythonda o‘zgaruchi qanday e’lon qilinadi?
2. Pythonda o‘zgaruvchilarga nom berishda nimalarga e’tibor berish kerak bo‘ladi?
3. O‘zgaruvchilarga nom berishning CamelCase usuli qanday?
4. O‘zgaruvchilarga nom berishning “Underscore notation” usulini tushuntirib bering.
5. Pythonda o‘zgaruvchilarning qaday turlari ishlatiladi?
6. Satr tipli o‘zgaruvchiga bir nechta satrni yuklash uchun qanday usuldan foydalaniladi?
7. Satr hosil qilinayotganda maxsus simvollar text ichida mavjud bo‘lsa nima sodir bo‘ladi? Misollar bilan tushuntirib bering.
8. Satr tarkibiga o‘zgaruvchilarning qiymatini chaqirish uchun qanday amal bajarish kerak?

2.3.8. Pythonda kiritish va chiqish funksiyalari

Konsolga xabar chiqarish funksiyasi - print()

print() pythonning standart funksiyasi ma’lumotni konsolga chiqarish uchun ishlatiladi. Ushbu funktsiya chaqirilganda, chiqish qiymati funksiyaga parametr sifatida beriladi:

```
print("Salom, QALAYSIZ?")
```

Ushbu kod konsolda "Salom, QALAYSIZ?" qatorini chop etadi.

Ushbu funktsiyaning ajralib turadigan xususiyati shundaki, u standart holatda qiymatni alohida satrda chiqaradi. Masalan:

```
print ("Salom dunyo")
print("Salom, QALAYSIZ?")
print ("Salom Python")
```

Har bir chaqirilgan print() funksiyasi xabarni alohida satrga joylashtiriladi:

```
Salom Dunyo
Salom qalaysiz?
Salom Python
```

Bu har doim ham qulay bo'la olmaydi. Masalan, biz barcha qiymatlarni bitta satrda ko'rsatishni xohlaymiz. Buning uchun **end** parametri yordamida har bir satrning oxirida qanday belgi bo'lishini aniqlash mumkin. End parametridan quyidagicha foydalaniladi:

```
print (qiymat, end=oxirgi_belgilar)
```

Odatda, end parametrining qiymati, keyingi qatorga o'tishni ko'rsatadigan "\n" belgisiga teng bo'ladi. Shuning uchun print() funksiyasi standart bo'yicha xabarni alohida satrda chop etadi.

Endi shart bo'yicha barcha xabarlar yonma-yon chiqishini ko'rib chiqaylik. Buning uchun end parametriga bitta probel - " " ni yuklab qo'yish kerak:

```
print ("Salom dunyo", end=" ")
print("Salom, QALAYSIZ?", end=" ")
print ("Salom Python")
```

Bu holda har bir qiymatlar bo'sh joy bilan ajratiladi:

```
Salom Dunyo Salom, QALAYSIZ? Salom Python
```

End parametri nafaqat bitta belgi, balki belgilar to'plamini qabul qilishi mumkin:

```
print("Salom dunyo", end=" va ")
print("Salom, QALAYSIZ?", end=" va ")
print("Salom Python")
```

Bunday holda, chiqish xabarlari " va " belgilari bilan ajratiladi:

```
Salom Dunyo va Salom, QALAYSIZ? va Salom Python
```

Print funksiyasining yana bir **sep** nomli ajoyib parametri mavjud. **Sep** separator soʻzidan olingan boʻlib, vazifasi print funksiyasiga berilgan bir nechta qiymatlar orasidagi ajratuvchi belgini oʻrnatib beradi:

```
A = "Salom do'stlar"
B = "Bu menman"
print(A, B, sep=" - ")
```

Bunday holda, chiqish xabarlarini " - " belgilari bilan ajratiladi:

```
Salom do'stlar - Bu menman
```

Kiritish operatori - input()

Konsolga chiqish bilan bir qatorda biz konsoldan foydalanuvchi ma'lumotlarini olishimiz, kirish ma'lumotlarini olishimiz ham kerak. Buning uchun Pythonda input() funksiyasi mavjud. Ushbu funktsiyaga kirish soʻrovi uzatiladi. Va biz kiritilgan natijani oʻzgaruvchida saqlashimiz mumkin boʻladi. Masalan, foydalanuvchi nomini kiritish uchun kodni aniqlaylik:

```
ism = input("Ismingizni kiriting:")
print(f" Sizning Ismingiz : {ism}")
```

Bunday holda, input() funksiyasiga "Ismingizni kiriting:" qatori koʻrinishidagi kiritish soʻrovi yuboriladi. Funktsiya natijasi - foydalanuvchi kiritgan natija **ism** oʻzgaruvchisiga uzatiladi. Keyinchalik print() funksiyasidan foydalanib, ushbu oʻzgaruvchining qiymatini konsolga chop etishimiz mumkin. Kod natijasi:

```
Ismingizni kiriting: Sanjarbek
Sizning Ismingiz: Sanjarbek
```

Bir nechta qiymatlar kiritishga misol:

```
ism = input("Ismingiz:")
yosh = input("Yoshingiz:")
print(f"Sizning ismingiz: {ism}, yoshingiz esa:
{yosh} da")
```

Dasturning ishlashiga misol:

```
Ismingiz: Anjilika
```

```
Yoshingiz: 20
```

```
Sizning ismingiz : Anjilika, yoshingiz esa: 20 da
```

Shuni yodda tutish kerakki, kiritilgan barcha qiymatlar str tipida qabul qilinadi, ya'ni satrlar sifatida qabul qilinadi. Va agar biz yuqoridagi koddagi ikkinchi holatda bo'lgani kabi raqamni kiritsak ham, Python kiritilgan qiymatni raqam sifatida emas, balki satr sifatida ko'rib chiqadi.

Nazorat savollari:

1. Ma'lumotni konsoldan kiritish qanday amalga oshiriladi?
2. Ma'lumotni konsol ekraniga chiqarish qanday amalga oshiriladi?
3. print() funksiyasining ishlash prinsipi va qabul qiladigan parametrlar haqida gapirib bering.
4. input() funksiyasining qabul qiladigan parametrlari haqida gapirib bering.
5. print() funsiyasi orqali stringlarning ichida o'zgaruvchining qiymatini chiqarish uchun qanday amallar bajarilishi kerak?

2.4.§. Arifmetik operatorlar

Python barcha turdagi arifmetik operatsiyalarni qo'llab-quvvatlaydi :

Qo'shish (+):	<code>print(6 + 2)</code>	# 8
Ayirish (-):	<code>print(6 - 2)</code>	# 4
Ko'paytirish (*):	<code>print(6 * 2)</code>	# 12
Bo'linish (/):	<code>print(6/2)</code>	# 3.0
Bo'lib butunni olish (//):	<code>print(7 // 2)</code>	# 3
Darajaga oshirish (**):	<code>print(6 ** 2)</code>	# 6 ² natija 36
Bo'lib qoldig'ini olish (%):	<code>print(7 % 2)</code>	# 1

Agar bitta arifmetik ifodada bir nechta arifmetik amallar ketma-ket foydalanilsa, amallarning prioritetiga muvofiq ketma-ketlikda bajariladi. Birinchi navbatda yuqori ustuvorlikdagi operatsiyalar amalga oshiriladi. Arifmetik amallarning prioritetlarining kamayish tartibida quyidagi 2-jadvalda ko'rsatilgan.

Operatsiya yo'nalishi:

1	**	o'ngdan chapga
2	*, /, //, %	Chapdan o'ngga
3	+, -	Chapdan o'ngga

Keling, quyidagi ifodani ko'rib chiqamiz:

```
raqam = 3 + 4 * 5 ** 2 + 7
print(raqam)    # 110
```

Bu yerda birinchi navbatda darajaga oshirish amali ($5 ** 2$) bajariladi, keyin natija 4 ga ($25 * 4$) ko'paytiriladi, keyin qo'shish ($3 + 100$) amali bajariladi va keyin yana qo'shish ($103 + 7$) amalga oshiriladi. Shunday qilib natija 110 ga teng bo'ladi.

Qavslar amallar tartibini qayta aniqlash uchun ishlatilishi mumkin:

```
raqam = (3 + 4) * (5 ** 2 + 7)
print(raqam)    # 224
```

Shuni ta'kidlash kerakki, arifmetik amallarda ham butun, ham kasr sonlar ishtirok etishi mumkin. Agar butun son (int) va vergulli (kasr) son (float) bir xil amalda ishtirok etsa, u holda butun son float turiga o'tkaziladi.

Arifmetik amallarning tenglik belgisi bilan ishlatilishi

Bu maxsus operatsiyalar amal bajarilgandan keyin natijani birinchi operandga yuklash imkonini beradi:

+=	Qo'shishdan hosil bo'lgan natijani o'zlashtirish
-=	Ayirishdan hosil bo'lgan natijani o'zlashtirish
*=	Ko'paytirishdan hosil bo'lgan natijani o'zlashtirish
/=	Bo'linishdan hosil bo'lgan natijani o'zlashtirish
//=	Bo'lib butunni olishdan hosil bo'lgan natijani o'zlashtirish
**=	Raqam darajaga oshirishdan hosil bo'lgan natijani o'zlashtirish
%=	Bo'lib qoldig'ini oshirishdan hosil bo'lgan natijani o'zlashtirish

Operatsiyalarni dasturda qo'llanilishi:

```
raqam = 10
raqam += 5
print(raqam)      # 10 + 5 = 15
raqam -= 3
print(raqam)      # 15 - 3 = 12
raqam *= 4
print(raqam)      # 12 * 4 = 48
raqam /= 2
print(raqam)      # 48 / 2 = 24.0
raqam //= 5
print(raqam)      # 24 // 5 = 4
raqam **= 3
print(raqam)      # 4 ** 3 = 64
raqam %= 6
print(raqam)      # 64 % 6 = 4
```

Yaxlitlash va “round” funksiyasi

Dasturlashda float tipli raqamlar bilan operatsiyalar bajarayotganda, natijalar aniq biz o‘ylaganimiz kabi bo‘lmasligi mumkin. Masalan :

```
bir_son = 2.0001
ikki_son = 5
uch_son = bir_son / ikki_son
print(uch_son) # 0.400020000000000004
```

Bunday holda, dasturchi 0,40002 kabi natijani olishni kutdi, lekin oxirida bir qator nollar orqali, qandaydir to‘rt paydo bo‘lmoqda. Yoki boshqa ifoda:

```
print(2.0001 + 0.2) # 2.10020000000000003
```

Yuqoridagi holatda, natijani yaxlitlash uchun o‘rnatilgan round() funksiyasidan foydalanishimiz mumkin:

```
bir_son = 2.0001
ikki_son = 0.1
uch_son = bir_son + ikki_son
print(round(uch_son)) # 2
```

Yaxlitlanadigan raqam round() funksiyasiga o‘tkaziladi. Agar yuqoridagi misoldagi kabi funksiyaga bitta raqam uzatilsa, u butun songa yaxlitlanadi.

`round()` funksiyasi ikkinchi raqamni ham olishi mumkin, natijada olingan sonda nechta o'nli kasr bo'lishi kerakligini ko'rsatadi:

```
bir_son = 2.0001
ikki_son = 0.1
uch_son = bir_son + ikki_son
print(round(uch_son, 4)) # 2.1001
```

Bu holda `uch_son` verguldan keying 4 ta raqamgacha yaxlitlanadi.

Agar funktsiyaga faqat bitta qiymat o'tkazilsa - faqat yaxlitlanadigan raqam, u eng yaqin butun songa yaxlitlanadi.

Yaxlitlash misollari:

```
# butun songacha yaxlitlash
print(round(2.49))
print(round(2.51))
```

Biroq, agar yaxlitlanadigan qism ikkita butun sondan bir xil masofada bo'lsa, yaxlitlash eng yaqin juft songa o'tadi:

```
print(round(2.5)) # 2
print(round(3.5)) # 4
```

`round()` funksiyasining ikkinchi parametrini beradigan bo'lsak, verguldan keyin shuncha xonagacha yaxlitlaydi. Yaxlitlagandayam tushirib qoldiriladigan raqam 5 dan katta yoki teng bo'lsa, unda oldingisiga birni qo'shib yaxlitlaydi:

```
# verguldan keyingi 2 ta raqamgacha yaxlitlash
print(round(2.554, 2)) # 2.55
print(round(2.5551, 2)) # 2.56
print(round(2.554999, 2)) # 2.55
print(round(2.499, 2)) # 2.5
```

Biroq, `round()` funksiyasi ideal vosita emasligini yodda tuting. Masalan, yuqorida, butun sonlarga yaxlitlashda, agar yaxlitlanadigan qism ikki qiymatdan bir xil masofada bo'lsa, yaxlitlash eng yaqin juft qiymatgacha amalga oshiriladi, degan qoida

qo'llaniladi. Pythonda, sonning kasr qismini suzuvchi raqam sifatida to'g'ri ko'rsatib bo'lmashligi sababli, bu butunlay kutilmagan natijalarga olib kelishi mumkin. Masalan:

```
# verguldan keyingi 2 ta raqamgacha yaxlitlash
print(dumaloq (2.545, 2))      # 2.54
print(dumaloq(2.555, 2))      # 2.56
print(dumaloq (2.565, 2))      # 2.56
print(dumaloq (2.575, 2))      # 2.58
print( dumaloq(2.655, 2))      # 2.65
print(dumaloq (2.665, 2))      # 2.67
print(dumaloq (2.675, 2))      # 2.67
```

Nazorat savollari:

1. pythonda qanday arifmetik amallar mavjud?
2. % - qanday amalni bajarish uchun ishlatiladi?
3. // - qanday amalni bajarish uchun ishlatiladi?
4. ** - qanday amalni bajarish uchun ishlatiladi?
5. Kvadrat ildiz qanday ishlatiladi?
6. Oddiy bo'lish amalini bajarilganda qanday tipli qiymat qaytaradi?
7. += , -=, *=, /=, **= va %= - operatorlarining vazifasi qanday?
8. Round() funksiyasining ishlash prinsipini tushuntirib bering.

2.5.§. Mantiqiy ifodalar

Python dasturlash tilida ham bir qancha mantiqiy ifodalar mavjud. Ushbu mantiqiy ifodalarning barchasi ikkita o'zgaruvchi bilan ishlatiladi va umumiy natija sifatida bool turidagi mantiqiy qiymat qaytaradi. Mantiqiy qiymatlar ikkita bo'lib, quyidagilar: **True** (rost) va **False** (yolg'on).

Solishtirish operatorlari. Eng oddiy shartli ifodalar ikki qiymatni taqqoslaydigan taqqoslash amallaridir. Python quyidagi taqqoslash operatsiyalarini qo'llab-quvvatlaydi:

<code>==</code>	Ikkala operand teng bo'lsa, True qiymatini qaytaradi. Aks holda False qaytaradi.
<code>!=</code>	Ikkala operand teng bo'lmasa, True qiymatini qaytaradi. Aks holda False qaytaradi.
<code>> (katta)</code>	Agar birinchi operand ikkinchisidan katta bo'lsa, True qiymatini qaytaradi.
<code>< (kichik)</code>	Agar birinchi operand ikkinchidan kichik bo'lsa, True qiymatini qaytaradi.
<code>>= (katta yoki teng)</code>	Birinchi operand ikkinchidan katta yoki teng bo'lsa, True qiymatini qaytaradi.
<code><= (kichik yoki teng)</code>	Birinchi operand ikkinchidan kichik yoki teng bo'lsa, True qiymatini qaytaradi.

Taqqoslash operatsiyalariga misollar:

```

a = 6
b = 9
res = 6 == 9 # natijani o'zgaruvchiga yuklash
print(res) # False
print(a > b) # False
print(a != b) # True
print(a < b) # True

boolean1 = True
boolean2 = False
print(boolean1 == boolean2) # False

```

Taqqoslash operatsiyalari turli ob'ektlarni - satrlarni, raqamlarni, mantiqiy amallarni solishtirishi mumkin, ammo operatsiyaning ikkala operandlari ham bir xil turni ifodalashi kerak.

Mantiqiy amallar. Turli murakkablikka ega bo'lgan mantiqiy ifodalarni yaratish uchun mantiqiy operatorlar ishlatiladi. Python quyidagi mantiqiy operatorlarni qo'llab quvvatlaydi:

and (mantiqiy ko'paytirish) operatori ikkita operandga qo'llaniladi:

```
x and y
```

Birinchidan, and operatori x ifodasini baholaydi va agar u False bo'lsa, uning qiymati qaytariladi. Agar u True bo'lsa, ikkinchi operand y baholanadi va y ning qiymati qaytariladi.

```
yosh = 20
vazn = 65
res = yosh > 19 and vazn == 65
print(res) # True
```

Bu holda **and** operatori ikkita ifoda natijalarini taqqoslaydi: yosh > 19 hamda vazn == 65. Agar bu ifodalarning ikkalasi ham True qiymatini qaytarsa, unda **and** operatori ham res o'zgaruvchisiga True qiymatini qaytaradi (rasmiy ravishda oxirgi operandning qiymati qaytariladi).

Lekin **and** operatorining operandlari True va False bo'lishi shart emas. Bu har qanday qiymat bo'lishi mumkin. Masalan:

```
natija = 14 va "s"
print(natija) # s, chunki 14 soni True ga ekvivalent,
#shuning uchun oxirgi operandning qiymati qaytariladi

natija = 0 va "s"
print(natija) # 0 , chunki 0 False ga ekvivalent
```

Bunday holda, 0 raqami va bo'sh "" qatori noto'g'ri deb hisoblanadi, qolgan barcha raqamlar va bo'sh bo'lmagan satrlar "True" ga ekvivalentdir.

or (mantiqiy qo'shish) ikkita operandga ham tegishli:

```
x or y
```

Or operatori avval x ifodasini baholaydi va agar u True bo'lsa, uning qiymati qaytariladi. Agar u False bo'lsa, ikkinchi operand y baholanadi va y ning qiymati qaytariladi. Masalan

```
yosh = 22
natija = yosh > 21 or False
print(natija) # True
```

Shuningdek, **OR** operatori har qanday qiymatlarga qo'llanilishi mumkin. Masalan:

```
res = 4 or "f"
print(res)      # 4, chunki 4 rost ga teng, shuning uchun birinchi
#operandning qiymati qaytariladi
natija = 0 or "f"
print(natija)    # f, chunki 0 False ga ekvivalent, shuning
uchun #oxirgi operandning qiymati qaytariladi
```

not (mantiqiy inkor - !)

Ifoda False bo'lsa, True qiymatini qaytaradi

```
yosh = 25
isSingle = False
print(not yosh > 21)    # False
print(not isSingle)     # True
print(not 45)           # False
print(not 0)            # True
```

in operatori

Agar ba'zi qiymatlar to'plamida ma'lum qiymat mavjud bo'lsa, **in** operatori "True" ni qaytaradi. U quyidagi shaklga ega:

```
qiymat in [qiymat_to`plamidagi]
```

Masalan, satr belgilar to'plamini ifodalaydi. Va in operatori bilan biz unda biron bir qiymat mos kelishini va mavjudligini tekshirishimiz mumkin:

```
xabar = "hello world!"
hi = "hello"
```

```
print(hi in xabar)
zol= "golden"
print(zol in xabar)
```

Agar qiymatlar to'plamida qiymat mavjud masligini tekshirishimiz kerak bo'lsa, unda biz **not in** modifikatsiyasidan foydalanishimiz kerak bo'ladi. Agar qiymat [qiymatlar to'plami] da mavjud bo'lmasa, u True qiymatini qaytaradi:

```
xabar = "hello world!"
hi = "hello"
print(hi not in xabar)  # False

golden = " golden "
print(golden not in xabar)  # True
```

Nazorat savollari:

1. Pythonda shart operatorining sintaksisi qanday?
2. Pythonda qanday taqqoslash operatorlar mavjud?
3. Pythonda qanday mantiqiy operatorlar mavjud?
4. Mantiqiy ko'paytirish va mantiqiy qo'shish operatorlarini solishtirib bering?
5. Shart operatorining ichida **in** operatorining ishlash prinsipi qanday?
6. **not in** operatorini qanday ishlaydi?
7. **in** va **not in** operatorlarining farqi nimada?

2.6.§. Shart operatori – IF

Shartli operatorlari shartli ifodalardan foydalanadi va ularning qiymatiga qarab, dasturning bajarilishini yo'llardan biri bo'ylab yo'naltiradi. Shunday operatorlardan biri **if** operatoridir. U quyidagi quyidagicha sintaksisga ega:

```
if mantiqiy_amal:
    [if bloki]
elif mantiqiy_amal:
    [elif bloki]
else:
    [else bloki]
```

Eng oddiy shaklda if kalit soʻzidan keyin mantiqiy ifoda keladi. Va agar bu mantiqiy ifoda "True" ni qaytarsa, u holda koʻrsatmalarning keyingi bloki bajariladi, ularning har biri blokga tegishlilikini oldiga 4 ta probel yoki 1 ta tab tashlanishi bilan aniqlanadi (4 boʻsh joy yoki boʻshliqlar sonini cheklash maqsadga muvofiqdir) bu 4 ga karrali):

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
print("Xayr")
```

Bu holda `til` oʻzgaruvchisining qiymati "uzbekcha" boʻlganligi sababli, if bloki bajariladi, unda faqat bitta operator mavjud - `print("Assalomu alaykum")`. Natijada, konsol quyidagi qatorlarni koʻrsatadi:

```
Assalomu alaykum
Xayr
```

Kodning oxirgi qatoriga eʼtibor bering, unda "Xayr" xabari koʻrsatiladi. Chunki `print("Xayr")` oldidan tab tashlanmagan, shuning uchun u if blokiga tegishli emas va if konstruktsiyasidagi ifoda False qiymatini qaytarsa ham bari bir `print("Xayr")` amali bajariladi.

Ammo agar biz uni cheklasak, u if konstruktsiyasiga ham tegishli boʻladi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
    print("Xayr")
block else:
```

Mabodo if ifodasi False qiymatini qaytarsa, unda else bloki ishga tushadi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu alaykum")
else:
    print("Hi")
```

```
print("Xayr")
```

Agar `til == "uzbekcha"` iborasi True qiymatini qaytarsa, u holda if bloki bajariladi, aks holda else bloki bajariladi. Va bu holda shart `til == "uzbekcha"` False qiymatini qaytarganligi sababli, else blokidagi operator bajariladi.

Bundan tashqari, else bloki tarkibiga kiruvchi operatorlarning oldidan ham satr boshidan tab (4ta probelga teng bo'sh joy) tashlanishi kerak. Misol uchun, yuqoridagi misolda `print("Xayr")` ning oldidan tab tashlanmagan, shuning uchun u else bloki tarkibiga kiritilmagan va `til` o'zgaruvchisining qiymati qaysi tilda bo'lishidan qat'iy nazar bajariladi. Ya'ni, konsol quyidagi qatorlarni ko'rsatadi:

```
Assalomu alaykum
Xayr
```

else bloki tarkibi bir nechta qatorlardan tashkil topgan bo'lishi ham mumkin:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Salom")
    print("Dunyo!")
else:
    print("Hello")
    print("World!")
```

Agar siz bir nechta aniq shartlarni kiritishingiz kerak bo'lsa, unda siz qo'shimcha elif bloklaridan foydalanishingiz kerak bo'ladi:

```
til = "uzbekcha"
if til == "uzbekcha":
    print("Assalomu")
    print("Alaykum")
elif til == "german":
    print("Hallo")
    print("Welton")
else:
    print("What's")
```

```
print("up")
```

Python bu yerda birinchi, if ning shartini tekshiradi. Agar rost bo'lsa, if blokidagi operatorlar bajariladi. Agar bu shart False qiymatini qaytarsa, Python elif shartini tekshiradi.

Agar elifning sharli ifodasi True bo'lsa, u holda elif blokidagi opertorlar bajariladi. Ammo agar u ham False qaytarsa, else blokidagi bajariladi.

Shartlarni elif operatoridan foydalangan holda ixtiyoriy davom ettirishingiz mumkin. Masalan :

```
til = "Uz"
if til == "english":
    print("Hello")
elif til == "german":
    print("Hallo")
elif til == "french":
    print("Salut")
else:
    print("Salom")
```

Ichma-ich joylashgan if operatori. if operatorining sharti bajarilganidan keyin yanayam aniqlashtirish uchun ichida yana bir necha shartga tekshiriladi:

```
til = "uzbekcha"
payt == "ertalabki vaqt"
if til == "uzbekcha":
    print("Uzbek")
    if payt == "ertalabki vaqt":
        print("Unda hayrli tong!")
    elif payt == "kechgi vaqt":
        print("Unda sizga hayrli kech!")
```

Bu yerda if konstruksiyasi ichki o'rnatilgan if/elif konstruktsiyalarini o'z ichiga oladi. Ya'ni, agar til o'zgaruvchisi "uzbekcha" ga teng bo'lsa, u holda ichki o'rnatilgan

if/elif konstruktsiyalarisi kunduzgi o'zgaruvchining qiymatini qo'shimcha ravishda tekshirishadi - **payt** o'zgaruvchisining qiymati **"ertalabki vaqt"** ga tengmi yoki **"kechgi vaqt"** ga tengligini tekshiradi va shunga qarab javob qaytariladi. Va bu holda, biz quyidagi konsol chiqishini olamiz:

```
Uzbek
```

```
Unda hayrli tong
```

Esda tutingki, ichki o'rnatilgan if ifodalarining oldidan tab qoldirish kerak, ichki o'rnatilgan konstruktsiyalardagi operatorlarning oldidan ham tab (bo'sh joy) qoldirilishi kerak. To'g'ri joylashtirilmagan tabulyatsiya dastur mantig'ini o'zgartirishi mumkin.

Xuddi shunday, ichki o'rnatilgan if/elif/else konstruktsiyalari elif va else bloklariga joylashtirilishi mumkin:

```
til = "uzbek"
payt = "ertalab"

if til == "uzbek":
    if payt == "ertalab":
        print("Hayrli tong")
    else:
        print("Hayrli kech")
else:
    if payt == "ertalab":
        print("Good morning")
    else:
        print("Good evening")
```

Nazorat savollari :

1. If operatori qanday vazifasi bajaradi?
2. If operatorining strukturasi qanday?
3. Elif operatorining vazifasi qanday?
4. Ichma – ich foydalaniladigan shart operatoriga misollar keltiring.
5. Bloklarning if ifodaga tegishliligini qanday bilish mumkin?

2.7.§. Pythonda takrorlanuvchi jarayonlarni dasturlash.

For, while, break, continue va else operatorlari bilan ishlash.

Sikllar biror bir shartning bajarilishiga qarab ba'zi harakatlarni bajarishga imkon beradi. Pythonda quyidagi turdagi sikllar mavjud: **while** va **for**.

WHILE sikli. while sikli qandaydir bir shartning to'g'riligini tekshiradi va agar shart True bo'lsa, u holda sikl tanasidagi blokni bajaradi. U quyidagicha sintaksisga ega:

```
while <shartli_ifoda>:  
    [sikl tanasi]
```

while kalit so'zidan keyin shartli ifoda keladi va bu ifoda True qiymat qaytarsa, sikl tanasiga kirib u yerdagi operatorlar ketma-ket bajariladi.

while sikliga tegishli barcha bloklar oldidan bo'sh joy qoldirilib ketishi va boshlanish qismlari bir ustunga joylashgan bo'lishi kerak va while kalit so'zidan bir xilda chekingan bo'lishi kerak.

```
son = 1  
while son < 5:  
    print(f"son = {son}")  
    son += 1  
print("Dastur tugatildi")
```

Bunday holda, **son** o'zgaruvchisi 5 dan kichik bo'lsa, while sikli ishlaydi. Aks holda sikldan keyingi amal bajariladi.

Bu sikl tanasi ikkita ifodadan iborat:

```
print(f"son = {son}")  
son += 1
```

Shuni ham yodda tutingki, oxirgi print("Dastur tugallandi") satr boshidan chekinmaydi, shuning uchun u while siklining bir qismi emas.

Siklning butun jarayonini quyidagicha ifodalash mumkin: Birinchi, **son** o'zgaruvchisining qiymati 5 dan kichik yoki kichik emasligini tekshiriladi. Va

o'zgaruvchi dastlab 1 ga teng bo'lganligi sababli, bu shart True ni qaytaradi va shuning uchun sikl operatorlari bajariladi.

Sikl ko'rsatmalari satr **son**=1 ni konsolga chiqaradi. Va keyin raqam o'zgaruvchisining qiymati bittaga oshiriladi - endi u 2 ga teng bo'ladi. sikl ko'rsatmalari blokini bir marta bajarish iteratsiya deb ataladi. Ya'ni, shu tarzda, birinchi takrorlash siklda amalga oshiriladi.

son < 5 sharti yana tekshiriladi. Yana True qaytariladi, chunki son 2 ga teng, shuning uchun sikl operatorlari yana bir marta bajariladi.

Sikl ko'rsatmalari konsolga satr **son** = 2 ni chiqaradi. Va keyin **son** o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 3 ga teng bo'ldi. Shunday qilib, ikkinchi takrorlash amalga oshiriladi.

son < 5 shart yana tekshiriladi. Yana True qiymat qaytariladi, chunki son = 3, shuning uchun sikl operatorlari bajariladi.

Sikl ko'rsatmalari konsolga satr **son** = 3 ni chiqaradi. Va keyin **son** o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 4 ga teng. Ya'ni uchinchi iteratsiya amalga oshiriladi.

son < 5 sharti yana tekshiriladi. Yana True qiymat qaytariladi, chunki son = 4, shuning uchun sikl ko'rsatmalari bajariladi.

Sikl ko'rsatmalari konsolga son= 4 qator raqamini chiqaradi. Va keyin son o'zgaruvchisining qiymati yana bittaga oshiriladi - endi u 5 ga teng. Ya'ni to'rtinchi takrorlash amalga oshiriladi.

Va shart **son < 5** yana tekshiriladi. Lekin endi u False qiymat qaytaradi, chunki son o'zgaruvchisining qiymati endi 5 ga teng, shuning uchun sikl tugatiladi. Shundan so'ng sikldan keyin operatorlar bajariladi. Shunday qilib, bu sikl to'rtta o'tish yoki to'rtta takrorlashni amalga oshiradi

Natijada, kodni bajarishda biz quyidagi konsol chiqishini olamiz:

```
son = 1
son = 2
son = 3
son = 4
```

Dastur tugatildi

While sikli bilan yana bir operator – else ham ishlatiladi. U while siklidagi shart False qiymat qaytarganida ishga tushadi va tarkibidagi blok bajariladi:

```
raqam = 1
while raqam < 5:
    print(f"raqam = {raqam}")
    raqam += 1
else:
    print(f"raqam = {raqam}. sikl tugallandi")
print("Dastur tugatildi")
```

Ya'ni, bu holda birinchi navbatda shart tekshiriladi va while operatorlari bajariladi. Keyin shart False ga aylanganda else blokidagi gaplar bajariladi. E'tibor bering, else blokidagi iboralar ham sikl konstruktsiyasining boshidan chekinadi. Natijada, bu holda biz quyidagi konsol chiqishini olamiz:

```
raqam = 1
raqam = 2
raqam = 3
raqam = 4
raqam = 5. sikl tugallandi
Dastur tugadi
```

For sikli. Sikl ning yana bir turi for konstruktsiyasidir. Ushbu sikl qiymatlar to'plamini takrorlaydi, har bir qiymatni o'zgaruvchiga qo'yadi va keyin siklda biz ushbu o'zgaruvchi bilan turli harakatlarni bajarishimiz mumkin. For siklining strukturasi quyidagicha:

```
for [o'zgaruvchi] in [qiymatlar_to'plami]:
    [sikl tanasi]
```

For kalit so'zidan keyin qiymatlar joylashtiriladigan o'zgaruvchining nomi keladi. Keyin in operatoridan keyin qiymatlar to'plami va ikki nuqta qo'yiladi.

Va keyingi qatordan sikl tanasidagi operatorlar boshlanadi. Bu operatorlar siklning bir qismi ekanligi uchun ular `for` kalit soʻzi boshidan ichkariga kiritilishi kerak.

Siklni bajarishda Python ketma-ket barcha qiymatlarni toʻplamdan oladi va ularni oʻzgaruvchiga uzatadi. Toʻplamdagi barcha qiymatlar takrorlangandan keyin sikl tugaydi.

Masalan, qiymatlar toʻplami sifatida siz aslida belgilar toʻplamini ifodalovchi satrni koʻrib chiqishingiz mumkin. Keling, bir misolni koʻrib chiqaylik :

```
string = "Hello"
for char in string:
    print(char)
```

char oʻzgaruvchisi siklda aniqlanadi, **in** operatoridan keyin **string** oʻzgaruvchisi "Hello" satrini saqlaydigan takrorlangan toʻplam sifatida koʻrsatiladi. Natijada, `for` sikli **string** tarkibidagi barcha belgilarni ketma-ketlikda takrorlaydi va ularni **char** oʻzgaruvchisiga oʻzlashtiradi. Sikl blokining oʻzi **char** oʻzgaruvchining qiymatini konsolga chop etadigan bitta funksiyadan iborat. Dasturning konsol chiqishi:

```
H
e
l
l
o
```

`For` siklida sikl tugagandan keyin bajariladigan qoʻshimcha `else` bloki ham boʻlishi mumkin:

```
string = "Hello"
for char in string:
    print(char)
else:
    print(f"oxirgi simvol: {char}")
print("Dastur yakunlandi")
```

Bunday holda, biz quyidagi konsol chiqishini olamiz:

```
H
e
l
l
o
oxirgi simvol: o
Dastur yakunlandi
```

Shuni ta'kidlash kerakki, `else` bloki `for` siklida aniqlangan barcha o'zgaruvchilarga kirish huquqiga ega.

Ichma-ich sikllar. Ba'zi sikllar o'zlarida boshqa sikllarni o'z ichiga olishi mumkin. Ko'paytirish jadvalining chiqishi misolini ko'rib chiqing:

```
a = 1
b = 1
while a < 10:
    while b < 10:
        print(a * b, end="\t")
        b += 1
    print("\n")
    b = 1
    a += 1
```

Tashqi sikl `while a < 10`: `a` o'zgaruvchisi 10 ga teng bo'lgunga qadar 9 marta ishlaydi. Bu sikl ichida ichki sikl `while b < 10`: ishlaydi. `b` o'zgaruvchisi 10 ga teng bo'lgunga qadar ichki sikl ham 9 marta ishlaydi. Bundan tashqari, ichki siklning barcha 9 ta takrorlanishi tashqi siklning bir iteratsiyasi ichida ishga tushiriladi.

Ichki halqaning har bir iteratsiyasida konsolda `a` va `b` raqamlarining mahsuloti ko'rsatiladi. Keyin `b` o'zgaruvchining qiymati bittaga oshiriladi. Ichki sikl o'z ishini tugatgandan so'ng, `b` o'zgaruvchining qiymati 1 ga o'rnatiladi va `a` o'zgaruvchining qiymati bittaga oshiriladi va tashqi siklning keyingi takrorlanishiga o'tish sodir bo'ladi. Va `a` o'zgaruvchisi 10 ga teng bo'lgunga qadar hamma narsa takrorlanadi.

Shunga ko‘ra, ichki sikl tashqi siklning barcha iteratsiyasi uchun faqat 81 marta ishlaydi. Natijada, biz quyidagi konsol chiqishini olamiz:

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

for siklini xuddi shunday aniqlanishi mumkin:

```
for c1 in "ab":
    for c2 in "ba":
        print(f"{c1}{c2}")
```

Bunday holda, tashqi sikl "ab" qatoridan o‘tadi va har bir belgini c1 o‘zgaruvchisiga qo‘yadi. Ichki sikl "ba" satrini takrorlaydi, satrdagi har bir belgini c2 o‘zgaruvchisiga qo‘yadi va ikkala belgi kombinatsiyasini konsolga chiqaradi. Ya’ni, oxirida biz a va b belgilarning barcha mumkin bo‘lgan kombinatsiyalarini olamiz:

```
ab
aa
bb
ba
```

Break va continue operatorlari. Biz siklni boshqarish uchun maxsus break va continue operatorlaridan foydalanishimiz mumkin. Break operatori siklni to'xtatadi. continue operatori esa siklning keyingi iteratsiyasiga o'tkazadi.

Agar siklda uning keyingi bajarilishiga mos kelmaydigan shartlar hosil bo'lsa, break operatoridan foydalanish mumkin. Quyidagi misolni ko'rib chiqing :

```
raqam = 0
while raqam <5:
    raqam += 1
    if raqam == 3: # agar raqam = 3 bo'lsa, sikldan chiqamiz
        break
    print(f"raqam = {raqam}")
```

Bu yerda while sikli raqam<5 shartini rostlikka tekshiradi. Va agar raqam o'zgaruvchisining qiymati hozircha 5 ga teng bo'lmasa, raqamning qiymati konsolda chop etiladi. Ammo ungacha sikl ichida yana bir shart ham tekshiriladi: if raqam == 3. Ya'ni **raqam**ning qiymati 3 ga teng bo'lsa, u holda break operatori yordamida sikldan chiqamiz. Va oxirida biz quyidagi konsol chiqishini olamiz:

```
raqam = 1
raqam = 2
```

Break operatoridan farqli o'laroq, **continue** operatori siklning keyingi iteratsiyasiga uni tugatmasdan o'tadi. Misol uchun, oldingi misolda break ni continue bilan almashtiramiz:

```
raqam = 0
while raqam <5:
    raqam += 1
```

```
if raqam == 3: # agar raqam = 3 bo'lsa, siklning yangi
iteratsiyasiga o'ting
    continue
print(f" raqam = {raqam}")
```

Va bu holda, agar raqam o'zgaruvchisining qiymati 3 ga teng bo'lsa, continue operatori siklni keyingi aylanishga o'tkazib yuboradi va navbatdagi qadam amalga oshiriladi:

```
raqam = 1
raqam = 2
raqam = 4
raqam = 5
```

Nazorat savollari :

1. Dasturlashda takrorlanuvchi jarayonlarni dasturlash qanday ro'l egallaydi?
2. Takrorlanuvchi jarayonlarni qanday operatorlar yordamida dasturini tuziladi?
3. **for** operatorining ishlash prinsipi qanday?
4. **while** operatorining sintaksisi va ishlash prinsipi qanday?
5. **for** va **while** operatorlarining bir biridan asosiy farqlari nimada?
6. **else** operatorini siklik jarayonlarda nima maqsadda foydalaniladi?
7. **range** operatorining vazifasi va ishlash prinsipi qanday?
8. **break** operatorining vazifasi nima?
9. **continue** operatori qanday holatlarda ishlatiladi?

2.8.§. Funktsiyalar

Funktsiyalar ma'lum bir vazifani bajaradigan va dasturning boshqa qismlarida qayta ishlatilishi mumkin bo'lgan kod blokini ifodalaydi. Funktsiyalardan oldingi mavzularni tushuntirish davomida allaqachon foydalaib o'tildi. Masalan, konsolga qiymatlarni chop etadigan **print()** funksiyasidan deyarli har bir mavzuda foydalanildi. Python ko'plab standart o'rnatilgan funktsiyalarga ega bo'lgan dasturlash tilidir. Bunday standart funktsiyalardan tashqari foydalanuvchi o'ziga kerakli

funktsiyalarni yaratish imkoni mavjud. Funktsiya yaratish uchun quyidagi sintaksisdan foydalaniladi:

```
def function_name([parametrlar]):  
    [funksiya tanasi]
```

Pythonda funksiya boshqa dasturlash tillaridan farqli ravishda `def` kalit soʻzi bilan boshlanadi. `Def` kalit soʻzidan keyin funksiya nomi va agar qabul qiluvchi parametrlari mavjud boʻlsa, ularni qavs ichida, vergul bilan ajratgan holda nomlari keltiriladi va qavslardan keyin osti-usti ikki nuqta qoʻyiladi. Va keyingi qatordan funksiya bajaradigan koʻrsatmalar bloki boshlanadi. Barcha funksiya tanasidagi operatorlar satr boshidan ichkariga chekingan holda bir ustundan boshlanishi kerak. Masalan, eng oddiy funktsiyaning koʻrinishi:

```
def say_hello():  
    print ("Salom")
```

Funktsiya nomi: `say_hello` deb ataladi. U hech qanday parametrlarga ega emas va konsolga "Salom" satrini chop etadigan bitta operatorni oʻz ichiga oladi. Eʼtibor bering, funksiya iboralari funksiya boshidan chekinishi kerak. Masalan:

```
def say_hello():  
    print("Salom")  
    print("Xayr")
```

Bu yerda `print("Xayr")` buyrugʻi `say_hello` funktsiyasi bilan bir ustunda va shuning uchun bu funktsiyaga tegishli boʻlmagan operator hisoblanadi. Funktsiya tarkibiga kiruvchi blok va funktsiyaga tegishli boʻlmagan operatorlar bir-biri bilan oldidagi qoldirilgan tab (boʻsh joy) bilan farq qiladi.

Funktsiyaga murojaat. Funktsiyani chaqirish uchun funksiya nomi va uning qabul qiluvchi parametrlariga mos qiymatlar qavs ichida koʻrsatiladi:

```
funktsiya_nomi ([parametrlar])
```

Masalan, salom tekstini konsolga chiqaruvchi bitta oddiy funksiya yaratamiz va uni chaqiramiz:

```
def hello():  
    print ("Salom")  
hello()  
hello()  
hello()
```

Bu yerda hello funksiyasi ketma-ket uch marta chaqirilmoqda. Natijada, konsolda quyidagi natijalar paydo bo'ladi:

```
Salom  
Salom  
Salom
```

Shuni esda tutish kerakki, funksiya yuqorida yaratiladi va keyin chaqiriladi.

Agar funksiya tarkibida faqat bitta operator mavjud bo'lsa, bu operatorni funksiya ta'rifining davomidan ikki nuqtadan keyin joylashtirilishi mumkin:

```
def hello(): print("Salom")  
hello ()
```

Boshqa funktsiyalarni ham xuddi shunday aniqlash va chaqirish mumkin. Masalan, bir nechta funktsiyalarni aniqlaymiz va bajaramiz:

```
def say_hello():  
    print ("Salom")  
def say_goodbye():  
    print ("Xayr")  
say_hello ()  
say_goodbye ()
```

Natijaning konsolda ko'rinishi :

```
Salom  
Xayr
```

Lokal funksiya. Ba'zi funktsiyalarni boshqa funktsiyalar ichida aniqlash mumkin. Bunday funktsiyalarni ichki funktsiyalar ya'ni **lokal funktsiyalar** deb ataladi. Lokal funktsiyalar faqat o'zi tegishli bo'lgan funksiya doirasida ishlatilishi mumkin. Masalan :

```
def print_msg():  
    def say_hello(): print("Salom")  
    def say_bye(): print("Xayr")  
    say_hello()  
    say_bye()  
print_msg()
```

Bu yerda `say_hello()` va `say_bye()` funksiyalari `print_msg()` funksiyasi ichida aniqlanadi va shuning uchun u uchun lokal fuksiya hisoblanadi. Shunga ko‘ra, ular faqat `print_msg()` funksiyasi ichidagina ishlatilishi mumkin.

main funksiyasi va undan qo‘llanilishi. Dasturda ko‘plab funktsiyalar ishtirok etishi mumkin. Va ularning barchasini birlashtirish uchun asosiy bitta funksiya ishlatiladi. Odatda bu funksiya `main` deb ataladi va unda boshqa funktsiyalar chaqiriladi:

```
def main_f ():  
    say_hello ()  
    say_bye ()  
def say_hello():  
    print("Salom")  
def say_bye():  
    print("Xayr")  
# Asosiy funksiyani chaqirish  
main_f()
```

Nazorat savollari:

1. Lokal funksiya deb qanday funksiyalarga aytiladi?
2. Funksiyalar dasturlashda qanday ro‘l o‘ynaydi?
3. Foydalanuvchi funksiyalari qanday yaratiladi?
4. Funksiya tanasi va funksiyaga tegishli bo‘lmagan operatorlar qanday farqlanadi?

2.9.§. Parametrli funktsiya

Funksiyalarni parametrli va parametrsiz funksiyalarga ajratishimiz mumkin. Parametrsiz funksiyalar tashqaridan hech qanday qiymat qabul qilmaydi. Parametrli funksiyalar esa funksiya chaqirilayotganda biror bir qiymatni berilishini kutadigan funksiyalardir. Ushbu parametrlar orqali funksiyaga turli ma'lumotlarni uzatish mumkin. Buni **print()** funksiyasi misolida ko'rib chiqamiz. **print()** parametr sifatida konsolga chop etilishi kerak bo'lgan xabarli qabul qiladi.

Endi bitta oddiy, ism paramrli funksiya yaratamiz va uni chaqiramiz:

```
def say_hello(ism):  
    print(f"Salom, { ism }")  
  
say_hello("Jamshid")  
say_hello("Boboy")  
say_hello("Asilbek")
```

Say_hello funksiyasi ism parametriga ega va funksiya chaqirilganda ushbu parametrba ba'zi qiymatlarni berish mumkin. Funksiya ichida parametrni oddiy o'zgaruvchi sifatida ishlatish mumkin. Masalan, ushbu parametrning qiymatini print funksiyasi bilan konsolga chop etish. Shunday qilib, ifodada:

```
say_hello("Jamshid ")
```

"Jamshid" qatori ism parametriga uzatiladi. Natijada, dasturni bajarishda quyidagi natijani konsol chiqaradi:

```
Salom, Jamshid  
Salom, Boboy  
Salom, Asilbek
```

Funktsiya chaqirilganda, qiymatlar pozitsiya bo'yicha parametrlarga o'tkaziladi. Masalan, bir nechta parametrli funksiyani quyidagicha yaratish va chaqirish mumkin:

```
def print_per (ism, yosh):  
    print(f"Ism: {ism}")  
    print(f"Yosh: {yosh}")  
print_per ("Jamshid", 25)
```

Bu yerda `print_per` funksiyasi ikkita parametrni qabul qiladi: `ism` va `yosh`. Funksiyani chaqirganda :

```
print_per("Jamshid", 25)
```

Funksiya chaqirilish vaqtidagi unga parametr sifatida beriladigan qiymatlar mos parametrlarga qabul qilinadi. Ya'ni birinchi qiymat – "Jamshid" birinchi parametrga, ya'ni `ism` parametriga qabul qilinsa, ikkinchi qiymat – 25 bo'lsa, `yosh` parametriga qabul qilinadi. Va funksiya ichida parametr qiymatlari konsolga chop etiladi:

```
Ism: Tom  
Yosh: 37
```

Standart qiymatlar

Funksiyani yaratish mobaynida, funksiyaning mavjud parametrlari uchun boshlang'ich qiymatlarni belgilash orqali funksiyaning ba'zi parametrlariga qiymat berishni ixtiyoriy qilinishi mumkin. Masalan :

```
def say_hello(ism="Jamshid"):  
    print(f"Salom, {ism}")  
say_hello()          # bu yerda ism = "Jamshid"  
say_hello("Bobo")    # bu yerda ism = "Bobo"
```

Bu yerda `ism` parametriga qiymat berish majburiy emas. Ya'ni, agar funksiya chaqirishda uning `ism` parametriga qiymat berilmasa standart qiymat – "Jamshid" qo'llaniladi. Ushbu dasturning konsolda ko'rinishi quyidagicha bo'ladi:

```
Salom, Jamshid  
Salom, Bobo
```

Agar funksiya bir nechta parametrlarga ega bo'lsa, qiymat berish ixtiyoriy bo'lgan parametrlar, qiymat berish majburiy bo'lgan qilinganlardan keyin kelishi kerak. Masalan :

```
def print_per(ism, yosh = 54):  
    print(f"Ism: {ism}    Yosh: {yosh}")  
print_per("Bobo")  
print_per("Jamshid", 25)
```

Bu yerda **yosh** parametri ixtiyoriy va standart bo'yicha 54 ga teng. Undan oldin qiymat berish majburiy bo'lgan **ism** parametri joylashgan. Shuning uchun funktsiyani chaqirishda **yosh** parametriga qiymat o'tkaza olmaymiz, lekin **ism** parametriga qiymat uzatish kerak.

Agar kerak bo'lsa, biz barcha parametrlarni ixtiyoriy qilishimiz mumkin:

```
def print_per(ism = "Jamshid", yosh = 18):  
    print(f"Ism: {ism}    Yosh: {yosh}")  
print_per()                # Ism: Jamshid Yosh: 18  
print_per("Bobo")          # Ism: Bobo    Yosh: 18  
print_per("Samat", 54)     # Ism: Samat   Yosh: 54
```

Nomlangan parametrlar. Yuqoridagi misollarda funktsiya chaqirilganda parametrlarga qiymatlar joylashish o'рни bo'yicha uzatildi. Ammo qiymatlarni parametrlarga nomi ko'ra uzatish ham mumkin. Buning uchun funktsiyani chaqirishda parametr nomi ko'rsatiladi va unga qiymat beriladi:

```
def print_per(ism, yosh):  
    print(f"Ism: {ism}    Yosh: {yosh}")  
print_per(yosh = 22, ism = "Jamshid")
```

Bunday holda, **yosh** va **ism** parametrlari nomiga ko'ra qiymatlarni topib olishadi. Funktsiya yaratilayotganda **ism** parametri birinchi o'rinda turibdi. Shunga qaramay, funktsiyani chaqirishda `print_per(yosh= 22, ism = "Jamshid")` kabi chaqirilsa ham bo'ladi va shu tariqa **yosh** parametriga 22 ni va **ism** parametriga "Jamshid" qiymatini berish mumkin.

Funksiya parametriga faqat nomi bo'yicha qiymat berish. * belgisi qaysi parametrlarga nom berilishini belgilash imkonini beradi. Ya'ni qiymatlarni faqat nomi bo'yicha uzatish mumkin bo'lgan parametrlar. * belgisining o'ng tomonida joylashgan barcha parametrlar qiymatlarni faqat nomi bilan qabul qiladi:

```
def print_per(ism, *, yosh, tashkilot):  
    print(f"Ism: {ism} Yosh: {yosh} Tashkilot: {tashkilot}")  
print_per("Bobo", yosh = 41, tashkilot = "Microsoft")
```

Bunday holda, **yosh** va **tashkilot** parametrlari nomi bilan qiymatlar qabul qilishadi.

Parametrlar ro'yxatiga ***** bilan prefiks qo'yish orqali barcha parametrlarni nomlash mumkin:

```
def print_person(*, ism, yosh, tashkilot):  
    print(f"Ism: {ism} Yosh: {yosh} Tashkilot: {tashkilot}")
```

Aksincha, siz qiymatlarni faqat pozitsiya bo'yicha, ya'ni pozitsion parametrlar bo'yicha qabul qilish mumkin bo'lgan parametrlarni belgilashingiz kerak bo'lsa, unda / belgisidan foydalanish mumkin.

Bunda / belgisidan olda joylashgan barcha parametrlar faqat joylashish o'rniga ko'ra qiymatlar va qabul qilishi mumkin.

```
def print_per(ism, /, yosh, tashkilot="Micros"):  
    print(f"Ism:{ism} Yosh: {yosh} Tashkilot: {tashkilot}")  
print_per("Jamshid", tashkilot="ShamsWeb", yosh = 11)  
print_per("Bobo", 41)
```

Bunday holda, **ism** parametri pozitsiondir.

Bitta funktsiyada bir vaqtning o'zida pozitsion va nomli parametrlar mavjud bo'lishi mumkin.

```
def print_per(ism, /, yosh = 18, *, tashkilot):  
    print(f"Ism: {ism} Yosh:{yosh} Tashkilot: {tashkilot}")  
print_per("Samat", tashkilot = "Google")  
print_per("Tomson", 37, tashkilot = "JetBrains")  
print_per("Bobo", tashkilot = "Micros", yosh= 42)
```

Bunday holda, **ism** parametri / belgisining chap tomonida joylashgan, shuning uchun u pozitsion, majburiydir hamda faqat pozitsiya bo'yicha qiymat berilishi mumkin.

Tashkilot parametri * belgisining o'ng tomonida joylashganligi sababli nomlangan parametr hisoblanadi. Yosh parametri nomi bo'yicha ham, joylashgan o'ri bo'yicha ham qiymat qabul qilishi mumkin.

Aniqlanmagan sondagi parametrlar. Funktsiyaga noma'lum miqdordagi qiymatlarni uzatish uchun yulduzcha belgisidan foydalaniladi. Bu funktsiyaga nomalumi miqdordagi, bir nechta qiymatlarni qabul qilishda qo'llaniladi. Masalan, sonlar yig'indisini hisoblash funksiyasini ko'rib chiqamiz:

```
def sum(*raqamlar):  
    natija = 0  
    for j in raqamlar:  
        natija += j  
    print(f"sum = {natija}")  
sum(1, 2, 3, 4, 5)      # sum = 15  
sum(3, 4, 5, 6)        # sum = 18
```

Bunday holda, `sum` funktsiyasi bitta parametрни oladi - * `raqamlar`, lekin parametr nomi oldidagi yulduzcha, aslida ushbu parametr o'rniga cheksiz miqdordagi qiymatlarni yoki qiymatlar to'plamini qabul qilish mumkinligini ko'rsatadi. Funktsiyaning o'zida `for` siklidan foydalanib, ushbu to'plamdan birma-bir elementlarni olish mumkin, ushbu to'plamdan har bir qiymatni `j` o'zgaruvchiga olish va u bilan qandaydir amallarni bajarish mumkin. Misol uchun, bu dasturda uzatilgan raqamlarning yig'indisi hisoblandi.

Nazorat savollari:

1. Parametrli va parametrsiz funksiyalarning bir-biridan farqi nimada?
2. Funksiya parametr sifatida qanday turdagi ma'lumotlarni qabul qilishi mumkin?
3. Funksiya parametrining tinchlik qiymati qanday beriladi va bu qanday hollarda kerak bo'ladi?
4. Nomlangan parametrlar deb qanday parametrlarga aytiladi?

2.10.§. Qiymat qaytaruvchi funktsiya

Natijani qaytarish. Odatda funktsiya bajargan amalning natjasidan dastur davomida foydalanishga zarurat paydo bo‘ladi. Bunday hollarda natija qaytaruvchi funksiyalardan foydalaniladi. Funktsiya natija qaytarishi uchun **return** operatoridan foydalaniladi, shundan so‘ng qaytarish qiymati ko‘rsatiladi:

```
def funksiya_nomi([parametrlar]):  
    [operatorlar bloki]  
    return qaytariladigan_qiymat
```

Qiymat qaytaruvchi eng oddiy funktsiyani yaratamiz:

```
def get_msg():  
    return "Salom, QALAYSIZ?"
```

Bu yerda, `get_msg()` funktsiyasi qaytaradigan qiymat **return** operatoridan keyin kelgan - "Salom, QALAYSIZ?" xabaridir.

Ushbu funktsiya natijasi keyinchalik o‘zgaruvchiga tayinlanishi yoki oddiy qiymat sifatida ishlatilishi mumkin:

```
def get_msg():  
    return "Salom, QALAYSIZ?"  
  
xabar = get_msg()  
print(xabar)                # Salom, QALAYSIZ?  
print(get_msg())            # Salom, QALAYSIZ?
```

Return operatoridan keyin murakkab hisoblash ifodasi ham kelishi mumkin. Uning natijasi funktsiyadan qaytariladi. Masalan, sonni ikki barobarga oshiradigan funktsiyani ko‘rib chiqaylik:

```
def ikkilantirish(son):  
    return 2 * son
```

Bu yerda `ikkilantirish` funktsiya `2 * son` ifodasi natijasini qaytaradi:

```
def ikkilantirish (son):  
    return 2 * son  
  
result1 = ikkilantirish (4)      # result1 = 8  
result2 = ikkilantirish (5)      # result2 = 10
```

```
print(f"result1 = {result1}")    # result1 = 8
print(f"result2 = {result2}")    # result2 = 10
```

Yoki berilgan ikkita raqamning yig'indisini hisblash funksiyasi:

```
def summasi(a, b):
    return a + b

natija = summasi(4, 6) # natija = 10

print(f"summasi(4,6)={natija}") # summasi(4, 6) = 10
print(f"summasi(3,5)={summasi(3,5)}") #summasi(3,5)= 8
```

Funktsiyadan chiqish. Return operatori nafaqat qiymatni qaytaradi, balki funktsiyadan chiqishni amalga oshiradi. Agar return operatori bajarilganidan keyin yana qandaydir operator joyashgan bo'lsa, unda bu operator bajarilmay qoladi. Chunki return qo'llanilganidan so'ng funksiya tugatiladi. Shuning uchun return operatori barcha amallar bajarilgandan keyin qo'llanilishi kerak. Masalan:

```
def get_msg():
    return "Salom, Qandaysan?"

    print("Funksiyaning tugatilishi")

print(get_msg())
```

Sintaksis nuqtai nazaridan, bu funksiya to'g'ri, lekin uning `print("Funksiyaning tugatilishi")` amali mantiqiy emas - u hech qachon bajarilmaydi, chunki u bajarilishidan oldin `return` operatori qiymatni qaytaradi va funktsiyadan chiqadi.

Biroq, hech qanday qiymat qaytarmaydigan funktsiyalarda `return` operatoridan qiymatsiz foydalaniladi. Bunday holda, `return` operatoridan keyin hech qanday qaytarish qiymati qo'yilmaydi. Odatiy holat - muayyan shartlarga qarab funktsiyadan chiqish:

```
def print_per(ism, yosh):
    if yosh > 120 or yosh < 1:
        print("Noto'g'ri yosh ")

    return
```

```
print(f"Ism: {ism}   Age: { yosh }")

print_per("Jamshid", 22)
print_per("Bobo", -102)
```

Bu yerda `print_per` funksiyasi parametr sifatida foydalanuvchi `ismi` va `yoshini` qabul qiladi. Biroq, funktsiyada birinchi navbatda yoshni ma'lum bir oraliqda (120 dan kam va 0 dan katta) ekanligini tekshiriladi. Agar yosh ushbu diapazondan tashqarida bo'lsa, noto'g'ri yosh haqida xabarni ko'rsatiladi va `return` operatori orqali funktsiyadan chiqiladi. Shundan so'ng, funktsiya o'z ishini tugatadi.

Biroq, agar yosh to'g'ri bo'lsa, foydalanuvchi ma'lumotlarini konsolga chop etiladi. Konsol chiqishi:

```
Ism: Jamshid   Yosh: 22
Noto'g'ri yosh
```

Nazorat savollari:

1. Qiymat qaytaruvchi va qaytarmaydigan funksiyalarning bir-biridan farqi nimada?
2. Funktsiyadan shartga asosan chiqib ketish uchun qaysi kalit so'zdan foydalanish kerak?
3. Funktsiya ichida `return` operatoridan keying operatorlar bloki yozilgan bo'lsa, qanday hodisa ro'y beradi?

2.11.§. Lambda funktsiya

Python dasturlash tilidagi lambda ifodalari - bu lambda operatori yordamida aniqlangan kichik anonim funksiyalar. Lambda ifodasining sintaksisi quyidagicha:

```
lambda [parametrlar]: operatorlar
```

Eng oddiy lambda ifodasini aniqlaymiz:

```
msg = lambda: print ("salom")
msg()      # salom
```

Bu yerda `lambda` ifodasi `msg` o'zgaruvchisiga yuklatilgan. Ushbu misolda `lambda` ifodasi hech qanday parametrga ega emas, hech narsa qaytarmaydi va

shunchaki konsolga "salom" xabarini chop etadi. Va msg o'zgaruvchisi orqali bu lambda ifodasini oddiy funktsiya kabi chaqirishimiz mumkin. Aslida, u quyidagi funktsiyaga o'xshaydi:

```
def msg():  
    print("salom")
```

Agar lambda ifodasi parametrlarga ega bo'lsa, ular lambda kalit so'zidan keyin aniqlanadi. Agar lambda ifodasi natijani qaytarsa, u holda u ikki nuqtadan keyin ko'rsatiladi. Masalan, sonning kvadratini qaytaruvchi lambda ifodasini yozaylik:

```
kvadrat = lambda a : a * a  
  
print(kvadrat(4))      # 16  
print(kvadrat(5))      # 25
```

Bunday holda, lambda ifodasi bitta a nomli parametрни qabul qiladi va undan keyingi Ikki nuqtaning o'ng tomonida qaytariladigan qiymat - $a * a$. Ushbu lambda ifodasi quyidagi funktsiyaning alternativi hisoblanadi:

```
def kvadrat2(a): return a * n
```

Xuddi shu usulda, bir nechta parametrlarni qabul qiluvchi lambda ifodalarini yaratish mumkin:

```
sum = lambda a, b: a + b  
  
print(sum(4, 5))      # 9  
print(sum(5, 6))      # 11
```

Lambda ifodalari funktsiya ta'riflarini biroz qisqartirishga imkon bersada, ular faqat bitta operatsiyani bajarishi mumkinligi bilan cheklangan. Biroq, parametr sifatida o'tish yoki boshqa funktsiyaga qaytish uchun funktsiyadan foydalanish kerak bo'lgan hollarda ular juda qulay bo'lishi mumkin. Masalan, lambda ifodasini parametr sifatida o'tkazish:

```
def do_operatsiya(a, b, operatsiya):  
    result = operatsiya(a, b)  
    print(f"result = {result}")
```

```
do_operatsiya(5, 4, lambda a, b: a + b) # result = 9
do_operatsiya(5, 4, lambda a, b: a * b) # result = 20
```

Bunday holda, oxirgi mavzuda bo'lgani kabi, ularni parametr sifatida jo'natish uchun funktsiyalarni belgilashimiz shart emas.

Xuddi shu narsa funktsiyalardan lambda ifodalarini qaytarish uchun ham amal qiladi:

```
def tanlash_funk(tanlangan):
    if tanlangan == 1:
        return lambda a, b: a + b
    elif tanlangan == 2:
        return lambda a, b: a - b
    else:
        return lambda a, b: a * b

operation = tanlash_funk (1) # operation = a+b
print(operation(10, 6)) # 16

operation = tanlash_funk (2) # operation = a-b
print(operation(10, 6)) # 4

operation = tanlash_funk (3) # operation = a*b
print(operation(10, 6)) # 60
```

Nazorat savollari:

1. Lambda funksiyasining vazifasi nima?
2. Lambda funksiyasining sintaksisi va ishlash prinsipi qanday?
3. Oddiy funktsiyadan qanday afzallik va kamchiliklarga ega?

2.12.8. O'zgaruvchilarning tiplarini o'zgartirish

Ma'lumotlar bilan ishlashda har xil turdagi qiymatlardan foydalanish mumkin. Masalan, int, float yoki strng va hokazo tiplardan. Ikkita sonli har xil tiplarni

bir-biri bilan qo‘shib yoki boshqa bir arifmetik amallarni bajarib ko‘rilsa, natijadagi hosil bo‘lgan son ularning umumiy bitta katta to‘laminig tipini qabul qilganini ko‘rishimiz mumkin. Masalan quyidagi misolda:

```
a = 2          # int turdagi raqam
b = 2,5        # float turdagi raqam
c = a + b

print(c)      # 4.5
```

Bunday holda, hech qanday xato bo‘lmaydi. Biroq, Python har doim ham har xil turdagi ma’lumotlarni o‘z ichiga olgan operatsiyalarni avtomatik ravishda bajara olmaydi. Keling, bu holatda qanday qoidalar qo‘llanilishini ko‘rib chiqaylik.

Yashirin konvertatsiyalar. Arifmetik amallardagi ikkala raqam ham bir xil turni ifodalashi kerak. Agar operatsiyaning ikkita operandlari turli xil ma’lumotlar turlarini ifodalasa, Python quyidagi qoidalarga muvofiq avtomatik ravishda turlardan biriga aylantirishga harakat qiladi:

- Agar operatsiya operandlaridan biri kompleks sonni (tur kompleksini) ifodalasa, boshqa operand tipi ham kompleks songa aylantiriladi.
- Agar, operandlardan biri float turiga ega bo‘lsa, ikkinchi operand ham float tipiga o‘tkaziladi. Aslida, bu yuqoridagi misolda sodir bo‘ldi, bu yerda a o‘zgaruvchisining qiymati float turiga aylantirildi.
- Agar, ikkala operand ham int tipida bo‘lsa, tiplar o‘zgarishsiz qoldiriladi va natija int tipida bo‘ladi.

Aniq konvertatsiyalar. Ba’zi hollarda o‘zgaruvchilarning tiplarini o‘zgartirishni qo‘lda bajarishga to‘g‘ri keladi. Masalan, bizda quyidagi kod bor deylik:

```
a = "2"
b = 3
c = a + b
```

Biz "2" + 3 ifoda 5 ga teng bo‘lishini kutamiz. Biroq, bu kod istisno qiladi, chunki birinchi raqam satrni ifodalaydi. Va kodni bajarishda biz shunga o‘xshash narsani ko‘ramiz:

Traceback (most recent call last):

File

"C:/Users/SHAdmin/AppData/Local/Programs/Python/Python310/ssssssssssssssss.py", line 3, in <module>

c = a + b

TypeError: can only concatenate str (not "int") to str
Python turini o'zgartirish uchun bir qator o'rnatilgan funktsiyalarni taqdim etadi:

int(): qiymatni butun songa aylantirish

float(): qiymatni floatga aylantiradi

str(): qiymatni satrga aylantirish

INT. Shunday qilib, oldingi misolda int() funksiyasi yordamida satrni raqamga aylantiramiz:

```
a = "2"
b = 3
c = int(a) + b
print(c)      # 5
```

int() yordamida konversiyalarga misollar:

```
a = int(15)      # a = 15
b = int(3.7)     # b = 3
c = int("4")     # c = 4
e = int(False)   # e = 0
f = int(True)    # f = 1
```

Biroq, agar qaysidir bir qiymatni int tipiga otirishni ilojisi bo'lmasa, u holda int() funksiyasi quyidagi xatolikni qaytaradi: *ValueError: invalid literal for int() with base 10*:

```
b = int("a1c")   # Xato
c = int("4.7")   # Xato
```

FLOAT. float() funksiyasi ham xuddi shunday ishlaydi, biroq vergulli sonlarni ya'ni kasr sonlarni raqamga aylantirish uchun xizmat qiladi. Masalan:

```
a = "2,7"
b = 3
c = float(a) + b

print(c) # 5.7
```

float() funksiyasi yordamida konversiya qilishga misollar:

```
a = float (15)          # a = 15,0
b = float (3,7)         # b = 3,7
c = float ("4,7")       # c = 4,7
d = float ("5")         # d = 5,0
e = float (False)       # e = 0,0
f = float (True)        # f = 1,0
```

Ammo yana, barcha qiymatlarni avtomatik ravishda floatga aylantirib bo'lmaydi. Shunday qilib, quyidagi holatda Python xatolik keltirib chiqaradi:

```
d = float("abcd")      # False
```

STR. str() funksiyasi barcha qiymatlarni satrga aylantirish maqsadida foydalaniladi. Quyida unga doir misollar keltirilgan:

```
a = str(False)         # a = "False"
b = str(True)          # b = "True"
c = str(5)              # c = "5"
d = str(5.7)           # d = "5.7"
```

str() funksiyasi juda ko'plab hollarda foydalaniladi. Masalan sonni satrga shunday holicha qo'shishda xatolik berganligi uchun str funksiyasi orqali sonni satrga aylantirib keyin satr va son birlashtiriladi. Bunga qiyida misollar keltirilgan:

```
yosh = 22
msg = "Yoshi: " + yosh      # xato
print(msg)
```


Agar raqamga raqam qo'shilsa, bu standart holat hisoblanadi. Agar satrga satr qo'shilsa, u holda satrlarni birlashtirish operatsiyasi deb tushiniladi va birlashtirish amali bajariladi. Ammo python turli tipdagi ya'ni, satr va raqamga nisbatan qo'shish amalini qanday bajarishni bilmaydi. Va agar bu holda biz satrlarni birlashtirish operatsiyasini bajarmoqchi bo'lsak, u holda raqamni `str()` funksiyasi yordamida satrga aylantirish talab qilinadi:

```
yosh = 22
msg = "Yoshi: " + str(yosh)      # Yoshi: 22
print(msg)
```

Nazorat savollari:

1. O'zgaruvchilarning tiplarini o'zgartirish dasturlashda qanday holatlarda ishlatiladi?
2. O'zgaruvchilarning turini qanday o'zgartiriladi?
3. Arifmetik amallar bajarilayorganda o'zgaruvchilarning turlari har xil bo'ladigan bo'lsa, operatsiyalar qanday amalga oshiriladi?
4. Yashirin konvertatsiya nima?
5. Aniq konvertatsiya nima?

III BOB. SATRLAR BILAN ISHLASH

3.1.8. Satrlar va ularni e'lon qilish

Satr deb mavjud belgilar ketma-ketligiga aytiladi. Satrlarni pythonda aniqlash uchun ham bittalik, ham qo'sh tirnoqlardan foydalaniladi. Masalan:

```
msg = 'Salom Dunyo!'
print(msg)    # Salom Dunyo!

nomi="Jeyson"
print(nomi)    # Jeyson
```

Agar satrdagi simvollar soni ko'p bo'lsa, uni qismlarga bo'lish va ularni turli xil kod satrlariga joylashtirish mumkin. Bunday holda, butun satr qavs ichiga olinadi va uning alohida qismlari tirnoq ichiga joylashtiriladi:

```
text = ("Salom hurmatli kursantlar"
        "Bugun biz sizlar bilan"
        "Satrlar ustida amallar bajarib ko'ramiz"
)
print(text)
```

Agar ko'p qatorli matnni hosil qilish kerak bo'lsa, unda bunday matn ikki tomondan uch qo'sh ("") yoki bittalik (``) tirnoq ichiga olinib yoziladi:

```
'''
Bu izoh
'''
text = '''Salom hurmatli kursantlar
Bugun biz sizlar bilan
Satrlar ustida amallar
bajarib ko'ramiz
'''
print(text)
```

Faqat shuni esda tutish kerakki uchta qo'shtirnoqdan izohlar uchun ham foydalaniladi. Agar uchta qo'shtirnoq ichidagi matn o'zgaruvchiga tayinlangan bo'lsa,

unda bu izoh emas, satr hisoblanadi. Izoh qoldirish uchun esa izoh bo'lishi kerak bo'lgan matnni uchta qo'shtirnoqqa o'rab olinadi holos.

Satrdagi qo'llaniladigan maxsus simvollar. Satr bir qator maxsus belgilarni o'z ichiga olishi mumkin. Ulardan ba'zilar quyida keltirilgan:

- ❖ `\`: satr tarkibiga slesh simbolini qo'shish imkonini beradi
- ❖ `'`: satr ichiga bitta tirnoq qo'shish imkonini beradi
- ❖ `"`: qatorga qo'sh tirnoq qo'shish imkonini beradi
- ❖ `\n`: Yangi qatorga o'tadi
- ❖ `\t`: Tabulyatsiya qo'shadi (4 ta probelga teng bo'sh joy)

Quyida ba'zi maxsus belgilardan satr ichida foydalanish bo'yicha misollar keltirilgan:

```
text = "Xabar:\n\"Salom Dunyo\""
print(text)
```

Dasturning konsoldagi ko'rinishi:

```
Xabar:
"Salom Dunyo"
```

Garchi bunday maxsus belgilar satrga bittalik yoki qo'sh tirnoq qo'yish, jadval tuzish, boshqa qatorga o'tkazish kabi ko'plab amallarni bajarishga imkon bersada, ba'zi hollarda ular dasturda xatolik keltirib chiqaradi. Masalan:

```
URL = "D:\python_course\name.txt"
print(URL)
```

Bu yerda URL o'zgaruvchisi faylga yo'lni o'zlashtirib oladi. Biroq, satr ichida `"\n"` maxsus belgisi mavjud bo'lib, garchi dasturchi bu belgini faylga yo'lni ko'rsatish

uchun foydalanayotgan bo'lsada, python interpretatori uni maxsus operator sifatida tarjima qiladi. Shunday qilib satrni ekranga chiqarishda natija quyidagicha bo'ladi:

```
c:\python_course  
ame.txt
```

Bunday vaziyatni oldini olish uchun satr oldiga r belgisi qo'yiladi:

```
URL = r"C:\python_course\name.txt"  
print(URL)
```

```
c:\python_course\name.txt
```

Qatorga qiymatlarni kiritish

Python boshqa o'zgaruvchilarning qiymatlarini satrga joylashtirish imkonini beradi. Buning uchun satr ichida o'zgaruvchilar jingalak qavslarga {} joylashtiriladi va satr oldidagi qo'shtirnoq belgisi oldiga f harfi qo'yiladi:

```
user_name = "Jeyson"  
user_age = 22  
user = f"Ismi: { user_name }   Yoshi: { user_age }"  
  
print(user_age)    # Ismi: Jeyson   Yoshi: 22
```

Bunday holda, user_name o'zgaruvchisining qiymati {user_name} o'rniga kiritiladi. Xuddi shunday, {user_age} o'rniga user_age o'zgaruvchisining qiymati kiritiladi.

Satr tarkibidagi belgilarga murojaat qilish. Python dasturlash tilida ham boshqa dasturlash tillaridagi kabi satrning tarkibidagi simvollarga murojaat qilib ular ustida turli amallar bajarish mumkin. Satr tarkibidagi belgilarga murojaat qilish uchun satr_nomi va kvadrat qavslar ichida simvolning indeksi kiritiladi. Indeks bu yerda simvolning satrdagi joylashgan o'rnini bo'lib, simvollar satrda 0 dan boshlab indexlangan bo'ladi. Demak satrdagi simvollar soni oxirgi simvolning indeksidan bittaga ortiq bo'ladi. Quyida satr tarkibidagi belgilarga murojaat qanday amalga oshirilishiga misol keltirilgan:

```
str = "hello world!"
belgi = str[0]    # h
print(belgi)

belgi = str[6]    # w
print(belgi)

belgi = str[11]   # error: IndexError: string index out of range
print(belgi)
```

Agar satrda mavjud bo'lmagan indeksdagi elementga murojaat qilinsa, `IndexError` xatoligi kelib chiqadi. Misol uchun, yuqoridagi holatda, `str` nomli satr 11 ta belgidan iborat va belgilari 0 dan 10 gacha indekslarga ega. Shuning uchun `str[11]` ifodasi xatolik qaytardi. Chunki satrda 11 ta element va oxirgi elementning indeksi 10 ga teng.

Satr oxiridan boshlab elementlarga murojlat qilish uchun manfiy indekslardan foydalanish mumkin. Shunday qilib, indeks -1 oxirgi belgini va indeks -2 oxirgi belgidan bitta oldingi ya'ni oxiridan 2- belgini ifodalaydi va hokazo. Masalan:

```
str = "salom dunyo!"
belgi = str[-1]

print(belgi)      #!

belgi2 = str[-5]

print(belgi2)     #u
```

Belgilar bilan ishlashda satr o'zgarmas (immutable) tip ekanligini hisobga olish kerak, shuning uchun satrning har qanday individual belgisini o'zgartirishga harakat qilsak, quyidagi holatda bo'lgani kabi xatoga duch kelamiz:

```
str = "Salom DASTURCHILAR!"
str[1] = "R"
```

TypeError: 'str' object does not support item assignment

Faqat satr qiymatini unga boshqa qiymat belgilash orqali butunlay qayta o'rnatishimiz mumkin.

Satrnin barcha elementlarini sikl yordamida olish. Satrdagi barcha belgilarni for tsiklini qo'llash orqali olish mumkin:

```
str = "salom dunyo"  
for belgi in str:  
    print(belgi)
```

```
s  
a  
l  
o  
m  
  
d  
u  
n  
y  
o
```

Matndan belgilar ketma-ketligini olish. Satr tarkibidan nafaqat bitta belgini, balki belgilar ketma-ketligini ham olish mumkin. Buning uchun quyidagi sintaksis qo'llaniladi:

str[:end] – ushbu ifoda 0 - indeksdan to end - indeksigacha belgilar ketma-ketligini chiqaradi. end – elementni o'zini olmaydi. End o'rnida son beriladi. str[:10]

str[start:end] – belgilar ketma-ketligi **start** dan to **end** - indeksigacha bo'lgan belgilar ketma-ketligini qaytaradi. Ammo ikkinchi index – end xisobga olinmaydi.

string[start:end:step] – belgilar ketma-ketligini **start** dan to **end** - indeksigacha bo'lgan belgilar ketma-ketligini **step** – qadami bilan qaytaradi.

Quyida `str` satrining tarkibidan belgilar ketma-ketligini olish uchun barcha variantlardan foydalanilgan dastur keltirilgan:

```
str = "salom dunyo"

# 0-indeksdan 5 - indeksigacha
sub_str1 = str[:5]
print(sub_str1)          # salom

# 2-indeksdan 5-indeksigacha
sub_str2 = str[2:5]
print(sub_str2)          #llo

# 2 - indeksdan 9-indeksigacha oraliqda 2 qadam bilan
sub_str3 = str[2:9:2]
print(sub_str3)          # lmdn
```

Satrlarni birlashtirish. Satrlar ustida eng keng tarqalgan amallardan biri ularni birlashtirish yoki qo'shish (konkatinatsiya) qilish. Starlar ustida arifmetik qo'shish amalini bajaradigan bo'lsak, satrlarni birlashtirish amalini bajargan bo'lamiz:

```
name = "Jeyson"
surname = "Stethom"

full_name = name + " " + surname

print(full_name)  # Jeyson Stethom
```

Ikki qatorni birlashtirish oson, lekin agar satr va raqam qo'shishi kerak bo'lsa-chi? Bunday holda `str()` funksiyasidan foydalanib raqamni satrga o'tkazish talab qilinadi:

```
name=" Jeyson "
age = 38

info_per = "Ism: " + name + " Yoshi: " + str(age)
print(info_per)  # Ism: Jeyson Yoshi: 38
```

Satrni takrorlash. Satrlarga arifmetik ko'paytirish amalini qo'llash, ularning shuncha marta takrorlaydi:

```
print("char" * 3)    # charcharchar
print("u" * 4)       # uuuu
```

Satrlarni taqqoslash. Satrlarni taqqoslashda asosiy diqqatni belgiga va ularning katta yoki kichik harfligiga qaratiladi. Demak, raqamli belgi shartli ravishda har qanday alifbo belgisidan kichik hisoblanadi. Katta harfdagi alifbo belgilari shartli ravishda kichik alifbo belgilaridan kichik hisoblanadi. Masalan:

```
str1 = "1a"
str2 = "aa"
str3 = "aa"
print(str1>str2) #False, chunki str1 ni 1- belgisi raqam
print(str2>str3) #Rost, chunki str2 ni 1- belgisi kichik harf
```

Shuning uchun "1a" qatori shartli ravishda "aa" qatoridan kichikdir. Avval birinchi belgi solishtiriladi. Agar ikkala satrning bosh belgilari raqamlarni ifodalasa, u holda kichikroq raqam kichikroq hisoblanadi, masalan, "1a" satri "2a" dan kichik.

Agar boshlang'ich belgilar bir xil holatda alifbo belgilarini ifodalasa, unda alifbo bo'yicha tartibiga qaratiladi. Shunday qilib, "aa" "da" dan kichik va "da" "fa" dan kichikdir.

Agar birinchi belgilar bir xil bo'lsa, ikkinchi belgilar solishtiriladi va hokazo.

Satrlarni katta kichik harflarini bir biri bilan solishtirishdan qochish maqsadida ularning harflarini bir xilga keltirib olish mumkin. Ya'ni katta yoki kichik registrga o'tkazib olish mumkin.

lower() funksiyasi satrdagi barcha harflarni kichik harfga, **upper()** funksiyasi esa satrdagi barcha harflarni bosh harfga o'zgartiradi. Bu funkmsiyalar satrning o'ziga o'zgartirish kiritadi.

```
str1 = "Tomas"
str2 = "TOMAS"
print(str1 == str2)    # False - satrlar teng emas

print(str1.lower() == str2.lower())    # Rost
```

ord va **len** funktsiyalari. Satr Unicode belgilarni o'z ichiga olganligi sababli, Unicode belgisining raqamli qiymatini olish uchun **ord()** funktsiyasidan foydalanish mumkin:


```
print(ord("B"))      # 66
```

String uzunligini olish uchun **len()** funksiyasidan foydalaniladi:

```
str = "salom dunyo!"  
uzunlik = len(str)  
print(uzunlik)      # 12
```

Satrdagi qidirish. **text in str** ifodasidan foydalanib, **str** qatoridan **text** satrini qidirib topish mumkin. Agar **str** satrida **text** topilsa, u holda ifoda **"True"** ni qaytaradi, aks holda **"False"** ni qaytaradi:

```
str = "salom dunyo"  
is_true = "salom" in str  
print(is_true)      # rost  
  
is_true = "sdunyo" in string  
print(is_true)      # False
```

Nazorat savollari:

1. Satr nima?
2. Satrlarni qanday e'lon qilinadi?
3. Satrning ichidan satr qidirish qanday amalga oshiriladi?
4. Satr uzunligini olish qanday amalga oshiriladi?

3.2.8. Satr bilan ishlovchi operatorlar va metodlardan foydalanish

Satrlar bilan ishlash uchun ko'p qo'llaniladigan asosiy metodlarni ko'rib chiqaylik:

isalpha (): agar satr faqat alfavit belgilaridan iborat bo'lsa, True qaytaradi;

islower (): Agar satr faqat kichik harflardan iborat bo'lsa, True qiymatini qaytaradi;

isupper (): Agar satrdagi barcha belgilar katta harflardan iborat bo'lsa, True qiymatini qaytaradi;

isdigit (): agar satrning barcha belgilari raqam bo'lsa, True qiymatini qaytaradi;

isnumeric(): agar satr raqam bo'lsa, True qiymatini qaytaradi;

startswith(str): agar satr pastki qator str bilan boshlansa, True qiymatini qaytaradi;

endwith(str): agar satr oxiri **str** bilan tugasa, True qiymatini qaytaradi;

low(): satrni kichik harfga o'zgartiradi;

upper(): satrni katta harfga aylantirish;

title(): Satrdagi barcha so'zlarning birinchi harflarini bosh harfga o'zgartiradi;

capitalize(): satrning faqat birinchi so'zining birinchi harfini bosh harf bilan yozadi;

lstrip(): satr boshidagi probellarni o'chirib, satrni tozalaydi;

rstrip(): satr oxiridagi probellarni o'chirib, satrni tozalaydi;

strip(): satrdan oldingi va keyingi bo'shliqlarni olib tashlaydi;

ljust(width): agar satr uzunligi width parametridan kichik bo'lsa, kenglik qiymatini to'ldirish uchun satrning o'ng tomoniga bo'shliqlar qo'shiladi va satrning o'zi oqlanadi;

rjust(width): agar satr uzunligi width parametridan kichik bo'lsa, kenglik qiymatini to'ldirish uchun satrning chap tomoniga bo'shliqlar qo'shiladi va satrning o'zi o'ng tomonga asoslanadi;

center(width): agar satrning uzunligi kenglik parametridan kichik bo'lsa, kenglik qiymatini to'ldirish uchun satrning chap va o'ng tomoniga bo'shliqlar teng ravishda qo'shiladi va satrning o'zi markazlashtiriladi;

find(str[, start [, end]]): satrning ko'rsatilgan intervalidan qism qatorini qidirish. Agar satrdan qism qator topilsa indeksini qaytaradi. Agar qator topilmasa, -1 raqami qaytariladi;

replace(old, new[, num]): satrdagi bir qism qatorni boshqasiga almashtiradi;

split([delimiter[, num]]): ajratuvchiga qarab qatorni qism qatorlarga ajratadi;

join(strs): bir nechta satrni ular orasiga ma'lum ajratuvchi qo'yib, bir qatorga birlashtiradi.

Yuqorida keltirilgan metodlarni dasturda misollar bilan ko'rib chiqaylik. Masalan, agar raqam klaviaturadan kiritilishi kerak bo'lsa, kiritilgan satrni raqamga aylantirishdan oldin isnumeric() usuli yordamida haqiqatda raqam kiritilganligini

tekshirishimiz mumkin va agar shunday bo'lsa, raqamga o'girish amalini bajariladi (**isnumeric()** metodi):

```
str = input(" Raqamni kiriting :")
if str.isnumeric():
    raqam = int(str)
    print(raqam)
```

Satr ma'lum bir qism qator bilan boshlanishi yoki tugashini tekshirish (**startswith()** metodi):

```
f = "hello.py"

startsWithHello = f.startswith("hello") #True
endsWithExe = f.endswith("exe") # False
```

Satrning boshida va oxiridagi bo'shliqlarni olib tashlash (**strip()** metodi):

```
str = "    Salom    dunyo!    "
str = str.strip()
print(str)                # salom    dunyo!
```

Bo'shliqlar va hizalamalar bilan satrni to'ldirish (**rjust()** metodi):

```
print("Redmi 7:", "500".rjust(10))
print("Nokia P10:", "360".rjust(10))
```

Konsol chiqishi:

```
Redmi 7:          500
Nokia N10:        360
```

Satrdagi qidirish

Satrdagi qism qatorni topish uchun Pythonda **find()** metodi mavjud, u satrda qism qatorning birinchi paydo bo'lish indeksini qaytaradi. **find()** metodining uchta ko'rinishi mavjud:

find(str): str qism satrini satr boshidan oxirigacha qidiriladi;

find(str, start): start parametri qidiriladigan boshlang'ich indeksni belgilaydi va start -indeksdan oxirigacha bo'lgan simvollar ketma-ketligidan qidiradi;

find(str, start, end): bu holda str qism satri satrning start va end indeksleri orasidan qidiradi.

Agar qism qator topilmasa, find metodi -1 qiymatni qaytaradi:

```
Hi_bye = "Salom dunyo! Xayr dunyo!"
indeks = Hi_bye.find("dun")
print(indeks)           #6

# 10 indeksdan qidirish
indeks = Hi_bye.find("dun",10)
print(indeks)           #21

# 10 dan 15 gacha indekslarni qidirish
indeks = Hi_bye.find("dun",10,15)
print(indeks)           # -1 - topilmaganini anglatadi
```

Satrdagi almashtirish (*replace*). Satrdagi bitta str satrini boshqasiga almashtirish uchun ***replace()*** metodidan foydalaning. Replace metodini bir nechta turda qo'llash mumkin:

***replace* (old, new)**: satrdagi **old** satrlarni **new** satr bilan almashtiradi;

***replace*(old, new, num)**: **num** parametri **old** satrlarning nechtasini **new** satri bilan almashtirishni belgilab beradi. Odatda **num** parametridan foydalanilmasa, u -1 qiymatni qabul qiladi, bu esa barcha **old** satrlarni **new** satri bilan almashtirilishini anglatadi.

```
tel_num = "+99-899-667-89-10"

# defislarni bo'sh joylar bilan almashtirish
tahrirlangan_telefon = tel_num.replace("-", " ")
print(tahrirlangan_telefon)      # +99 899 667 89 10

# defislarni olib tashlash
tahrirlangan_telefon = tel_num.replace("-", "")
```

```

print(tahrirlangan_telefon)          # +998996678910
# faqat birinchi defisni almashtirish
tahrirlangan_telefon = tel_num.replace("-", "", 1)
print(tahrirlangan_telefon)          # +99899-667-89-10

```

Qism qatorlarga ajratish. *Split()* metodi chegaralovchiga qarab qatorni qism qatorlar ro'yxatiga ajratadi. Ajratuvchi har qanday belgi yoki belgilar ketma-ketligi bo'lishi mumkin. Ushbu metod quyidagi ko'rinishlarda qo'llanilishi mumkin:

split(): ajratuvchi sifatida bo'sh joy qilinadi. Va probeldan probelgacha bo'lgan joylarni element qilib oladi.

split(delimiter): cheklovchi sifatida delimiter dan foydalaniladi.

split(delimiter, num): num parametri bo'linish uchun delimiter (ajratuvchi) ning necha marta ishlatilishini belgilaydi. Qatorning qolgan qismi qismlarga ajratilmasdan ro'yxatga qo'shiladi.

```

text = "Bu katta, ikki bo'yli eman, shoxlari singan
va po'stlog'i singan"
# probellardan ajratiladi

splitted_text = text.split()
print(splitted_text)
print(splitted_text[6])  #singan

# vergul ustida tanaffus
splitted_text = text.split(",")
print(splitted_text)
print(splitted_text[1])  # ikki bo'yli eman

# birinchi besh bo'shliqqa bo'lingan
splitted_text = text.split(" ", 5)
print(bo'lingan_matn)
print(bo'lingan_matn[5])

# shoxlari singan va po'stlog'i singan

```

Satrlarni birlashtirish. Satrlar ustidagi eng oddiy amallarni ko'rib chiqq turib, qo'shish amali yordamida qatorlarni birlashtirishni ko'rsatdik. Satrlarni

birlashtirishning yana bir imkoniyati - ***join()* usuli**: u bir nechta satrlarni birlashtirish uchun ishlatiladi.

Bundan tashqari, ushbu metod chaqirilgan joriy satr ajratuvchi sifatida ishlatiladi:

```
strings = ["Let's", "go", "speaking", "from", "my",
"heart", "into", " My box"]

# ajratuvchi - bo'sh joy
sentence = " ".join(strings)
print(sentence)
# Let's go speaking from my heart into My box

# ajratuvchi - vertikal chiziq
sentence = " | ".join(strings)
print(sentence)
# Let's | go | speaking | from | my | heart | into | My box
```

Ro'yxat o'rniga oddiy satr qo'shilish usuliga o'tkazilishi mumkin, keyin ajratuvchi ushbu satr belgilari orasiga kiritiladi:

```
str = "hello"
joined_str = "\".join(str)

print(joined_str)      # h\e\l\l\o
```

Nazorat savollari:

1. satrlarning asosiy metodlariga misollar keltiring.
2. Satrda qidirish metodi qaysi?
3. Split() metodining vazifasi qanday va u parametr sifatida qanday qiymatlar qabul qiladi?
4. Len() – metodining vazifasi qanday?
5. Join() - metodining vazifasi qanday?

6. isalpha () - metodining vazifasi qanday?
7. islower () - metodining vazifasi qanday?
8. isupper () - metodining vazifasi qanday?
9. isdigit () - metodining vazifasi qanday?
- 10.startswith () - metodining vazifasi qanday?
- 11.endswith () - metodining vazifasi qanday?
- 12.replace () - metodining vazifasi qanday?

3.3.§. Format metodining qo'llanilishi

Oldingi mavzularda satr oldiga f belgisini qo'yish orqali ba'zi qiymatlarni qanday kiritish mumkinligi ko'rib chiqilgan edi. Masalan:

```
first_name="Jeyson"
text = f"Hello, {first_name}."
print(text)          # Hello, Jeyson.

name="Bobo"
age=73
info = f"Ismi: {name}\t Yoshi: {age}"
print(info)          # Ismi: Bobo  Yoshi: 73
```

Satrga o'zgaruvchining qiymatini kiritish uchun maxsus parametrlardan foydalaniladi, ular jingalak qavslar ({}) bilan o'rab olinadi.

Nomlangan parametrlar. Formatlanayotgan satrga chaqirilayotgan o'zgaruvchilarni **format()** metodiga parametr sifatida beriladi. Format metodi esa ushbu parametrlarning qiymatlarini satr tarkibida jingalak qavslarda ko'rsatilgan mos nomlarga uzatadi:

```
txt = "Salom, {first_name}.".format(first_name="Jeyson")
print(txt)          # Salom, Jeyson.

info = "Ismi: {name}\t Yoshi: {age}.".format(name="Bobo", age=73)
print(info)          # Ismi: Bobo  Yoshi: 73
```

Demak, formatlash metodida parametrlar qatordagi parametrlar bilan satr tarkibidagi jingalak qavslar ichidagi nomlar ham bir xil bo'lishi talab qilinadi. Shunday qilib, agar parametr yuqorida ko'rsatilga dasturda bo'lgani kabi `first_name` deb ataladigan bo'lsa, u holda qiymat tayinlangan argument ham `first_name` deb ataladi.

Joylashgan o'rni bo'yicha parametrlar. Shuningdek, ketma-ket argumentlar to'plamini formatlash usuliga o'tkazish va bu argumentlarni format satrining o'ziga, ularning indeksini jingalak qavslar ichida ko'rsatish mumkin (indekslash noldan boshlanadi):

```
info = "Ismi: {0}\t Age: {1}".format("Bobo", 73)
print(info)      # Ismi: Bobo  Age: 73
```

Bunday holda, argumentlar satrga bir necha marta chaqirilishi ham mumkin:

```
txt = "Salom, {0} {0} {0}.".format("Jeyson")
Salom, Jeyson Jeyson Jeyson
```

Almashtirishlar. Formatlangan qiymatlarni satrga o'tkazishning yana bir usuli - o'rniga ma'lum qiymatlar qo'shiladigan almashtirishlar yoki maxsus to'ldiruvchilardan foydalanish. Formatlash uchun quyidagi to'ldiruvchilardan foydalanish mumkin:

s: String tipli ma'lumot kiritish uchun;

d: Double tipli sonlarni kiritish uchun;

f: Float tipli sonlarni kiritish uchun. Bu tur uchun nuqta orqali kasr sonini aniqlash ham mumkin;

%: qiymat 100 ga ko'paytiriladi va foiz belgisini qo'shib qo'yiladi.

To'ldiruvchining umumiy sintaksisi quyidagicha:

```
{:[maxsus to'ldiruvchi] }
```

To'ldiruvchiga qarab qo'shimcha parametrlar qo'shilishi mumkin. Masalan, float raqamlarini formatlash uchun siz quyidagi variantlardan foydalanishingiz mumkin

```
{:[belgilar_soni][vergul][.sonning_verguldan_keyingi_qismi] to'ldiruvchi}
```


Format metodini chaqirganda, qiymatlar unga argumentlar sifatida uzatiladi, ular to'ldiruvchilar o'rniga kiritiladi. String almashtirish (`{:s}`) ga misol:

```
hi = "Salom sizga {:s}"
name = "Jeyson"
formatted_txt = hi.format(name)
print(formatted_txt)          # Salom sizga Jeyson
```

`Format()` usuli natijada yangi formatlangan qatorni qaytaradi.

`{:d}` - butun son formatlashga misol:

```
txt = "{:d} ta simvol"
number = 12
formatted_txt = txt.format(number)
print(formatted_txt)        # 12 ta simvol
```

Agar formatlash kerak bo'lgan raqam 999 dan katta bo'lsa, vergulni minglik ajratuvchi sifatida ishlatmoqchi ekanligimizni to'ldiruvchi ta'rifida belgilash (`{:,d}`) mumkin:

```
txt = "{:,d} belgi"
print(txt.format(300))      # 300 belgi
```

Bundan tashqari, to'ldiruvchilar f-satrlarida ham ishlatilishi mumkin:

```
n = 300
txt = f"{n:,d} belgi"
print(txt)          # 200 belgi
```

Kasr sonlar uchun, sonning kasr qismida qancha belgi ko'rinishini aniqlash mumkin. Buning uchun verguldan keyin ko'rinishi kerak bo'lgan sonlar soni, nuqtadan keyin to'ldiruvchidan oldin kiritiladi. Masalan:

```
num = 115.2548695
print("{:.2f}".format(num))      # 115.25
print("{:.3f}".format(num))      # 115.254
print("{:.4f}".format(num))      # 115.2548
print("{:,.2f}".format(12582.23664))  # 12,582.23
```

Yana bir parametr formatlangan qiymatning minimal kengligini belgilar bilan belgilash imkonini beradi:

```
print("{:10.2f}".format(115.2548695))      #      115.25
print("{:8d}".format(25))                  #      25
```

format metodini o'rnini bosuvchi **f** usuli:

```
n1 = 115.2548695
print(f"{n1:10.2f}")      #      115.25
n2 = 26
print(f"{n2:8d}")         #      26
```

Foizlarni ekranga chiqarish uchun **"%"** kodini ishlatish yaxshiroqdir :

```
num = .123456
print("{:%}".format(num))      # 12.3456000%
print("{:.0%}".format(num))    # 12%
print("{:.1%}".format(num))    # 12.3%

print(f"{num:%}")              # 12.3456000%
print(f"{num:.0%}")            # 12%
print(f"{num:.1%}")            # 12.3%
```

Format metodini ishlatmasdan formatlash. Quyidagi sintaksis bilan formatlashning yana bir usuli ham mavjud:

```
string%(param1, param2, ..paramN)
```

Ya'ni, boshida yuqorida ko'rib chiqilgan bir xil to'ldiruvchilarni o'z ichiga olgan qator mavjud (% to'ldiruvchidan tashqari) satrdan keyin foiz - % belgisi qo'yiladi va keyin qatorga kiritiladigan qiymatlar ro'yxati keltiriladi. Aslida, foiz belgisi yangi qatorga olib keladigan operatsiyani anglatadi:

```
info = " Ism : %s \t Yoshi : %d" % ("Jeck", 32)
print(info)      # Ism : Jeck      Yoshi : 32
```

To'ldiruvchidan keyin foiz belgisi qo'yiladi va format funksiyasidan farqli o'laroq, bu yerda jingalak qavslar kerak emas.

Bundan tashqari, bu yerda raqamlarni formatlash usullari ham qo'llaniladi:

```
raqam = 23.8689578
print("%0.2f    - %e" % (raqam, raqam))
# 23.87    - 2.386896 e +01
```

Nazorat savollari:

1. Format metodining qo'llanilishi qanday?
2. Format metodining "Nomlangan parametrlar" usulini tushuntirib bering.
3. Format metodining "Pozitsiya bo'yicha parametrlar" usulini tushuntirib bering
4. Almashtirishlar – bu qanday vazifani bajaradi?
5. Format metodisiz formatlash qanday amalga oshiriladi?

IV BOB. RO'YXAT, KORTEJ VA LUG'ATLAR BILAN ISHLASH

4.1.8. Pythonda ro'yxat tushunchasi va u bilan ishlovchi metodlar bilan tanishish

Ma'lumotlar to'plamlari bilan ishlash uchun Python ro'yxat, kortej va lug'atlar kabi o'rnatilgan turlarni taqdim etadi.

Ro'yxat (list) - elementlar to'plami yoki ketma-ketligini saqlaydigan ma'lumotlar turi. Ko'pgina dasturlash tillarida massiv deb ataladigan o'xshash ma'lumotlar tuzilmasi mavjud.

Ro'yxat yaratish

Ro'yxatni yaratish uchun kvadrat qavslar [] ishlatiladi. Ularning ichida ro'yxat elementlari vergul bilan ajratilgan holda saqlanadi. Masalan, raqamlar ro'yxatini yarataylik:

```
raqamlar = [0, 2, 12, 44, 51]
```

Xuddi shunday, boshqa turdagi ma'lumotlar bilan ro'yxatlarni belgilash mumkin, masalan, satrlar (ya'ni string turdagi ma'lumotlar) ro'yxatini yaratish quyidagicha amalga oshiriladi:

```
odamlar = ["Jasur", "Akmal", "Bobur"]
```

Ro'yxat yaratish uchun **list()** konstruktor funksiyasidan ham foydalanish mumkin :

```
raqamlar1 = []  
raqamlar2 = list()
```

Ushbu ikkala ro'yxat ta'riflari o'xshash - ular bo'sh ro'yxat yaratadi.

Ro'yxatda faqat bir xil turdagi ob'ektlar bo'lishi shart emas. Bir vaqtning o'zida satrlarni, raqamlarni va boshqa ma'lumotlar turlarining ob'ektlarini bir xil ro'yxatga joylashtirish mumkin:

```
obyektlar = [1, 2.26, "Salom dunyo", True, False]
```

Ro'yxat elementlarini ekranga chiqarish uchun standart print funksiyasidan foydalanish mumkin. Bu ro'yxat tarkibini to'liq shaklda konsolga chop etadi:

```
raqamlar = [1, 2, 3, 4, 5]  
odamlar = ["Jasur", "Akmal", "Bobur"]
```

```
print(raqamlar)    # [1, 2, 3, 4, 5]
print(odamlar)     # ["Jasur", "Akmal", "Bobur"]
```

`list()` konstruktori qiymatlar to'plamini qabul qilishi mumkin, ular asosida ro'yxat tuziladi:

```
raqamlar1 = [1, 2, 3, 4, 5]
raqamlar2 = list(raqamlar1)
print(raqamlar2)    # [1, 2, 3, 4, 5]

harflar = list("Salom")
print(harflar)      # ['S', 'a', 'l', 'o', 'm']
```

Agar bir xil qiymat bir necha marta takrorlanadigan ro'yxatni yaratish kerak bo'lsa, unda yulduzcha - * belgisidan foydalanish mumkin, ya'ni aslida mavjud ro'yxatga ko'paytirish amalini qo'llaniladi:

```
raqamlar = [15]*6          # 15 marta 6 takrorlash
print(raqamlar)            # [15, 15, 15, 15, 15, 15]

odamlar = ["Jasur"]*3      # "Tom" ni 3 marta takrorlash
print(odamlar)             # ["Jasur", "Jasur", "Jasur"]

talabalar = ["Bobur", "Saman"] * 2
# "Bobur", "Saman" ni 2 marta takrorlash

print (talabalar)
# ["Bobur", "Saman", "Bobur", "Saman"]
```

Ro'yxat elementlariga murojaat qilish. Ro'yxat elementlariga murojaat qilish uchun ro'yxatdagi element sonini ifodalovchi indekslardan foydalanish kerak. Indekslar noldan boshlanadi. Ya'ni, birinchi element indeksi 0, ikkinchi element indeksi 1 ga teng bo'ladi va hokazo. Elementlarga teskari tartibda ya'ni oxiridan boshiga qarab murojaat qilish uchun manfiy indeks tushunchasi kiritilgan. Bu manfiy indekslar -1 dan boshlanadi. Ya'ni, oxirgi element -1 indeksiga ega bo'ladi, oxirgidan bitta oldingisi esa -2 indeksiga ega bo'ladi va hokazo.

```
odamlar = ["Jasur", "Samat", "Bobur"]
# ro'yxat boshidan elementlarni olish

print(odamlar[0])    # Jasur
print(odamlar[1])    # Samat
print(odamlar[2])    # Bobur

# ro'yxat oxiridagi elementlarni olish
print(odamlar[-2])   # Samat
print(odamlar[-1])   # Bobur
print(odamlar[-3])   # Jasur
```

Ro'yxat elementini o'zgartirish uchun unga yangi qiymat yuklash kifoya:

```
odamlar = ["Jasur", "Samat", "Bobur"]

odamlar[1] = "Akmal"    # ikkinchi elementni o'zgartirish
print(odamlar[1])      # Akmal
print(odamlar)          # ["Jasur", " Akmal", "Bobur"]
```

Ro'yxat elementlarini ajratish. Python sizga ro'yxatni alohida elementlarga ajratish imkonini beradi:

```
odamlar = ["Jasur", "Bobur", "Samat"]

jasur, bobur, samat = odamlar

print (jasur)          # Jasur
print (bobur)          # Bobur
print (samat)          # Samat
```

Bunday holda, **jasur**, **bobur** va **samat** o'zgaruvchilari odamlar ro'yxatidan ketma-ket tayinlangan elementlardir. Biroq, o'zgaruvchilar soni tayinlangan ro'yxat elementlari soniga teng bo'lishi talab qilinadi. Aks xolda xatolik kelib chiqadi.

Ro'yxat elementlarini olish uchun **for** yoki **while** tsiklidan foydalanish mumkin.

For tsikli bilan takrorlash:

```
odamlar = ["Tomson", "Sems", "Bob"]
for odam in odamlar:
    print (odam)
```

Bu yerda odamlar ro'yxati takrorlanadi va har bir element shaxs o'zgaruvchisiga joylashtiriladi.

Iteratsiyani while tsikli bilan ham bajarish mumkin:

```
odamlar = ["Tomson", "Sems", "Bob"]
i = 0
while i < len(odamlar):
    print(odamlar[i])
    # elementni olish uchun indeksdan foydalanish
    i += 1
```

len() funksiyasidan foydalanib takrorlash uchun ro'yxat uzunligini olinadi. Bu dasturda *i* - hisoblagich qiymati ro'yxat uzunligiga teng bo'lguncha chop etadi. *i* ning qiymati esa har safar birga oshirilib boraveradi.

Ro'yxatni taqqoslash. Ikki ro'yxat, agar ular bir xil elementlar to'plamini o'z ichiga olsa, teng hisoblanadi:

```
raqamlar1 = [1, 2, 3, 4, 5]
raqamlar2 = list ([1, 2, 3, 4, 5])

if raqamlar1 == raqamlar2:
    print("1 - raqamlar 2 - raqamlarga teng")
else:
    print("1-raqamlar 2-raqamlarga teng emas")
```

Bunday holda, ikkala ro'yxat ham teng bo'ladi.

Ro'yxatlar bilan ishlovchi metodlar va funksiyalar

Ro'yxat elementlarini boshqarish uchun bir qancha metodlar mavjud. Ulardan asosan ko'p marta murojaat qilinadiganlarini quyida keltirilgan:

append(element): *element* - ni ro'yxat oxiriga qo'shish;
insert (indeks, element): ro'yxatga *element*ni *indeks* – element qilib qo'shish;
extend (items): ro'yxat oxiriga *items* to'plamini qo'shish;
remove(item): ro'yxatdan *item* elementini olib tashlaydi. Ro'yxatdan birinchi uchragan *item* elementi o'chiriladi. Agar element topilmasa, ValueError xatoligi qaytariladi;
clear(): ro'yxatdagi barcha elementlarni o'chirish;

index(item): *item* elementining indeksini qaytaradi. Agar element topilmasa, ValueError xatoligi qaytariladi;

pop([indeks]): *indeks* indeksdagi elementni olib tashlaydi va qaytaradi. Agar indeks ko'rsatilmay faqat pop() holida foydalanilsa, oxirgi elementni olib tashlanadi.

count(item): ro'yxatdagi *item* elementining takrorlanishlar sonini qaytaradi. Agar mavjud bo'lmas, 0 qaytaradi;

sort([key]): ro'yxat elementlarini tartiblaydi. Standart bo'yicha o'sish tartibida saralanadi. Lekin *key* parametr yordamida saralashni qanday tartibda bo'lishini boshqarish mumkin.

reverse(): Ro'yxatdagi barcha elementlarni teskari tartibda qayta joylashtiradi.

copy(): ro'yxatni nusxalaydi.

Bundan tashqari, Python ro'yxatlar bilan ishlash uchun bir qator standart funktsiyalarni taqdim etadi:

len(list): ro'yxat uzunligini aniqlash;

sorted(list, [key]): tartiblangan ro'yxatni qaytaradi;

min(list): ro'yxatning eng kichik elementini topish;

max(ro'yxat): ro'yxatdagi eng katta elementni topish.

Ro'yxatga element qo'shish uchun append() , extension va insert metodlari, ro'yxatdan elementni olib tashlash uchun **remove()**, **pop()** va **clear()** usullari qo'llaniladi.

Foydalanish usullari:

```
list1= ["Timur", "Bobur"]

# ro'yxat oxiriga qo'shish
list1.append("Elmurod")    #["Timur", "Bobur", "Elmurod"]

# ikkinchi pozitsiyaga qo'shish
```



```

list1.insert(1, "Bilol") #["Timur", " Bilol", "Bobur",
"Elmurod"]

# elementlar to'plamini qo'shish ["Malik", "Samat"]
list1.extend(["Malik", "Samat"])
# ["Timur", " Bilol", "Bobur", "Elmurod", "Malik", "Samat"]

# element indeksini olish
index_of_tom = list1.index("Bobur") # 2

# ushbu indeksda o'chirish
removed_item = list1.pop(index_of_tom)
# ["Timur", " Bilol", "Elmurod", "Malik", "Samat"]

# oxirgi elementni olib tashlang
oxirgi_element = list1.pop() # "Samat"
print(list1) # ["Timur", " Bilol", "Elmurod", "Malik"]

# "Elmurod" elementini olib tashlash
list1.remove("Elmurod")
print(list1) # ["Timur", " Bilol", "Malik"]

# barcha elementlarni o'chirish
list1.clear()
print (list1) # []

```

IN kalit so'zining ishlatirishi

Agar biron bir element topilmasa, olib tashlash va indekslash usullari xatolik qaytaradi. Bunday vaziyatni oldini olish uchun elementni ishlatishdan oldin **in** kalit so'zidan foydalanib uning mavjudligini tekshirish lozim:

```

list_odamlar = ["Timur", "Bobur", "Samat"]
if "Ali" in list_odamlar:

```

```
list_odamlar.remove("Ali")

print(list_odamlar) # ["Timur", "Bobur", "Samat"]
```

Odamlar ro'yxatida "Ali" elementi bo'lsa, **if "Ali" in list_odamlar** ifodasi "True" qiymati qaytaradi. Shunday ekan bu vaziyatda False qiymati qaytadi va **list_odamlar.remove("Ali")** funksiyasi bajarilmaydi va **list_odamlar** nomli ro'yxat tarkibi o'zgarishsiz qoladi.

DEL metodi

Pythonda ro'yxatdan elementni o'chirish uchun yana bir **del** nomli operatoridan ham foydalaniladi. O'chiriladigan element yoki elementlar to'plami ushbu operatorga parametr sifatida uzatiladi:

```
list_odamlar = ["Timur", "Bilol", "Bobur",
"Elmurod", "Malik", "Samat"]

# ikkinchi elementni olib tashlash
del list_odamlar [1]
print (list_odamlar) # ["Timur", "Bobur", "Elmurod",
"Malik", "Samat"]

# to'rtinchi elementgacha o'chirish
list_odamlar [:3]
print(list_odamlar) #["Elmurod", "Malik", "Samat"]

del list_odamlar[1:]#ikkinchi elementdan oxirigacha
print(list_odamlar) #["Timur"]
```

COUNT() metodi

Agar element ro'yxatda necha marta borligini bilish kerak bo'lsa, **count()** metodidan foydalanish maqsadga muvofiq bo'ladi:

```
odamlar = ["Bobur", "Bilol", "Bobur", "Elmurod",
"Malik", "Bobur"]
```

```
odamlar_soni = odamlar.count("Bobur")  
print(odamlar_soni)          # 3
```

SORT() metodi

Sort() metodi standart holatda o'sish tartibida saralash uchun ishlatiladi:

```
odamlar = ["Timur", "Bilol", "Bobur", "Elmurod",  
"Malik", "Samat"]  
odamlar.sort()  
print(odamlar)  
# ['Bilol', 'Bobur', 'Elmurod', 'Malik', 'Samat', 'Timur']
```

Agar ma'lumotlarni teskari tartibda saralash zarur bo'lsa, **sort()** metodidan keyin **reverse()** usulini qo'llash mumkin:

```
odamlar = ["Timur", "Bilol", "Bobur", "Elmurod",  
"Malik", "Samat"]  
  
odamlar.sort()  
odamlar.reverse()  
print(odamlar)  
# ['Timur', 'Samat', 'Malik', 'Elmurod', 'Bobur', 'Bilol']
```

Saralash aslida ikkita ob'ektni taqqoslaydi va qaysi biri "kichik" bo'lsa, "katta" ob'ektdan oldin joylashtiriladi. "katta" va "kichik" tushunchalari albatta nisbiydir. Va agar ro'yxatning barcha elementlari raqamlardan tarkib topgan bo'lsa, unda barchasi oddiy - raqamlar o'sish tartibida joylashtiriladi. Agar ro'yxatda satrlar va boshqa ob'ektlar mavjud bo'lsa va saralash kerak bo'lsa, unda vaziyat yanada murakkabroq bo'ladi. Bunday holda harflar sonlardan oldin joylashtiriladi. Albatta sonli satrlar haqida gap ketmoqda. Chunki butun tipli ma'lumot bilan string tipli ma'lumot solishtirish xatolik keltirib chiqaradi. Demak, string tiplarni solishtirishda, agar ularning birinchi belgilar teng bo'lsa, ikkinchi belgilar solishtiriladi va hokazo. Bunda

raqamli belgi alifbodagi bosh harfdan "kichik" deb hisoblanadi va o'z navbatida bosh harf ham kichik harfdan kichik hisoblanadi.

Shunday qilib, agar ro'yxat tarkibida katta va kichik harflar mavjud bo'lsa va saralash amali bajarilsa, unda unchalik to'g'ri bo'lmagan natijalarni olinishi mumkin, chunki asosan "bob" matni "Tom" matnidan oldin kelishi kerak. Bunday holda sort metodining standart tartiblashini o'zgartirish uchun unga parametr sifatida funktsiyani beriladi:

```
odamlar = ["Timur", "bilol", "Bobur", "elmurod",
"Malik", "Samat"]

odamlar.sort()      # standart tartiblash
print(odamlar)      # ['Bobur', 'Malik', 'Samat', 'Timur',
'bilol', 'elmurod']

odamlar.sort(key=str.lower)
# kichik katta-kichik tartiblash

print ( odamlar )
# ['bilol', 'Bobur', 'elmurod', 'Malik', 'Samat', 'Timur']
```

SORTED metodi

Saralash usuliga qo'shimcha ravishda ikkita shaklga ega bo'lgan tartiblangan standart funksiyadan foydalanish mumkin:

sorted(list): list ro'yxatini tartiblaydi

sorted(list, key): list ro'yxati elementlariga **key** funksiyasini qo'llash orqali ro'yxatni tartiblaydi

```
list_per = ["Tom", "bob", "alis", "Sem", "Bill"]

sorted_per = sorted(list_per, key=str.lower)
print (sorted_per)
# ["alisa", "Bill", "bob", "Sem", "Tom"]
```

Ushbu funktsiyadan foydalanganda shuni yodda tutish kerakki, bu funksiya tartiblangan ro'yxatni o'zgartirmaydi, ammo u barcha tartiblangan elementlarni yangi ro'yxatga joylashtiradi va natijani qaytaradi.

MIN va MAX funksiyalari

Pythonning **min()** va **max()** funksiyalari mos ravishda minimum va maksimum qiymatlarni topishga imkon beradi:

```
raqamlar = [9, 221, 12, 10, 3, 125, 18]

print (min(raqamlar))      # 3
print (max(raqamlar))      # 221
```

COPY() funksiyasi

Ro'yxatlarni nusxalashda, ro'yxatlar o'zgaruvchan tur ekanligini yodda tutish kerak. Shuning uchun ikkala o'zgaruvchi ham bir xil ro'yxatga ishora qilsa, bitta o'zgaruvchini o'zgartirish boshqa o'zgaruvchiga ta'sir qiladi:

```
list_per = ["Timur", "Bobur", "Alisey"]
list_per2 = list_per
list_per2.append("Samat") # elementni ikkinchi ro'yxatga
qo'shish

# kishi1 va odamlar2 bir xil ro'yxatga ishora qiladi
print(list_per)      # ["Timur", "Bobur", "Alisey", " Samat"]
print(list_per2)      # ["Timur", "Bobur", "Alisey", "
Samat"]
```

Bu yuqorida keltirilgan dasturdagi bir o'zgaruvchining qiymatini ikkinchisiga yuklab qo'yish holati nusxalash deb atalmaydi. Nusxalash maqsadida bunday hatti harakatni qilmagan ma'qul. Elementlarni nusxalash uchun chuqurroq nusxalash talab qilinadi. Shundagina bir vaqtning nusxalangan va aslnusxadagi ro'yxatlar qiymatlari bir xil bo'lsada, aslida turli xil ro'yxatlarni ifodalaydi. Buning uchun oddiygina **copy()** metodidan foydalanib nusxalash kerak bo'ladi:

```
list_per = ["Tom", "Bob", "Alis"]
list_per2 = list_per.copy() # elementlarni list_per
dan list_per2 ga nusxalash

# elementni FAQAT ikkinchi ro'yxatga qo'shish
list_per2.append("Sem")

# list_per va list_per2 turli ro'yxatlarni bildiradi
print(list_per)      # ["Tom", "Bob", "Alis"]
print(list_per2)     # ["Tom", "Bob", "Alis", "Sem"]
```

Ro'yxatning bir qismini nusxalash

Agar butun ro'yxatni emas, balki uning ma'lum bir qismini nusxalash kerak bo'lsa, unda quyidagi shakllarni qabul qilishi mumkin bo'lgan maxsus sintaksisdan foydalanish kerak bo'ladi:

list[:end]: ro'yxat boshidan ***end*** elementigacha bo'lgan oraliqni tanlash.

list[start:end]: ro'yxatning ***start*** va ***end*** elementlari oralig'ini belgilaydi

list[start:end:step]: ro'yxatning ***start*** va ***end*** elementlari oralig'ini ***step*** qadam bilan belgilaydi. Odatiy bo'lib, bu ***step*** parametr 1 ga teng bo'ladi.

```
list_per = ["Tomson", "Bobi", "Elison", "Semmi",
"Timati", "Billi"]

slice_people1 = list_per [ :3 ] # 0 dan 3 gacha
print(slice_people1)      # ['Tomson', 'Bobi', 'Elison']

slice_people2 = list_per [ 1:3 ] #1 dan 3 gacha
print(slice_people2)      # ["Bobi", "Elison"]

slice_people3 = list_per[ 1:6:2 ] #1 dan 6 gacha 2 ga oshib
print(slice_people3)      # ["Bobi", "Semmi", "Billi"]
```

Manfiy indeks

Ro'yxatning oxirgi elementidan boshlab murojaatni boshlash kerak bo'lganda manfiy indeksdan foydalanish kerak bo'ladi. Masalan, -1 – element oxirgi element, -2 – element oxiridan bitta oldingi va hokazo.

```
list_per = ["Tomson", "Bobi", "Elison", "Semmi",
"Timati", "Billi"]

# nolinchidan oxirgisigacha, oxirgi xisobga kirmaydi
slice_people1 = list_per[:-1]
print(slice_people1)

# ['Tomson', 'Bobi', 'Elison', 'Semmi', 'Timati']

# oxirgi uchinchi elementdan oxirgisigacha, oxirgisini olmaydi
slice_people2 = list_per [-3:-1]
print(slice_people2)      # ["Semmi", "Timati"]
```

Ro'yxatlarni ulash

Qo'shish (+) operatori ro'yxatlarni birlashtirish uchun ishlatiladi:

```
people1 = ["Tomson", "Bobi", "Alison"]
people2 = ["Tomson", "Semmi", "Timati", "Billi"]
people3 = people1 + people2
print (people3)

# ["Tomson", "Bobi", "Elison", "Tomson", "Semmi", "Timati",
"Billi"]
```

Ichma-ich Ro'yxatlar

Ro'yxatlar satrlar, raqamlar kabi standart ma'lumotlardan tashqari, boshqa ro'yxatlarni ham o'z ichiga olishi mumkin. Bunday ro'yxatlarni jadvallar bilan bog'lash mumkin, bu yerda ichki ro'yxatlar qatorlar vazifasini bajaradi. Masalan:

```
odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
```

```

]

print(odamlar[0])          # ["Tomson", 29]
print(odamlar[0][0])       # "Tomson"
print(odamlar[0][1])       # 29

```

Ichki ro'yxatning elementiga murojaat qilish uchun bir juft indeksdan foydalanish kerak: `odamlar[0][1]` - birinchi joylashtirilgan ro'yxatning ikkinchi elementiga murojaat.

Umumiy ro'yxatni, shuningdek, ichki ro'yxatlarni qo'shish, o'chirish va o'zgartirish oddiy (bir o'lchovli) ro'yxatlar kabi amalga oshiriladi:

```

odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
]

# ichki ro'yxat yaratish
odam = list()
odam.append("Billi")
odam.append(41)

# ichki ro'yxat qo'shish
odamlar.append(odam)
print(odamlar[-1])    # ["Billi", 41]

# ichki ro'yxatga qo'shish
odamlar[-1].append("+79996543210")

print(odamlar[-1])    # ["Billi", 41, "+79996543210"]

# oxirgi elementni o'rnatilgan ro'yxatdan olib tashlash

```



```

odamlar[-1].pop()
print(odamlar[-1])           # ["Billi", 41]

# oxirgi kiritilgan ro'yxatni butunlay o'chirish
odamlar.pop(-1)

# birinchi elementni o'zgartirish
odamlar[0] = ["Semmi", 18]
print (odamlar)
# [ ["Semmi", 18], ["Alison", 33], ["Bobi", 27]]

```

Ichki ro'yxatlar ustida takrorlash:

```

odamlar = [
    ["Tomson", 29],
    ["Elison", 33],
    ["Bobi", 27]
]

for odam in odamlar:
    for item in odam:
        print(item, end=" | ")

```

Konsol chiqishi :

```
Tomson | 29 | Elison | 33 | Bobi | 27 |
```

Nazorat savollari:

1. Ro'yxat deb nimaga aytiladi?
2. Ro'yxatlarni qanday yaratiladi?
3. Ro'yxatlarning elementlari bir turga mansub bo'lishi shartmi?
4. ro'yxat elementlariga murojaatlar qanday amalga oshiriladi?
5. Ro'yxat uzunligini qanday aniqlanadi?
6. Ro'yxat elementini biro songa ko'paytirish nimani beradi?
7. Manfiy index tushunchasi nimani anglatadi?
8. tomson, bobi, semmi = odamlar – ushbu ifodaning ma'nosi nima?

9. Ro'yxatga ma'lumotni sikl orqali kiritish qanday amalga oshiriladi?
10. Ro'yxat elementlarini saralash uchun qaysi metodni ishlatiladi?
11. Ro'yxatga ma'lumot qo'shish uchun qaysi metodni ishlatiladi?
12. `remove()` , `pop()` va `clear()` matodlari qo'llanilishi haqida gapirib bering.
13. `if "Alison" in people` - ifodasining manosi tushuntirib bering.
14. `del` – metodi qanday ishlatiladi?
15. `count()` metodining vazifasi qanday?
16. `reverse()` metodini nima maqsadda foydalaniladi va uning parametrlari qanday?
17. Ro'yxatlarni nusxalash qanday amalga oshiriladi?

4.2.8. Pythonda kortej tushunchasi va u bilan ishlovchi metodlar bilan tanishish

Kortej xuddi ro'yxat kabi elementlar ketma-ketligini ifodalaydi. Bundan tashqari kortej o'zgarimas tur hisoblanadi. Shuning uchun kortejdagi elementlarni qo'shish, olib tashlash yoki uni o'zgartirishning ilojisi yo'q.

Tuple yaratish uchun turli qiymatlarni vergul bilan ajratib oddiy qavslar ichiga joylab qo'yish kifoya:

```
tommi= ("Tomson",23)
print (tommi)          # ("Tomson", 23)
```

Shuningdek, kortejni yaratish uchun qavslarsiz vergul bilan ajratilgan qiymatlarni ro'yxatga olish kerak bo'ladi:

```
tommi = "Tomson", 23
print (tommi)          # ("Tomson", 23)
```

Agar mabodo kortej bitta elementdan iborat bo'lib qolsa, u holda kortejning yagona elementidan keyin vergul qo'yish kerak:

```
tommi = ("Tomson",)
```

Ro'yxat kabi boshqa elementlar to'plamidan kortej yaratish uchun ro'yxatni **tuple()** funksiyasiga uzatiladi, bu esa ro'yxatni kortejni aylantiradi va kortej qaytaradi:

```
data = ["Tomson", 37, "Google"]
tommi = tuple (data)
print(tommi)          # ("Tomson", 37, "Google")
```

LEN() funksiyasi

Pythonning standart len() funksiyasidan foydalanib, kortej uzunligini aniqlash mumkin:

```
tommi = ("Tomson", 37, "Google")
print (len(tommi))      # 3
```

Kortej elementlariga murojaat

Kortejdagi elementlarga murojaat ro'yxatdagi kabi indeks bo'yicha amalga oshiriladi. Indeksflash, shuningdek, ro'yxat boshidan elementlarni olishda noldan va ro'yxat oxiridagi elementlarni olishda -1 dan boshlanadi:

```
tommi = ("Tomson", 37, "Google", "software
engineereeng")
print(tommi[0])        # "Tomson"
print(tommi[1])        # 37
print(tommi[-1])       # "software engineereeng"
```

Lekin kortej o'zgarmas tur bo'lgani uchun uning elementlarini o'zgartirishning ilojisi yo'q. Ya'ni, quyidagi yozuv ishlamaydi:

```
tommi[1] = "Tomsonjon"
```

Agar zarurat tug'lsa, kortejni alohida o'zgaruvchilarga ajratish mumkin:

```
name, age, company, position = ("Tomson", 35,
"Google", "software engineereeng")
print(name)            # Tomson
print(age)             # 35
print(position)        # software engineereeng
print(company)         # Google
```

Subtuplelarni olish

Ro'yxatlarda bo'lgani kabi, kortejning bir qismini boshqa kortej sifatida olish mumkin:

```
tommi = ("Tomson", 35, "Google", "software enginireeng")

# 1 dan 3 gacha bo'lgan elementni olish, 3-element xisoblanmaydi
print(tommi[1:3])      # (35, "Google")

# 0 dan 3 gacha bo'lgan elementni olish, 3-element xisoblanmaydi
print(tommi[:3])       # ("Tomson", 35, "Google")

# 1 dan oxirgi elementgacha subtuple olish
print (tommi[1:])
# (35, "Google", "software enginireeng")
```

Kortejni funksiyaga parametr sifati berish va natijasi sifatida qabul qilish

Funktsiyadan bir vaqtning o'zida bir nechta qiymatlarni qaytarish kerak bo'lganda, kortejlardan foydalanish qulaydir. Funktsiya bir nechta qiymatlarni qaytarsa, u aslida kortejni qaytaradi:

```
def get_user():
    name="Tomson"
    yosh = 25
    kompaniya = "Google"
    return name, yosh, kompaniya

foydalanuvchi = get_user()
print(foydalanuvchi)      # ("Tomson", 25, "Google")
```

* operatori yordamida funksiyaga kortejni uzatishda uni funktsiya parametrlariga o'tkaziladigan individual qiymatlarga ajratish mumkin:

```
def print_person(ism, yosh, kompaniya):
    print(f"Ism:{ism}Yosh:{yosh} Kompaniya: {kompaniya}")

tommi = ("Tomson", 25)
print_person(*tommi, "Microsoft")
# Ism: Tomson   Yosh: 25   Kompaniya: Microsoft

bob = ("Bobi", 41, "Apple")
print_person(*bob)
# Ism: Bobi    Yoshi: 41   Kompaniya: Apple
```

Kortejda sikldan foydalanish

To'plam elementlariga murojaat qilishda ularning har bir elementiga alohida murojaat qilish, o'ta noto'g'ri bo'ladi. Buning uchun standart for va while sikllaridan foydalaniladi. **for** tsikli orqali kortej elementlarini olish quyidagicha amalga oshiriladi:

```
tom = ("Tomson", 22, "Google")

for element in tom:
    print (element)
```

While siklidan foydalanib kortej elementlariga murojaat qilish:

```
tom = ("Tomson", 22, "Google")

i = 0
while i < len(tom):
    print (tom[i])
    i += 1
```

Kortej tarkibida element mavjudligini tekshirish

Ba'zi masalalarda elementga murojaat qilish kerak bo'ladi va murojnat natijasi xatolik bilan tugashi mumkin. Bunday hollarda elementni mavjudligini birinchi tekshirib olish kerak bo'ladi va keyin biror amal bajarilsa, maqsadga muvofiq bo'ladi. Ushbu murojaat amalga oshiriladigan elementni mavjudligini tekshirish uchun **in** operatoridan foydalaniladi va u quyidagicha foydalaniladi:

```
foydalanuvchi = ("Tomson", 22, "Google")
nomi="Tomson"
if nomi in foydalanuvchi:
    print("Foydalanuvchi nomi Tomson")
else:
    print("Foydalanuvchining nomi boshqacha ekan!")
```

Nazorat savollari:

1. Kortej deb nimaga aytiladi?
2. Kortejlarning ro'yxatlardan farqi nimada?
3. Kortejlarning yaratilishi usullari qanday?
4. Kortejdagi ma'lumotlarga qanday murojaat qilinadi?
5. Kortejdagi ma'lumotlarni qanday qilib o'zgartirish mumkin?
6. Parametr va funksiya natijasi sifatida kortejni qanday qo'llash mumkin?
7. Kortej elementlarini tsikl orqali chiqarish qanday amalga oshiriladi?
8. Kortejni parametr sifatida uzatishda "*" (yulduzcha) belgisining ro'li qanday?

4.3.§. Pythonda lug'at tushunchasi va u bilan ishlovchi

metodlar bilan tanishish

Pythondagi lug'at (dictionary) elementlar to'plamini saqlaydi. Lug'at tarkibidagi har bir element o'ziga xos, yagona kalitga va u bilan bog'liq bo'lgan qiymatga ega bo'ladi.

Lug'at ta'rifi quyidagi sintaksisga ega:

```
dictionary={kalit1: qiymat1, kalit2: qiymat2, ....}
```

Lug'atning barcha elementlari jingalak qavslar ichiga joylanadi. Har bir element bir-biri bilan vergul orqali ajratildi. Har bir elementning yagona kaliti va unga bog'langan qiymat mavjud bo'ladi. Elementning kaliti birinchi keladi va undan keyin qiymat keladi. Kalit va qiymatini ikki nuqta ajratib turadi.

Lug'atni aniqlash:

```
users = {1: "Tomson", 2: "Bobi", 3: "Billi"}
```

`users` lug‘ati raqamlarni kalit sifatida va satrlarni qiymat sifatida ishlatadi. Ya’ni, 1-kalitga ega bo‘lgan element "Tomson" qiymatiga teng, 2-kalitga ega element "Bobi" qiymatiga ega va hokazo.

Lug‘atdagi elementlar tarkibi bir xil tipli ma’lumotlar bo‘lishi ham mumkin.

Masalan:

```
e_emails = {"tomson@gmail.com": "Tomson",  
"bobi@gmail.com": "Bobi", "semmi@gmail.com": "Semmi"}
```

`e_emails` lug‘atida kalit sifatida satrlardan foydalanildi - foydalanuvchi elektron pochta manzillari va qiymat sifatida ham satrlardan - foydalanuvchi nomlaridan foydalanildi.

Lekin kalitlar va satrlar bir xil turdagi bo‘lishi shart emas. Ular turli xil turlarni ko‘rsatishi mumkin:

```
objects = {1: "Tomson", "2": False, 3: 150.64}
```

Bundan tashqari, umuman elementsiz, bo‘sh lug‘atni aniqlash ham mumkin:

```
dict_object = {}
```

yoki shunga o‘xshash:

```
dict_object = dict()
```

Ro‘yxatlar va kortejlarni lug‘atga aylantirish

Lug‘at va ro‘yxat tuzilmaviy jihatdan bir-biriga o‘xshamaydigan turlar bo‘lsa-da, standart **dict()** funksiyasi yordamida ma’lum turdagi ro‘yxatlarni lug‘atga aylantirish mumkin. Buning uchun ro‘yxat ichki ro‘yxatlar to‘plamini saqlashi kerak. Har bir ichki ro‘yxat ikkita elementdan iborat bo‘lishi kerak - lug‘atga aylantirilganda birinchi element kalitga, ikkinchisi esa qiymatga aylanadi:

```
user_list=[  
    ["+554513455", "Tomson"],  
    ["+002549557", "Bobi"],  
    ["+445454512", "Elison"]  
]  
user_dict = dict(user_list)
```

```
print(user_dict) # {"+554513455", "Tomson:", "+002549557",  
#"Bobi", "+445454512": "Elison"}
```

Xuddi shunday, ikkita kortej lugʻatga aylantirilishi mumkin, oʻz navbatida lugʻat ikkita kortejni oʻz ichiga oladi:

```
user_tuple = (  
    ("+554513455", "Tomson"),  
    ("+002549557", "Bobi"),  
    ("+445454512", "Elison")  
)  
  
user_dict = dict (user_tuple)  
print (user_dict)
```

Lugʻat elementlariga murojaat qilish va oʻzgartirish

Lugʻat elementlariga murojaat qilish uchun uning nomidan keyin element kaliti kvadrat qavs ichida koʻrsatiladi:

```
Dictionary_name[key]
```

Lugʻatdagi elementlarni olish va oʻzgartirish uchun:

```
dict_users = {  
    "+11002255": "Tomson",  
    "+15245999": "Bobi",  
    "+15415154": "Elison"  
}  
  
# "+11002255" kaliti bilan elementni olish  
print (dict_users ["+11002255"]) # Tomson  
  
# element qiymatini "+15245999" tugmasi bilan belgilash  
dict_users["+15245999"] = "Bob Smit"  
print (dict_users["+15245999"]) # "Bob Smit"
```

Agar bunday kalit bilan elementning qiymatini belgilashda lugʻatda hech qanday element boʻlmasa, u qoʻshiladi:


```
dict_users["+5555555"] = "Semmi"
```

Ammo agar lugʻatda mavjud boʻlmagan kalit bilan qiymat olishga harakat qilinsa, Python **KeyError** xatoligini chiqaradi:

```
foydalanuvchi=foydalanuvchilar["+4444444"] # KeyError
```

Bunday holatni oldini olish maqsadida elementga murojaat qilishdan oldin **"key in dict"** ifodasi yordamida lugʻatda element mavjudligini tekshirish mumkin. Agar kalit lugʻatda boʻlsa, u holda bu ifoda **"True"** ni qaytaradi:

```
key = "+00000044"
if key in dict_users:
    dict_user = dict_users[key]
    print(dict_user)
else:
    print ("Element topilmadi")
```

Shuningdek lugʻat elementlariga murojaat qilish uchun **get** metodidan ham foydalaniladi. **get()** metodidan ikki usulda foydalanish mumkin:

get(key): lugʻatdan kalit tugmasi boʻlgan elementni qaytaradi. Agar ushbu kalitga ega boʻlgan element mavjud boʻlmasa, u holda **"False"** qiymati qaytariladi;

get(key, default): lugʻatdan key kaliti boʻlgan elementni qaytaradi. Agar bunday kalitga ega element boʻlmasa, u **default** qiymatni qaytaradi.

```
foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

user1 = users.get("+15415154")
print (user1)      # Alison
user2=users.get("+15245999", "Noma'lum foydalanuvchi")
print (user2 )     #Bobi
```

```
user3=users.get("+0000000", "Noma'lum foydalanuvchi")
print(user3)      # Noma'lum foydalanuvchi
```

DEL funksiyasi

Del funksiyasi elementni kalit bilan olib tashlash uchun ishlatiladi :

```
foydalanuvchilar = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

del foydalanuvchilar["+15415154"]
print (foydalanuvchilar)
# { "+11002255": "Tomson", "+15245999": "Bobi"}
```

Ammo shuni yodda tutish kerakki, agar bunday kalit lug'atda bo'lmasa, `KeyError` xatoligi qaytariladi. Shuning uchun, o'chirilishi kerak bo'lgan elementni oldin, berilgan kalitli element mavjudligini tekshirish tavsiya etiladi.

```
dict_user = {
    "+11002255": "Tomson",
    "+15245999": "Bobi",
    "+15415154": "Elison"
}

key = "+99990000"
if key in dict_user:
    del dict_user[key]
    print(f"{key} kaliti element olib tashlandi")
else:
    print (f"{key} kalitli element topilmadi")
```

POP() metodi

O‘chirishning yana bir usuli - **pop()** usuli. Pop metodidan ikki xil ko‘rinishda foydalanish mumkin:

- 1) **pop(key):** *key* kalit berilgan elementni olib tashlaydi va olib tashlangan elementni qaytaradi. Agar berilgan kalit bilan element topilmasa, **KeyError** xatoligi qaytariladi.
- 2) **pop(key, default):** *key* kalitli elementni o‘chiradi va olib tashlangan elementni qaytaradi. Agar berilgan kalit bilan element bo‘lmasa, **default** qiymati qaytariladi.

```
foydalanuvchilar = {  
    "+11002255": "Tomson",  
    "+15245999": "Bobi",  
    "+15415154": "Elison"  
}  
  
key = "+11002255"  
user1 = users.pop (key)  
print (user1)          # Tomson  
  
user1 = users.pop("+00000000", "Noma'lum  
foydalanuvchi")  
print (user1)          # Noma'lum foydalanuvchi
```

Agar barcha elementlarni olib tashlash kerak bo‘lsa, ya’ni bir so‘z bilan aytganda tozalash kerak bo‘lsa, unda **clear()** metodidan foydalanish kerak bo‘ladi:

```
per.clear()
```

Lug‘atlarni nusxalash va birlashtirish

copy() usuli lug‘at tarkibini nusxalab, yangi lug‘atni qaytaradi:

```
foydalanuvchilar={"998888":"Tomson","998771": "Bobi"}  
talabalar = users.copy()  
print (talabalar) # {"998888": "Tomson", "998771": "Bobi"}
```

update() metodi ikkita lugʻatni birlashtiradi:

```
foydalanuvchilar={"998888":"Tomson","998771": "Bobi"}

users2 = {"222222": "Semmi", "666000": "Kail"}
users.update(users2)

print(users)      # {"998888": "Tomson", "998771": "Bobi",
"222222": "Semmi", "666000": "Kail"}

print(users2)     # {"222222": "Semmi", "666000": "Kail"}
```

Bunday holda, users2 lugʻati oʻzgarishsiz qoladi. Faqat foydalanuvchilarning lugʻati oʻzgartiriladi, unga boshqa lugʻat elementlari qoʻshiladi. Ammo agar ikkala manba lugʻati ham oʻzgarishsiz qolishi kerak boʻlsa va birlashmaning natijasi uchinchi lugʻat boʻlsa, avval bitta lugʻatni boshqasiga nusxalanishi kerak:

```
users3 = users.copy()
users3.update(users2)
```

Lugʻatni takrorlash

Lugʻat tarkibini sikldan foydalanib olish uchun for siklidan foydalanish mumkin:

```
users = {"998888": "Tomson", "998771": "Bobi"}
for a in users:
    print(f"id: {a}    Foydalanuvchi: {users[a]} ")
```

Takrorlash orqali elementlarga murojaat qilishda joriy elementning kalitini elementning oʻzini olish uchun ishlatiladi.

Elementlarni olishning yana bir usuli - **items()** metodidan foydalanish:

```
users = {"998888": "Tomson", "998771": "Bobi"}
for key, val in users.items():
    print(f"id: {key}    Foydalanuvchi: {val} ")
```

items() usuli kortejlar to‘plamini qaytaradi. Har bir kortej elementning kaliti va qiymatini o‘z ichiga oladi, ularga murojaat qilishda **key** va **val** o‘zgaruvchilarini chaqirilsa yetarli.

Shuningdek, kalitlar ustida takrorlash va qiymatlarni takrorlash uchun alohida usul ham mavjud. Lug‘atdagi kalitlarni olish uchun **keys()** metodidan foydalaniladi:

```
for key in dict_users.keys():  
    print(key)
```

Faqat qiymatlarni takrorlash uchun lug‘atning *values()* metodidan foydalaniladi:

```
for val in dict_users.values():  
    print(val)
```

Murakkab lug‘atlar

Raqamlar va satrlar kabi eng oddiy ob’ektlardan tashqari, lug‘atlar murakkabroq ob’ektlarni ham saqlashi mumkin. Masalan bir xil ro‘yxatlar, kortejlar yoki boshqa lug‘atlarni o‘z ichiga element sifatida joylashtirishi mumkin:

```
dict_users = {  
    Tomson: {  
        "id": "745",  
        "e_main": "tomson12@gmail.com"  
    },  
    Bobi: {  
        "id": "444",  
        "e_mail": "bobi@gmail.com",  
        "skype": "bob123"  
    }  
}
```

Bunda lug‘atning har bir elementining qiymati o‘z navbatida alohida lug‘atni ifodalashi mumkin.

Ichki lug‘at elementlariga kirish uchun mos ravishda ikkita kalitdan foydalanish kerak:

```
old_e_mail = dict_users["Tomson"]["e_mail"]
dict_users["Tomson"]["e_mail"] =
"supertomson@gmail.com"

print (dict_users["Tomson"])

# { "id": "745", "e_main": "supertomson@gmail.com" }
```

Ammo agar lugʻatda boʻlmagan kalit orqali qiymat olishga harakat qilinsa, Python **KeyError** xatoligini chiqaradi:

```
tom_skype = dict_users["Tom"]["skype"]      # KeyError
```

Xatolikka yoʻl qoʻymaslik uchun lugʻatda kalit mavjudligini tekshirish kerak:

```
key = "skype"
if key in dict_users["Tomson"]:
    print(dict_users["Tomson"]["skype"])
else:
    print("skype lugʻatdan topilmadi!")
```

Boshqa barcha jihatlarida murakkab va ichki lugʻatlar bilan ishlash oddiy lugʻatlar bilan ishlash kabi bajariladi.

Nazorat savollari:

1. Pythonda tugʻat tushunchasiga tarif bering.
2. Pythonda lugʻatlar qanday usullar bilan yaratilishi mumkin?
3. Roʻyxatlar va kortejlarni lugʻatga aylantirish qanday amalga oshiriladi?
4. Lugʻat Elementlariga murojaat qilish va oʻzgartirish qanday amalga oshiriladi?
5. **get()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
6. **del** metodining vazifasi va ishlash prinsipi haqida gapirib bering.
7. **pop()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
8. **clear()** metodining vazifasi va ishlash prinsipi haqida gapirib bering.
9. **copy()** metodining vazifasi qanday va uning qabul qiluvchi parametrlari qanday?
10. **upgrate()** metodining vazifasi va ishlash prinsipi haqida gapirib bering
11. **items()** metodining vazifasi va ishlash prinsipi haqida gapirib bering

12.**keys ()** metodining vazifasi va ishlash prinsipi haqida gapirib bering

13.Murakkab lug‘atlar qanday hosil qilinadi?

4.4.§. To‘plamlar bilan ishlash

To‘plam (set) shunday to‘plamki, uning har bir elementi to‘plamda yagona, takrorlanmas hisoblanadi. Bunday turdagi to‘plamni aniqlash uchun jingalak qavslardan foydalaniladi:

```
set_list= {"Tomson", "Bobbi", "Alisey", "Tomson"}  
print(set_list)    # {"Tomson", "Bobbi", "Alisey"}
```

E’tibor berish kerakki, print funksiyasi yordamida to‘plamni ekranga chiqarilganda "Tomson" elementini bir marta chiqarildi. Ammo e’lon qilinganda "Tomson" ham, to‘plam ta’rifi ushbu elementni ikki marta o‘z ichiga oladi.

Buning sababi shundaki, to‘plam faqat yagona qiymatlarni o‘z ichiga oladi.

Bundan tashqari, to‘plamni aniqlash uchun elementlar ro‘yxati yoki to‘plami uzatiladigan **set()** funksiyasidan foydalanish mumkin:

```
set_persons = ["Maya", "Billi", "Teddi"]  
set_users = set(set_persons)  
print(set_users)      # {"Maya", "Billi", "Teddi"}
```

set() funksiyasi bo‘sh to‘plam yaratish uchun quyidagicha foydalaniladi:

```
set_users= set()
```

Boshqa turdagi toplamlar singari **set()** to‘plamida ham uzunligini aniqlovchi **len()** funksiyasi mavjud:

```
foydalanuvchilar = {"Tomson", "Bobi", "Alison"}  
print (len(foydalanuvchilar))      # 3
```

add() metodi

To‘plamga bitta element qo‘shish uchun **add()** metodidan foydalaniladi:

```
set_users = set()  
set_users.add("Sammi")  
print (set_users) # "Sammi"
```

remove() metodi

Element olib tashlash. Bitta elementni olib tashlash **remove()** metodi yordamida amalga oshiriladi va olib tashlanadigan element qaytariladi. Ammo shuni yodda tutish kerakki, agar bunday element to‘plamda bo‘lmasa, unda xatolik yuzaga keladi. Shuning uchun, o‘chirishdan oldin, **in** operatori yordamida element mavjudligini tekshirish kerak:

```
set_users = {"Tomson", "Bobi", "Alison"}
set_user = "Tomson"
if set_user in set_users:
    set_users.remove(set_user)
print(set_users)          # {"Bobi", "Alison"}
```

discard () metodi

Elementni olib tashlash uchun discard () usulini ham qo‘llash mumkin. Ammo element mavjud bo‘lmasa, xatolik qaytariladi:

```
set_users = {"Tomson", "Bobi", "Alison"}

set_users.discard("Tim") # element "Tim" mavjud emas va metod
hech narsa qilmaydi
print(set_users)        # {"Tomson", "Bobi", "Alison"}

set_users.discard("Tomson") # "Tomson" elementi mavjud va
usul elementni olib tashlaydi

print(set_users)        # {"Bobi", "Alison"}
```

clear() metodi

Barcha elementlarni olib tashlash uchun **clear()** usuli chaqiriladi :

```
set_users.clear()
```

To‘plam elementlariga sikl yordamida murojaat qilish

Elementlarga murojaat qilish uchun for siklidan foydalanish mumkin :


```
set_users = {"Tomson", "Bobi", "Alison"}  
for user in set_users:  
    print (user)
```

Takrorlash paytida har bir element **user** oʻzgaruvchisiga uzatiladi va u oʻz navbatida ushbu elementlarni konsolga chiqaradi.

copy () metodi

Toʻplamlarni nusxalash. **Copy ()** usulidan foydalanib, bir toʻplam elementlarini boshqa oʻzgaruvchiga nusxalash mumkin:

```
set_users = {"Tomson", "Bobi", "Alison"}  
talabalar = set_users.copy()  
print (talabalar)          # {"Tomson", "Bobi", "Alison"}
```

union() metodi

Toʻplamlarni birlashtirish. **union()** usuli ikkita toʻplamni birlashtiradi va yangi toʻplamni qaytaradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}  
users2 = {"Semmi", "Keyt", "Bobbi"}  
  
users3 = set_users.union (users2)  
print (users3)  
# {"Tomson", "Bobi", "Alison", "Semmi", "Keyt"}
```

intersection() metodi

Toʻplamlarning kesishishi. Toʻplamlarning kesishishi faqat ikkala toʻplamda bir vaqtning oʻzida boʻlgan elementlarni olish imkonini beradi. **Intersection()** metodi toʻplamlar kesishish funksiyasini bajaradi va kesishmadan hosil boʻlgan yangi toʻplamni qaytaradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}  
users2 = {"Sammi", "Keyt", "Bobbi"}  
  
users3 = set_users.intersection(users2)
```

```
print (users3)      # {"Bobbi"}
```

and operatori

Kesishish usuli o‘rniga mantiqiy ko‘paytirish operatsiyasi (**and**) dan foydalanish ham mumkin:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

print (set_users and users2)      # {"Bobbi"}
```

Bu ikkala holda ham, bir xil natijaga erishiladi.

intersection_update() metodi

intersection_update() metodi birinchi to‘plamni kesishish elementlari bilan almashtiradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

set_users.intersection_update(users2)
print(set_users)      # {"Bobbi"}
```

DIFFERENCE() metodi.

Ushbu metod to‘plamlar orasidagi farqlanuvchi elementlarni qaytaradi. Ya’ni birinchi to‘plamda bo‘lgan, ammo ikkinchisida yo‘q bo‘lgan elementlarni qaytaradi. To‘plamlar farqini olish uchun **difference** usuli yoki ayirish operatsiyasidan foydalaniladi mumkin:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

users3 = set_users.difference(users2)
print(users3)      # {"Tomson", "Alison"}
print (set_users - users2)      # {"Tomson", "Alison"}
```

symmetric_difference() metodi

To'plam farqining alohida turi - simmetrik farq **symmetric_difference()** usuli yoki $\wedge$ operatsiyasi yordamida amalga oshiriladi. U ikkala to'plamning umumiy elementlardan tashqari barcha elementlarini qaytaradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2 = {"Sammi", "Keyt", "Bobbi"}

users3 = set_users.symmetric_difference(users2)
print(users3)          # {"Tomson", "Elison", "Sammi", "Keyt"}

users4 = set_users ^ users2
print(users4)          # {"Tomson", "Alison", "Sammi", "Keyt"}
```

issubset metodi

issubset metodi joriy to'plam boshqa to'plamining kichik to'plami (ya'ni qismi) ekanligini aniqlash imkonini beradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2={"Sammi", "Keyt", "Bobbi", "Alison", "Tomson"}

print (set_users.issubset(users2))      # True
print (users2.issubset(set_users))      # False
```

issuperset metodi

Agar joriy to'plam boshqa to'plamning super to'plami (ya'ni o'z ichiga olgan) bo'lsa, **issuperset** usuli **True** qiymatini qaytaradi:

```
set_users = {"Tomson", "Bobbi", "Alison"}
users2={"Sammi", "Keyt", "Bobbi", "Alison", "Tomson"}

print(set_users.issuperset(users2))      # False
print (users2.issuperset(set_users))      # True
```

frozenset matodi

Muzlatilgan to‘plam (*frozen set*) turidagi to‘plamlar o‘zgartirib bo‘lmaydigan to‘plamlar turiga kiradi. Uni yaratish uchun **frozenset** funksiyasidan foydalaniladi:

```
set_user= frozenset ({ "Tommi", "Bobbi", "Elison" })
```

Frozenset funksiyasiga elementlar to‘plami - ro‘yxat, kortej boshqa to‘plamlar uzatiladi.

Bunday to‘plamga yangi elementlarni qo‘shish, undagi mavjud elementlarini o‘chirib tashlash kabi amallarni bajarishning ilojisi yo‘q. Aynana shuning uchun **frozenset** to‘plami operatsiyalarning cheklangan to‘plamini qo‘llab-quvvatlaydi:

len(s): to‘plam uzunligini aniqlash;

x in s : agar *s* to‘plamida *x* elementi mavjud bo‘lsa, True qiymatini qaytaradi

x not in s: agar *x* *s* to‘plamining tarkibida bo‘lmasa, True qaytaradi

s.issubset(t): agar *t* *s* to‘plamini o‘z ichiga olgan bo‘lsa, True qiymatini qaytaradi

s.issuperset(t): agar *t* *s* ichida bo‘lsa, True qiymatini qaytaradi

s.union(t): *s* va *t* to‘plamlarning birlashuvini qaytaradi

s.intersection(t): *s* va *t* ning kesishishini qaytaradi

s.difference(t): *s* va *t* to‘plamlari orasidagi farqni qaytaradi

s.copy(): *s* to‘plamining nusxasini qaytaradi

Nazorat savollari:

1. To‘plamlarni yaratishning qanday usullarini bilasiz?
2. set() metodining vazifasi qanday?
3. To‘plamlarni lug‘atlardan farqi nimada?
4. To‘plamga element qo‘shish qanday amalga oshiriladi?
5. To‘plamga element olib tashlash qanday amalga oshiriladi?
6. **discard()** metodining vazifasi qanday?
7. **clear()** metodining vazifasi qanday?

8. **copy()** metodi nima maqsadda ishlatiladi va uning afzallik hamda kamchiliklari qanday?
9. **union()** metodi haqida gapirib bering.
10. **intersection()** qanday metod?
11. **intersection_update()** metodining vazifasi va **intersection()** metodidan farqi haqida gapirib bering.
12. **symmetric_difference()** metodi haqida gapirib bering.
13. **issubset()** – metodining ishlash prinsipi qanday?
14. **issuperset()** – metodidan qanday foydalaniladi?
15. **frozenset()** funksiyasining vazifasi nima?

V BOB. PYTHONDA FAYLLAR

5.1.§. Pythonda fayllar bilan ishlashning oddiy usuli

Python dasturlash tilidagi fayllar bilan o‘zaro aloqada bo‘lish foydalanuvchiga ilova tomonidan qayta ishlangan ma’lumotlarni saqlash imkoniyatini beradi va foydalanuvchi keyinchalik istalgan qulay vaqtda faylga murojaat qilishi mumkin bo‘ladi. Ushbu python dasturlash tilining asosiy funktsiyalari fayllarni yaratish, faylga ma’lumotlarni yozish va fayldan ma’lumotlarni o‘qishni ancha osonlashtiradi.

Faylni yaratish va ochish

Fayl bilan ishlash uchun albatta avval uni yaratish kerak. Buni standart operatsion tizim vositalari yordamida kerakli katalogga o‘tish va ***.txt** formati bilan yangi hujjat yaratish orqali amalga oshirish ham mumkin. Biroq, shunga o‘xshash harakat Python dasturlash tilida **open()** metodi yordamida amalga oshiriladi. Bu metodga **fayl nomi** va uni **qayta ishlash rejimi** parametrlar sifatida berilishi kerak.

Quyidagi kod fayl o‘zgaruvchisini yangi hujjatga ishora qilishini ko‘rsatadi. Agar ushbu dasturni ishga tushirilsa, u manba kodi saqlanadigan papkada **test.txt** matn faylini yaratadi.

```
file_txt = open("test.txt", "w")  
file_txt.close()
```

Agar **test.txt** nomli fayl kod katalogida allaqachon mavjud bo‘lsa, dastur yangi hujjat yaratmasdan u bilan ishlashni davom ettiradi. Ko‘rib turganingizdek, fayl nomi **open()** metodining birinchi parametridir. Undan so‘ng ma’lumotlarni **qayta ishlash usulini** ko‘rsatadigan maxsus belgi parametr sifatida keladi. Bu yerda ikkinchi parametr sifatida kelgan “w” - fayl faqat yozish uchun ochilayotganini bildiradi.

Endi ma’lumot yo‘qolmasligini ta’minlash uchun fayl ustida turli amallarni bajarilgandan so‘ng, uni **close()** funktsiyasidan foydalanib yopish kerak bo‘ladi.

Oldingi misolda faylga kirish uchun nisbiy yo‘l ishlatilgan bo‘lib, u qattiq diskdagi ob‘yektning joylashuvi haqida to‘liq ma’lumotni o‘z ichiga olmaydi. Ularni o‘rnatish uchun **open()** funksiyaga birinchi argument sifatida mutlaq yo‘lni

ko'rsatilishi kerak. Bunday holda, test.txt hujjati dastur papkasida emas, balki D diskidagi qaysidir bir katalogga joylashgan bo'ladi.

```
file_txt = open(r"D:\test\test.txt", "w")  
file_txt.close()
```

Bu yerda faylga yo'lni ko'rsatishda satrdan oldin **r** belgisi qo'yiladi. Aks holda, kompilyator "\" ketma-ketligini maxsus operator sifatida ko'rib chiqadi va xatolik kelib chiqadi.

Faylni ochilish rejimlari

Barcha dasturlash tillaridagi kabi Python dasturlash tilida ham faylni ochishda ishlatiladigan maxsus belgilardan foydalaniladi. Ular faylni ochish rejimi deb ataladi va faylni qanday maqsadda ochishni aniq beradi. Ularning barchasi quyidagi jadvalda keltirilgan, unda ularning belgisi va qisqacha mazmuni keltirilgan:

5-jadval

Belgi	Mazmuni
«r»	O'qish uchun ochiq (standart)
«w»	Yozish uchun ochiladi. Agar fayl ko'rsatilgan katalogda mavjud bo'lmasa, yangisi yaratiladi. Yozilgan ma'lumot hujjatdagi barcha ma'lumotlarni o'rniga yoziladi
«a»	Yozish uchun ochiladi. Yozilgan ma'lumot hujjatning oxiriga qo'shiladi
«b»	Faylni ikkilik rejimda ochish
«t»	Faylni matn rejimida ochish (standart)
«+»	O'qish va yozish uchun bir vaqtning o'zida ochish. Ushbu amal alohida o'zi ochish rejimi sifatida ishlatilmaydi. a+, r+, w+ kabi ishlatiladi.

Open() metodining ikkinchi argumentidan foydalanib, turli xil fayl rejimlarini birlashtirish mumkin. Masalan, yozma ma'lumotlarni ikkilik rejimda o'qish uchun "rb" ni belgilash kerak.

Yana bir misol: "r+" va "w+" o'rtasidagi farq shundaki, "w+" - agar fayl mavjud bo'lmasa, yangi fayl yaratiladi. Birinchi holda, xatolik kelib chiqadi. "r+" va "w+" dan foydalanish faylni o'qish va yozish uchun ochadi.

Usullari

open() funksiyasi tomonidan qaytarilgan ob'yekt mavjud faylga tavsifni o'z ichiga olgan to'rtta parametrni qaytaradi. Quyidagi jadvalda ularning barchasi nomlari va ma'nolari keltirilgan:

6-jadval

Nomi	Ma'nosi
name	fayl nomini qaytaradi
mode	ochilgan rejimni qaytaradi
closed	fayl yopiq bo'lsa true ni, ochiq bo'lsa false qiymatini qaytaradi
softspace	agar faylning chiqishi alohida bo'sh joy belgisini o'z ichiga olmasa, true qiymatini qaytaradi

Faylning xususiyatlarini ko'rsatish uchun kirish operatoridan, ya'ni nuqtadan foydalanish va keyin buni **print()** funktsiyasiga parametr sifatida berish kifoya bo'ladi.

Masalan :

```
f = open(r"D:\test\test.txt", "w")
print(f.name)
f.close()
```

Natijasi:

```
D:\test\test.txt
```

Faylga ma'lumot yozish

Faylga ma'lumot yozish **write()** usuli yordamida amalga oshiriladi. Usul mavjud faylga ishora qiluvchi ob'yektda chaqiriladi. Shuni esda tutish kerakki, buni amalga oshirish uchun avval hujjatni ochish funktsiyasidan foydalanib ochish va "w" belgisi bilan yozish rejimini belgilash kerak bo'ladi. **Write()** usuli argument sifatida matn

fayliga yoziladigan ma'lumotlarni qabul qiladi. Quyidagi kod misolida "Salom dunyo!" satrini kiritilishi ko'rsatilgan:

```
file_txt = open("test.txt", "w")
file_txt.write("Salom dunyo!")
file_txt.close()
```

Agar ilgari yozilgan ma'lumotlarga yangi ma'lumotlarni qo'shish kerak bo'lsa, u holda ochish rejimi sifatida "a" belgisini ko'rsatib, **open** funksiyasini qayta chaqirish kerak. Aks holda, **test.txt** faylidagi barcha ma'lumotlar butunlay o'chirib tashlanadi. Quyidagi kod misolida faylga ma'lumot qo'shish uchun matnli hujjatni ochadi. Shundan so'ng unda " Salom dunyo!" satri hujjat oxiriga joylashtiriladi. Shunday qilib, **test.txt** faylida "Salom dunyo! Salom dunyo!" satri paydo bo'ladi. Bularning barchasidan so'ng, faylni majburiy yopish haqida unutmang.

```
file_txt = open("test.txt", "a")
file_txt.write(" Salom dunyo!")
file_txt.close()
```

Matn fayliga ma'lumotlarni yozishning eng oddiy protsedurasi shunday amalga oshiriladi. Shuni ta'kidlash kerakki, Python dasturlash tilida hujjatlar bilan yanada ilg'or ishlash uchun ko'plab qo'shimcha vositalar mavjud bo'lib, ular yaxshilangan yozishni ham o'z ichiga oladi.

Ikkilik ma'lumotlarni yozing

Ikkilik ma'lumotlarni yozishda "wb" rejimidan foydalanish kerak. Utf8 kodlashda satr yozishga misol:

```
file_txt = open('test.dat', 'wb')
file_txt.write(bytes('наш строка', 'utf8'))
file_txt.close()
```

Fayldan o‘qish

Fayldan ma’lumotni o‘qish uchun mavjud faylga ishora qiluvchi ob’yektda **read()** metodini chaqirish kerak bo‘ladi. Matn faylini ochishda **open()** funksiyaning ikkinchi parametri sifatida "r" belgidini ko‘rsatishni ham unutmaslik kerak.

Quyidagi misolda o‘qish **test.txt** dan ma’lumotni **print()** funksiyasiga qaytaradi, so‘ngra ma’lumotni ekranga chop etadi. Avvalgidek, dastur hujjatni **close()** usuli bilan yopish bilan yakunlanadi. **Read()** metodi butun son parametrini ham qabul qilishi mumkin, bu o‘qish uchun belgilar sonini uzatish uchun ishlatiladi. Misol uchun, agar 5 ni kiritilsa, dastur faqat “*Salom*” satrini o‘qiydi.

```
try:
    file_txt = open("test.txt", "r")
    print(file_txt.read())
    file_txt.close()
except FileNotFoundError:
    print('Fayl topilmadi!')
except IOError:
    print('Qandaydir xatolik!')
```

Iltimos, ochishda agar ko‘rsatilgan fayl topilmasa xatolik yuz berishi mumkinligini unutmang. Shunday qilib, try bloki bilan orab olish kerak edi. Pythonda buni oldini olish uchun **with** konstruktoridan foydalanish mumkin. Bu holda xatoliklarni ko‘rib chiqish va hatto faylni yopish ham shart emas.

Ikkilik ma’lumotlarni o‘qish

Agar ma’lumotlar ikkilik turda bo‘lsa, **open()** funksiyasining ikkinchi parametri sifatida "rb" dan foydalanish kerak:

```
try:
    f_txt = open("test.dat", "rb")
    b = f_txt.read(1)
    str = ""
    while True:
```

```
b = f_txt.read(1)
if b == b'':
    break
str += b.hex()
print(str)
f_txt.close()
except IOError:
    print('error')
```

Natijasi:

```
81d182d180d0bed0bad0b081d182d180d0bed0bad0b0
```

Bu yerda fayl bayt bo'yicha o'qiladi. Har bir bayt o'n oltilik sanoq sistemasi ko'rinishdagi satr shaklida natija beradi. **Print()** funksiyasi olingan qatorni chop etadi.

with as konstrukturi

Matnli fayllarni qayta ishlashni biroz avtomatlashtirish uchun bir qator **with as** iboralaridan foydalanish tavsiya etiladi. Ushbu operatrolardan foydalanilsa, yopilishi kerak bo'lgan hujjatda **close()** usulini chaqirishning hojati yo'q, chunki bu avtomatik ravishda sodir bo'ladi. Buni quyidagi qism dasturda ko'rib chiqamiz. Odatdagidek, ekranga satr ma'lumotlarini chop etish uchun **print()** usuli qo'llaniladi:

```
with open('test.txt', 'r') as file_txt:
    print(file_txt.read())
```

Bundan tashqari, bu holda istisno bilan shug'ullanish kerak emas. Agar ko'rsatilgan nomga ega fayl bo'lmasa, **with** operatoridagi ichki kodli satrlar bajarilmaydi.

Python dasturlash tilining ko'rib chiqilgan xususiyatlari yordamida foydalanuvchi ma'lumotlarni fayllarga o'qish va yozishning asosiy operatsiyalarini osongina bajarishi mumkin.

Nazorat savollari:

1. Qanday turdagi fayllar mavjud?
2. Fayl yaratishning qanday usullari mavjud?
3. Faylni ochishning qanday rejimlari mavjud?

4. Faylni faqat yozish uchun ochish qanday amalga oshiriladi?
5. Faylni faqat o‘qish uchun ochish rejimi qanday?
6. Faylni faqat o‘qish va yozish uchun ochish rejimi qanday?
7. Faylni ochish rejimlariga “+” belgisi qo‘shilsa qanday o‘zgarish bo‘ladi?
8. with as operatorining qo‘llanilishi qanday?

5.2.8. Pythonda matnli fayllar bilan ishlash

Yuqori darajadagi dasturlash tili Python 3 o‘z foydalanuvchilariga matnli fayllar bilan ishlash uchun juda ko‘p foydali vositalarni taqdim etadi. Platformaning o‘rnatilgan metodlari nafaqat qator ma’lumotlarini saqlash uchun fayllarni yaratish, balki ulardan ma’lumotlarni o‘qish, shuningdek, ularning tarkibini har tomonlama o‘zgartirish, o‘chirish va boshqarish imkonini beradi.

Fayl yaratish, ochish va yopish

Pythonda har qanday matnli fayl bilan ishlash maxsus ochiq funksiyaga qo‘ng‘iroq qilishdan boshlanadi, u parametr sifatida kompyuterdagi ob‘yektga yo‘lni va ishlov berish rejimini oladi. Hujjatga yo‘l mutlaq (kompyuterning qattiq diskidagi manzil manzili) yoki nisbiy (loyiha papkasiga nisbatan koordinatalar) sifatida ko‘rsatilishi mumkin. Kerakli faylni qayta ishlash rejimini tanlashda uning maqsadini hisobga olish kerak (yozish uchun "w", ma’lumotlarni o‘qish uchun "r"). Quyidagi misol matn hujjati bilan oddiy o‘zaro aloqani ko‘rsatadi.

Faylga satrma-satr yozish

Pythonda faylga satr bo‘yicha yozish kerakli qatorni yozish va undan keyin yangi satr belgisi ‘\n’ yozish orqali amalga oshiriladi. Ro‘yxat elementini element bo‘yicha yozish misolini ko‘rib chiqaylik. Har bir element yangi qatorga yoziladi:

```
lines = ["birinchi", "ikkinchi", "uchinchi"]
with open(r"D:\test\test.txt", "w") as file_txt:
    for line in lines:
        file_txt.write(line + '\n')
```

Yuqoridagi kichik dastur misolida uchta qator elementini o‘z ichiga olgan kichik massiv qatorlarini yaratish ko‘rsatilgan: "birinchi", "ikkinchi" va "uchinchi". **Open()** funksiya va **as** operatori orqali **test.txt** matn fayli D qattiq diskining **test** katalogida ochiladi. Sikl yordamida ro‘yxatning barcha elementlariga murojaat qilinadi va yozish uchun **write()** buyrug‘idan foydalaniladi. Har bir yozuvni yangi qatorga joylashtirish uchun yangi qator belgisi “\n” ni qo‘shiladi.

Faylga yozish uchun **writelines()** funksiyasidan ham foydalanish mumkin. Agar ro‘yxatni uning parametri sifatida berilsa, u holda u elementlarni bir qatorga birin-ketin yozadi. U quyidagicha bajariladi: “**generator**”ni parametr sifatida quyidagi tarzda berish kerak:

```
lines = ["birinchi", "ikkinchi", "uchinchi"]
with open(r"D:\test\test.txt", "w") as file_txt:
    file_txt.writelines("%s\n" % line for line in lines)
```

Bu misol oldingisidan farqi shundaki, argument sifatida generatorni oladigan **writelines** usuli chaqiriladi. Hujjatdagi har bir so‘z alohida satrda bo‘lishi uchun ketma-ketlik elementlarini ajratish uchun bu yerda "\n" ishlatiladi.

Fayldan satrma-satr o‘qib olish

Python tilida matn fayli mazmunini satr bo‘yicha o‘qib olish juda oson, buning uchun odatda **for** tsiklidan foydalaniladi. Lekin birinchi navbatda, avvalgi holatda bo‘lgani kabi, **open()** va **as** funksiyalari bilan faylni ochish kerak. Biroq, bu safar o‘qish ma’lumotlariga kirish uchun "r" ni tanlab, boshqa faylni qayta ishlash rejimini belgilash kerak.

```
with open(r"D:\test\test.txt", "r") as file_txt:
    for line in file_txt:
        print(line)
```

Ushbu misol **test.txt** faylini ochadi, u allaqachon yozilgan kichik satrlar qatorini o‘z ichiga oladi. Fayldan ma’lumotni satr bo‘yicha o‘qish **for** tsiklida amalga oshiriladi. Har bir iteratsiya **print()** funksiyasidan foydalangan holda yangi satrdan

ma'lumot chiqaradi. Barcha elementlar avval **test.txt** da joylashtirilganidek, ekranda satr bo'yicha ko'rsatiladi.

Satrlar sonini aniqlash

Python tilining imkoniyatlari matnli fayllarning ichki mazmuni bilan ancha murakkab turdagi dasturiy operatsiyalarni bajarishga imkon beradi. Misol uchun, agar Pythonda fayldagi satrlar sonini aniqlash kerak bo'lsa, faylni ochib, undagi qatorlar soni sikl bo'yicha aylanib chiqish mumkin. Buning uchun oddiy dasturlashdagi summani hisoblash dasturidagi kabi oldindan e'lon qilingan o'zgaruvchi bunga yordam beradi va uning boshlang'ich qiymati nolga teng bo'lishi kerak va keyin har bir yangi qatorga o'tilganida bittadan oshirib boriladi.

```
Count_str = 0
with open(r"D:\test\test.txt", "r") as file_txt:
    for line in file_txt:
        count_str += 1
        print(count_str)
```

Fayldagi satrlarni sanashni boshqa yo'l bilan amalga oshirish mumkin. **readlines()** metodi yordamida faylni ro'yxatda o'qiladi. Keyin ro'yxat hajmini aniqlash uchun **len** funktsiyasidan foydalaniladi. Bu qatorlar sonini qaytaradi:

```
with open(r"D:\test\test.txt", "r") as file_txt:
    print(len(file_txt.readlines()))
```

Fayldan satrni qidirish

Matn faylida satrni topish uchun faylning ichki tarkibini avtomatik ravishda o'qish imkonini beruvchi o'rnatilgan **readlines()** usulidan foydalanish mumkin. Buning uchun oldingidek, avval **test.txt** faylini o'qish uchun ochish kerak, shundan so'ng undan olingan ma'lumotlarni maxsus **lines** nomli massivga joylashtirish kerak. O'rnatilgan **index()** usuli kerakli ob'yektni, masalan, "**ikkinchi**" qatorni parametr sifatida berish orqali ketma-ketlikdagi elementning sonini topishga imkon beradi.

```
with open(r"D:\test\test.txt", "r") as file_txt:
    lines = file_txt.readlines()
    print(lines.index("ikkinchi\n"))
```

test.txt faylidagi barcha elementlar alohida satrlarda joylashganligi sababli, **index()** usuliga argument sifatida uzatilishi kerak bo'lgan satrga "\n" konstruktsiyasini qo'shish kerak.

Fayldan satrni olib tashlash

Pythonida fayldan keraksiz qatorni olib tashlash uchun bir vaqtning o'zida ikkita faylni qayta ishlash rejimidan foydalanish kerak bo'ladi: o'qish va yozish. Undan olingan ma'lumotlarni alohida **lines** nomli massivga joylash uchun avval **test.txt** faylini o'qish uchun ochish kerak. Keyinchalik, **del** operatori yordamida ketma-ketlik elementlaridan birini olib tashlash kerak. Buning uchun **del** operatoriga kvadrat qavs ichida kerakli satrning indeksini ko'rsatiladi. O'zgartirilgan ya'ni ba'zi elementlari o'chirilgan ob'yektlar massivi asl matn fayliga qayta joylashtirilishi kerak, lekin bu safar fayl yozish rejimida ochilishi va eski ma'lumotlar o'chirilishi kerak.

```
with open(r"D:\test\test.txt", "r") as file_txt:
    lines = file_txt.readlines()
del lines[1]
with open(r"D:\test\test.txt", "w") as file_txt:
    file_txt.writelines(lines)
```

Yuqoridagi kichik dastur misoli fayldan satrni o'chirishni ko'rib chiqildi. Bunda fayldan satrlar massivini o'qish va ro'yxatning alohida elementlarini ajratib o'sha yerga yozish imkonini beruvchi **readlines** va **writelines** usullari qo'llanildi.

Shunday qilib, Python dasturlash tilidan foydalangan holda matnli fayllar tarkibi bilan ishlash juda oson.

Nazorat savollari:

1. Faylni yaratish, ochish va yopish uchun qanday operatorlardan foydalaniladi?

2. Satrlarni faylga yozish qanday amalga oshiriladi?
3. Fayldan satrlarni o‘qish qanday amalga oshiriladi?
4. Fayldagi qatorlar sonini aniqlash uchun qanday usuldan foydalaniladi?
5. Qatorni qidirish uchun qaysi metoddan foydalaniladi?
6. Qatorni o‘chirish uchun qanday metoddan foydalaniladi?

5.3.§. OS modulidan foydalanib fayllar bilan ishlash

Python dasturlash tilining standart kutubxonasidagi OS moduli odatda o‘rnatilgan OT va shaxsiy kompyuter fayl tizimi bilan ishlash uchun qo‘llaniladi. Unda qattiq diskda saqlanadigan fayllar va kataloglar bilan ishlash uchun juda ko‘p foydali o‘rnatilgan metodlar mavjud. OS moduli bilan ishlaydigan dasturlar OT turiga bog‘liq bo‘lmasdan, boshqa platformalarga osongina ko‘chiriladi.

OS moduli nima?

Pythondagi OS moduli operatsion tizim bilan ishlash uchun kerak bo‘ladigan turli funktsiyalar kutubxonasidir. Unga kiritilgan usullar **operatsion tizim turini aniqlash, muhit o‘zgaruvchilariga kirish, katalog va fayllarni boshqarish** imkonini beradi:

- berilgan yo‘lda ob‘yektning mavjudligini tekshirish;
- baytlarda o‘lchamlarni aniqlash;
- o‘chirish;
- nomini o‘zgartirish va boshqalar.

OS funksiyalarini chaqirayotganda, ularning ba‘zilari joriy operatsion tizim tomonidan qo‘llab-quvvatlanmasligi mumkin.

OS moduli metodlaridan foydalanish uchun kutubxonani “import os” orqali chaqirish kerak. Ushbu ko‘rsatmani dasturning boshida joylashtirish tavsiya etiladi.

OS modulidagi mavjud metodlardan foydalanuvchi turli maqsadlarda foydalanishi mumkin. Quyida operatsion tizim haqida ma’lumot olish imkonini beruvchi eng mashhur metodlar keltirilgan. Shuningdek, kompyuterning qattiq diskidagi xotirada saqlangan fayl va papkalar haqida ma’lumot olish mumkin.

OT haqida ma'lumot olish

Joriy OT nomini bilish uchun faqat **name()** metodidan foydalaning. O'rnatilgan platformaga qarab, u o'zining qisqa nomini satr ko'rinishida qaytaradi. Quyidagi dastur Windows 10 kompyuterida ishga tushirildi, shuning uchun **name()** funksiyasining natijasi sifatida **nt** qaytarildi. Buni odatiy **print()** funksiyasi bilan ko'rish mumkin.

```
import os #os modulini chaqirish
print(os.name) #OT nomini bilish
```

Natijasi:

```
nt
```

environ() metodidan foydalanib, kompyuterning konfiguratsiyasi bilan bog'liq ma'lumotlarni olish mumkin. Undan **os** kutubxonasini chaqirish orqali foydalanuvchi konsolga yoki satr o'zgaruvchisiga chiqadigan muhit o'zgaruvchilari bilan katta lug'at turidagi ma'lumot oladi. Shunday qilib, tizim diskining nomini (C:\...), katalogining manzilini, tizim nomini va boshqa ko'plab ma'lumotlarni bilib olish mumkin. Quyidagi misol **environ()** metodidan foydalanishni ko'rsatadi.

```
import os
print(os.environ)
environ({'ALLUSERSPROFILETEXT': 'C:\\ProgramData', ...})
```

getenv funksiyasidan foydalanib, turli xil muhit o'zgaruvchilariga kirish mumkin. Buning uchun quyidagi misoldagi kabi kerakli o'zgaruvchi nomini argument sifatida beriladi. Bunda **print** displeydagi TMP haqidagi ma'lumotlarni konsolga chop etadi.

```
import os
print(os.getenv("TMP"))
```

Natijasi:

```
C:\Users\IT\AppData\Local\Temp
```

Ishchi katalogni o'zgartirish

Odatda dasturning ishchi katalogi uning manba kodi bilan hujjatni o'z ichiga olgan katalog bo'ladi. Buning yordamida, agar u ushbu papkada joylashgan bo'lsa, faylga mutlaq yo'lni aniqlashning iloji bo'lmaydi. Qattiq diskdagi ishchi katalogning to'liq manzilini qaytaruvchi **getcwd** funksiyasidan foydalanib joriy katalog haqida ma'lumot olinadi. Quyidagi qism dastur, agar ushbu usulning natijasini chop etish uchun o'tkazsa nima bo'lishini ko'rsatadi. Ko'rib turganingizdek, ishchi katalog C tizim diskidagi dastur katalogidir.

```
import os  
print(os.getcwd())
```

Natijasi:

```
C:\Users\IT\source\repos\program
```

Agar so'ralsa, ishchi katalogni **os** kutubxonasidan **chdir** usuli yordamida xohishga ko'ra sozlash mumkin. Buni amalga oshirish uchun uni yangi katalogning mutlaq manzilini parametr sifatida o'tkazish kerak. Agar ko'rsatilgan yo'l haqiqatda mavjud bo'lmasa, dastur xatolik qaytaradi va dastur ishlashi to'xtatiladi. Quyidagi kod misoli D diskidagi papka deb nomlangan yangi ishchi katalogga o'tishni ko'rsatadi.

```
import os  
os.chdir(r"D:\folder")
```

Natijasi:

```
D:\folder
```

Yo'l mavjudligini tekshirish

Dastur tomonidan qayta ishlanishi kerak bo'lgan ma'lum bir fayl yoki katalogning yo'qligi bilan bog'liq xatolarga yo'l qo'ymaslik uchun avval mavjud usuli yordamida ularning mavjudligini tekshirish kerak. Unga argument sifatida kerakli fayl yoki papkaga yo'lni o'tkazib, ko'rsatilgan ob'yektning kompyuter xotirasida mavjudligini yoki yo'qligini ko'rsatuvchi mantiqiy True/False qiymat ko'rinishidagi

ixcham javob orqali amalga oshirish mumkin. Quyidagi misol D disk katalogidan **test.txt** matn faylini tekshiradi va **True** qiymatini qaytaradi.

```
import os  
print(os.path.exists("D:/test/test.txt"))
```

Natijasi:

```
True
```

Agar ob'yekt diskda haqiqatda mavjud bo'lsa, bu har doim ham uning keyingi ishlov berish uchun mos keladigan shaklga ega ekanligini anglatmaydi. Muayyan ob'yekt fayl ekanligini tekshirish uchun uning manzilini oladigan **isfile()** funksiyasi yordam beradi. Quyidagi misolda ushbu funksiyaning ishlashini ko'rishimiz mumkin, bunda **print()** funksiyasi ekranda **test.txt** fayli uchun **True** qiymatini ko'rsatadi:

```
import os  
print(os.path.isfile("D:/test/test.txt"))
```

Natijasi:

```
True
```

Shu kabi amallarni ob'yektning katalog sinfiga tegishli ekanligini aniqlash uchun uning manzili **isdir()** funksiyasi orqali amalga oshiriladi. Bu holda **print()** funksiyasi **False** qiymatini chop etadi, chunki **test.txt** - bu katalog emas.

```
import os  
print(os.path.isdir("D:/test.txt"))
```

Natijasi:

```
False
```

Kataloglar yaratish

OS modulining imkoniyatlari nafaqat xotirada mavjud bo'lgan ob'yektlar haqidagi ma'lumotlarni ko'rsatish, balki butunlay yangilarini yaratish imkonini beradi. Masalan, **mkdir** usulidan foydalanib, unga kerakli yo'lni berish orqali yangi

katalog yaratish mumkin. Quyidagi misolda **mkdir** orqali D diskining tarkibidagi katalogda “*katalog*” deb nomlangan yangi papka yaratilgan:

```
import os  
os.mkdir(r"D:\katalog ")
```

Biroq, kataloglarni yaratish imkoniyatlari shu bilan tugamaydi. **makedirs** funksiyasi yordamida bir vaqtning o‘zida cheksiz sonda bir bir birining ichida joylashgan kataloglar yaratish mumkin. Shunday qilib, quyidagi misolda ichma-ich joylashgan uchta katalog yaratilgan:

```
import os  
os.makedirs(r"D:\katalog\test\bir\ikki")
```

Fayllar va kataloglarni o‘chirish

REMOVE() usuli yordamida keraksiz fayllarni o‘chirib yuborish mumkin. Ushbu funksiyaga argument sifatida ob‘yektga mutlaq yoki nisbiy yo‘l beradi. Quyidagi qisqa kod **test.txt** hujjatini kompyuterdagi D diskining tarkibidagi katalogidan o‘chirishni ko‘rsatadi.

```
import os  
os.remove(r"D:\test\test.txt")
```

Katalogni xotiradan o‘chirish uchun o‘rnatilgan **rmdir** funksiyasidan foydalanish kerak, unga ob‘yekt manzilini ko‘rsatiladi. Biroq, bu yerda ma’lum nuanslar mavjud, chunki dastur boshqa ob‘yektlar saqlanadigan katalogni erkin o‘chirishga imkon bermaydi. Bunday holda, ekranda xatolik ko‘rsatiladi va istisno chiqariladi. Quyidagi misol **rmdir** usuli yordamida bo‘sh papkani olib tashlaydi xalos.

```
import os  
os.rmdir(r"D:\katalog")
```

Ko‘p bo‘sh papkalarni tezda o‘chirish uchun **redirs** funksiyasini chaqiriladi. U bir vaqtning o‘zida diskdagi bir nechta kataloglardan qutulish imkoniyatini beradi.

Agar ularning barchasi bir-birining ichiga joylashtirilgan bo'lsa. Shunday qilib, maqsad papkaga yo'lni ko'rsatib, barcha asosiy kataloglarni osongina o'chirish.

Misol to'rt xil papkani bir vaqtda o'chirish misolini ko'rib chiqaylik: katalog, test, bir, ikki.

```
import os  
os.removedirs(r"D:\katalog\test\bir\ikki")
```

Alohida ochishni ishga tushirish

Os kutubxonasining o'rnatilgan funktsiyalari to'g'ridan-to'g'ri dasturdan alohida fayl va papkalarni ishga tushirish imkonini beradi. **startfile()** usuli kerakli ob'yekt manziliga o'tishi kerak bo'lgan ushbu vazifani mukammal darajada bajaradi. Hujjatni ochish uchun foydalaniladigan dastur avtomatik ravishda muhit tomonidan aniqlanadi. Masalan, oddiy **test.txt** faylini ishga tushirishda, quyidagi misoldagi kabi, standart bloknot ishlatiladi. Agar katalogga yo'lni funksiyaga parametr sifatida berilsa, u o'rnatilgan fayl menejeri yordamida, alohida ochiladi.

```
import os  
os.startfile(r"D:\test\test.txt")
```

Fayl va katalog nomini olish

Ba'zan hujjat bilan ishlash uchun uning to'liq nomini, shu jumladan ruxsatni olish kerak bo'ladi. Lekin diskdagi mutlaq yo'lni olish bundan mustasno. OS kutubxonasining tarkibida joylashgan **basename** funksiyasi ob'yekt manzilini nomga aylantirish imkonini beradi. Shunday qilib, quyidagi misolda **test.txt** fayli yo'lni oddiy fayl nomiga aylantirish ko'rsatilgan:

```
import os  
print(os.path.basename("D:/test/test.txt"))
```

Natijasi:

```
test.txt
```

Teskari holat foydalanuvchi faqat faylga yo'lni olishi kerak bo'lganda, ob'yektning o'zi nomisiz sodir bo'ladi. Bu quyida keltirilgan kichik misolda ko'rsatilganidek, **string** ko'rinishida berilgan hujjatga yo'lni qaytaradigan **dirname** usulini yaratishga yordam beradi. Bu yerda **print()** test papkasidagi matn hujjatining manzilini ko'rsatadi.

```
import os  
print(os.path.dirname("D:/test/test.txt"))
```

Natijasi:

```
D:/test
```

O'lchamni hisoblash

Hujjat yoki papka hajmini aniqlash uchun **test.txt** fayli uchun quyidagi misolda ko'rsatilganidek, **getsize** funksiyasidan foydalaniladi. **Print()** funksiyasi berilgan hujjat hajmini baytlarda chop etadi. Kataloglar hajmini o'lchash uchun **getsiz** funksiyasidan ham foydalanish mumkin.

```
import os  
print(os.path.getsize("D:\\test\\test.txt"))
```

Natijasi:

```
1362262
```

Nomini o'zgartirish

Os kutubxonasi **rename** usuli yordamida istalgan fayl yoki katalog nomini tezda o'zgartirish imkoniyatini beradi. Bu funksiya bir vaqtning o'zida ikki xil argumentni oladi. Ulardan birinchisi eski hujjat nomiga boradigan yo'lni, ikkinchisi esa uning yangi nomini qabul qiladi. Misol "katalog" nomli papkaning nomini "kataloglar"ga o'zgartirishni ko'rib chiqaylik. Belgilangan yo'lda fayl bo'lmasa, metod xatolik qaytaradi.

```
import os  
os.rename(r"D:\katalog", r"D:\kataloglar")
```

Faqat bitta katalog emas, balki bir vaqtning o'zida bir nechta papkalarni ham nomini o'zgartirish mumkin. Faqat ularning barchasi bir xil ierarxik zanjirda bo'lishi kerak.

Buni amalga oshirish uchun faqat **renames** usulini chaqiring va uni birinchi argumenti sifatida kerakli katalogga yo'lni beriladi. Ikkinchi parametrqa ichma-ich joylashgan papkalarning boshqa nomlar bilan nomlangan zanjiri beriladi. Quyidagi misol **renames** funksiyasidan to'g'ri foydalanishni ko'rsatib o'tilgan, bu esa katalog papkasini bir va ikki kataloglarini, kataloglar, one va two kataloglariga o'zgartiradi.

```
import os  
os.rename(r"D:\katalog\bir\ikki", r"D:\kataloglar\one\two")
```

Katalog tarkibi

listdir funksiyasi katalogda ma'lum ob'yektlar mavjudligini tekshirish imkonini beradi. Uning yordami bilan ro'yxat shaklida fayllar va papkalar haqida ma'lumot olish mumkin. Quyidagi dastur **listdir** metodi D diskidagi **katalog** nomli papkasiga yo'lni parametr sifatida qanday qabul qilishini ko'rsatilgan va keyin avval ichki papka nomini va test.txt hujjatini chop etadi, konsolga chiqish **print()** funksiyasi yordamida amalga oshiriladi:

```
import os  
print(os.listdir(r"D:\katalog"))
```

Natijasi:

```
['birinchi', 'test.txt']
```

WALK() metodidan foydalanib, berilgan katalogga tegishli barcha ichki papkalar va fayllarning nomlari va yo'llariga kirish mumkin. Bitta tashqi **for** sikli, shuningdek ikkita ichki o'rnatilgan maxsus **direktories** va **files** nomli ro'yxatlari orqali

“**katalog**” papkasidagi ob’yektlar haqida ma’lumot olish oson. Ma’lumot `print()` funksiyasini qayta-qayta chaqirish orqali konsolga chiqariladi.

```
import os
for root, directories, files in os.walk(r"D:\katalog"):
    print(root)
    for directory in directories:
        print(directory)
    for file in files:
        print(file)
```

Natijasi:

```
D:\katalog
test
D:\katalog\test
bir
D:\katalog\test\bir

D:\katalog\test\bir\ikki
test.txt
```

Fayllar va kataloglar haqida ma’lumot

stat metodi yordamida ob’jekt haqidagi asosiy ma’lumotlarni ekranda yoki istalgan boshqa joyda ko’rsatiladi. Unga kompyuter diskidagi fayl yoki papkaning joylashuvini parametr sifatida beriladi va bir qancha ma’lumotni qabul qilib olish mumkin. Bu yerda ob’jektning baytdagi o’lchami haqidagi ma’lumotlarni, shuningdek kirish va uning ishlash rejimini ko’rsatadigan ba’zi raqamli qiymatlarni topish mumkin:

```
import os
print(os.stat(r"D:\test\test.txt"))
os.stat_result(st_mode=332006, ...)
```


Fayl yoki katalogga bo'lgan yo'lni boshqarish

OS kutubxonasining **path** sinfiga tegishli **split** funktsiyasini faylga yo'lni va fayl nomini turli qatorlarga ajratishni juda osonlashtiradi. Bu quyidagi misolda “**test**” papkasida **test.txt** matn hujjati bilan ko'rsatilgan:

```
import os
print(os.path.split(r"D:\test\test.txt"))
```

Natijasi:

```
('D:\\test', 'test.txt')
```

Teskari harakat **join()** funktsiyasi tomonidan amalga oshiriladi, bu hujjatga yo'lni uning nomi bilan birlashtirishni osonlashtiradi. Ushbu kodning chiqishidan ko'rinib turibdiki, **print()** funktsiyasi orqali papkaga yo'l ekranda ko'rsatiladi, bu D diskning test papkasida joylashgan test.txt matn fayliga ishora qiladi.

```
import os
print(os.path.join(r"D:\test", "test.txt"))
```

Natijasi:

```
D:\test\test.txt
```

Bular Python dasturlarida o'rnatilgan OTni boshqarishning ko'plab o'rnatilgan usullari orqali amalga oshirilgan OS modulining asosiy xususiyatlari edi. Shunday qilib, ular nafaqat platforma haqida foydali ma'lumotlarni olish, balki disk tarkibi bilan ishlash, yangi katalog va fayllarni yaratish, ularni qayta nomlash va butunlay o'chirish imkonini beradi.

Nazorat savollari:

1. OS modulining vazifasi qanday?
2. OS modulining xususiyatlari haqida gapirib bering.
3. Ishchi katalogni o'zgartirish qanday amalga oshiriladi?
4. Yo'lning mavjudligini tekshirish uchun qaysi metoddan foydalaniladi?
5. Kataloglarni yaratish qanday amalga oshiriladi?

6. Fayllar va kataloglarni o‘chirish qanday amalga oshiriladi?
7. Fayl nomi va katalogini olish uchun qaysi metodlar qo‘llaniladi?
8. Fayl o‘lchamini hisoblash qanday amalga oshiriladi?
9. Fayl nomini o‘zgartirish qanday bajariladi?
10. Kataloglar tarkibini aniqlash uchun qanday usul mavjud?
11. Fayllar va kataloglar haqida ma’lumotlar qaytaruvchi metod qaysi?
12. Fayllar va kataloglarga yo‘lni qayta ishlash uchun qaysi metoddan foydalaniladi?

5.4.§. Fayllar bilan ishlovchi metodlar

Python dasturlash tilining asosiy xususiyatlari nafaqat matnli fayllardagi ma’lumotlarni qayta ishlash, balki ularni har tomonlama boshqarish imkonini beradi. Buning uchun bir nechta maxsus kutubxonalar mavjud bo‘lib, ularning o‘rnatilgan funktsiyalari kompyuterdagi fayllar bilan **nusxa ko‘chirish, o‘chirish, nomini o‘zgartirish** va boshqa turdagi operatsiyalarni ta’minlaydi.

Fayl mavjudligini tekshirish

Kompyuterning qattiq diskida uning yo‘qligi bilan bog‘liq bo‘lishi mumkin bo‘lgan matnli hujjat bilan ishlashda oddiy xatoliklarga yo‘l qo‘ymaslik OS kutubxonasining **exists()** metodidan foydalaniladi. Ushbu funksiya orqali ko‘rsatilgan yo‘lda fayl mavjudligini tekshirish mumkin va natijas sifatida mantiqiy True yoki False qiymatlarni qaytaradi. Quyidagi Python misoli D qattiq diskining **test** nomli katalogida **test.txt** va **test10.txt** fayllari mavjudligini tekshiradi. **PRINT()** funksiyasi esa D da faqat birinchi hujjat mavjudligini ko‘rsatadi.

```
import os
print(os.path.exists("D:\\test\\test.txt"))
print(os.path.exists("D:\\test\\test10.txt"))
```

Natijasi:

```
True
False
```

Ba'zan, hujjatlar bilan ishlashda, berilgan yo'l bo'ylab nafaqat ob'yektning mavjudligini tekshirish, balki fayl yoki papka ekanligini tekshirish ham kerak bo'ladi. Bunday holda OS kutubxonasining **isfile** funksiyasi dasturchiga kompyuterning qattiq diskidagi ma'lum bir manzilda olingan ob'yekt papka emas, balki fayl ekanligiga ishonch hosil qilish imkoniyatini beradi. Bu usul **path** sinfida ham mavjud. Quyidagi misolda **isfile** funksiyasi **test.txt** faylini va D diskning test papkasini argument sifatida qabul qilganda qanday natija ko'rsatadi. **PRINT()** funksiyasining chiqishidan ko'rinib turibdiki, birinchi holatda True, keyin esa False ko'rsatiladi:

```
import os
print(os.path.isfile("D:\\test\\test.txt"))
print(os.path.isfile("D:\\test\\folder"))
```

Natijasi:

```
True
False
```

Faylni nusxalash

shutil kutubxonasi qattiq diskdagi ob'yektlarning nusxalarini yaratish uchun bir nechta foydali funktsiyalarni o'z ichiga oladi. Faylni manba katalogiga tezda nusxalash uchun **shutil** kutubxonasini qo'shgandan so'ng **copyfile()** usulidan foydalanish kerak. Bu yerda birinchi argument asl hujjat, ikkinchi argument esa taklif qilingan yangi fayldir. Esda tutingki, **meta ma'lumotlar** emas, faqat faylning tarkib nusxalanadi. Quyidagi misolda D diskidagi test.txt dan test2.txt ga ma'lumotlarni ko'chiriladi. **copyfile()** funktsiyasi yangi yaratilgan hujjatning manzilini ham qaytaradi.

```
import shutil
shutil.copyfile("D:\\test\\test.txt", "D:\\test\\test2.txt")
```

shutil modulidan o'rnatilgan **copy()** usuli Pythonga faylni asl nomini saqlab qolgan holda belgilangan papkaga nusxalash imkonini beradi. Quyidagi kod misoli **test.txt** dan ma'lumotni D diskida joylashgan **folder** deb nomlangan katalogdagi

ob'jektga qanday ko'chirishni ko'rsatadi. **Copy()** funksiyasi bilan oldingi holatda bo'lgani kabi, faqat ichki ma'lumotlar uzatiladi, lekin hujjatni yaratish va tahrirlash sanasi haqida ma'lumotlar qaytarilmaydi.

```
import shutil  
shutil.copy("D:\\test\\test.txt", "D:\\folder")
```

Matn faylidan ma'lumotni, shuningdek, u haqidagi barcha ma'lumotlarni to'liq nusxalash uchun tayyor nusxa **copy2** usulidan foydalanish kerak. Uni ishlatish usuli **copy** funksiyasi bilan bir xil. Birinchi parametr o'rniga bu asl faylning manzili, ikkinchi parametr esa yangi hujjatning joylashuvi va nomini belgilaydi. Quyida kontent va metama'lumotlar folser papkasidan **test2.txt** ga ko'chiriladigan misol keltirilgan:

```
import shutil  
shutil.copy2("D:\\test\\test.txt", "D:\\folder\\test2.txt")
```

Fayl o'chirish

Ob'jektning nomi va diskdagi aniq joylashuvini bilib olinsa, undan qutulish juda oson. Yuqorida aytib o'tilgan operatsion tizim kutubxonasidan **remove** usuli bu vazifani hal qilishga yordam beradi. Buning uchun keraksiz faylning to'liq manzilini unga parametr sifatida beriladi.

Quyida D diskining **test** katalogidagi **test.txt** faylini o'chirish uchun **remove** metodidan qanday foydalanishga misol keltirilgan:

```
import os  
os.remove("D:\\test\\test.txt")
```

Fayl hajmini bilib olish

Fayl hajmini baytlarda qaytaradigan OS modulidagi **getsize** standart funksiyasidan foydalanib, qattiq diskdagi istalgan ob'jektning aniq hajmini aniqlash mumkin. Bu yerda **getsize** funksiyasi argumenti hujjatning kompyuter xotirasidagi joylashuvidir. **getsize** funksiyasi ma'lumotlariga ko'ra, **test.txt** faylining o'lchami 7219. **Print()** funksiyasi buni ekranga chop etadi:

```
import os
print(os.path.getsize("D:\\test\\test.txt"))
```

Natijasi:

7219

Pythonda fayl hajmini hisoblashning yana bir usuli - uni **open** funksiyasi orqali ochish va **seek** funksiyasini chaqirish. U faylning boshidan oxirigacha ma'lumotlarni o'qish uchun maydonni parametr sifatida uzatilishi kerak. Natijada, matnli faylga yo'l orqali **tell** usulini chaqirish kerak va keyin uning ishining natijasini konsolga chiqarish uchun **print()** funksiyasi uchun yuborish kerak.

```
import os
with open("D:\\test\\test.txt") as file:
    file.seek(0, os.SEEK_END)
    print(file.tell())
```

Natijasi:

7219

Fayl nomini o'zgartirish

Hujjat nomini nafaqat tizim vositalari, balki OS modulining tayyor funksiyalari yordamida ham o'zgartirish mumkin. Buning uchun **rename()** metodidan foydalanish maqsadga muvofiq bo'ladi. Bu metod asl va yangi fayl nomlarini parametr sifatida qabul qiladi. Quyidagi misolda D diskining **test** katalogidagi **test.txt** fayli nomi **test1.txt** deb o'zgartirish ko'rsatilgan:

```
import os
os.rename("D:\\test\\test.txt", "D:\\test\\test1.txt")
```

Xuddi shunday, **shutil** modulidan **copy** metodi yordamida Pythonda fayl nomini o'zgartirish mumkin. Buning uchun ushbu kutubxonani dasturga import qilish, funktsiyaga hujjatning joylashuvi va yangi nomini berish kifoya qiladi:

```
import shutil  
  
shutil.move("D:\\test\\test.txt", "D:\\test\\test1.txt")
```

Shunday qilib, Python tilidagi fayllar bilan ishlash bo'yicha asosiy operatsiyalar bir nechta o'rnatilgan kutubxonalar, jumladan **os** va **shutil** yordamida amalga oshiriladi. Ushbu modullarning funktsiyalari diskda **fayl mavjudligini tekshirish**, uni bir nechta **turli rejimlarda nusxalash**, shuningdek **o'chirish**, **nomini o'zgartirish** va **hajmini ko'rsatish** imkonini beradi.

Nazorat savollari:

1. Fayl mavjudligini tekshirish qaysi metod orqali amalga oshiriladi?
2. Faylni nusxalash qanday amalga oshiriladi?
3. Faylni o'chirish uchun qaysi metoddan foydalaniladi?
4. Fayl hajmini olish uchun qanday amallarni bajarish kerak?
5. Fayl nomini o'zgartirish qanday bajariladi?

5.5.§. Faylga yo'l, fayl nomi va fayl kengaytmasi bilan ishlash

Ko'pincha, faylning to'liq nomi mavjud bo'lganda, uning kengaytmasini bilish kerak bo'ladi. Yoki foydalanuvchi kerakli formatni kiritgan yoki kiritmaganligi noma'lum bo'lsa, kerakli formatni qo'shish kabi masalalar mavjud. Ushbu mavzu fayl nomi bilan ishlashning asosiy usullari haqida bo'ladi.

Faylga mutlaq yo'l

Pythonda faylga mutlaq yo'lni bilish uchun OS modulidan foydalanish kerak bo'ladi. **Path** sinfida faylga absolyut yo'lni qaytaruvchi - **abspath** metodi mavjud:

```
import os  
  
r = os.path.abspath('file.txt')  
  
print(r)
```

Natijasi:

```
C:\python\file.txt
```

Buning uchun standart **pathlib** kutubxonasidan ham foydalanish mumkin. U Python 3.4 dan boshlab asosiy kutubxonalarga kiritilgan. Undan oldin uni pip

`install pathlib` buyrug'i yordamida o'rnatish kerak edi. U turli xil operatsion tizimlarda fayl tizimi yo'llari bilan ishlash uchun mo'ljallangan va bu muammoni hal qilish uchun juda mos keladi.

```
import pathlib

r = pathlib.Path('file.txt ')

print(r)
```

Natijasi:

```
C:\python\file.txt
```

Fayl nomi

To'liq yo'l qatoridan fayl nomini olish uchun `os` modulining **`basename()`** metodidan foydalaniladi.

```
import os

file_name = os.path.basename(r'C:\python\file.txt ')

print(file_name)
```

Natijasi:

```
file.txt
```

Bu yerda maxsus belgilarni oddiy belgilar sifatida ishlatish uchun qatordan oldin **`r`** belgisi qo'yilgan.

Kengaytmasiz fayl

Endi Pythonda kengaytmasiz fayl nomini qanday topish mumkinligini aniqlaylik. Bunday hollarda `os` modulining **`splittext`** metodidan foydalaniladi:

```
from os import path

full_file_name = path.basename(r'C:\python\file.tar.gz ')

name = path.splitext(full_file_name)[0]

print(name)
```

Natijasi:

```
file.tar
```

Ko‘rinib turibdiki, **gz** arxivatorining oxirgi kengaytmasi o‘chirilgan, siqilmagan **tar** arxivining kengaytmasi esa fayl nomida qolgan.

Agar faqat nom kerak bo‘lsa, birinchi nuqtadan keyin kelgan qabul qilingan satrning barcha belgilarini olib tashlash mumkin. Nuqta belgisini ham olib tashlaylik. Oldingi misolni quyidagi kod bilan to‘ldiraylik:

```
index = name.index('.')  
print(name[:index])
```

Natijasi:

```
file
```

Fayl kengaytmasi

Pythonda fayl kengaytmasini xuddi shu **splitext** funksiyasidan foydalanib olish mumkin. Ushbu metod o‘zidan kortej tipli ma’lumot qaytaradi. Kortejning birinchi elementi fayl nomi bo‘lsa, ikkinchi element kengaytmadir. Bunday holda, bizga ikkinchi element kerak. Ikkinchi elementning indeksi birga teng, chunki kortej elementlari ham massivlar elementlari kabi noldan indexlanadi.

```
from os import path  
full_file_name = path.basename(r'C:\python\file.tar.gz ')  
name = path.splitext(full_file_name)[1]  
print(name)
```

Natijasi:

```
.gz
```

Xuddi shunday, **pathlib** kutubxonasiining **suffix** metodidan foydalanish ham mumkin:

```
from pathlib import Path  
print(Path(r'C:\python\file.tar.gz').suffix)
```

Natijasi:

```
.gz
```


Ammo ushbu holatlarda ikkita kengaytma mavjud. Ularni **suffixes()** funktsiyasi yordamida aniqlab olish mumkin. U elementlari kengaytmalar bo'lgan ro'yxatni qaytaradi. Quyida kengaytmalar ro'yxatini olish misoli keltirilgan:

```
from pathlib import Path  
print(Path(r'C:\python\file.tar.gz').suffixes)
```

Natijasi:

```
[ '.tar', '.gz' ]
```

To'liq yo'ldan fayl nomi yoki kengaytmasini olish yoki faylga mutlaq yo'lni olish uchun **os** va **pathlib** kutubxonalaridan foydalanish tafsila etiladi.

Nazorat savollari:

1. Mutlaq fayl yo'li qanday olinadi?
2. Fayl nomini qanday olish mumkin?
3. Fayl kengaytmasini qanday olish mumkin?
4. Fayl kengaytmasini o'zgartirish qanday amalga oshiriladi?

5.6.§. Faylni tozalash usullari

Ba'zida ma'lumotlarni o'z ichiga olgan faylga yozish kerak bo'lgan vaziyatlar yuzaga keladi. Yoki shunchaki barcha tarkibni o'chirib tashlashga extiyoj paydo bo'ladi. Ushbu mavzuda Python yordamida faylni qanday tozalashni ko'rib chiqilgan. Python tilida faylni quyidagi usullar bilan tozalash mumkin:

- Ochayotganda ko'rsatkich fayl boshida turgan rejimdan foydalanish.
- Ko'rsatkichni qo'lda boshlang'ich holatiga o'tkazish.
- Operatsion tizim yordamida fayl tarkibini qayta o'rnatish.

Ochayotganda ko'rsatkich fayl boshida turgan rejimdan foydalanish

Fayl yozish uchun ochilganda, ko'rsatkichi hujjatning boshida yoki oxirida joylashgan bo'lishi mumkin. Agar ko'rsatkich oxirida bo'lsa, u holda ma'lumotlar qo'shiladi. Bizni ko'rsatkich boshida joylashgan bo'lsachi, unda qanday hodisa sodir bo'ladi?

```
f_txt = open('test.txt', 'w')  
f_txt.close()
```

Bu yerda w - boshida kursor bilan matn rejimida yozish uchun faylni ochish rejimini bildiradi. Ushbu kodni bajargandan so'ng, agar ko'rsatilgan fayl mavjud bo'lsa, uning mazmuni o'chiriladi. Agar u yo'q bo'lsa, unda yangi bo'sh joy yaratiladi.

Yopishdan oldin ma'lumot qo'shilishi mumkin. U faylning boshidan yoziladi, oxiriga qo'shilmaydi.

```
f_txt = open('test.txt', 'w')  
f_txt.write('Nimadir yozildi!')  
f_txt.close()
```

Agar ikkilik faylga ma'lumotlarni yozish kerak bo'lsa, "wb" rejimidan foydalanish kerak.

Aks holda, faylning oxiriga ma'lumot qo'shish kerak. Shu bilan birga, eski ma'lumotlar qolishi mumkin. Bunday holda, rejimga + belgisi qo'shilishi kerak. Matnli hujjatni ochish rejimi "w+" va ikkilikda esa "wb+" bo'ladi.

Ko'rsatkichni siljitish

Agar faylni yozish uchun ochgan bo'lsak va ko'rsatgich qayerda joylashganligini bilmasak. Ehtimol, allaqachon ba'zi ma'lumotlarni yozilgan bo'lishi mumkin. Shunchaki ko'rsatgichni boshiga olib, uni yopish mumkin. Bunday holda, hujjat bo'sh bo'ladi.

```
f_txt = open('test.txt', 'w+')  
f_txt.seek(0)  
f_txt.close()
```

Ushbu misolda, ochish maxsus qayta yozish rejimida ochilgan. Yopgandan so'ng, faylda ma'lumotlar bo'lsa ham, ular o'chiriladi.

Mana yana bir misol, bu yerda ma'lumotlarni yoziladi, keyin ko'rsatgichni fayl boshiga o'tkaziladi. Shundan so'ng yana faylga ma'lumot yoziladi. Natijada, ish

oxirida faqat oxirgi yozilgan yozuv faylda qoladi. Boshida kiritilgan ma'lumotlar o'chiriladi.

```
f_txt = open('test.txt', 'w+')  
f_txt.write('bir qancha satrlarni qushish')  
f_txt.seek(0)  
f_txt.write('yangi satr qushish')  
f_txt.close()
```

Operatsion tizim vositalari

Operatsion tizim vositalari yordamida tozalash uchun standart operatsion tizim kutubxonasidan foydalaniladi. Birinchidan, u **import os** ifodasi yordamida dasturga ulanishi kerak. Dastur Linux tizimida quyidagicha ko'rinishda bo'ladi:

```
import os  
os.system(r' >file.txt')
```

Buning uchun qo'shimcha tarzda **cp** yoki **cat** buyruqlaridan foydalanish mumkin. Bu yerda **cat** funksiyasi yordamida masalaning yechimi keltirilgan:

```
os.system(r'cat /dev/null>file.txt')
```

Agar kod Windowsda bajarilgan bo'lsa, unda yechim quyidagicha bo'lishi mumkin:

```
os.system(r'nul>file.txt')
```

Nazorat savollari:

1. Faylni tozalashning qanday usullari mavjud?
2. Faylni tozalashning ko'rsatgichni siljitish usulidan qanday foydalaniladi?
3. Faylni tozalashning OS moduli yordamida qanday amalga oshiriladi?

VI BOB. PYTHONDA OB'YEKTGA MO'ljALLANGAN DASTURLASH ASOSLARI

6.1.8. Pythonda ob'yektga mo'ljallangan dasturlash

Ob'yektga yo'naltirilgan dasturlash (OYD) - tegishli xususiyatlar va xatti-harakatlarni alohida ob'yektlarga joylashtirish orqali dasturni tizimlashtirish metodi hisoblanadi. Ushbu bobda Pythonda ob'yektga yo'naltirilgan dasturlash asoslarining asosiy qismlari ko'rib chiqilgan. Har doimgiday ushbu OYD bo'lim yangi o'rganuvchilar uchun biroz tushunarsiz bo'lib kelgan. Ammo bu qo'llanmada OYD tushunchalarini sodda va izchil tushuntirishga harakat qilingan, shunda yosh Python dasturchilari ushbu qo'llanmani o'qiganlaridan so'ng o'z ob'yektlarini yaratishlari va yangi sinflarni yozishni boshlashlari mumkin.

Konseptual jihatdan ob'yektlar tizim komponentlariga o'xshaydi. Dasturni zavodning o'ziga xos yig'ish konveyeri sifatida tasavvur qiling. Noveyerning har bir bosqichida tizimning tarkibiy qismi ba'zi materiallarni qayta ishlaydi va oxir-oqibat xom ashyoni tayyor mahsulotga aylantiradi.

Ob'yektda yig'ish konveyerining har bir bosqichida oldindan ishlangan materiallar, shuningdek, odatiy xatti-harakatlar (masalan, yig'ish liniyasining har bir komponenti bajaradigan harakatlar) kabi ma'lumotlar mavjud.

Pythonda ob'yektga yo'naltirilgan dasturlash nima?

Ob'yektga yo'naltirilgan dasturlash - bu dasturlash paradigmasi bo'lib, dasturlarni xususiyatlar va xatti-harakatlar alohida ob'yektlarga birlashtiriladigan tarzda tuzilishiga imkon beradi.

Masalan, predmet ism, yosh va manzil kabi xususiyatlarga ega, yurish, gapirish, nafas olish, yugurish kabi xulq-atvor xususiyatlariga ega bo'lgan shaxsni ifodalashi mumkin.

Boshqacha qilib aytganda, ob'yektga yo'naltirilgan dasturlash - bu avtomobillar kabi aniq, haqiqiy narsalarni, shuningdek, kompaniyalar va xodimlar, talabalar va o'qituvchilar va boshqalar o'rtasidagi munosabatlarni modellashtirishga yondashuv. OYD real dunyo ob'yektlarini ular bilan bog'langan ba'zi ma'lumotlarga

ega bo'lgan va muayyan funktsiyalarni bajara oladigan dasturiy ob'yektlar sifatida modellashtiradi.

Yana bir keng tarqalgan dasturlash paradigmasi protsessual dasturlash bo'lib, u dasturni retsept kabi tuzadi, vazifani bajarish uchun ketma-ket bajariladigan funktsiyalar va kod bloklari ko'rinishidagi qadamlar to'plamini ta'minlaydi.

Asosiy nuqta shundaki, ob'yektlar Pythonda ob'yektga yo'naltirilgan dasturlashning markazida bo'lib, ular nafaqat protsessual dasturlashda bo'lgani kabi, balki ma'lumotlarni ham ifodalaydi, balki umumiy dastur tuzilmasida ham asosiy ro'l o'ynaydi.

Nazorat savollari:

1. Ob'yektga yo'naltirilgan dasturlash deganda nimani tushunasiz?
2. Python bilan boshqa dasturlash tillaridagi ob'yektga yo'naltirilgan dasturlash qanday farqi mavjud?

6.2.8. Pythonda classlar va ob'yektlar

Pythonda dasturda foydalanish mumkin bo'lgan int, str va shunga o'xshash ko'plab o'rnatilgan turlar mavjud. Ammo Python, sinflar yordamida o'z tiplarimizni aniqlashga imkon beradi.

Quyidagi o'xshashlikni ham o'tkazish mumkin. Har birimiz ismi-sharifi, yoshi, va shu kabi boshqa xususiyatlarga ega bo'lgan inson haqida qandaydir tasavvurga egamiz. Inson muayyan harakatlarni bajarishi mumkin: yurish, yugurish, o'ylash va hokazo. O'ziga xos xususiyatlar va harakatlar majmuasini o'z ichiga olgan bu ko'rinishni **class** deb atash mumkin. Aniq bir xususiyatlarni o'zida jamlagan bu class obyektlar uchun shablon xisoblanadi va turli ob'yektlar bir-biri bilan farq qiladi. masalan: bir odamning ism bunaqa bo'lsa, boshqalari boshqa ismga ega.

Sinf class kalit so'zi yordamida aniqlanadi:

```
class sinf_nomi:
    sinf_atributlari
    class_methods
```

Sinf ichida uning atributlari aniqlanadi, ular sinfning turli xususiyatlarini saqlaydi va metodlar sinfning funktsiyalari vazifasini bajarishadi.

Keling, oddiy sinf yarataylik :

```
class Person:
    pass
```

Bunda shartli ravishda shaxsni ifodalovchi Person sinfi aniqlanadi. Bunday holda, sinfda hech qanday metodlar yoki atributlar aniqlanmaydi. Biroq, unda biror narsa aniqlanishi kerakligi sababli, pass iborasi sinf funktsionalligi o'rnini bosuvchi sifatida ishlatiladi. Ushbu bayonot sintaktik jihatdan ba'zi kodlarni aniqlash uchun zarur bo'lganda ishlatiladi, lekin biz buni xohlamaymiz va ma'lum bir kod o'rniga biz pass bayonotini kiritamiz.

Sinf yaratilgandan so'ng, ushbu sinf ob'yektlarini aniqlash mumkin. Masalan :

```
class Person:
    pass

tomson = Person()      # tomson ob'yektini aninqlash
bobbi = Person()       # bobbi ob'yektini aninqlash
```

Person sinfi aniqlangandan so'ng, Person sinfining ikkita - tomson va bobbi ob'yektlari yaratildi. Klass ob'yektini yaratish uchun maxsus funktsiyadan - konstruktordan foydalaniladi. U class nomi bilan chaqiriladi va class ob'yektini qaytaradi. Ya'ni, bu holda, Person() chaqiruvi konstruktor chaqiruvini ifodalaydi. Har bir yaratilgan classda parametrlarsiz standart konstruktor mavjud bo'ladi:

```
tomson = Person()  # Person() - bu Person sinfining ob'yektini
qaytaruvchi konstruktor chaqiruvi
```

Class methodlari

Class metodlari aslida class ichida belgilangan va uning xususiyatlarini belgilaydigan funktsiyalarni ifodalaydi. Masalan, bitta metod bilan Person sinfini aniqlaylik:

```
class Person:          # Person klasining aniqlanishi
    def say_hello(self):
        print("Salom")
```

```
tom = Person()
tom.say_hello()      # Salom
```

Bu yerda **say_hello()** metodi aniqlandi, u shartli ravishda Salom xabarini konsolga chop etadi. Classga tegishli har qanday metodlarini aniqlashda shuni yodda tutish kerakki, ularning barchasi birinchi parametr sifatida **self** kalit so'zini qabul qilishadi. Self bu o'zi degan ma'noni anglatadi va ushbu klasga tegishli ekanligini bildiradi. Ammo metod chaqirilganda, bu marametr hisobga olinmaydi hamda qiymat berilmaydi.

ob'yekt nomidan foydalanib, classning metodlariga murojaat qilish mumkin. Mavjud metodlarni chaqirish uchun nuqta belgisi qo'llaniladi - ob'yekt nomidan keyin nuqta qo'yiladi va undan keyin metod chaqiruvi keladi:

```
Ob'yekt.metod([metod qabul qiladigan parametrlar])
```

Masalan, konsolga salomlashishni chop etish uchun say_hello() metodini chaqirish:

```
tomson.say_hello()      # Salom
```

Natijada, ushbu dastur konsolga "Salom" qatorini chop etadi.

Agar metod self dan tashqari boshqa parametrlarni qabul qiladigan bo'lsa, ular self parametridan keyin belgilanadi va bunday metodni chaqirishda, ushbu parametrlariga qiymatlar berish talab qilinadi:

```
class Person:           # Person klasining aniqlanishi
    def say(self, msg):   # method
        print(msg)

tom = Person()
tom.say("Hello , HOW ARE YOU?") # Hello , HOW ARE YOU?
```

Say() metodi bu yerda aniqlanadi. Bu metod ikkita parametr oladi: **self** va **msg**. Va ikkinchi parametr uchun - **msg**, metodni chaqirganda ushbu parametrga qiymat berilishi kerak.

Self kalit so‘zi orqali classning atributi va funksiyalariga murojaat qilish mumkin:

```
self.attribute      # methodning atributiga murojaat qilish  
self.method         # method chaqiruvi
```

Masalan, Person sinfida ikkita metodni aniqlaymiz :

```
class Person:  
    def say(self, msg):  
        print(msg)  
  
    def say_hello(self):  
        self.say("Hello JOB")  
  
tom = Person()  
tom.say_hello()      # Hello work
```

Bu yerda **say_hello()** metodi ichida turib **say()** metodiga murojaat amalga oshirilmoqda

```
self.say("Hello JOB")
```

Say() metodi o‘ziga qo‘shimcha ravishda parametrlarni (msg parametrini) qabul qilganligi sababli, metod chaqirilganda ushbu parametr uchun qiymat berish kerak.

Bundan tashqari, metod ichida turib, ob’yekt metodini chaqirganda, self kalit so‘zini ishlatishimiz kerak. Aks holda xatolik kelib chiqadi:

```
def say_hello(self):  
    say("Hello job")    # Xatolik
```

Konstruktorlar. Konstruktor sinf ob’yektini yaratish uchun ishlatiladi. Shunday qilib, yuqorida, Person sinfining ob’yektlarini yaratganda, hech qanday parametrlarni qabul qilmaydigan va barcha sinflar bilvosita ega bo‘lgan standart konstruktordan foydalanildi:

```
tom = Person()
```


Biroq, `__init__()` deb nomlangan maxsus metod yordamida sinflarda konstruktorni aniq belgilash mumkin (har bir tomonda ikkita past chiziqcha). Masalan, keling, Person sinfini unga konstruktor qo‘shish orqali o‘zgartiramiz:

```
class Person:
    # konstruktor
    def __init__(self):
        print("Person obyektining yaratildi")

    def say_hello(self):
        print("Hello")

tom = Person()          # Person obyekti yaratildi
tom.say_hello()         # Hello
```

Demak, bu yerda Person sinfining kodida konstruktor va say_hello () metodi aniqlangan. Birinchi parametr sifatida konstruktor ham metodlar kabi – **self** ni qabul qiladi. Odatda konstruktorlar obyekt yaratilganda bajariladigan amallarni aniqlash uchun ishlatiladi:

```
tom = Person()
```

Person sinfidagi `__init__()` konstruktori chaqiriladi, u konsolga " Person obyekti yaratildi " matnini chop etadi.

Ob'yekt atributlari. Atributlar ob'yekt holatini saqlaydi. Class ichidagi atributlarni aniqlash va o‘rnatish uchun **self** so‘zidan foydalanish mumkin. Masalan , quyidagi Person sinfini aniqlaymiz :

```
class Person:

    def __init__(self, nom):
        self.nom = nom      # shaxning nomi
        self.yosh = 1       # shaxsning yoshi
```

```

tomson = Person("Tomson")

# Atributga murojaat
# qiymatni qabul qilish
print(tomson.nom)          # Tom
print(tomson.yosh)         # 1

# Qiymatni o'zgartirish
tomson.yosh = 37
print(tomson.yosh)         # 37

```

Endi Person klassi konstruktori yana bitta parametr - **nom** ni qabul qiladi. Ushbu parametr orqali yaratilgan person ob'yektining nomi konstruktorga uzatiladi.

Konstruktor ichida ikkita atribut o'rnatilgan – **nom** va **yosh** (shartli ravishda, shaxsning ismi va yoshi):

```

def __init__(self, nom):
    self.nom = nom
    self.yosh = 1

```

self.nom atributi **nom** o'zgaruvchisiga tenglashtirib qo'yildi. **yosh** atributining boshlang'ich qiymati 1 ga teng deb qabul qilindi.

Agar sinfda **__init__** konstruktoridan foydalanilgan bo'lsa, endi standart konstruktorni chaqira olmaymiz. Endi biz aniq belgilangan **__init__** konstruktorimizni chaqirishimiz kerak, unga **nom** parametri uchun qiymat berilishi kerak:

```

tomson = Person("Tomson")

```

Keyinchalik ushbu nom orqali ob'yektning atributlariga murojaat qilib, uni olish va o'zgartirish mumkin:

```

print(tomson.nom) # nom atributining qiymatini olish
tomson.yosh = 37 # yosh atributining qiymatini o'zgartirish

```

Asosan, class ichida atributlarni aniqlamasdan uni ob'yektning o'ziga biriktirib qo'yish mumkin. Python bu atributni dinamik ravishda ob'yektga qo'shishga imkon beradi:

```

class Person:

```

```

def __init__(self, nom):
    self.nom = nom      # shaxsning nomi
    self.yosh = 1       # shaxsning yoshi

tomson = Person("Tomson")
tomson.company = "Microsoft"
print(tomson.company)  # Microsoft

```

Bu yerda kompaniya atributi dinamik ravishda oʻrnatildi. Bu atribut person obʼyektining ish joyini oʻzida saqlaydi. Va oʻrnatgandan soʻng, uning qiymatiga murojaat qilib, qiymatini olish mumkin. Shuni esda turish kerakki, agar atributni aniqlashdan oldin unga murojlat qilishga harakat qilinsa, dastur xatolik qaytaradi:

```

tomson = Person("Tomson")
print(tomson.company)  # ! Xatolik - AttributeError:
# Person object has no attribute company

```

Sinf ichidagi obʼyektning atributlariga murojaat qilish uchun uning metodlarida self soʻzi ham qoʻllaniladi:

```

class Person:
    def __init__(self, nom):
        self.nom = nom      # shaxsning nomi
        self.yosh = 1       # shaxsning yoshi

    def display_info(self):
        print(f"Nomi: {self.nom}   Yoshi: {self.yosh}")

tomson = Person("Tomson")
tomson.display_info()      # Nomi: Tomson   Yoshi: 1

```

Bu konsolga maʼlumotlarni chop etadigan **display_info()** metodini belgilaydi. Metoddagi obʼyekt atributlariga murojaat qilish uchun esa **self** soʻzi ishlatiladi: **self.nom** va **self.yosh**

Obʼyektlarni yaratish. Yuqorida bitta obʼyekt yaratilgan. Ammo shunga oʻxshash tarzda class ning boshqa obʼyektlarni yaratish mumkin:

```

class Person:
    def __init__(self, nom):
        self.nom = nom      # shaxsning nomi
        self.yosh = 1       # shaxsning yoshi
    def display_info(self):
        print(f"Nomi: {self.nom}   Yoshi: {self.yosh}")

tomson = Person("Tomson")
tomson.yosh = 37
tomson.display_info()      # Nomi: Tomson   yoshi: 37

bobbi = Person("Bobbi")
bobbi.yosh = 41
bobbi.display_info()      # Nomi: Bobbi   Yosh: 41

```

Ushbu dasturda Person sinfining ikkita obyekti: tomson va bobbi yaratilgan. Ushbu ob'yektlar Person sinfga tegishli bo'lib, ular bir xil atributlar va metodlar to'plamiga egalar. Biroq ular bir-biridan xususiyatlari bilan farq qilishadi.

Dastur bajarilganda, Python self parametri dinamik ravishda aniqlanadi - bu chaqiriladigan ob'yektni ifodalaydi. Masalan, satrda:

```

tomson.display_info()      # Nomi: Tomson   yoshi: 37
bobbi.display_info()      # Nomi: Bobbi   Yosh: 41

```

Nazorat savollari:

1. Pythonda ob'yekt tushunchasiga ta'rif bering.
2. Pythonda class tushunchasiga tarif bering.
3. Classning methodlari qanday yaratiladi?
4. Classga murojaat qanday amalga oshiriladi?
5. Class ob'yekti qanday hosil qilinadi?
6. Metodga qanday murojaat qilinadi?

6.3.§. Pythonda “Inkapsulyatsiya” tushunchasi

Odatda class atributlari hammaga kirish uchun ruxsat berilgan bo‘ladi. Ya’ni dasturning istalgan joyidan ob’yektning atributini chaqirish va uni o‘zgartirishga imkon berilgan bo‘ladi. Masalan :

```
class Person:

    def __init__(self, nom):

        self.nom = nom           # nomini o'rnatish

        self.yosh = 1           # yoshni o'rnatish

    def display_info(self):

        print(f"Nomi: {self.nom} \Yoshi: {self.yosh}")

tomson = Person("Tomson")

tomson.nom = "O'rgimchak odam"
# nom atributini #o'zgartirish

tomson.yosh = -125           # yosh atributining o'zgartirilishi
tomson.display_info()       # Nom: O'rgimchak odam Yosh: -125
```

Ammo bu holda, Person class atributlari **yosh** yoki **nom** ga noto‘g‘ri qiymat berilishi mumkin, masalan, manfiy yosh ko‘rsatilishi mumkin. Bunday qiymat berilishi ma’qul bo‘lmaydi. Shuning uchun ob’yekt atributlariga kirishni nazorat qilishga zarurat paydo bo‘ladi.

Inkapsulyatsiya tushunchasi bu muammo bilan chambarchas bog‘liq bo‘lib, ob’yektga yo‘naltirilgan dasturlashda asosiy tushuncha hisoblanadi. Inkapsulyatsiya class atributlariga to‘g‘ridan-to‘g‘ri kirishni nazorat qiladi.

Python dasturlash tilida bevosita inkapsulyatsiya, class atributlarini maxfiy yoki shaxsiy qilib yashirish va ularga xossalar deb ataladigan maxsus metodlar orqali kirishni cheklashga imkon beradi.

Yuqorida keltirilgan dasturni inkapsulyatsiyadan foydalanib qayta terib chiqamiz:

```

class Person:
    def __init__(self, nom):
        self.__nom = nom # nomni o'rnatish
        self.__yosh = 1 # yoshni o'rnatish

    def set_age(self, yosh):
        if 1 < yosh < 110:
            self.__yosh = yosh
        else:
            print("Ruxsat etilgan yosh")

    def get_yosh(self):
        return self.__yosh

    def get_nom(self):
        return self.__nom

    def display_info(self):
        print(f"Nom:{self.__nom}\tYosh:{self.__yosh}")

tomson = Person("Tomson")
tomson.display_info() # Nom: Tomson Yosh: 1
tomson.set_yosh(-3486) # Ruxsat etilmagan yosh
tomson.set_yosh(25)
tomson.display_info() # Nom: Tomson Yosh: 25

```

Shaxsiy atribut yaratish uchun uning oldiga qo'sh past chiziqcha qo'yiladi: **self.__nom**. Bunday atributga faqat bir class dan kirish mumin. Ya'ni ushbu classdan tashqarida murojaatni amalga oshirishning ilojisi bo'lmaydi. Masalan, ushbu **yosh** atributiga classdan tashqarida turib qiymat berilgani hech narsani bermaydi:

```
tomson.__yosh = 43
```

Chunki bu holda yangi **__yosh** atributi dinamik tarzda tashqaridan aniqlanadi, ammo ushbu **__yosh** atributi **self.__yosh** atributiga hech qanday aloqasi yo'q.

Va uning qiymatini olishga urinish xatolik kelib chiqishiga sabab bo'ladi (agar **__yosh** o'zgaruvchisi avval aniqlanmagan bo'lsa):

```
print(tomson.__yosh)
```

Biroq, hali ham foydalanuvchining yoshini tashqaridan belgilash kerak bo'lishi mumkin. Xususiyatlar aynan shu maqsadda ishlatiladi. Bitta xususiyatdan foydalanib, atributning qiymatini olish mumkin:

```
def get_yosh(self):  
    return self.__yosh
```

GETTER metodi

Yoshni o'zgartirish uchun boshqa xususiyat belgilangan:

```
def set_yosh(self, yosh):  
    if 1 < yosh < 110:  
        self.__yosh = yosh  
    else:  
        print("Noto'g'ri yosh")
```

Bu **setter** metodi yana **mutator** deb ham ataladi. Bu yerda shart asosida, yosh atributi qiymatini qayta o'rnatish kerakmi yoki yo'qligini hal qilish mumkin.

Har bir xususiyy atribut uchun bunday juft xususiyatlarni yaratish shart emas. Shunday qilib, yuqoridagi misolda shaxsning ismini faqat konstruktordan belgilash mumkin. Va qiymatini olish **get_nom** metodi orqali amalga oshiriladi.

Yuqorida xususiyatlarni qanday yaratishni ko'rib chiqildi. Ammo Python ham xususiyatlarni aniqlashning yana bir, yanada optimalroq metodiga ega. Bu usul **@** belgisidan foydalanish orqali amalga oshiriladi.

Getter xususiyatini yaratish uchun uning tepasiga, ya'ni tepa qatoriga

@property annotatsiyasi qo'yiladi .

Keling, annotatsiyalar yordamida Person class ini qayta yozamiz:

```
class Person:  
    def __init__(self, nom):  
        self.__nom = nom    # Nomni o'rnatish  
        self.__yosh = 1     # yoshni o'rnatish
```

```

@property
def yosh(self):
    return self.__yosh

@yosh.setter
def yosh(self, yosh):
    if 1 < yosh < 110:
        self.__yosh = yosh
    else:
        print("Noto'g'ri yosh")

@property
def nom(self):
    return self.__nom

def display_info(self):
    print(f"Nomi:{self.__nom}\tYoshi:{self.__yosh}")

tomson = Person("Tom")

tomson.display_info()    # Nomi: Tomson  Yoshi: 1
tomson.yosh = -3486      # Noto'g'ri yosh
print(tomson.yosh)       # 1
tomson.yosh = 36
tomson.display_info()    # Nomi: Tomson  Yoshi: 36

```

Birinchidan, **setter** xususiyati **getter** xususiyatidan keyin aniqlanganligiga etibor qaratish kerak.

Ikkinchidan, **setter** ham, **getter** ham bir xil nomlanadi: **yosh**. Qabul qiluvchi yoshi deb atalganligi sababli, **@age.setter** izohi sozlagich ustida o'rnatiladi .

Shundan so'ng **setter** ga ham, **getter** ga ham **tomson.yosh** ifodasi orqali murojaat qilinishi mumkin.

Nazorat savollari:

1. Pythonda "Inkapsulyatsiya" tushunchasiga tarif bering;
2. Inkapsulyatsiya qanday amalga oshiriladi?
3. Getter metodi nima?
4. Setter metodi nima
5. @ - setter va getter orqali ishlaganda qanday vazifani bajaradi?

6.4.§. Python tilidagi "Merosxo'rlik" tushunchasi, atributlari va xossalari

OOP da merosxo'rlik konsepsiyasi mavjud class asosida yangi class yaratish imkonini beradi. Inkapsulyatsiya bilan bir qatorda merosxorlik ob'yektga yo'naltirilgan dasturlashning asoslaridan hisoblanadi.

Merosxorlik konsepsiyasining asosiy tushunchalari: **subklass** va **superclass** lardir. **Subklass** barcha umumiy atributlar va metodlarni yuqori classdan meros qilib oladi. Shuningdek superclass, asosiy (**base class**) yoki ota (**parent class**) deb atalsa, subclass - voris class (**derived class**) yoki bola class (**child class**) deb ataladi.

Merosxorlik classini yaratish sintaksisi quyidagicha ko'rinishda bo'ladi:

```
class subclass (superclass):  
    subclass_metodlari
```

Masalan, shaxsni ifodalovchi Person sinfi mavjud bo'lsin:

```
class Person:  
    def __init__(self, nom):  
        self.__nom = nom  
  
    @property  
    def nom(self):  
        return self.__nom
```

```
def display_info(self):  
    print(f"Nomi: {self.__nom} ")
```

Aytaylik, bizga qandaydir korxonada ishlaydigan ishchilar classi kerak. Biz noldan yangi sinf yaratishimiz mumkin, masalan, *Employee* sinfi:

```
class Employee:  
    def __init__(self, nom):  
        self.__nom = nom  
  
    @property  
    def nom(self):  
        return self.__nom  
  
    def display_info(self):  
        print(f"Nomi: {self.__nom} ")  
  
    def work(self):  
        print(f"{self.nom} works")
```

Biroq, Employee classi *Person* classi bilan bir xil atribut va metodlarga ega bo'lishi mumkin, chunki xodimlar ham inson classiga mansub ob'yeklar hisoblanadi. Shunday qilib, yuqoridagi ***Employee*** sinfida faqat ***work*** metodi qo'shiladi va qolgan kod *Person* sinfining funksionalligidan nusxalab olinadi. Agar merosxorlikda foydalanilmasa, bir xil funktsionallik ikkala sinfda ham takrorlanaveradi. Bunday holda merosxorlik konsepsiyasidan foydalanish ancha kodni tejaliishiga olib keladi.

Shunday qilib, ***Person*** sinfidan ***Employee*** sinfini meros qilib olaylik :

```
class Person:  
    def __init__(self, nom):  
        self.__nom = nom
```

```

@property
def nom(self):
    return self.__nom

def display_info(self):
    print(f"Nomi: {self.__nom} ")

class Employee(Person):
    def work(self):
        print(f"{self.nom} works")

tomson = Employee("Tomson")
print(tomson.nom)          # Tomson
tomson.display_info()     # Nomi: Tomson
tomson.work()              # Tomson works

```

Employee sinfi **Person** sinfining funksiyalarini to‘liq o‘ziga nusxalab oladi, faqat **work()** metodi *Employee* sinfiga qo‘shimcha tarzda qo‘shiladi. Shunga ko‘ra, *Employee* sinfi ob‘yektini yaratishda **Person** dan meros qilib olingan konstruktordan foydalanish mumkin :

```
tomson = Employee ("Tomson")
```

Shuningdek, meros qilib olingan atributlar/ xususiyatlar va metodlarga murojaat qilish mumkin:

```

print(tomson.nom)          # Tomson
tomson.display_info()     # Nomi: Tomson

```

Shuni yodda tutish kerakki, *Employee* sinfi uchun *Person* sinfining yopiq atributi bo‘lgan **__nom** atributiga kirishga ruxsat yo‘q. Masalan, **work()** metodi tarkibida **self.__nom** xususiy atributiga murojaat qila olmaymiz :

```
def work(self):  
    print(f"{self.__nom} works")    # Xatolik!
```

Ko‘p merosxorlik

Python tilining ajralib turadigan xususiyatlaridan biri - bu bir ko‘p merosxorlik konsepsiyasini qo‘llab-quvvatlashligidir. Ya’ni bitta sinf bir nechta sinflardan meros olishi mumkin:

```
class Employee:  
    def work(self):  
        print("Employee jobs")  
  
class Students:  
    def study(self):  
        print("Students studying")  
  
class WorkingStudents(Employee, Students):  
    pass  
  
tomson = WorkingStudent()  
tomson.job()          # Employee jobs  
tomson.study()        # Students studying
```

U firma xodimini ifodalovchi **Employee** sinfini va talabalarini ifodalovchi **Students** sinfini belgilaydi. Ishlayotgan talabani ifodalovchi **WorkingStudents** sinfi tarkibida hech qanday metod va atributlar aniqlanmaganligi **pass** operatori bilan to‘ldirib qo‘yilgan holos.

WorkingStudents sinfi ikkita sinfdan **Employee** va **Students** funktsionallikni meros qilib oladi. Shunday ekan ushbu sinf ob’yektida ikkala sinfning metodlarini chaqirish imkoniyati mavjud.

Shu bilan birga, meros qilib olingan sinflar funktsional jihatdan murakkabroq bo‘lishi mumkin, masalan:

```

class Employee:
    def __init__(self, nom):
        self.__nom = nom

    @property
    def nom(self):
        return self.__nom

    def work(self):
        print(f"{self.nom} works")

class Students:
    def __init__(self, nom):
        self.__nom = nom

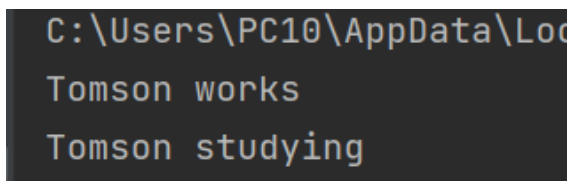
    @property
    def nom(self):
        return self.__nom

    def study(self):
        print(f"{self.nom} studying")

class WorkingStudents(Employee, Students):
    pass

tomson = WorkingStudents("Tomson")
tomson.work()
tomson.study()

```



```

C:\Users\PC10\AppData\Loc
Tomson works
Tomson studying

```

22-rasm. Dastur natijasi.

Nazorat savollari:

1. Merosxorlik tushunchasiga tarif bering;
2. Pythonda merosxorlik qanday amalga oshiriladi?
3. Inkopsulyatsiya nima maqsadda ishlatiladi?
4. Merosxor classning qanday afzalliklari mavjud?
5. Subclass va Superclass tushunchasiga tarif bering.

VII BOB. PYTHONDA MA'LUMOTLAR BAZASI. SQLITE ASOSLARI

7.1.§. SQLite ning kelib chiqishi

SQLite - bu ma'lumotlar bazasi mexanizmi bo'lib, uning negizi C dasturlash tilida yozilgan va Hwachi kompaniyasi tomonidan qo'llab-quvvatlanadi. Bu alohida ilova emas, bu oddiy bir kutubxona bo'lib, dastur ishlab chiqilayotganda uni chaqirib foydalaniladi. Shunday qilib, u standart ma'lumotlar bazalari oilasiga tegishli. SQLite eng ko'p ishlatiladigan ma'lumotlar bazasi mexanizmi bo'lib, u bir nechta etakchi veb-brauzerlar, operatsion tizimlar, mobil telefonlar va boshqa tizimlar tomonidan ko'lamli foydalaniladi.

Kelib chiqish tarixi. D. Richard Xipp 2000 yilning bahorida General Dynamics kompaniyasida AQSh harbiy-dengiz floti shartnomasi bo'yicha ishlagan vaqtida SQLite-ni ishlab chiqdi. Hipp dastlab IBM Informix ma'lumotlar bazasi bilan HP-UX-dan foydalanilgan, boshqariladigan raketa esminetslarida zararni nazorat qilish tizimi uchun ishlatiladigan dasturiy ta'minotni ishlab chiqdi. O'sha paytlarda SQLite Tcl ning kengaytmasi sifatida boshlangan.

2000 - yilning avgust oyida SQLite ning birinchi 1.0 versiyasi gdbm (GNU ma'lumotlar bazasi menejeri) asosida saqlash bilan chiqarildi. 2001-yil sentabrida SQLite 2.0 tranzaksiya qobiliyatini qo'shib, gdbm ni o'zining B-daraxt dasturi bilan almashtirdi. 2004 yil iyun oyida SQLite 3.0 xalqarolashtirish, manifest yozish va qisman America Online tomonidan moliyalashtirilgan boshqa muhim funksiyalarni qo'shdi. 2011-yilda Hipp SQLite-ga NoSQL interfeysini qo'shish rejalarini e'lon qildi, shuningdek, ma'lumot almashishga yo'naltirilgan ma'lumotlar bazalari uchun mo'ljallangan SQLning funktsional superto'plami bo'lgan UnQLni e'lon qildi. 2018-yilda SQLite Avliyo Benedikt qoidasiga asoslangan xulq-atvor kodeksini qabul qildi, bu ba'zi bahs-munozaralarga sabab bo'ldi va keyinchalik axloq kodeksi deb o'zgartirildi.

SQLite - ma'lumotlar to'plamini uzoq muddatli saqlash uchun tavsiya etilgan to'rtta formatdan biri bo'lib, Kongress kutubxonasi tomonidan foydalanish uchun tasdiqlangan.

Nazorat savollari:

1. SQLite qanday ma'lumotlar bazasi turiga kiradi?
2. SQLite ma'lumotlar bazasining yaratilish tarixi qanday?
3. SQLite ning hozirga qanday versiyalari mavjud?
4. SQLite qaysi dasturlash tilida yozilgan?

7.2.8. SQLite moduli bilan tanishish

Ushbu bobda SQLite ma'lumotlar bazasidan foydalangan holda Python ilovalarini qanday ishlab chiqishni ko'rsatib o'tilgan.

Pythonda SQLitega ulanish. Ushbu bo'limda **sqlite3** moduli yordamida ma'lumotlar bazasini qanday yaratish va unga ulanishni ko'rib chiqiladi.

Ma'lumotlar bazasi bilan bog'lanish uchun ma'lumotlar bazasi nomini ko'rsatish kerak. Agar mavjud bo'lgan, ya'ni oldindan yaratilgan ma'lumotlar bazasi nomini ko'rsatilsa, u holda ushbu MB ga ulanish sodir bo'ladi. Aks holda, ya'ni ko'rsatilgan nomdagi baza mavjud bo'lmasa, unda SQLite ko'rsatilgan nomda yangi ma'lumotlar bazasini yaratadi.

SQLitega ulanish uchun quyidagi amallarni bajariladi:

- **sqlite3** modulining **connect()** metodidan foydalaniladi va ma'lumotlar bazasi nomini ushbu metodga argument sifatida beriladi;
- Python dan SQLite so'rovlarini bajarishning oldingi usulini qaytaradigan **connect** ob'yekti yordamida **cursor** ob'yektini yaratiladi;
- Ish tugagach, **cursor** ob'yektini yopish;
- Agar ulanish jarayonida xatolik yuzaga kelsa, ma'lumotlar bazasi xatoligini ushlab.

Quyidagi dastur **sqlite_python.db** ma'lumotlar bazasi faylini yaratadi va SQLite versiyasining tafsilotlarini chop etadi.

```
import sqlite3
try:
    sqlite_connection = sqlite3.connect('sqlite_python.db')
    cursor = sqlite_connection.cursor()
    print("Ma'lumotlar bazasi yaratildi va SQLitega
muvaffaqiyatli ulandi")
```

```

sqlite_select_query = "select sqlite_version()"
cursor.execute(sqlite_select_query)
record = cursor.fetchall()
print("SQLite ma'lumotlar bazasi versiyasi: ", record)
cursor.close()

except sqlite3.Error as error:
    print("sqlite ga bog'lanishda xatolik", error)
finally:
    if (sqlite_connection):
        sqlite_connection.close()
        print("SQLite ulanishi yopildi")

```

Ulangandan so'ng, quyidagi xabar paydo bo'lishi kerak.

```

Ma'lumotlar bazasi yaratildi va SQLite-ga muvaffaqiyatli ulandi
SQLite ma'lumotlar bazasi versiyasi:  [('3.31.1',)]
SQLite ulanishi yopildi

```

Yuqoridagi dasturni tahlili:

1) *import sqlite3*

- a. Ushbu qatorda `sqlite3` modulini dasturga import qilinmoqda. Bu modulning sinflari va metodlaridan foydalanib, SQLite ma'lumotlar bazasi bilan o'zaro aloqa qilish mumkin.

2) *sqlite3.connect()*

- a. ***sqlite3*** modulining ***connect()*** metodi yordamida ma'lumotlar bazasiga ulanish amalga oshiriladi. Bu metod SQLite ga ulanish ob'ektini qaytaradi.
- b. ***sqlite3*** moduli oqimlar orasidagi ulanishni almashishga ruxsat bermaydi. Agar buni qilishga harakat qilinsa, xatolik qaytadi.
- c. ***connect()*** metodi turli argumentlarni qabul qiladi. Ushbu misolda ma'lumotlar bazasi nomi berilgan.

3) *cursor=sqliteConnection.cursor()*

- a. Ulanish ob'yekti yordamida Python tilidan SQLite so'rovlarini bajarish imkonini beruvchi cursor ob'yektini yaratiladi;
- b. Bitta ulanish uchun cheksiz miqdordagi kursorlar yaratish mumkin. Bundan tashqari, ma'lumot oqimlari xavfsizligi ta'minlangan emas. Modul kursor ob'ektlarini oqimlar o'rtasida almashishga ruxsat bermaydi. Agar buni qilishga urunish holatlari bo'lsa, unda xato sodir bo'ladi.

Shundan so'ng, ma'lumotlar bazasi versiyasini olish uchun so'rov yaratiladi.

4) ***cursor.execute()***

- a. ***cursor*** ob'ektining ***execute()*** usulidan foydalanib, Python dasturidan ma'lumotlar bazasiga so'rovini bajarish mumkin. U parametr sifatida SQLite so'rovini qabul qiladi va ***resultSet*** - ma'lumotlar bazasi qatorlarini qaytaradi.
- b. So'rov natijasini ***resultSet*** metodi yordamida olish mumkin, masalan, ***fetchAll()***
- c. Ushbu dasturda ***SELECT version()*** - SQLite ma'lumotlar bazasi versiyasini olish vazifasini bajaradi.

5) ***try-except-finally***: Barcha kod try-except blokida joylashgan bo'lib, bu jarayonda yuzaga kelishi mumkin bo'lgan xatoliklar va ma'lumotlar bazasi xatolarini boshqarish imkonini beradi.

- a. ***sqlite3.Error*** sinfidan foydalanib, Python dan SQLite bilan ishlashda yuzaga kelishi mumkin bo'lgan har qanday xato va istisnolarni boshqarish mumkin.
- b. Bu esa ilovani xatolarga yanada chidamliroq bo'lishini ta'minlaydi. Xatoning mohiyatini tushunishga ***sqlite3.Error*** klassi yordam beradi. Ushbu class xatolik haqidagi xabarni va xatolik kodini qaytaradi.

6) ***cursor.close()*** va ***connection.close()***

- a. Ma'lumotlar bazasi bilan bog'liq muammolarni oldini olish uchun ish tugagach, **connection** va **cursor** ob'ektlarini close metodi orqali yopish kerak bo'ladi.

Pythonda SQLite jadvalini yaratish

Ushbu bo'limda Python va **sqlite3** moduli yordamida SQLite ma'lumotlar bazasida jadvallarni qanday yaratishni tushuntirilgan. Jadval yaratish - bu Python dan bajariladigan DDL so'rovidir.

Ushbu misolda **sqlite_python.db** ma'lumotlar bazasida **sqlitedb_developers** ma'lumotlar bazasini yaratiladi.

Python yordamida SQLite modulida jadval yaratishni qadamma-qadam ko'rib chiqamiz:

- Ma'lumotlar bazasiga **sqlite3.connect()** yordamida ulaniladi. Bu birinchi bo'limda muhokama qilib ketilgan edi.
- Jadval yaratish so'rovini tayyorlash.
- **cursor.execute(query)** yordamida so'rovni bajariladi.
- Ma'lumotlar bazasi bilan bog'lanishni va cursor ob'ekti bilan aloqani close metodi orqali yopiladi.

```
import sqlite3

try:
    sqlite_connection=sqlite3.connect('sqlite_python.db')
    sqlite_create_table_query = '''CREATE TABLE
sqlitedb_developers (
                                id INTEGER PRIMARY KEY,
                                name TEXT NOT NULL,
                                email text NOT NULL UNIQUE,
                                joining_date datetime,
                                salary REAL NOT NULL);'''
```

```

cursor = sqlite_connection.cursor()
print("Ma'lumotlar bazasi SQLitega ulandi")
cursor.execute(sqlite_create_table_query)
sqlite_connection.commit()
print("SQLite jadvali yaratildi")

cursor.close()

```

```

except sqlite3.Error as error:

```

```

    print("sqlitega ulanishda xatolik", error)

```

```

finally:

```

```

    if (sqlite_connection):
        sqlite_connection.close()
        print("SQLite ulanishi yopildi")

```

```

Ma'lumotlar bazasi SQLitega ulandi
SQLite jadvali yaratildi
SQLite ulanishi yopildi

```

Структура БД				
Данные				
Прагмы				
SQL				
Table: sqlitedb_developers				
Добавить запись				
Удалить запись				
id	name	email	joining_date	salary
Фильтр	Фильтр	Фильтр	Фильтр	Фильтр

23-Rasm. Sqlite ma'lumotlar bazasini yaratish.

SQLite ma'lumotlar turlari va Pythonda ma'lumotlar turini mosligi.

Python'dan SQLite CRUD operatsiyalarini bajarishga o'tishdan oldin, birinchi navbatda SQLite ma'lumotlar turlarini va jadvaldagi ma'lumotlarni saqlash va o'qishga yordam beradigan, ularga mos keladigan Python ma'lumotlar turlarini anglab olish kerak bo'ladi.

SQLite dvijokida qiymatlarni saqlash uchun bir nechta sinflar mavjud. Ma'lumotlar bazasida saqlangan har bir qiymat quyidagi tur yoki sinflardan biridir.

SQLite ma'lumotlar turlari:

- NULL — NULL qiymati
- INTEGER — raqamli qiymatlar. Butun sonlar qiymatga qarab 1, 2, 3, 4, 6 va 8 baytlarda saqlanadi.
- REAL – vergulli sonlar yoki kasr sonlar, masalan, 3.14, Pi
- TEXT — matn qiymatlari. UTF-8, UTF-16BE yoki UTF-16LE kodlashda saqlanishi mumkin
- BLOB — ikkilik ma'lumotlar. Rasmlar va fayllarni saqlash uchun

Python-dan quyidagi ma'lumotlar turlari osongina SQLite-ga o'zgartiriladi.

Konvertatsiya qilish uchun faqat ushbu jadvalni eslab qolish kerak:

7-jadval

Python turi	SQLite turi
None	NULL
int	INTEGER
float	REAL
str	TEXT
bytes	BLOB

EXECUTESCRIPT funksiyasi bilan SQL so'rovlarini bajarish

SQLite skriptlari umumiy vazifalar uchun juda yaxshi. Bu faylda (.sql formatida) saqlangan SQL buyruqlar to'plamidir. Bitta faylda bir yoki bir nechta SQL iboralari mavjud bo'lib, ular buyruq satridan bajariladi.

SQL skriptlarini ishlatish uchun bir nechta umumiy senariylar:

- Bir vaqtning o'zida bir nechta ma'lumotlar bazalarining zaxira nusxalarini yaratish.
- Ikki xil ma'lumotlar bazasining qatorlar sonini bitta sxema bilan solishtirish.
- Barcha jadvallarni bitta skriptda yaratish, bu istalgan serverda kerakli sxemani yaratish imkonini beradi

Skriptni SQLite buyruq satridan **.read** buyrug'i yordamida bajarish mumkin :

```

sqlite> .read sqlitescript.sql

CREATE TABLE fruits (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    price REAL NOT NULL
);

CREATE TABLE drinks (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    price REAL NOT NULL
);

```

Endi uni Python tilidan qanday bajarishni ko‘rib chiqamiz.

```

import sqlite3

try:
    sqlite_connection = sqlite3.connect('sqlite_python.db')
    cursor = sqlite_connection.cursor()
    print("SQLite-ga ulangan ma'lumotlar bazasi")

    with open('sqlite_create_tables.sql', 'r') as sqlite_file:
        sql_script = sqlite_file.read()

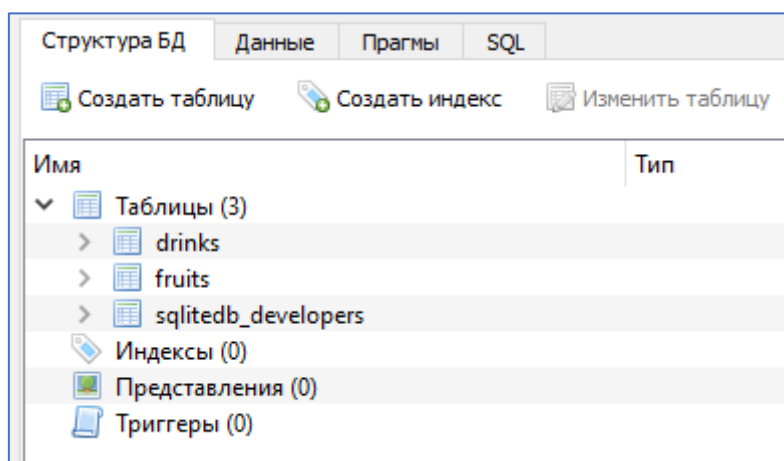
    cursor.executescript(sql_script)
    print("SQLite skripti muvaffaqiyatli yakunlandi")
    cursor.close()

except sqlite3.Error as error:
    print("sqlite ga bog'lanishda xatolik", error)

finally:
    if (sqlite_connection):
        sqlite_connection.close()

```

```
print("SQLite ulanishi yopildi")
```



24-rasm. *executescript* funksiyasi bilan qilingan SQL so‘rovi natijasi

SQLite jadvallari Python dan skriptni bajarish orqali yaratiladi. Xulosa:

```
SQLite-ga ulangan ma'lumotlar bazasi
SQLite skripti muvaffaqiyatli yakunlandi
SQLite ulanishi yopildi
```

Eslatma: SQLitega ulangandan so‘ng, faylning barcha tarkibi o‘zgaruvchida saqlanadi. Keyin ***cursor.executescript(script)*** buyrug‘i barcha ko‘rsatmalarni bir vaqtning o‘zida bajarish uchun ishlatiladi.

SQLite ma'lumotlar bazasi istisnolari

- **sqlite3.Warning.** Istisno Exception sinfiga tegishli bo‘lib, agar kerak bo‘lsa, dastur bajarilishini to‘xtatmasligi uchun uni e‘tiborsiz qoldirish mumkin.
- **sqlite3.Error.** Ushbu istisno Exception sinfiga tegishli bo‘lib, sqlite3 modulining boshqa istisnolari uchun asosiy class hisoblanadi.
- **sqlite3.DatabaseError.** Ma'lumotlar bazasi xatolarida qaytariladigan istisnodir. Misol uchun, agar faylni sqite3 ma'lumotlar bazasi sifatida ochishga harakat qilinsa, u mavjud bo‘lmasada, u holda "*sqlite3.DatabaseError: file is encrypted or is not a database*" xatosi qaytariladi.
- **sqlite3.IntegrityError.** bu istisno DatabaseError sinfiga tegishli va ma'lumotlar bazasidagi munosabatlarga ta'sir qilganda qaytariladi, Masalan, chet el kalitini tekshirish muvaffaqiyatsiz tugaganda.

- **sqlite3.ProgrammingError.** DatabaseError sinfining subclassi bo'lib, ushbu xato dasturchi xatolari tufayli yuzaga keladi. Masalan: allaqachon band qilingan nom bilan jadval yaratish, SQL so'rovlarida sintaksis xatosini keltirib chiqaradi.

- **sqlite3.OperationalError.** DatabaseError sinfining subclassi bo'lib, bu xatoni nazorat qilib bo'lmaydi. Bu ma'lumotlar bazasining ishlashi bilan bog'liq bo'lgan vaziyatlarda paydo bo'ladi, masalan, uzilgan ulanish, bo'sh server, ma'lumotlar manbai bilan bog'liq muammolar va boshqalar.

- **sqlite3.NotSupportedError.** Ushbu istisno ma'lumotlar bazasi tomonidan qo'llab-quvvatlanmaydigan API-dan foydalanishga harakat qilganda yuzaga keladi. Misol: tranzaktsiyalarni qo'llab-quvvatlamaydigan ulanishda rollback() usulini chaqirish. Jadval yaratish buyrug'idan keyin topshiriqni chaqirish.

Shunday qilib, har doim ma'lumotlar bazasini boshqarish kodini try blokiga yozish tavsiya etiladi, shunda istisnolarni ushlab qolish va ularni engishga yordam beradigan harakatlar qilish mumkin bo'ladi.

Misol uchun, mavjud bo'lmagan jadvalga ma'lumotlarni qo'shishga harakat qilaylik va Pythonidan butun istisnolar stekini chop qilaylik.

```
import sqlite3
import traceback
import sys

try:
    sqlite_connection = sqlite3.connect('sqlite_python.db')
    cursor = sqlite_connection.cursor()
    print("SQLite-ga ulangan ma'lumotlar bazasi")

    sqlite_insert_query = """INSERT INTO unknown_table_1
                           (id, text) VALUES (1, Test
text')"""

    count = cursor.execute(sqlite_insert_query)
    sqlite_connection.commit()
    print("yozuv sqllitedb_developers jadvaliga muvoffaqiyatli
joylashtirildi", cursor.rowcount)
    cursor.close()

except sqlite3.Error as error:
    print("Sqlite jadvaliga ma'lumotlarni kiritib bo'lmadi")
```

```

        print("Istisno sinf: ", error.__class__)
        print("Istisno", error.args)
        print("SQLite istisno tafsilotlarini chop etish: ")
        exc_type, exc_value, exc_tb = sys.exc_info()
        print(traceback.format_exception(exc_type, exc_value,
exc_tb))
    finally:
        if (sqlite_connection):
            sqlite_connection.close()
            print("SQLite ulanishi yopildi")

```

```

SQLite-ga ulangan ma'lumotlar bazasi
Sqlite jadvaliga ma'lumotlarni kiritib bo'lmadi
Istisno sinf:
Istisno ('no such table: unknown_table_1')
SQLite istisno tafsilotlarini chop etish:
['Traceback (most recent call last):\n', '  File
"C:\\Users\\demo\\AppData\\Local\\Programs\\Python\\Python38\\sqli
tet.py", line 13, in \n      count =
cursor.execute(sqlite_insert_query)\n', 'sqlite3.OperationalError:
no such table: unknown_table_1\n']
SQLite ulanishi yopildi

```

Pythondan ma'lumotlar bazasiga ulanishda *timeout* o'zgarishi

SQLite ma'lumotlar bazasiga bir nechta ulanishlar mavjud bo'lib, ulardan biri ma'lum bir o'zgarishlarni amalga oshiradi. Buning uchun ulanish bloklanishi kerak. Ushbu holat ma'lumotlar bazasi tranzaksiya tugaguniga qadar davom etadi.

Ulanish paytida o'rnatilgan ***timeout*** parametri istisno qaytarilishidan oldin ulanish bloklanishning bekor qilinishini qancha vaqt davomida kutishini aniqlaydi.

Ushbu parametr uchun standart qiymat 5.0 (5 soniya) ga teng. Uni o'rnatish shart emas, chunki u standart qiymatdir. Shunday qilib, Pythondan ma'lumotlar bazasiga ulanishda, agar 5 soniya ichida javob olinmasa, istisno qaytariladi. Biroq, shunday bo'lsada, sqlite3.connect funktsiyasi orqali parametrغا qiymat berish mumkin. Keling, buni Python-dan qanday qilishni ko'rib chiqaylik.


```

import sqlite3

def read_sqlite_table():
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db',
        timeout=20)
        cursor = sqlite_connection.cursor()
        print("SQLite ga ulangan")

        sqlite_select_query = """SELECT count(*) from
sqlitedb_developers"""
        cursor.execute(sqlite_select_query)
        total_rows = cursor.fetchone()
        print("Jami qatorlar: ", total_rows)
        cursor.close()

    except sqlite3.Error as error:
        print("sqlitega ulanishda xatolik", error)
    finally:
        if (sqlite_connection):
            sqlite_connection.close()
            print("SQLite ulanishi yopildi")

read_sqlite_table()

```

Natija:

```

SQLite ga ulangan
Jami qatorlar:      (0,)
SQLite ulanishi yopildi

```

Ma'lumotlar bazasiga ulanishda o'zgarishlarni olish.

Statistik ma'lumotlar uchun ulanish ochilgandan beri **kiritilgan**, **o'chirilgan** yoki **o'zgartirilgan** ma'lumotlar bazasi qatorlari sonini topishni xohlashingiz mumkin. Buning uchun sqlite3 modulining connection.total_changes funksiyasidan foydalaniladi.

Ushbu usul **kiritilgan**, **o'chirilgan** yoki **o'zgartirilgan** qatorlarning umumiy sonini qaytaradi. Keling, bir misolni ko'rib chiqaylik .

```

import sqlite3

try:
    sqlite_connection = sqlite3.connect('sqlite_python.db')
    cursor = sqlite_connection.cursor()
    print("SQLite ga ulangan ")

    sqlite_insert_query = """INSERT INTO sqlitedb_developers
                        (id, name, email, joining_date, salary)
                        VALUES (4, 'Alex', 'sale@gmail.com',
'2020-11-20', 8600);"""
    cursor.execute(sqlite_insert_query)

    sql_update_query = """Update sqlitedb_developers set
salary = 10000 where id = 4"""
    cursor.execute(sql_update_query)

    sql_delete_query = """DELETE from sqlitedb_developers
where id = 4"""
    cursor.execute(sql_delete_query)

    sqlite_connection.commit()
    cursor.close()

except sqlite3.Error as error:
    print("SQLite bilan ishlashda xotolik", error)
finally:
    if (sqlite_connection):
        print("Ma'lumotlar bazasiga ulangandan so'ng
o'zgartirilgan jami qatorlar: ",
sqlite_connection.total_changes)
        sqlite_connection.close()
        print("SQLite ulanishi yopildi ")

```

SQLite ga ulangan

Ma'lumotlar bazasiga ulangandan so'ng o'zgartirilgan jami
qatorlar: 3

SQLite ulanishi yopildi

Ma'lumotlar bazasi zahira nusxasini saqlash

Python'dagi `sqlite3` moduli SQLite ma'lumotlar bazasining zaxira nusxasini saqlash funksiyasini ta'minlaydi. **`Connection.backup()`** usulidan foydalanib, siz SQLite ma'lumotlar bazasining zaxira nusxasini yaratishingiz mumkin.

```
connection.backup(target, *, pages=0, progress=None,  
name="main", sleep=0.250)
```

Bu xususiyat SQLite ma'lumotlar bazasining to'liq zaxira nusxasini yaratadi. O'zgarishlar maqsadli argumentga yoziladi, bu boshqa ulanishning namunasi bo'lishi kerak.

Odatda `pages` parametri 0 yoki manfiy qiymat qabul qilsa, butun ma'lumotlar bazasi bir bosqichda ko'chiriladi. Aks holda, usul bir vaqtning o'zida ko'rsatilgan sahifalarni sikl asosida ko'chirib oladi.

`name` argumenti zaxiralanadigan ma'lumotlar bazasini belgilaydi. **`sleep`** argumenti - qolgan sahifalarni saqlash uchun ketma-ket urinishlar orasidagi soniyalar sonini aniqlaydi. **`sleep`** argumenti butun son yoki suzuvchi nuqta sonlarni qabul qilishi mumkin.

Ma'lumotlar bazasini boshqasiga nusxalashning bir misolini ko'rib chiqamiz.

```
import sqlite3  
  
def progress(status, remaining, total):  
    print(F'Nusxalandi {total-remaining} / {total}...')  
  
try:  
    sqlite_con = sqlite3.connect('sqlite_python.db')  
    backup_con = sqlite3.connect('sqlite_backup.db')  
    with backup_con:  
        sqlite_con.backup(backup_con, pages=3,  
progress=progress)  
        print("Zaxiralash muvaffaqiyatli yakunlandi")  
except sqlite3.Error as error:  
    print("Zaxiralashda xatolik: ", error)  
finally:  
    if(backup_con):  
        backup_con.close()  
        sqlite_con.close()
```

Eslatmalar:

- SQLite ga ulangandan so‘ng ikkala ma’lumotlar bazasi ikkita turli ulanishlar yordamida ochiq holatda bo‘ladi;
- Keyin birinchi ulanish yordamida ***connection.backup()*** usuli bajariladi. Bitta iteratsiyada ko‘chiriladigan sahifalar soni ham belgilab ketiladi.

Nazorat savollari:

1. SQLite tili bilan Python tilining bog‘lash uchun qanday kutubxonadan foydalaniladi?
2. SQLite bilan bog‘lanishda qanday parametrlardan foydalaniladi?
3. URI parametridan nima maqsadda foydalaniladi?
4. SQLite va Pythonning ma’lumot turlarini solishtiring.
5. `con.cursor` – metodining vazifasi nima?
6. Ma’lumotlar bazasiga so‘rovlar qanday amalga oshiriladi?
7. Jadval yaratish qanday amalga oshiriladi?
8. Ma’lumotlar bazasini qanday qilib bittada zaxiralash mumkin?
9. Ma’lumotlar bazasidan belgilangan satrlarni zaxiralash qanday amalga oshiriladi?

7.3.§. Ma’lumotlar bazasiga ma’lumotlarni yozish

Ushbu mavzuda jadvalga yangi qatorlar qo‘shish uchun SQLite moduli yordamida Insert operatsiyasini qanday bajarishni ko‘rib chiqiladi:

- ✓ SQLite jadvaliga bir yoki bir nechta qator qo‘shish;
- ✓ SQLite jadvaliga butun sonlar, satrlar, kasr sonlar va ***datetime*** tipli ma’lumotlarni qo‘shish;
- ✓ Python o‘zgaruvchilarini jadvalga dinamik ma’lumotlar sifatida qo‘shish uchun parametr so‘rovlaridan foydalanish.

Quyidagi dasturlarni bajarishdan oldin jadval nomini, shuningdek, uning ustunlari haqidagi ma’lumotlarni bilish kerak bo‘ladi. Ushbu misolda oldingi mavzuda yaratilgan ***sqllitedb_developers*** jadvalidan foydalaniladi.

SQLite jadvaliga qator qo'shish

`sqlitedb_developers` jadvali hozirda bo'sh va uni to'ldirish kerak. Buning uchun quyidagi amallarni bajariladi:

- SQLite ulanishini o'rnatish;
- Ulanish ob'yekti yordamida `cursor` ob'yektini yaratish.
- Jadval va ustunlar haqidagi ma'lumotlarni bilgan holda INSERT so'rovini yaratiladi.
- `cursor.execute()` yordamida so'rovni amalga oshiriladi.
- Muvaffaqiyatli tugatgandan so'ng, ma'lumotlar bazasiga o'zgartirish kiritishni unutmashlik kerak.
- Ma'lumotlar bazasiga so'rovlar albatta `try` bloki ichida bajarilishini ta'minlash kerak.
- Nihoyat, jadvaldan ma'lumotlarni qaytarish orqali natijani tekshirish kerak.

Keling, dasturni ko'rib chiqaylik:

```
import sqlite3
try:
    sqlite_connection = sqlite3.connect('sqlite_python.db')
    cursor = sqlite_connection.cursor()
    print("SQLite ga ulanish")

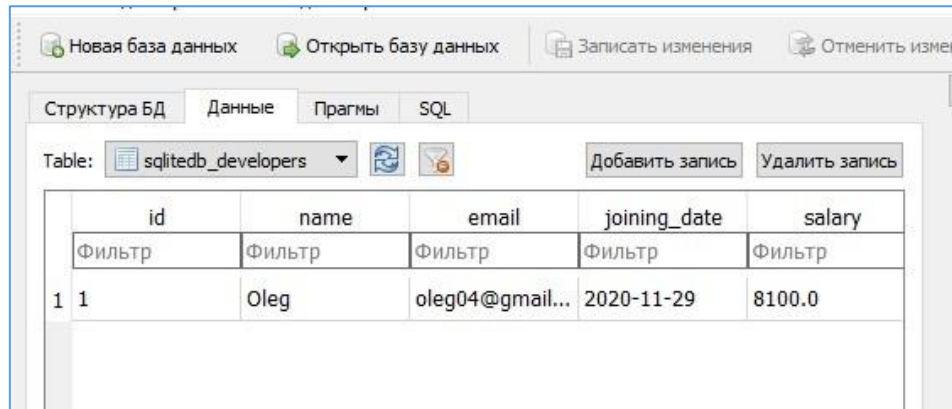
    sqlite_insert_query = """INSERT INTO sqlitedb_developers
                                (id, name, email, joining_date, salary)
                                VALUES
                                (1, 'Temurbek', 'temurbek04@gmail.com',
'2023-03-29', 8100);"""
    count = cursor.execute(sqlite_insert_query)
    sqlite_connection.commit()
    print("Yozuv sqlitedb_developers 1 -jadvaliga
muvaffaqiyatli kiritildi ", cursor.rowcount)
    cursor.close()

except sqlite3.Error as error:
    print("SQLite bilan ishlashda xatolik sodir bo'ldi",
error)
finally:
    if sqlite_connection:
        sqlite_connection.close()
        print("SQLite bilan bog'lanish tugallandi!")
```

Natija:

```
SQLite ga ulanish
Yozuv sqllitedb_developers 1 -jadvaliga muvaffaqiyatli kiritildi
SQLite bilan bog'lanish tugallandi!
```

Endi DB Browser dasturi orqali jadvalga qarab natijani tekshirish mumkin.



The screenshot shows the DB Browser for SQLite interface. The 'Table:' dropdown is set to 'sqlitedb_developers'. The table structure is displayed with columns: id, name, email, joining_date, and salary. A single record is shown with id 1, name Oleg, email oleg04@gmail..., joining_date 2020-11-29, and salary 8100.0.

	id	name	email	joining_date	salary
1	1	Oleg	oleg04@gmail...	2020-11-29	8100.0

25-rasm. Jadvalga ma'lumot joylashtirish

Kodni tahlil qilish

1. **import sqlite3:**

- Bu qator sqlite3 modulini dasturga import qiladi.
- Moduldagi sinflar va usullardan foydalanib, SQLite ma'lumotlar bazasi bilan o'zaro aloqani ta'minlash mumkin.

2. **sqlite3.connect()** va **connection.cursor():**

- sqlite3.connect()** metodi SQLite ma'lumotlar bazasiga ulanishni o'rnatadi.
- Keyinchalik, ulanish ob'ektidan **cursor** ob'ektini olish uchun **connection.cursor()** metodidan foydalaniladi.

3. Shundan so'ng, jadvalga ma'lumotlarni kiritish uchun INSERT so'rovi tayyorlanadi. Unda ustunlarning nomlari va ularning qiymatlari ko'rsatilgan bo'ladi. Ushbu jadvalda jami 5 ta ustun mavjud.

4. **cursor.execute():**

- cursor** ob'ektining **execute()** usuli INSERT so'rovini bajaradi.
- Ma'lumotlar bazasidagi o'zgarishlarni saqlash uchun **connection.commit()** dan foydalanish kerak .

c. ***cursor.rowcount*** yordamida tahrirlangan qatorlar sonini bilib olish mumkin.

Operatsiya tugagandan so'ng, ***finally*** blokida ***cursor*** va ***connect*** ob'ektlari yopiladi.

INSERT so'rovi

Ba'zan jadval ustuniga Python o'zgaruvchisining qiymatini kiritish kerak bo'ladi. Bu o'zgaruvchining tipi turlicha bo'lishi mumkin: integer, string, float, datetime.

Masalan, ro'yxatdan o'tishda foydalanuvchi o'z ma'lumotlarini kiritadi. Ularni olish va SQLite jadvaliga kiritish kerak.

Buning uchun parametrli so'rov mavjud bo'lib, ushbu so'rov to'ldiruvchilar (?) o'rniga Python o'zgaruvchilaridan foydalanish imkonini beradi. Misol:

```
import sqlite3

def insert_variable_into_table(dev_id, name, email, join_date,
salary):
    try:
        sqlite_connection=sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLite ga ulanish")

        sqlite_insert_with_param = """INSERT INTO
sqlitedb_developers
(id, name, email, joining_date, salary)
VALUES (?, ?, ?, ?, ?);"""

        data_tuple = (dev_id, name, email, join_date, salary)
        cursor.execute(sqlite_insert_with_param, data_tuple)
        sqlite_connection.commit()
        print("Python o'zgaruvchilari sqlitedb_developers
jadvaliga muvaffaqiyatli kiritildi")

        cursor.close()
```

```

except sqlite3.Error as error:
    print("SQLite bilan ishlashda xatolik", error)
finally:
    if sqlite_connection:
        sqlite_connection.close()
        print("SQLite bilan ulanish yopildi ")

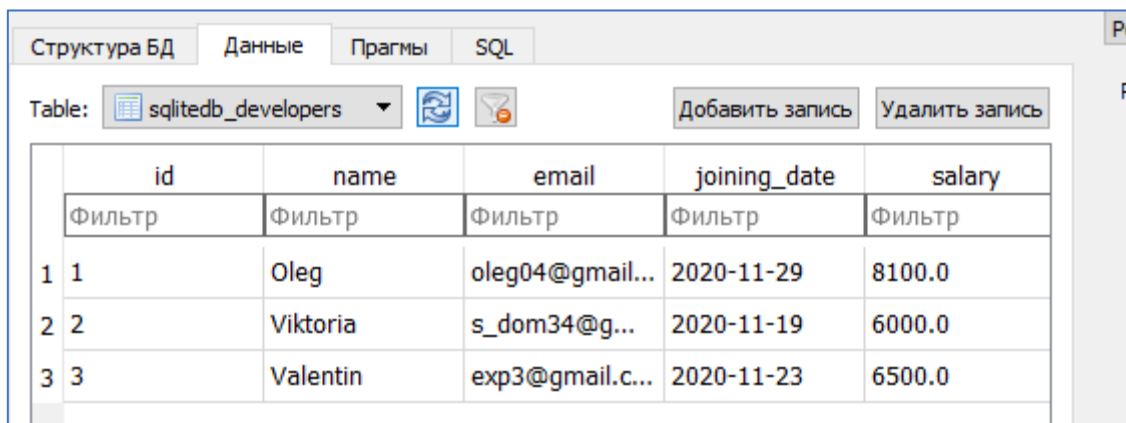
insert_variable_into_table(2, 'Viktoria', 's_dom34@gmail.com',
'2020-11-19', 6000)
insert_variable_into_table(3, 'Valentin', 'exp3@gmail.com',
'2020-11-23', 6500)

```

Natija: Python o'zgaruvchisini ustun qiymati sifatida kiritgandan so'ng sqlitedb_developers jadvali.

SQLite-ga ulangan
 Python o'zgaruvchilari sqlitedb_developers jadvaliga
 muvaffaqiyatli kiritildi
 SQLite bilan ulanish yopildi

Jadvaldan ma'lumotlarni olish orqali natijani tekshirish mumkin.



The screenshot shows a database management interface with tabs for 'Структура БД', 'Данные', 'Прагмы', and 'SQL'. The 'Данные' tab is active, showing a table named 'sqlitedb_developers'. The table has columns: id, name, email, joining_date, and salary. There are three rows of data:

	id	name	email	joining_date	salary
1	1	Oleg	oleg04@gmail...	2020-11-29	8100.0
2	2	Viktoria	s_dom34@g...	2020-11-19	6000.0
3	3	Valentin	exp3@gmail.c...	2020-11-23	6500.0

26-rasm. Insert so'rovi natijasi.

EXECUTEMANY() bilan bir nechta qatorlarni kiritish

Oldingi misolda jadvalga bitta yozuv qo'shish uchun kursor ob'ektining execute() usuli ishlatilgan, lekin ba'zida bir nechta qatorlarni kiritish kerak bo'ladi.

Masalan, CSV kabi kengaytmali faylni o'qiyotganda undagi barcha yozuvlarni SQLite jadvaliga qo'shish mumkin. Har bir yozuv uchun INSERT so'rovini bajarish o'rniga, bitta so'rovda barcha amallarni bajarish mumkin. Kursor ob'ektining

executemany() metodi yordamida SQLite jadvaliga bir nechta yozuvlarni qo‘shish mumkin.

Ushbu metod ikkita argumentni qabul qiladi: SQL so‘rovi va yozuvlar ro‘yxatini.

```
import sqlite3

def insert_multiple_records(records):
    try:
        sqlite_connection=sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLite-ga ulangan")

        sqlite_insert_query = """INSERT INTO
sqlitedb_developers
                        (id, name, email, joining_date, salary)
                        VALUES (?, ?, ?, ?, ?);"""

        cursor.executemany(sqlite_insert_query, records)
        sqlite_connection.commit()
        print("Yozuvlar sqlitedb_developers jadvaliga
muvaffaqiyatli kiritilgan ", cursor.rowcount)
        sqlite_connection.commit()
        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan ishlashda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bilan ulanish yopildi")

records_to_insert = [
    (4, 'Jaroslav', 'idebylos@gmail.com', '2020-11-14', 8500),
    (5, 'Timofei', 'ullegyddomm@gmail.com', '2020-11-15', 6600),
    (6, 'Nikita', 'aqillysso@gmail.com', '2020-11-27', 7400)
]

insert_multiple_records(records_to_insert)
```

Jadvaldan ma’lumotlarni olish orqali yana tekshiring.

SQLite-ga ulangan

Yozuvlar sqlitedb_developers 3 jadvaliga muvaffaqiyatli kiritildi

SQLite bilan ulanish yopildi

Структура БД

Данные

Прагмы

SQL

Table:

sqllitedb_developers

Добавить запись

Удалить запись

	id	name	email	joining_date	salary
	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр
1	1	Oleg	oleg04@gmail...	2020-11-29	8100.0
2	2	Viktoria	s_dom34@g...	2020-11-19	6000.0
3	3	Valentin	exp3@gmail.c...	2020-11-23	6500.0
4	4	Jaroslav	idebylos@gm...	2020-11-14	8500.0
5	5	Timofei	ullegyddomm...	2020-11-15	6600.0
6	6	Nikita	aqillysso@gm...	2020-11-27	7400.0

27-rasm. executemany() metodining natijasi

Dasturning tahlili:

- Ma'lumotlar bazasiga ulangandan so'ng, jadvalga kiritish uchun yozuvlar ro'yxati tayyorlanadi. Ularning har biri satrlardan iborat bo'ladi;
- SQLite INSERT intruksiyasidagi parametrlari bo'lmish: `id`, `name`, `email`, `joining_date`, `salary` lar so'roq belgilari o'rniga ketma-ketlikda joylashtiriladi. Ya'ni bu yerda har bir savol belgisi qiymat bilan almashtiriladi;
- Bundan tashqari, **`cursor.executemany(sqlite_insert_query, recordList)`** yordamida jadvalga bir nechta yozuvlar kiritiladi;
- Kiritilgan qatorlar sonini bilish uchun **`cursor.rowcount`** metodi qo'llaniladi. Barchasini tugatgach, ma'lumotlar bazasiga kiritilgan o'zgarishlarni saqlashni saqlash kerak bo'ladi.

Nazorat savollari:

1. SQLite ma'lumotlar bazasiga ma'lumot kiritish qanday amalga oshiriladi?
2. SQLite jadvaliga qator qo'shish qanday amalga oshiriladi?
3. **`cursor.execute()`** metodining vazigasi qanday?
4. **INSERT** – bu kalit so'zning vazifasi qanday?
5. **datetime** – ob'yektining vazifasi qanday?
6. **commit** – bu metodning vazifasi qanday?
7. **`cursor.rowcount`** metodi qanday vazifani bajaradi?

7.4.§. SQLite jadvalidan ma'lumotlarni olish

Ushbu mavzuda SQLite jadvalidan ma'lumotlarni olish buyrug'i - SELECT haqida so'z boradi. Ushbu mavzuda: 1) barcha qatorlarni ***cursor.fetchall()*** yordamida olishni; 2) cheklangan qatorlar sonini olish uchun ***cursor.fetchmany(size)***, bittasi qator uchun ***cursor.fetchone()*** metodidan qanday foydalanishni; 3) dinamik ma'lumotlarni uzatish uchun SQLite so'rovida Python o'zgaruvchilaridan qanday foydalanishni ko'rib chiqiladi.

Ushbu Python dasturi bilan ishlashdan oldin ishlayotgan SQLite jadvalining **nomi** va ustun haqidagi ma'lumotlarni bilish kerak bo'ladi.

Ushbu misolda oldingi mavzularda yaratilgan ***sqllitedb_developers*** jadvalidan foydalaniladi.

SQLite jadvalidan qatorlarni olish qadamlari. Python dan SELECT operatsiyasini bajarish uchun quyidagi amallarni bajarish kerak:

- 1) SQLite ma'lumotlar bazasiga ulanish o'rnatish;
- 2) Jadval nomi va ustunlar haqida ma'lumotlar asosida SELECT buyrug'i bilan SQLite MBga so'rov yaratish;
- 3) ***cursor.execute()*** metodi yordamida SELECT so'rovini amalga oshirish;
- 4) ***cursor*** ob'yekti va ***cursor.fetchall()*** metodi yordamida qatorlarga murojaat qilish;
- 5) Qatorlar bo'ylab o'tish va har bir qatorga mos qiymatlarini olish;
- 6) ***cursor*** ob'ektini va SQLite ma'lumotlar bazasiga ulanishni yopish;
- 7) baza bilan ishlash vaqtida yuzaga kelishi mumkin bo'lgan istisnolarni boshqarish uchun kerakli so'rovlarni ***try*** bloki ichiga olish.

sqllitedb_developers jadvalidagi barcha qatorlarni olish uchun misol.

```
import sqlite3

def read_sqlite_table(records):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLite-ga ulangan")
```

```

        sqlite_select_query = """SELECT * from
sqlitedb_developers"""
        cursor.execute(sqlite_select_query)
        records = cursor.fetchall()
        print("Jami satrlar: ", len(records))
        print("Har bir satrning chiqishi")
        for row in records:
            print("ID:", row[0])
            print("Nom:", row[1])
            print("Pochta:", row[2])
            print("Qo'shilgan:", row[3])
            print("Ish haqi:", row[4], end="\n\n")

        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan ishlashda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bilan ulanish yopildi")

read_sqlite_table()

```

Natijasi:

```

SQLitega ulangan
Jami satrlar: 6
Har bir satrning chiqishi
ID: 1
Ism: Olga
Pochta: olga04@gmail.com
Qo'shilgan: 2023-02-27
Ish haqi: 8100.0

...

```

```
ID: 6
Ism: Nikola
Pochta: aqilyso@gmail.com
Qo'shilgan: 2023-02-27
Ish haqi: 7400.0

SQLite bilan ulanish yopildi
```

Ushbu misol to'g'ridan-to'g'ri satr va uning ustun qiymatini ko'rsatadi. Agar dasturda ustun qiymatlaridan foydalanish kerak bo'lsa, ularni Python o'zgaruvchilarida saqlash mumkin. Buning uchun, quyidagicha yozish kerak: `name = row[1]`.

Misolni tahlil qilish

import sqlite3:

- Bu qator `sqlite3` modulini dasturga import qiladi.
- Ushbu modulning sinflari va usullaridan foydalanib, ma'lumotlar bazasi bilan bevosita aloqada bo'lish mumkin.

***sqlite3.connect()* va *connection.cursor()*:**

- ***sqlite3.connect()*** SQLite ma'lumotlar bazasiga ulanishni o'rnatadi.
- Shundan so'ng ***sqlitedb_developers*** jadvalidagi barcha qatorlarni olish uchun `SELECT` so'rovi tayyorlanadi va jadvaldagi beshta ustunni qamrab oladi.
- ***connection.cursor()*** usuli ulanish ob'yektidan ***cursor*** ob'yektini olish uchun ishlatiladi.

***cursor.execute()* va *cursor.fetchall()*:**

- `SELECT` operatsiyasi ***cursor*** ob'yektining ***execute()*** usuli yordamida bajariladi .
- Muvaffaqiyatli so'rovdan so'ng, ***cursor.fetchall()*** metodi ***sqlitedb_developers*** jadvalidagi barcha yozuvlarni olish uchun ishlatiladi.
- Oxirida barcha yozuvlarni birma-bir konsolga chiqarish uchun sikl operatori ishlatiladi.

Barcha yozuvlar olingandan so'ng, ***cursor*** ob'yekti ***cursor.close()*** bilan yopiladi va ma'lumotlar bazasi ulanishi ***sqliteConnection.close()*** bilan yopiladi .

Eslatma:

- So‘rovni bajarish uchun ***cursor.execute()*** buyrug‘idan foydalanish maqsadga muvofiq bo‘ladi;
- ***cursor.fetchall()*** - barcha qatorlarni tanlash va olishda foydalaniladi;
- ***cursor.fetchone()*** - bir qator uchun;
- ***cursor.fetchmany(size)*** - chegaralangan qatorlar uchun.

SELECT so‘rovi parametrlari sifatida o‘zgaruvchilardan foydalanish

Ko‘pincha ma’lum bir shartni tekshirish uchun o‘zgaruvchini SELECT so‘rovi tarkibida o‘zgaruvchilardan foydalanishga ehtiyoj paydo bo‘ladi.

Faraz qilaylik, dastur o‘z identifikatoridan foydalangan holda ishlab chiquvchilar haqida ma’lumot olish uchun so‘rov yubormoqchi. Bu esa o‘z navbatida parametrlarga ega bo‘lgan so‘rov jo‘natilishini talab qiladi. Bu shunday so‘rovki, unda to‘ldiruvchilar (?) parametrlar o‘rniga joylashtiriladi. Keyinchalik ular haqiqiy qiymatlar bilan almashtiriladi.

```
cursor.execute("SELECT salary FROM  
sqlitedb_developers WHERE id = "Dasturdan ID")
```

Keling, bir misolni ko‘rib chiqaylik.

```
import sqlite3

def get_developer_info(id):
    try:
        sqlite_connection = sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLite-ga ulangan")

        sql_select_query = """SELECT * from sqlitedb_developers
where id = ?"""
        cursor.execute(sql_select_query, (id,))
        records = cursor.fetchall()
        print("ID ni kiriting:", id)
        for row in records:
```

```

        print("ID:", row[0])
        print("Nom:", row[1])
        print("Pochta:", row[2])
        print("Qo'shilgan:", row[3])
        print("Ish haqi:", row[4], end="\n\n")

    cursor.close()

except sqlite3.Error as error:
    print("SQLite bilan ishlashda xatolik", error)
finally:
    if sqlite_connection:
        sqlite_connection.close()
        print("SQLite bilan ulanish yopildi")

get_developer_info(2)

```

Xulosa :

```

SQLitega ulangan
ID ni kiriting: 2
ID: 2
Nom: Viktoria
Pochta: s_dom21@gmail.com
Qo'shilgan: 2023-02-19
Ish haqi: 6000.0
SQLite bilan ulanish yopildi

```

Jadvaldan bir nechta qatorlarni olish

Ba'zi hollarda, jadvaldan barcha qatorlarni olishga urinish juda uzoq vaqtni sarflashni talab etadi, ayniqsa ular minglab bo'lsa.

Barcha qatorlarni olish, o'z navbatida ko'proq resurslar talab etadi: xotira va qayta ishlash uchun vaqt. Va bunday hollarda ishlashni yaxshilash uchun **cursor** sinfining **fetchmany(size)** usulidan qat'iy belgilangan qatorlarni olish tavsiya etiladi.

cursor.fetchmany(size) metodi yordamida nechta qatorni o'qishni belgilash mumkin:

```

import sqlite3

def read_limited_rows(row_size):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLitega bog'lanildi")

        sqlite_select_query = """SELECT * from
sqlitedb_developers"""
        cursor.execute(sqlite_select_query)
        print ( row_size, " satrni", " o'qimoqda")
        records = cursor.fetchmany(row_size)
        print("har bir satrning chiqishi \n")
        for row in records:
            print("ID:", row[0])
            print("Nom:", row[1])
            print("Pochta:", row[2])
            print("Qo'shilgan:", row[3])
            print("Ish haqi:", row[4], end="\n\n")

        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan ishlashda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bilan bog'lanish yopildi")

read_limited_rows(2)

```

Ekranida ko'rinishi :

```

SQLitega bog'lanildi
2  satrni o'qimoqda
Har bir satrni kiritish

```



```
ID: 1
Nom: Oleg
Pochta: oleg0044@gmail.com
Qo'shilgan: 2023-02-29
Ish haqi: 8100.0
ID: 2
Nom: Viktoria
Pochta: s_dom21@gmail.com
Qo'shilgan: 2023-02-19
Ish haqi: 6000.0
SQLite bilan bog'lanish yopildi.
```

Eslatma: Ushbu dastur bazadan ikkita qatorni bazaga so'rov jo'natish orqali oladi. Ammo jadvalda ulardan kamroq bo'lsa, aynan shu raqam qaytariladi.

Jadvaldan bitta qatorni olish

SQLite jadvalidan bitta qatorni olish kerak bo'lib qolsa, **cursor** sinfining **fetchone()** metodidan foydalanish kerak bo'ladi. Ushbu usul bazaga jo'natiladigan so'rovdan foydalanib bitta qatorni olish ma'lum bo'lganda kerak bo'ladi.

cursor.fetchone() faqat natijadan keyingi qatorni oladi. Agar qatorlar bo'lmasa, u holda None qaytariladi . Misol :

```
import sqlite3

def read_single_row(developer_id):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLitega bog'lanildi ")

        sqlite_select_query = """SELECT * from
sqlitedb_developers where id = ?"""
        cursor.execute(sqlite_select_query, (developer_id, ))
        print("Bitta satrni o'qish \n")
        record = cursor.fetchone()
        print("ID:", record[0])
        print("Nom:", record[1])
        print("Pochta:", record[2])
        print("Qo'shilgan:", record[3])
```

```

        print("Ish haqi:", record[4])

    cursor.close()

except sqlite3.Error as error:
    print("SQLite bilan ishlashda xatolik", error)
finally:
    if sqlite_connection:
        sqlite_connection.close()
        print("SQLite bilan bog'lanish yopildi")

read_single_row(3)

```

Ekraida ko'rinishi:

```

SQLitega bog'lanildi
Bitta satrni o'qish

ID: 3
Nom: Valentin
Pochta: exp33@gmail.com
Qo'shilgan: 2023-02-23
Ish haqi: 6500.0
SQLite bilan bog'lanish yopildi

```

Nazorat savollari:

1. SQLite jadvalidagi ma'lumotlarni chiqarib olish qanday amalga oshiriladi?
2. **sqlite3.connect()** va **connection.cursor()** metodlarining vazifasi qanday?
3. **cursor.execute()** va **cursor.fetchall()** metodlarining vazifasi qanday?
4. SQLite jadvalidan bitta qatorni olish qanday amalga oshiriladi?

7.5.§. Yozib olingan ma'lumotlarni yangilash

Ushbu mavzuda Python ilovasidan SQLite jadvalini yangilash uchun UPDATE so'rovidan foydalanish haqida so'z ketadi. SQLite jadvalini yangilash uchun o'rnatilgan sqlite3 modulidan qanday foydalanishni ko'rib chiqiladi.

Mavzu davomida quyidagilarni ko'rib chiqiladi:

- 1) Bir yoki bir nechta ustunlarni yangilash.

2) Yangilash so‘rovida dastur ishlayotgan vaqtda qiymatni o‘tkazish uchun parametrlar bilan so‘rovdan foydalanish.

3) Yangilash operatsiyasini bajarish va orqaga qaytarish.

4) date-time va timestamp qiymatlari yordamida ustunni yangilash.

5) Bitta so‘rovda barcha ustunlarni yangilashni amalga oshirish.

Quyidagi SQLite jadvalini yangilash operatsiyalarini bajarishdan oldin uning nomini, shuningdek, ustunlar nomlarini bilish talab qilinadi.

Ushbu misollarda ham, oldingi mavzulardagi **sqlitedb_developers** jadvalidan foydalaniladi.

SQLite jadvalidagi bitta satrni yangilash

sqlitedb_developers jadvali hozirda olti qatordan iborat, shuning uchun id 4 orqali dasturchining “Ish_haqi”ni yangilaylik. Pythondan UPDATE so‘rovini ishga tushirish uchun quyidagi amallarni bajaring:

- Avval Pythondan SQLite ulanishini o‘rnatish kerak;
- Keyinchalik ulanish ob’ekti yordamida kursor ob’ektini yaratish kerak;
- Shundan so‘ng, UPDATE so‘rovini yaratiladi. Buni amalga oshirish uchun yangilamoqchi bo‘lgan jadval va ustun nomlarini bilish talab etiladi.
- Keyinchalik so‘rov **cursor.execute()** yordamida amalga oshiriladi .
- So‘rov muvaffaqiyatli bajarilgandan so‘ng, ma’lumotlar bazasiga o‘zgartirish kiritishni unutmash kerak.
- Ma’lumotlar bazasiga ulanishni yopiladi.
- Bundan tashqari, barcha SQLite istisnolarini boshqarish uchun dasturning kerakli qismlarini try bloki ichiga yozishni unutmash kerak.
- Nihoyat, jadvaldan ma’lumotlarni olish orqali operatsiya muvaffaqiyatli o‘tganligini tekshirish kerak.

Keling, dasturni ko‘rib chiqaylik.

```
import sqlite3

def update_sqlite_table():
```

```

try:
    sqlite_connection= sqlite3.connect('sqlite_python.db')
    cursor = sqlite_connection.cursor()
    print("SQLitega bog'lanildi")

    sql_update_query = """UPDATE sqlitedb_developers set
salary = 10000 where id = 4"""
    cursor.execute(sql_update_query)
    sqlite_connection.commit()
    print("Satr muvoffaqiyatli o'zgartirildi")
    cursor.close()
except sqlite3.Error as error:
    print("SQLite bilan ishlashda xatolik ", error)
finally:
    if sqlite_connection:
        sqlite_connection.close()
        print("SQLite bilan bog'lanish yopildi")
update_sqlite_table()

```

Natija: Python-dan qator yangilangandan keyin sqlitedb_developers jadvali .

```

SQLitega bog'lanildi
Satr muvoffaqiyatli o'zgartirildi
SQLite bilan bog'lanish yopildi

```

Misolni tahlili

import sqlite3:

- Bu qator sqlite3 modulini dasturga import qiladi.
- Moduldagi sinflar va usullardan foydalanib, ma'lumotlar bazasi bilan o'zaro aloqada bo'lish mumkin.

sqlite3.connect() va connection.cursor() :

- **SQLite3.connect()** Python dan SQLite ma'lumotlar bazasiga ulanishni o'rnatadi.
- Keyinchalik ulanish ob'ektidan kursor ob'ektini olish uchun **connect.cursor()** usuli qo'llaniladi .

Shundan so'ng, jadvallardagi qatorni yangilash uchun UPDATE so'rovi yaratiladi. UPDATE so'rovi o'z navbatida ustun **nomi** va yangi qiymatini qabul qiladi. Jadvalda beshta ustun bor, lekin kod ulardan faqat bittasini yangilaydi.

cursor.execute() :

- UPDATE so'rovida saqlangan operatsiya ***cursor*** ob'ektining ***execute()*** usuli yordamida amalga oshiriladi .
- ***connection.commit()*** metodi esa ma'lumotlar bazasiga ma'lumotni saqlash uchun ishlatiladi.

Nihoyat, ***cursor*** ob'ektlari va ulanish yangilash jarayoni tugagandan so'ng ***finally*** bloki orqali yopiladi.

Eslatma: agar ommaviy yangilash jarayoni davom etayotgan bo'lsa va kamida bitta xatolik yuz berganda o'zgarishlarni orqaga qaytarish zarurati tug'ilsa, ulanish sinfining ***rollback()*** funksiyasidan foydalanish kerak bo'ladi. Uni ***except*** bloki ichida qo'llash kerak.

UPDATE so'rovida Python o'zgaruvchilaridan foydalanish

Ko'pincha jadvalni yangilash dastur ishga tushganda oladigan qiymatlar bilan amalga oshirilishi kerak. **Masalan**, foydalanuvchilar o'z profillarini GUI orqali yangilaganlarida, ular tegishli jadvalda o'rnatgan qiymatlarni yangilash kerak bo'ladi.

Bunday holda, parametrlar bilan so'rovdan foydalanish tavsiya etiladi. Bunday so'rovlar to'g'ridan-to'g'ri SQL operatorlari ichida to'ldiruvchidan(?) foydalanadi. Bu qiymatlarni o'zgaruvchilar bilan yangilash va SQL in'ektsiyalarining oldini olishga yordam beradi.

```
import sqlite3

def update_sqlite_table(dev_id, salary):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLitega bog'lanildi")
```

```

        sql_update_query = """UPDATE sqlitedb_developers set
salary = ? where id = ?"""
        data = (salary, dev_id)
        cursor.execute(sql_update_query, data)
        sqlite_connection.commit()
        print("Satr muvoffaqiyatli o'zgartirildi")
        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan ishlashda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bilan bog'lanish yopildi")

update_sqlite_table(3, 7500)

```

Natija: sqlitedb_deveopers jadvali Python o'zgaruvchisi va parametrlari bilan so'rov bilan yangilangandan keyin.

```

SQLitega bog'lanildi
Satr muvoffaqiyatli o'zgartirildi
SQLite bilan bog'lanish yopildi

```

SQLite jadvalining bir nechta qatorlarini yangilash

Oxirgi misolda bitta qiymatni yangilash uchun **cursor** ob'ekting **execute()** metodi ishlatilgan, lekin ba'zida Python ilovalarida bir nechta satrlarni yangilash kerak bo'ladi. Misol uchun, ko'pchilik ishchilarning ish haqini 20% ga oshirish kerak bo'ladi.

Har bir yozuv uchun har safar UPDATE operatsiyasini bajarish o'rniga, bitta so'rovda ommaviy yangilashni amalga oshirish mumkin. **cursor.executemany()** metodi yordamida bir so'rovda **SQLite** jadvalidagi bir nechta yozuvlarni o'zgartirish mumkin.

cursor.executemany(query, seq_param) metodi ikkita argumentni qabul qiladi: SQL so'rovi va yangilanadigan yozuvlar ro'yxati.

Keling, bir misolni ko‘rib chiqaylik. Bu yerda 3 ta ishchining maoshi yangilanadi.

```
import sqlite3

def update_multiple_records(record_list):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLitega bog'lanildi")

        sqlite_update_query = """Update sqlitedb_developers
set salary = ? where id = ?"""
        cursor.executemany(sqlite_update_query, record_list)
        sqlite_connection.commit()
        print(cursor.rowcount, "satr muvoffaqiyatli
yangilandi")

        sqlite_connection.commit()
        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan ishlashda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bilan bog'lanish yopildi")

records_to_update = [(9700, 4), (7800, 5), (8400, 6)]
update_multiple_records(records_to_update)
```

Natija: Python dan bir nechta qatorlarni yangilagandan so‘ng *sqlitedb_developers* jadvali.

```
SQLitega bog'lanildi
3 satr muvoffaqiyatli yangilandi
SQLite bilan bog'lanish yopildi
```

Kodning tahlili:

1) SQLite jadvaliga ulangandan so'ng, ikkita to'ldiruvchi (ish haqi va id ustunlari) va kortej formatida yangilanadigan yozuvlar ro'yxati bilan SQLite so'rovi tayyorlanadi.

2) Har bir element har bir yozuv uchun kortej hisoblanadi. Har bir kortejda ikkita qiymat mavjud bo'ladi: ish haqi va ishlab chiquvchi id si.

3) ***cursor.executemany(sqlite_update_query, record_list)*** funksiyasi SQLite jadvalining bir nechta satrlarini yangilash uchun chaqiriladi.

4) Yozuvlar o'zgartirilganligini bilish uchun ***cursor.rowcount*** funksiyasidan foydalaniladi.

5) Nihoyat, ma'lumotlar ***connect*** sinfining ***commit*** usuli yordamida ma'lumotlar bazaga saqlanadi.

SQLite jadvalining bir nechta ustunlarini yangilash

Bitta so'rovda SQLite jadvalining bir nechta ustunlarini yangilash mumkin. Buni amalga oshirish uchun faqat parametrlar va to'ldiruvchilar bilan so'rov tayyorlash kerak bo'ladi:

```
import sqlite3

def update_multiple_columns(dev_id, salary, email):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLitega bog'lanildi")

        sqlite_update_query = """Update sqlitedb_developers
set salary = ?, email = ? where id = ?"""
        column_values = (salary, email, dev_id)
        cursor.execute(sqlite_update_query, column_values)
        sqlite_connection.commit()
        print("Birqancha ustunlar muvoffaqiyatli yangilandi!")
        sqlite_connection.commit()
        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan ishlashda xatolik", error)
```



```
finally:
    if sqlite_connection:
        sqlite_connection.close()
        print("SQLite bilan bog'lanish tugatildi")

update_multiple_columns(3, 2500, 'bec9988@gmail.com')
```

```
SQLitega bog'lanildi
Birqancha ustunlar muvoffaqiyatli yangilandi!
SQLite bilan bog'lanish tugatildi
```

Nazorat savollari:

1. Jadvalga yozilgan ma'lumotlarni yangilash uchun qanday amallarni bajarish kerak bo'ladi?
2. SQLite jadvalidagi bitta yozuvni yangilash uchun

7.6.§. SQLite jadvalidan yozuvlarni o'chirish

Ushbu mavzuda Python ilovasidan SQLite jadvalida SQL DELETE operatsiyasini bajarish haqida gap boradi.

Mavzu davomida quyidagilarni ko'rib chiqiladi:

- Python yordamida SQLite jadvalidan bitta, bir nechta yoki barcha satr yoki ustunlarni o'chirish;
- SQLite dan **o'chirish** operatsiyasini bajarish uchun parametrlarli so'rovdan foydalanish;
- Oxirgi operatsiyani bajarish va bekor qilish;
- Bitta so'rovdan foydalanib barcha ma'lumotlarni o'chirish.

Quyidagi amallarni bajarishdan oldin SQLite jadvali nomi va uning ustunlarini bilish kerak bo'ladi. Ushbu mavzuda **sqlitedb_developers** jadvalidan foydalanadi.

SQLite jadvalidan bitta qatorni o'chirish misoli

Endi **sqlitedb_developers** jadvali oltita qatorni o'z ichiga oladi va identifikatori 6 bo'lgan ishchini o'chirib tashlash amalini bajarib ko'raylik. Buning uchun quyidagi amallarni ketma-ketlikda bajarish kerak:

- Python'dan **SQLite** ma'lumotlar bazasiga ulanish;

- SQLite ulanish ob'yekti yordamida **cursor** ob'yektini yaratish;
- SQLite uchun **DELETE** so'rovini yaratiladi buning uchun jadval va ustunlar nomlarini bilish kerak bo'ladi;
- **cursor.execute()** yordamida **DELETE** so'rovini amalga oshirish;
- So'rov bajarilgandan so'ng, ma'lumotlar bazasiga o'zgartirishlar kiritiladi;
- Ma'lumotlar bazasiga ulanishni yopish;
- Xatoliklar kelib chiqishi ehtimoli bo'lgan bloklarni try-except blokiga olish va istisnolarni boshqarish;

Misol:

```
import sqlite3

def delete_record():
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLite ga ulanildi")

        sql_delete_query = """DELETE from sqllitedb_developers
where id = 6"""
        cursor.execute(sql_delete_query)
        sqlite_connection.commit()
        print("Yozuv muvaffaqiyatli o'chirildi")
        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bazasi bilan ishlashda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bazasi bilan ishlash yopildi")

delete_record()
```

Natija: Pythondan qator o'chirilgandan keyin **sqlitedb_developers** jadvali.

```
SQLite ga ulanildi  
Yozuv muvaffaqiyatli o'chirildi  
SQLite bazasi bilan ishlash yopildi
```

Dasturni batafsil tahlil qilish

import sqlite3:

- Bu satrda **sqlite3** moduli dasturga import qilinmoqda;
- **sqlite3** moduli tarkibidagi sinflar va usullardan foydalanib, **SQLite** ma'lumotlar bazasi bilan o'zaro aloqani o'rnatish.

sqlite3.connect() va **connection.cursor()** :

- **sqlite3.connect()** SQLite ma'lumotlar bazasiga ulanishni o'rnatadi;
- **sqliteConnection.cursor()** *cursor* ob'yektini olish uchun ishlatiladi.

Shundan so'ng, jadvaldagi oltinchi qatorni o'chirish uchun **DELETE** so'rovi yaratiladi (id 6 ga teng bo'lgan ishchi uchun).

cursor.execute() :

- Operatsiya **Cursor** ob'ektining **execute()** usuli yordamida **DELETE** so'rovidan amalga oshiriladi;
- Yozuv muvaffaqiyatli o'chirilgandan so'ng, o'zgartirishlar **connection.commit()** yordamida ma'lumotlar bazasiga kiritiladi.

Nihoyat, Cursor va SQLite ulanishi **finally** blokida yopiladi.

Eslatma: Agar bir nechta o'chirish jarayoni davom etayotgan bo'lsa va ulardan kamida bittasi bajarilmasa, barcha o'zgarishlarni orqaga qaytarish uchun ulanish sinfining **rollback()** funksiyasidan foydalaniladi. Ushbu funksiyadan try-except bloki ichida foydalanish kerak.

Qatorni o'chiruvchi so'rovlarda o'zgaruvchilardan foydalanish.

Ko'pgina hollarda, SQLite jadvalidagi satrlarni ish vaqtida allaqachon uzatilgan kalit yordamida o'chirish kerak. Misol uchun, foydalanuvchi o'z narsasini o'chirganida, ular jadvaldan olib tashlanishi kerak.

Bunday hollarda parametrli so'rovdan foydalanish kerak. Bunday so'rovlarda qiymatlar o'rniga to'ldiruvchilar (?) qo'yiladi. Bu dastur ishlayotgan vaqtda

qiymatlarni olish orqali yozuvlarni o‘chirishga yordam beradi va SQL in’ektsiya muammolarini oldini oladi. Mana id=5 bo‘lgan ishchini o‘chirishga misol.

```
import sqlite3

def delete_sqlite_record(dev_id):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLite ga ulanish amalga oshirildi")

        sql_update_query = """DELETE from sqlitedb_developers
where id = ?"""
        cursor.execute(sql_update_query, (dev_id, ))
        sqlite_connection.commit()
        print("Yozuv muvaffaqiyatli o‘chirildi")
        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan bog‘lanishda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bilan bog‘lanish yakunlandi")

delete_sqlite_record(5)
```

Natija: Parametrli so‘rov bilan qatorni o‘chirishdan keyin *sqlitedb_developers* jadvali quyidagi ma’lumotlarni qaytaradi.

```
SQLite ga ulanish amalga oshirildi
Yozuv muvaffaqiyatli o‘chirildi
SQLite bilan bog‘lanish yakunlandi
```

Dastur tahlili:

1) Dastur ishlayotgan vaqtda ishlab chiquvchining identifikatorini olish va uni joyiga qo'yish uchun parametrlari bilan so'rov ishlatilganmi? U olib tashlanadigan yozuvning identifikatorini belgilaydi.

2) Shundan so'ng Python o'zgaruvchilari yordamida ma'lumotlar majmuasi yaratiladi.

3) Bundan tashqari, **DELETE** so'rovi ma'lumotlar bilan birga **cursor.execute()** usuliga uzatiladi.

4) Nihoyat, o'zgarishlar **connection** sinfining **commit()** usuli yordamida ma'lumotlar bazasiga kiritiladi.

Bir nechta qatorlarni o'chirish uchun o'chirish operatsiyasi

Yuqoridagi misolda bitta yozuvni o'chirish uchun **Cursor** ob'ektining **execute()** usuli ishlatilgan. Lekin ko'pincha bir vaqtning o'zida bir nechtasini o'chirish kerak bo'ladi.

Har bir yozuv uchun har safar **DELETE** so'rovini bajarish o'rniga, bitta so'rovda ommaviy o'chirish operatsiyasini bajarish mumkin. Buning uchun **cursor.executemany()** usulidan foydalaniladi.

cursor.executemany(query, seq_param) usuli ikkita argumentni oladi: SQL so'rovi va o'chiriladigan yozuvlar ro'yxati.

Quyidagi misolda bir vaqtning o'zida uchta ishchini olib tashlash keltirilgan:

```
import sqlite3

def delete_multiple_records(ids_list):
    try:
        sqlite_connection= sqlite3.connect('sqlite_python.db')
        cursor = sqlite_connection.cursor()
        print("SQLitega ulangan")

        sqlite_update_query = """DELETE from
sqlitedb_developers where id = ?"""
        cursor.executemany(sqlite_update_query, ids_list)
        sqlite_connection.commit()
```

```

        print("O'chirilgan yozuvlar:", cursor.rowcount, " ta")
        sqlite_connection.commit()
        cursor.close()

    except sqlite3.Error as error:
        print("SQLite bilan ishlashda xatolik", error)
    finally:
        if sqlite_connection:
            sqlite_connection.close()
            print("SQLite bilan ishlash yakunlandi")

ids_to_delete = [(4,), (3,)]
delete_multiple_records(ids_to_delete)

```

```

SQLitega ulangan
O'chirilgan yozuvlar: 2 ta
SQLite bilan ishlash yakunlandi

```

Dastur tahlili:

1) SQLite ma'lumotlar bazasiga ulangandan so'ng, parametrlar va bitta to'ldiruvchi bilan SQL so'rovi tayyorlanadi. Shu bilan birga, kortej formatidagi id ro'yxati ham uzatiladi.

2) Ro'yxatning har bir elementi faqat har bir qatorning kortejidan iborat bo'ladi. Har bir kortejda ishchi identifikatori mavjud bo'ladi. Ushbu misolda uchta kortej mavjud - ya'ni uchta ishchi o'chiriladi.

3) Keyinchalik, jadvaldan bir nechta yozuvlarni o'chirish uchun ***cursor.executemany(sqlite_delete_query, ids_list)*** funksiyasi chaqiriladi. Tayyorlangan **so'rov** ham, **idlar** ro'yxati ham ***cursor.executemany()*** ning argumentlari sifatida uzatiladi.

4) O'zartirilgan, o'chirilgan yozuvlar sonini ko'rish uchun ***cursor.rowcount*** metodidan foydalaniladi. Va nihoyat, o'zgarishlar ***Connection*** sinfining ***commit*** usuli yordamida ma'lumotlar bazasiga kiritiladi .

Nazorat savollari:

1. SQLite jadvalidan yozuvlarni o'chirish qanday amalga oshiriladi?
2. Jadvaldan ma'lumotlarni o'chirish uchun qaysi operatorlardan foydalaniladi?

3. Jadvaldagi o‘chirilgan ma’lumotlar ro‘yxatini qanday olish mumkin?

4. Jadvaldagi ma’lumotlarning barchasini bitta so‘rovda o‘chirish qanday amalga oshiriladi?

5. `cursor.executemany(sqlite_delete_query, ids_list)` - ushbu metodning parametrlarini tushuntirib bering

VIII BOB. PYTHONDA GUI

8.1.§. Pythonda turli grafik modullar.

Python, boshqa dasturlash tillari kabi, grafikalar bilan ishlash qobiliyatiga ega. Quyida Pythonning eng yaxshi 5 ta grafik kutubxonasi haqida foydali ma'lumotlar keltirilgan:

1. PyQt5b
2. Python Tkinter
3. PySide 2
4. Kivi
5. wxPython

Dasturlash tillaridan foydalanib turli dasturlar tuzish davomida GUI dasturlar tuzish degan tushunchaga duch kelinadi. Xo'sh GUI o'zi nima?

GUI (Graphics User Interface)- bu shakllar, hujjatlar, testlar va boshqalar kabi turli vaziyatlarda foydalanuvchilardan javob olish uchun dasturchi tomonidan ishlab chiqilgan interaktiv muhit. Bu foydalanuvchiga an'anaviy Buyruqlar qatori interfeysi (CLI) ga qaraganda chiroyli interaktiv ekranni taqdim etadi.

Python dasturlash tili uchun eng yaxshi GUI kutubxonalarini ko'rib chiqaylik:

1) **PyQt5**. PyQt5 - bu Python uchun grafik foydalanuvchi interfeysi (GUI). Bu dasturchilar orasida juda mashhur va GUI kodlash yoki QT dizayner dasturi tomonidan ishlab chiqilishi mumkin. QT freymworki foydalanuvchi interfeyslarini yaratish uchun vidjetlarni bevosita sichqoncha yordamida tanlab kerakli joyga joylashtirish imkonini beruvchi vizual tizimdir.

Bu bepul va ochiq kodli bog'lovchi dasturiy ta'minot bo'lib, crossplatformali ilovalarni ishlab chiqish uchun xizmat qiladi. U Windows, Mac, Android, Linux, Arduino va Raspberry PI da qo'llanilishi mumkin.

PyQt5 ni o'rnatish uchun quyidagi buyruqdan foydalaniladi:

```
pip install pyqt5
```

Bu yerda oddiy koddan foydalanib, PyQt5 kutubxonasidan qanday foydalanishni ko'rsatib o'tilgan:


```

from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

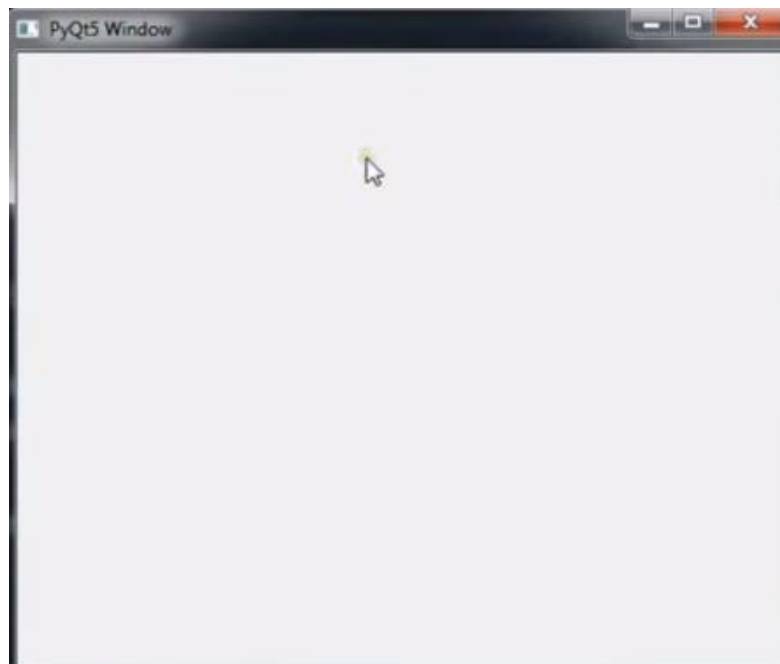
class Window(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setGeometry(300, 300, 600, 400)
        self.setWindowTitle("PyQt5 window")
        self.show()

app = QApplication(sys.argv)
window = Window()
sys.exit(app.exec_())

```

Dastur natijasi:



28-rasm. PyQt 5 kutubxonasi interfeysi ko‘rinishi

PyQt – bu python dasturlash tili va Qt platformasini bir-biri bilan hamjihatlikda ishlashiga imkon beruvchi mukammal bir kutubxona. U murakkab grafik interfeyslar uchun maxsus vidjetlar yaratish uchun o‘rnatilgan vidjetlar va asboblarning boy

tanlovini, shuningdek, ma'lumotlar bazalariga ulanish va ular bilan o'zaro ishlash uchun mustahkam SQL ma'lumotlar bazasini qo'llab-quvvatlaydi.

2) **Python Tkinter**. Tkinter – bu python dasturlash tilining o'rnatilgan grafik kutubxonasi. Tkinter desktop ilovalarini ishlab chiqish uchun eng mashhur Python GUI kutubxonalaridan biri bo'lib, u standart grafik interfeys TK va pythonning kombinatsiyasi hisoblanadi.

Tkinter GUI ilovasida ishlatiladigan teglar, tugmalar, matn maydonlari, tasdiqlash yacheykalari kabi turli xil vidjetlarni taqdim etadi.

Tugmalarni boshqarish vidjetlari ilovalarni ko'rsatish va loyihalash uchun, *canvas* vidjeti esa ilovada *chiziqlar*, *ko'pburchaklar*, *to'rtburchaklar* va shu kabi shakllarni chizish uchun ishlatiladi.

Tkinter Pythonning o'rnatilgan kutubxonasi bo'lganligi uchun uni boshqa grafik kutubxonalar kabi alohida o'rnatishni talab qilinmaydi. Quyida Tkinter kutubxonasi yordamida oddiy dastur kodi keltirilgan.

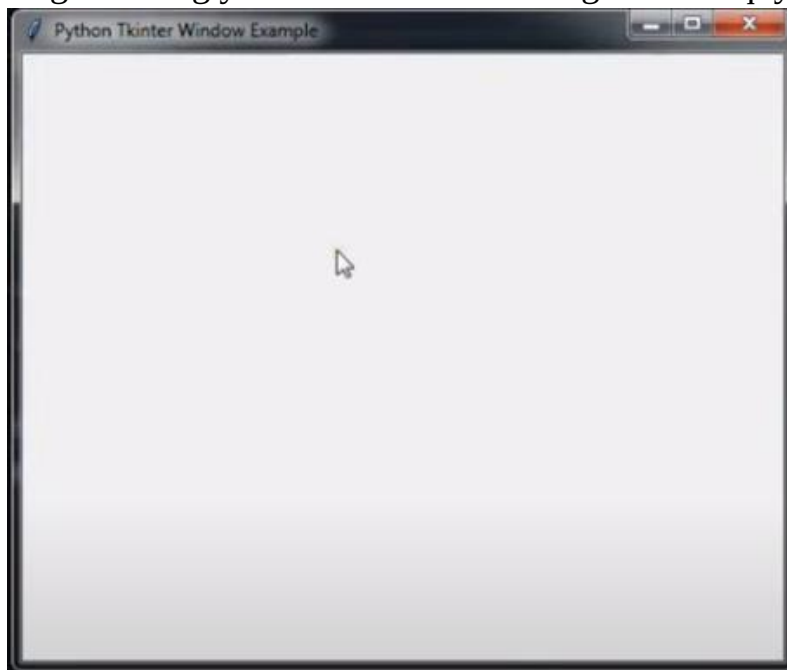
```
from tkinter import *

class Root(Tk):
    def __init__(self):
        super(Root, self).__init__()

        self.title("Python Tkinter")
        self.minsize(500,400)

root = Root()
root.mainloop()
```

Yuqoridagi kodning yuklanishidan hosil bo‘lgan ekran quyidagicha bo‘ladi:



29-rasm. Tkinter kutubxonasi interfeysining ko‘rinishi

3) **PySide 2**. Python dasturlash tilining navbatdagi GUI kutubxonasi – bu PySide2 yoki uni boshqacha qilib Python uchun QT deb nomlash ham mumkin. PySide Python ilovalarida API ishlatilishiga ruxsat berish uchun Qt (PySide2) uchun rasmiy Python ulanishlarini va Pythonda C++ loyihalarini ifodalash uchun ishlatilishi mumkin bo‘lgan bog‘lovchi generator vositasini (Shiboken2) taklif etadi.

Python Qt platformasi bilan LGPLv3/GPLv3 litsenziyasi va Qt tijorat litsenziyasi ostida birgalikda ishlashi mumkin.

Shunday qilib, ushbu paket pythonning o‘rnatilgan paketlari tarkibida emasligi uchun alohida o‘rnatish talab qilinadi. Quyida PySide paketining o‘rnatilish jarayoni va undan boshlang‘ich foydalanish uchun oddiy misol keltirilgan:

Shunday qilib **PySide** paketini kompyuterga o‘rnatish uchun konsolga quyidagi komandani terish kerak bo‘ladi (Albatta kompyuterga python o‘rnatilgan bo‘lishi kerak):

```
pip install PySide2
```

PySide2 kutubxonasi o‘rnatilgandan so‘ng. Quyidagi oddiy dastur kodini kompyuteringizga kiritib ishlatib ko‘rish mumkin.

```

from PySide2.QtWidgets import QtWidgets, QApplication
import sys

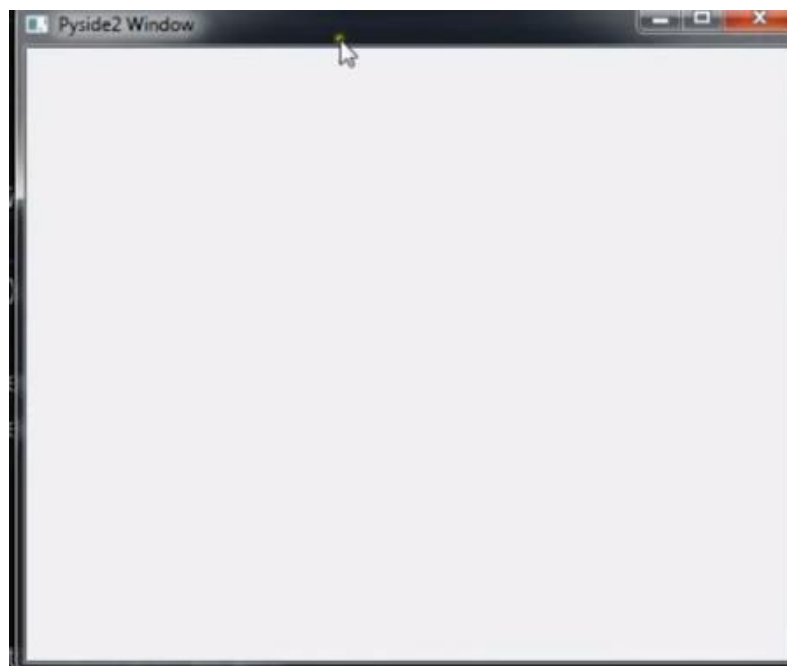
class Window(QtWidgets):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Pyside2 Window")
        self.setGeometry(300,300,500,400)

app = QApplication(sys.argv)
window=Window()
window.show()
app.exec_()

```

Yuqoridagi kodning ishga tushirishdan hosil bo'lgan oyna quyidagicha ko'rinishda bo'ladi:



30-rasm. PySide 2 kutubxonasi interfeysining ko'rinishi

4) **Kivy**. Kivy - bu ko'p sensorli ilovalar kabi innovatsion foydalanuvchi interfeyslaridan foydalanadigan, tez rivojlanayotgan ilovalar uchun ochiq manbali Python kutubxonasi.

Kivy Linux, Windows, OS X, Android, iOS va Raspberry Pi tizimlarida keng foydalaniladi. Barcha qo'llab-quvvatlanadigan platformalarda bir xil kodni ishlatish mumkin. U WM_Touch, WM_Penn, Mac OS X trek paneli va Magic Mouse, Netdev,

Linux HID yadrosi kabi ko'pgina kirishlar, protokollar va qurilmalardan tabiiy ravishda foydalanishi mumkin.

Kivy MIT litsenziyasi ostida foydalanish uchun 100% bepul python kutubxonasi hisoblanadi.

Asboblarni to'plamini professional tarzda ishlab chiqilgan, turli grafik ilovalar yaratishni qo'llab-quvvatlaydi va foydalaniladi. Kivy paketining tijorat mahsulotidan bimalol foydalanish mumkin. Freymvork stabil va yaxshi hujjatlashtirilgan API, shuningdek, boshlashga yordam beradigan dasturlash yo'riqnomasiga ega.

Kivy grafik dvigateli OpenGL ES 2 da zamonaviy va tezkor grafik konveyeridan foydalangan holda ishlab chiqilgan.

Ushbu kutubxonaning asboblarni to'plamini 20 dan ortiq vidjetlar bilan birga keladi, ularning barchasi imkoniyat jihatdan kengaytiriladi. Ko'p qismlar Cython yordamida C tilida yozilgan va regressiya testlari bilan sinovdan o'tkazilgan.

Kivyni o'rnatishdan oldin "glew" komponentasini o'rnatish kerak bo'ladi. Buning uchun konsolga quyidagi pip buyrug'ini yozish kerak bo'ladi:

```
pip install docutils pygments pypiwin32 kivy.deps.sdl2  
kivy.deps.glew
```

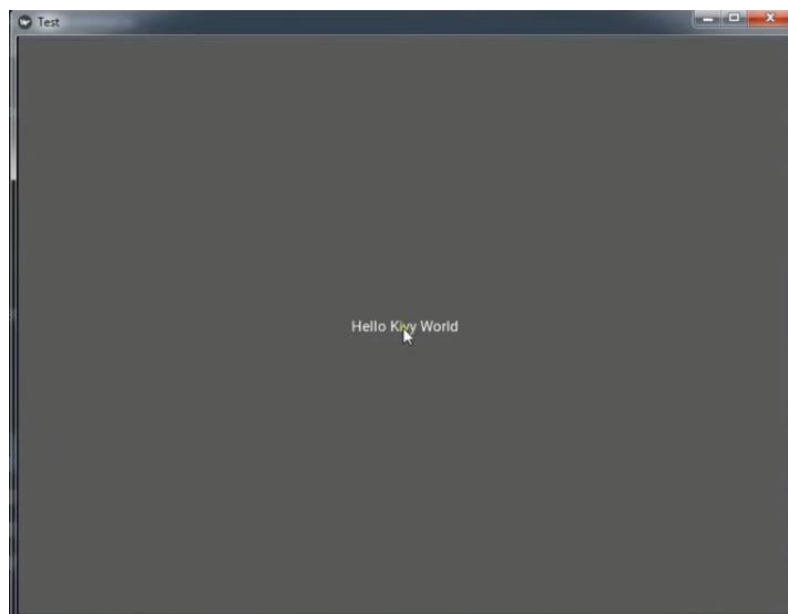
Shundan so'ng, Kivy paketini o'rnatish uchun ushbu buyruqni konsolga kiritiladi:

```
pip install Kivy
```

Barcha o'rnatishlar tugaganidan so'ng, Kivy paketidan foydalanib quyidagi dasturni ishlatishimiz mumkin:

```
from kivy.app import App  
from kivy.uix.button import Button  
  
class TestApp(App):  
    def build(self):  
        return Button(text= " Hello Kivy World ")  
  
TestApp().run()
```

Yuqoridagi kodning chiqishi quyidagicha ko'rinadi:



31-rasm. Kivy kutubxonasi interfeysining ko‘rinishi

5) **wxPython**. wxPython - bu Python dasturlash tili uchun platformalararo grafik asboblarni to‘plam.

Bu Python dasturchilariga sodda va oson mustahkam, yuqori funktsional grafik foydalanuvchi interfeysi bilan dasturlar yaratish imkonini beradi. U C++ tilida yozilgan mashhur **wxWidgets** krossplatforma kutubxonasining GUI komponentlarini qamrab olgan Python kengaytma modullari to‘plam sifatida amalga oshiriladi.

Python va **wxWidgets** singari, **wxPython** ham python tilining ochiq manbali kutubxonasi hisoblanadi.

wxPython - bu python dasturlash tilining krossplatformali to‘plam. Bu shuni anglatadiki, bitta dastur bir nechta platformalarda - Microsoft Windows, Mac OS X va macOS va Linux hech qanday o‘zgartirishlar kiritilmasdan ishlashi mumkin.

wxPython paketini kompyuterga o‘rnatish quyidagicha amalga oshiriladi: birinchi navbatda kompyuterga python o‘rnatilganligiga ishonch hosil qilish uchun konsolga: `python -V` buyrug‘i yoziladi va agar konsolga pythonning versiyasi haqidagi ma’lumot chop etilsa, demak python tili kompyuterga o‘rnatilgan bo‘ladi.

Shundan so‘ng **wxPython** paketini o‘rnatish uchun konsolga quyidagi buyruqni kiritiladi:

```
pip install wxPython
```

wxPython kutubxonasidan foydalanib quyidagi oddiy dasturni tuzaylik:

```

import wx
class MyFrame(wx.Frame):
    def __init__(self, parent, title):
super(MyFrame, self).__init__(parent, title=title, size=(400, 300))
    self.panel=MyPanel(self)

class MyPanel(wx.Panel):
    def __init__(self, parent):
        super(MyPanel, self).__init__(parent)

class MyApp(wx.App):
    def OnInit(self):
        self.frame=MyFrame(parent=None, title= "wxPython Window")
        self.frame.show()
        return True
app = MyApp()
app.MainLoop()

```

Yuqoridagi kodning ishga tushirishdan hosil bo‘lgan dastur oynasi:



32-rasm. **wxPython** kutubxonasi interfeysining ko‘rinishi

Shunday qilib, yuqorida ko‘rib chiqilgan Python GUI kutubxonalaridan birortasini tanlash uchun avvalambor tuziladigan dasturning qanday vazifa bajarishiga qarash kerak bo‘ladi. Ko‘p hollarda universal kutubxonalardan foydalangan ma’qulroq bo‘ladi va qolgan kutubxonalardan nisbatan kuchliroq bo‘lgan GUI freymvorklarini tafsiya qilish mumkin: PySide2 va PyQt5 kutubxonalarini. Ammo ushnbu kutubxonalardan to‘laonli foydalanish uchun ularning litsenziyasini sotib olish kerak bo‘ladi. Va bu holat ularning nima uchun boy xususiyatlarga ega ekanligini

tushunishga asos bo‘ladi. Qolgan GUI kutubxonalari: Tkinter, Kivy va wxPython lar bepul GUI kutubxonalar hisoblanadi.

Nazorat savollari:

1. GUI deganda nimani tushunasiz?
2. Tkinter paketi va uning xususiyatlari.
3. Qanday grafik paketlarni bepul o‘rnatish mumkin?
4. Tkinter paketi va uning xususiyatlari haqida gapirib bering.
5. PySide2 paketining xususiyatlari qanday?

8.2.§. PyQt5 paketini o‘rnatish.

PyQt - Python dasturlash tili uchun Qt grafik freymvorki uchun Python kengaytmasi sifatida amalga oshirilgan kengaytmalar (bog‘lashlar) to‘plami.

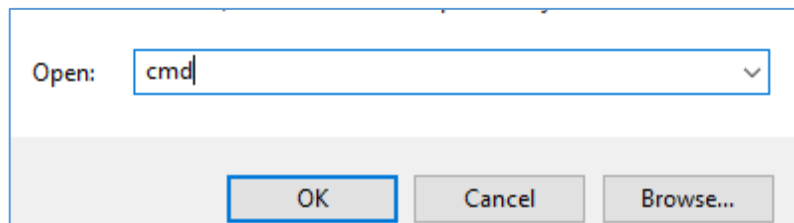
PyQt Britaniyaning Riverbank Computing kompaniyasi tomonidan ishlab chiqilgan python kutubxonasi. PyQt Qt tomonidan qo‘llab-quvvatlanadigan barcha platformalarda ishlaydi: Linux, UNIX, macOS va Windows va shu kabi operatsion tizimlar. Hozirgi kungacha PyQt ning 3 ta versiyasi ishlab chiqilgan: PyQt4, PyQt5 va PyQt6. PyQt kutubxonasi GPL (pythonning 2 va 3 versiyalari) va tijorat litsenziyalari ostida tarqatiladi.

PyQt deyarli to‘liq Qt imkoniyatlarini qamrab olgan. Bu esa 600 dan ortiq sinflar, 6000 dan ortiq funktsiyalar va metodlar degani.

PyQt shuningdek, QtDesigner nomli GUI dizaynerini ham o‘z ichiga oladi. QtDesigner dasturida yaratilgan *.ui formatli dizayn faylini Python faylga o‘girib, qayta ishlash yoki to‘g‘ridan-to‘g‘ri *.ui fayli bilan *.py fayllari o‘rtasida bog‘lanish hosil qilish uchun **pyuic** kutubxonasidan foydalaniladi. QtDesigner dasturi PyQtni tez prototiplash uchun juda foydali vositaga aylantiradi. Bundan tashqari, Qt Designer dasturiga Pythonda yozilgan yangi grafik boshqaruv elementlarini qo‘shish mumkin.

PyQt5 paketini o‘rnatish. Pythonning PyQt5 paketini Windows operatsion tizimiga o‘rnatish uchun quyidagi amallarni bajarish kerak:

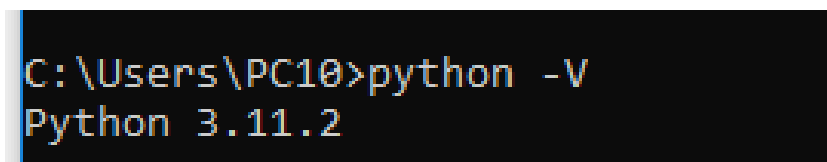
1-qadam: Python operatsion tizimga oʻrnatilganligiga ishonch hosil qilish kerak. Buning uchun klaviaturadan **WIN+R** klaviaturalarini bir vaqtda bosiladi va ekranda **Выполнить** oynasi paydo boʻladi. Bu oynaga **CMD** buyrugʻi yoziladi va ok tugmachasini bosiladi:



33-rasm. **Выполнить** oynasi koʻrinishi

Shundan soʻng consol oynasi ochiladi va u yerga operatsion tizimda python oʻrnatilganligini bilish maqsadida oddiy pythonning versiyasini bilib beruvchi buyruqni kiritamiz: **python -V**

Shundab soʻng, agar tizimda python allaqachon oʻrnatilgan boʻlsa, buyruqning natijasi pythonning versiyasi boʻladi :



34-rasm. Python versiyasini aniqlash

Aks holda, xatolik yuzaga keladi va python oʻrnatilishi kerak boʻladi.

2-qadam. Kompyuterga PyQt5 oʻrnatilmaganligiga ishonch hosil qilish.

Bunda foydalanuvchi PyQt avvaldan oʻrnatilmaganligiga ishonch hosil qilish uchun quyidagi buyruqni kiritishi kerak.

Consolni ochiladi va u yerga pythonning oʻrnatilgan barcha paketlari roʻyxatini chiqaruvchi buyruqni yoziladi: **pip list**.

Shunday qilib PyQt5 tizimga oʻrnatilmagan yoki oʻrnatilganligini bilib olinadi.

3-qadam. PyQt-ni oʻrnatish. Demak operatsion tizimga python dasturi oʻrnatilganligini va PyQt5 paketi oʻrnatilmaganligiga ishonch hosil qilindi.

Endi PyQt5 paketini tizimga oʻrnatish uchun foydalanuvchi quyida keltirilgan buyruqni konsolga kiritishi kerak: **pip install PyQt5**

```

PS C:\Users\ksahn> pip list
Package                                Version
-----
charset-normalizer                     2.0.9
cyclor                                 0.11.0
fonttools                              4.28.3
idna                                    3.3
imutils                                0.5.4
kiwisolver                             1.3.2
matplotlib                             3.5.1
mysql-connector-python                 8.0.27
numpy                                   1.21.4
opencv-contrib-python                  4.5.4.6
packaging                              21.3
Pillow                                 8.4.0

```

35-rasm. O'rnatilgan paketlar ro'yxatini aniqlash.

Shundan so'ng hech qanday xatoliklar sodir bo'lmasa, PyQt5 tizimga muvaffaqiyatli o'rnatiladi va 2-qadamni takrorlash orqali foydalanuvchi PyQt o'rnatilganligini tekshirishi mumkin.

```

PS C:\Users\ksahn> pip install PyQt5
Collecting PyQt5
  Downloading https://files.pythonhosted.org/packages/3b/d3/76693500af5d323160/PyQt5-5.13.0-5.13.0-cp35.cp36.cp37.cp38-none-win32.whl (49.7MB)
    100% |#####| 49.7MB 97kB/s
Requirement already satisfied: PyQt5_sip<13,>=4.19.14 in c:\python38\lib\site-packages (from PyQt5)
Installing collected packages: PyQt5
Successfully installed PyQt5-5.13.0
You are using pip version 19.0.3, however version 19.1.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

```

36-rasm. PyQt 5 paketini o'rnatish

Shunday qilib Windows muhitida Python uchun PyQtni muvaffaqiyatli o'rnatildi.

QtDesignerni windowsga o'rnatish. QtDesigner dasturi **pyqt5-tools** to'plami bilan birga o'rnatiladi. Va uni o'rnatish juda oddiy. Kompyuterning konsol oynasiga quyidagi buyruqni yoziladi va ishga tushiriladi:

```
pip install pyqt5-tools
```

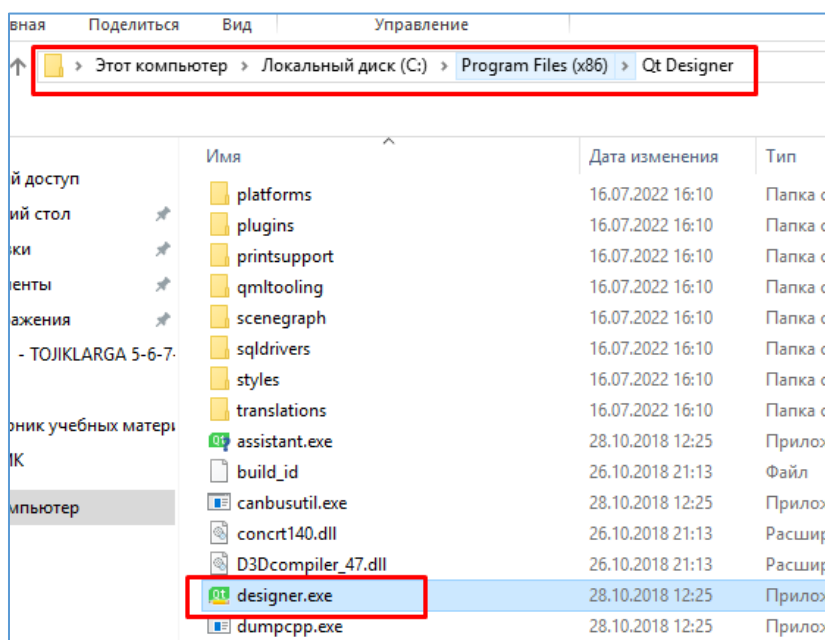
Agar yuqoridagi buyruq ishlamasa, unda quyidagi- pip buyrug'i pip3 buyrug'I bilan almashtirilgan buyruqni sinab ko'rish tavsiya etiladi:

```
pip3 install pyqt5-tools
```

QtDesigner-ni ishga tushiring.

Agar hammasi normada o'rnatilgan bo'lsa, unda bimalol dasturni ishga tushirib undan foydalanish mumkin bo'ladi. Buning uchun oldin QtDesigner dasturi o'rnatilgan joyni topish kerak. U odatda python o'rnatilgan kotalogda bo'ladi:

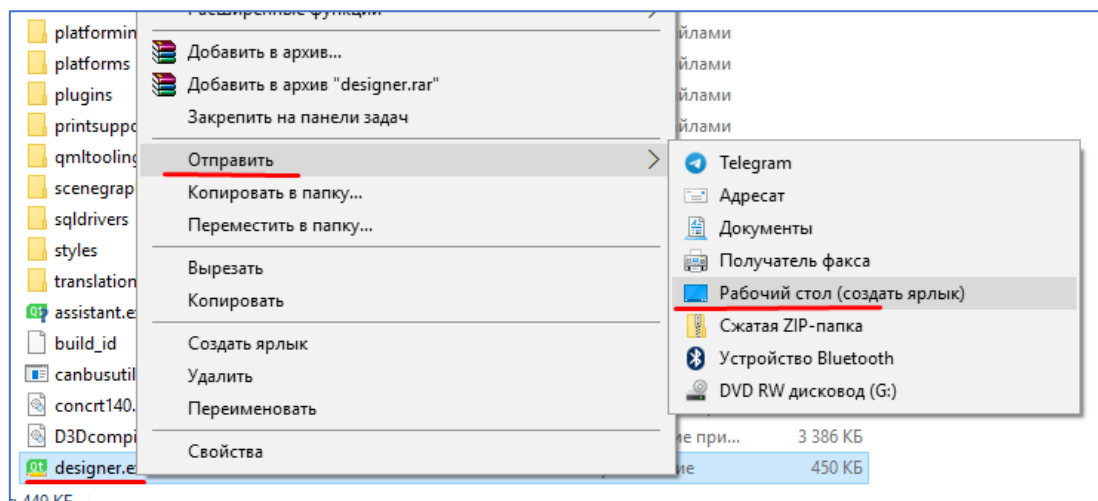
C:\Program Files (x86)\Qt Designer



37-rasm. Qt Designer dasturini ishga tushirish .

Odatda yorliq yaratish va uni ish stoliga joylashtirish tavsiya etiladi (38-rasm):

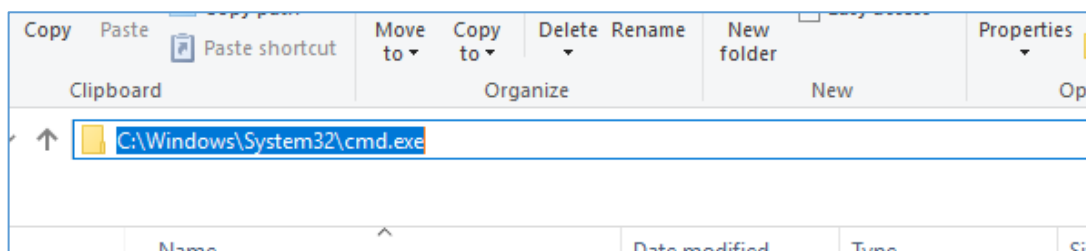
Endi dizayn dasturini ishga tushirish va python tilidagi dasturning grafik interaktiv qismini yaratishni boshlash mumkin.



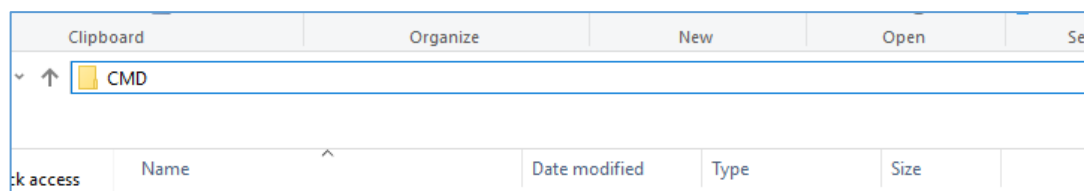
38-рasm. Qt Designer yorliq dasturini yaratish

QtDesigner dasturidan yaratilgan, ilova ko‘rinishini o‘zida saqllovchi faylni ichida python kodlarini joylashtirish uchun ushbu faylni python faylga o‘girib olish talab qilinadi. Buning uchun quyidagi amallarni bajarish kerak bo‘ladi:

1. QtDesigner dasturida yaratilgan **.ui** formatli faylni topiladi;
2. Ushbu fayl saqlangan kotalogdan turib konsolni ochiladi. Buning uchun kotlaogning **URL** manzili yoziladigan joyga **CMD** deb yoziladi va **enter** tugmasini bosish orqali konsolni kerakli kotologda ochish amali bajariladi (39-, 40-rasmlar):



39-rasm. Papkaning URL manzili yoziladigan joyi



40-rasm. CMD buyrug‘i yozilishi kerak bo‘lgan joy.

3. Ochilgan konsol oynasiga quyidagi buyruqni yoziladi. Faqat `ui_file_name` yozuvi o‘rniga QtDesigner dasturining fayli nomi yozilsa, `python_file_name` o‘rniga hosil bo‘ladigan python fayl nomi yozilishi kerak bo‘ladi:

```
pyuic5 -x "ui_file_name".ui -o "python_file_name".py
```

Shundan so‘ng ushbu kotalogda ko‘rsatilgan nomdagi .py formatdagi fayl ham paydo bo‘lganini ko‘rishingiz mumkin. Va ushbu python faylning tarkibiga turli python kodlarini qo‘shish, vidgetlarining qiymatlarini olish va ularni ishga tushirish uchun turli funksiyalar class, metodlar qo‘shish imkoniyati mavjud.

Nazorat savollari :

1. Python PyQt 5 paketining xususiyatlarini tavsiflab bering .
2. Windows muhiti uchun PyQt5 paketi qanday o‘rnatiladi?
3. QtDesigner dasturini o‘rnatish qanday amalga oshiriladi?
4. QtDesigner dasturining .ui faylini .py formatli faylga o‘girish uchun qanday amallarni bajarish kerak bo‘ladi?
5. Nima uchun .ui formatli faylni python faylga og‘irish kerak?

8.3.§. PyQt5 da dastur tuzilishi

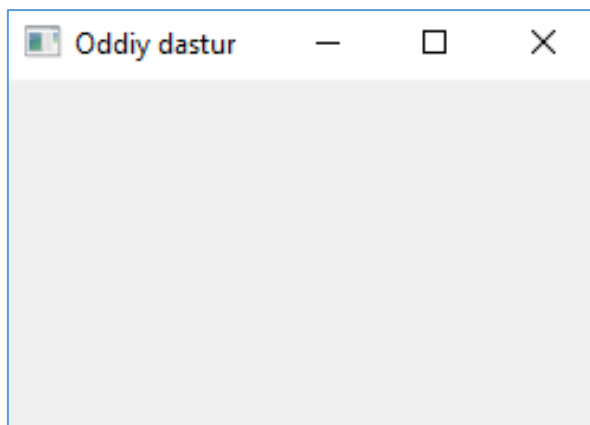
Bu kichik oynani ko‘rsatadigan oddiy dastur. Biroq, bu oynaning imkoniyatlari ko‘p va ushbu oyna yordamida juda ko‘plab turli vazifalarni amalga oshirish mumkin. Masalan oddiygina uning o‘lchamlarini o‘zgartirish, kengaytirish yoki yig‘ish mumkin.

PyQt5 yuqori darajadagi dasturiy vositadir. Agar PyQt da yozilgan kodlarning barchasini past darajada yozadigan bo‘lsak, ya’ni tayyor funksiyalarda emas balki oddiy kodlardan foydalanib yozadigan bo‘lsak, u holda quyidagi kod misoli osongina yuzlab satrlarni qamrab olishi mumkin. Ammo PyQt5 paketida bu funksiyalarni alohida yaratishga xojat yo‘q:

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget

if __name__ == "__main__":
    app = QApplication(sys.argv)
    w = QWidget()
    w.resize(250, 150)
    w.move(300, 300)
    w.setWindowTitle(Oddiy dastur)
    w.show()
    sys.exit(app.exec_())
```

Dastur tahlili: Yuqoridagi kodni ishga tushirish orqali ekranda kichik oyna paydo qiladi:



41-rasm. Dastur natijasi.

Dasturda birinchi kerakli kutubxonalarini chaqiriladi va tizimli funksiyalar uchun `sys` kutubxonasi va `PyQt5` kutubxonalarini import qilib olinmoqda. `PyQt5` kutubxonasining `QtWidgets` paketi - vidjetlar bilan ishlashga ruxsat beruvchi paket hisoblanadi.

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget
```

- Ushbu **QtWidgets** paketining **QApplication** va **QWidget** sinflari alohida-alohida vazifalarni bajarishadi. **QApplication** sinfi kerakli oynani boshqarish uchun ishlatilganda, **QWidget** sinfi orqali kerakli vidjetlarni boshqariladi.

```
app = QApplication(sys.argv)
```

- Har bir `PyQt5` ilovasida dastur ob'ekti (`QApplication` sinfining ob'yekti) yaratish kerak. `sys.argv` -bu parametr bo'lib, buyruq qatori argumentlari ro'yxatini anglatadi. Bu shuni anglatadiki Python skriptlari buyruq satridan - konsoldan ishga tushirilishi mumkin.

```
w = QWidget()
```

- **QWidget** vidjeti `PyQt5` dagi barcha foydalanuvchi interfeysi obyektlari uchun asosiy sinf hisoblanadi va **QWidget** uchun standart konstruktorni taqdim etiladi. Ushbu standart konstruktor boshqa birorta widgetga tegishli bo'lmaydi. Bunday vidjet oyna deb ataladi.

```
w.resize(250, 150)
```

- `Resize()` metodi vidjet o'lchamini o'zgartiradi. Uning kengligi 250 piksel va balandligi 150 pikselga tenglashtiriladi.

```
w.move(300, 300)
```

- `move()` metodi ikkita parametрни: `x` va `y` ni qabul qiladi. `x` – bu gorizontal ya'ni chapdan o'ngga siljishi bo'lsa, `y` – vertikal ya'ni tepadan pastga qarab siljishi tushuniladi. Bu yerda vidjetni ekranda `x=300`, `y=300` ga siljitadi.

```
w.setWindowTitle('Oddiydastur')
```

- Bu yerda `setWindowTitle` metodi yordamida ilova oynasining sarlavhasini o'rnatiladi.

```
w.show()
```

- `show()` metodi vidjetlarni ekranda aks ettiradi. Vidjet avval xotirada yaratiladi va shundan keyingina (`show` metodi yordamida) ekranda ko'rsatiladi.

```
sys.exit(app.exec_())
```

- Nihoyat, asosiy dastur tsikliga yetib keldik. Hodisalarni qayta ishlash shu nuqtadan boshlanadi. Asosiy tsikl oyna tizimidan hodisalarni qabul qiladi va ularni ilova vidjetlariga tarqatadi. Agar `exit()` metodini chaqirilsa yoki asosiy vidjet yo'q qilingan bo'lsa, asosiy tsikl tugaydi. `sys.exit()` metodi bo'lsa, dastur yuklanishini kafolatlaydi.

`exec_()` metodi ikkita pastki chiziqqa ega ekanligining sababi, pythonning oldingi 2 – versiyasida ham `exec()` metodi mavjud ekanligida.

Ilova icon

Ilova belgisi odatda sarlavhaning yuqori chap burchagida ko'rsatiladigan kichik rasmdir. Bu rasmni hohlagan bir boshqa rasmga o'zgartirish mumkin. Buning uchun `setIcon()` metodidan foydalaniladi va parametr sifatida tanlangan rasmning URL manzili ko'rsatiladi.

Nazorat savollari:

1. PyQt5 da dasturga qanday kutubxonalarni import qilish talab qilinadi?

2. QApplication(sys.argv) – sinfining vazifasi nima?
3. QWidget() – sinfi qanday vazifalarni amalga oshiradi?
4. Ilova ekrani o‘lchamlarini o‘zgartirish qanday amalga oshiriladi?
5. Ilovaning joylashgan joyini qanday o‘zgartirish mumkin?
6. Ilova sarlavha belgisini o‘zgartirish qanday amalga oshiriladi?

8.4.§. PyQt5 paketining asosiy vidjetlari

PyQt5 paketi bilan tanishishga bag‘ishlangan ushbu navbatdagi mavzuda quyidagi vidjetlar bilan tanishib chiqiladi: **QLabel**, **QPushButton**, **QLineEdit**, **QComboBox**, **QCheckBox**, **QRadioButton**, **QTextEdit**, **QInputDialog**, **QMessageBox**, **QProgressBar**, **QSlider**, **QFileDialog**, **QPixmap** va **QSplitter**.

8.4.1.§. QLabel vidjeti

QLabel matn **satrlarini** ko‘rsatish qobiliyatiga ega PyQt-dagi eng oddiy vidjetlardan biridir. Ushbu vidjet xohlagan vaqtda ushbu matnni olish va yangilash imkonini beruvchi ko‘plab yordamchi funksiyalar va usullar bilan birga keladi.

QLabel bilan bog‘liq barcha usullarning to‘liq ro‘yxatini ushbu sahifaning pastki qismida topish mumkin.

QLabel yaratish. Quyidagi misolda PyQt5 paketi yordamida oddiy QLabel yaratilishini ko‘rsatib o‘tilgan:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,500,300)
win.setWindowTitle("CodersLegacy")
label = QtWidgets.QLabel(win)
```



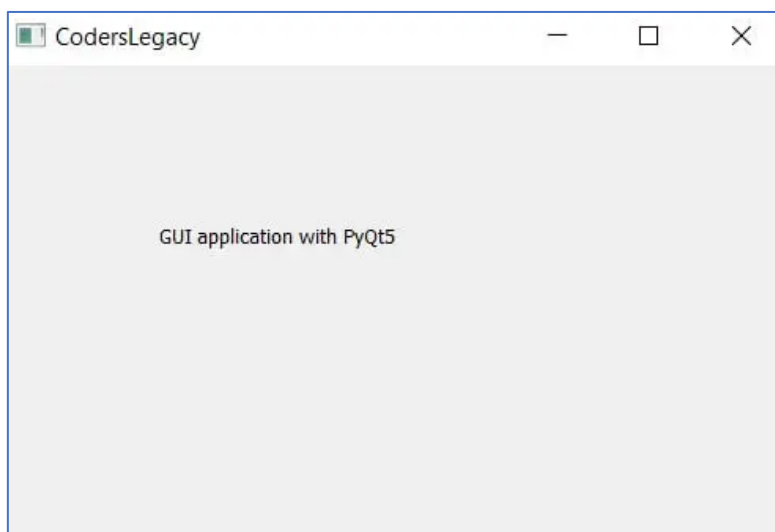
```

label.setText("GUI application with PyQt5")
label.adjustSize()
label.move(100,100)

win.show()
sys.exit(app.exec_())

```

Ushbu kodning chiqishi quyida ko‘rsatilgan:



42-rasm. PyQt5 QLabel

QLabel bilan bog‘liq kodni satr bo‘yicha muhokama qilib chiqaylik.

```
label = QtWidgets.QLabel(win)
```

QtWidgets.QLabel() ga murojaat qilish GUI uchun foydalanish mumkin bo‘lgan yorliq ob'ektini yaratadi. Biz yaratgan (win) oyna ob'ektini uning parametrlari sifatida berilishi yorliq ushbu oynaning bir qismi ekanligini anglatadi.

```
label.setText("GUI application with PyQt5")
```

Yuqoridagi oddiy kod labelga matn o‘rnatadi.

```
label.adjustSize()
```

adjustSize() QLabel o‘z o‘lchamini avtomatik ravishda sozlashiga imkon beradi.

```
label.move(100,100)
```

Ushbu **move (100, 100)** labening dastur ekranining yuqori chap burchagidagi (100 , 100) koordinatalarida paydo bo‘lishini ta’minlaydi. Yodda tutingki, **ekranning** yuqori chap burchagida (0, 0) pozitsiya mavjud.

QLabel qiymatlarini olish va yangilash

Ushbu bo‘limda hozirda QLabel da ko‘rsatilgan matnni qanday yangilashni, shuningdek QLabel da ko‘rsatilayotgan joriy matnni qanday qilib olish mumkinligini ko‘rib chiqamiz. Bunday ishlarni qilish uchun QPushButton vidjetidan foydalanish kerak, u kerakli buyruqlarni o‘z ichiga olgan funksiyalarni chaqiradi.

QLabelni yangilash uchun avvalgi **setText()** usulidan foydalaniladi va matn qiymatini olish uchun ushbu **text()** usulidan foydalaniladi:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

def update():
    matn.setText("Yangilandi!")

def retrieve():
    print(matn.text())

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,500,300)
win.setWindowTitle("QLabel widgetining ishlashi")

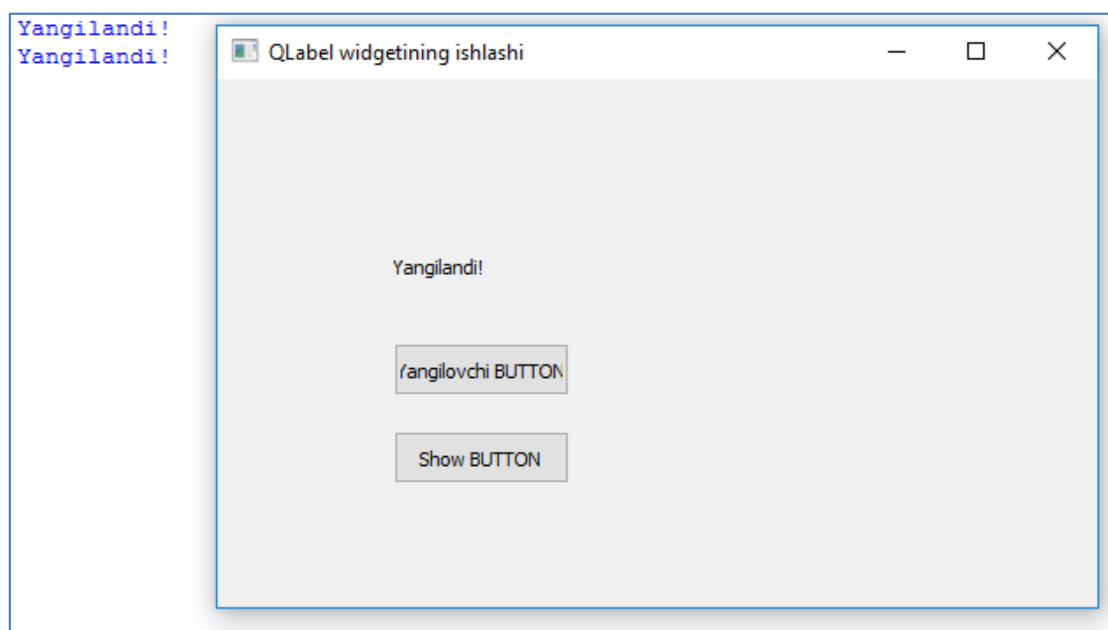
matn = QtWidgets.QLabel(win)
matn.setText("PyQt5 paketida GUI dasturlari")
matn.adjustSize()
matn.move(100,100)

upd_btn = QtWidgets.QPushButton(win)
upd_btn.clicked.connect(update)
upd_btn.setText("Yangilovchi BUTTON")
upd_btn.move(100,150)

show_btn = QtWidgets.QPushButton(win)
show_btn.clicked.connect(retrieve)
show_btn.setText("Show BUTTON")
show_btn.move(100,200)

win.show()
sys.exit(app.exec_())
```

Yuqoridagi kodni ishga tushirgandan so‘ng ikkita QPushButton va QLabel vidjetlari orqali ishlovchi ilova paydo bo‘ladi. Unda QLabelga **“PyQt5 paketida GUI dasturlari”** yozuvi, buttonlarning biriga **“Yangilovchi BUTTON”**, ikkinchisiga esa **“Show BUTTON”** yozuvlari yozilgan bo‘ladi. Ilovaning ishlashi quyidagicha: **“Yangilovchi BUTTON”** tugmachasi bosilganda labelning matnini **“Yangilandi!”** yozuviga o‘zgartiruvchi update funksiyasini ishga tushiradi. **Show BUTTON** tugmachasi bosish orqali esa **“retrieve”** funksiyasini ishga tushirish orqali QLabel ning text xususiyatidagi yozuvni konsolga chiqariladi.



43-rasm. QLabel uchun yozilgan dasturning natijasi.

Label vidjeti uchun mavjud bo‘lgan eng muhim metodlar:

8-jadval

Metod nomi	Tavsif
.setAlignment()	Label ichidagi matnni tekislash. U 4 xil qiymatlardan birini olishi mumkin: <i>Qt.AlignLeft</i> , <i>Qt.AlignRight</i> , <i>Qt.AlignCenter</i> va <i>Qt.AlignJustify</i> .
.setIndent()	Label matni uchun “tab”larni belgilaydi.
.setPixmap()	Labelga qo‘yilgan rasmni ko‘rsatish uchun ishlatiladi.
.text()	Label uchun (ko‘rsatilgan) matn qiymatini qaytaradi.
.setText()	Labelda ko‘rsatiladigan matnni o‘rnatadi.

Metod nomi	Tavsif
.selectedText()	Foydalanuvchi tomonidan tanlangan matnni qaytaradi. (Buning ishlashi uchun textInteractionFlag TextSelectableByMouse xususiyatlariga o'rnatilishi kerak).
.setWordWrap()	Label matnini avtomatik ravishda keying qatorga tushadigan xususiyatini yoqish.
.adjustSize()	Label o'lchamini ko'rsatilayotgan matnga avtomatik moslashtiradi.

Nazorat savollari:

1. QLabel vidjetidan qanday maqsadda foydalaniladi?
2. QLabel vidjetining adjustSize() metodining vazifasi qanday?
3. QLabel vidjetining move() metodining vazifasi qanday?
4. Label ichidagi matnni tekislash uchun qaysi metod ishlatiladi?

8.4.2.§. QPushButton vidjeti

PyQt5-dagi eng asosiy va keng tarqalgan vidjetlardan biri bu **QPushButton**. Nomidan ko'rinib turibdiki, bu tugma bosilganda (clicked) funktsiyani ishga tushiradi.

QPushButton vidjetini yaratish

QtWidgets.QPushButton() funktsiyasidan foydalanib, QPushButton vidjetini yaratish mumkin. Keyin ushbu vidjet matni va joylashuvi *setText()* va *move()* kabi metodlar yordamida sozlanadi.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

WIDTH = 500
HEIGHT = 300

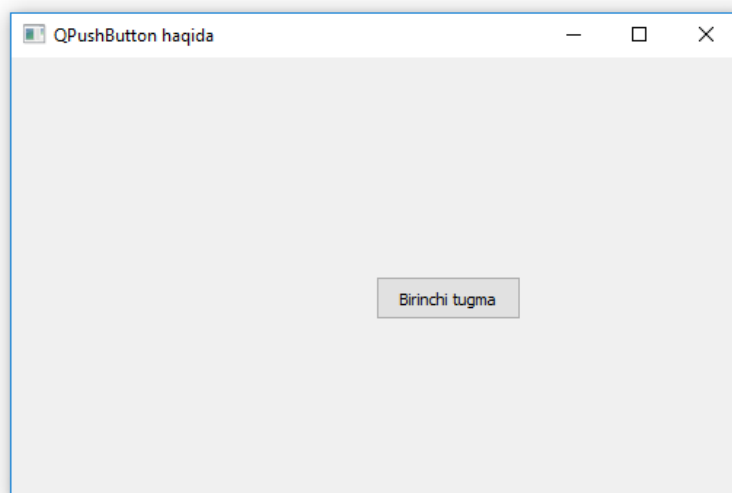
app = QApplication(sys.argv)
win = QMainWindow()
```

```
win.setGeometry(400,400,WIDTH,HEIGHT)
win.setWindowTitle("QPushButton haqida")

button = QtWidgets.QPushButton(win)
button.setText("Birinci tugma")
button.move(WIDTH//2, HEIGHT//2)

win.show()
sys.exit(app.exec_())
```

Quyida yuqoridagi kodning chiqishi keltirilgan. Bu tugma hozircha biron bir funksiyaga bog‘lanmadi, shuning uchun uni bosish orqali hech qaday amal bajarilmaydi.



44-rasm. QPushButton vidjeti ko‘rinishi

Ushbu vidjetni ekranning markaziga joylashtirish uchun WIDTH va HEIGHT o‘zgaruvchilarini yaratib, biroz qo‘shimcha kod hosil qilingan. Biroq, move() funksiya QPushButtonni unga berilgan koordinatalar bo‘yicha markazlashtirmaydi, balki u yerdan vidjetni chizishni boshlaydi. Boshqacha qilib aytganda, yuqorida yaratgan tugmaning yuqori chap burchagi ekranning markazida joylashgan bo‘ladi.

Tugmalarni funksiyalarga bog‘lash qanday amalga oshiriladi?

Hech qanday funksiya bilan bog'lanmagan tugma foydasiz hisoblanadi. Tugmalar aslida bosilgandan so'ng qandaydir amal bajarilishi uchun mo'ljallangan vidjetdir. Bu amallar esa funksiyaga yuklaniladi va tugma bosilganda funksiya yuklanilishi orqali amalga oshiriladi.

Misol: PyQt5-da **QPushButton** vidjetining oynani yopish, label matnini yangilash va label vidjeti qiymatlarini olish kabi amallarni bajarish uchun kichkina dasturcha tuzildi:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

def yangilash():
    lbl.setText("Yangilandi!")

def print_txt():
    print(lbl.text())

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,500,300)
win.setWindowTitle("Buttonlar va label birgalikda")

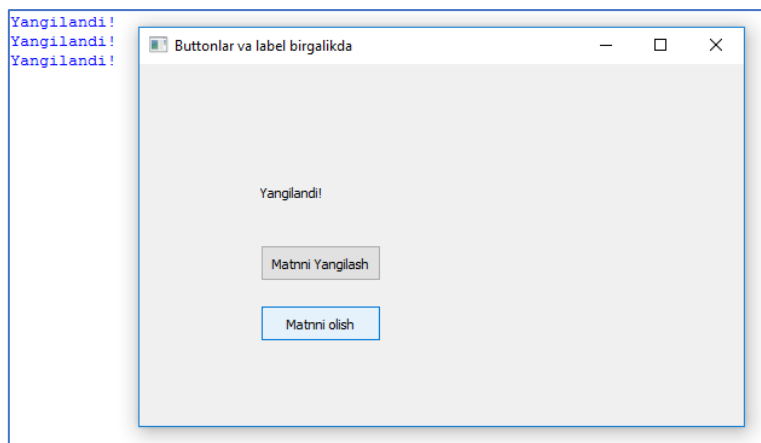
lbl = QtWidgets.QLabel(win)
lbl.setText("PyQt5 paketida GUI applicationlar")
lbl.adjustSize()
lbl.move(100,100)

btn = QtWidgets QPushButton(win)
btn.clicked.connect(yangilash)
btn.setText("Matnni Yangilash")
btn.move(100,150)

btn2 = QtWidgets QPushButton(win)
btn2.clicked.connect(print_txt)
btn2.setText("Matnni olish")
btn2.move(100,200)

win.show()
sys.exit(app.exec_())
```

Yuqoridagi kodni ishga tushirishda hosil bo'lgan oynaning ko'rinishi quyidagicha bo'ladi:



45-rasm. Yuqoridagi dastur kodining natijasi

PyQt ilovasini button yordamida yopish

QPushButtondan yana bir mashhur foydalanish – bu bosilganda PyQt5 ilovasini yopadigan “Chiqish tugmasi” ni yaratish. Buni qanday amalga oshirish mumkin?

Buning uchun shunchaki `close()` funksiyasi yordamida `QMainWindow()` yaratgan oynadagi usuldan foydalanamiz

Buning uchun alohida `close()` funktsiyani yaratib chiqmasdan, yorliqni olib, `win.close` ulanish funksiyasiga beriladi. Qavsni olib tashlashni unutmang, aks holda funktsiya avtomatik ravishda o'zini chaqiradi.

Biz, shuningdek, `QPushButton` hajmini o'zgartirish uchun ishlatiladigan qo'shimcha `resize()` metodini kiritdik. Unga parametr sifatida yangi kenglik va yangi balandlikni kiritiladi.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

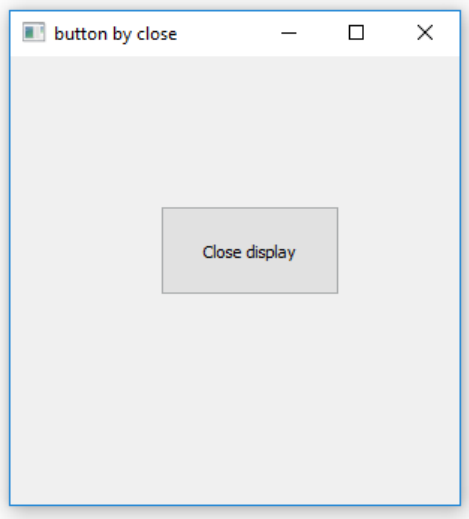
app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(300,300,300,300)
```

```
win.setWindowTitle("button by close")

close_btn = QtWidgets.QPushButton(win)
close_btn.clicked.connect(win.close)
close_btn.setText("Close display")
close_btn.resize(120,60)
close_btn.move(100,100)

win.show()
sys.exit(app.exec_())
```

Ushbu dasturninsh natijasi quyidagicha ko‘rinishda bo‘ladi:



46-rasm. Buttonni **close** tugmachasi sifatida foydalanish.

Tasvirni sozlash

QIcon vidjetidan foydalanib, tugmachaga tasvirni yanada zamonaviy ko‘rinishga keltirish mumkin. Faqat fayl yo‘lini unga qiymat sifatida beriladi.

```
from PyQt5 import QtGui

button = QPushButton(win)
button.setIcon(QtGui.QIcon('myImage.jpg'))
```

QPushButton vidjetining asosiy metodlari:

9-jadval

Metodlar	Tavsif
isDefault()	Foydalanuvchi Enter tugmasini bosganda, bu xususiyat rost (ya'ni, dialog oynasining <i>standart</i> tugmasi) bilan avtomatik ravishda bosiladi.
setIcon()	Tugmadagi rasm belgisini ko'rsatadi.
setEnabled()	Standart bo'yicha True qiymat qabul qiladi. Agar "False" qiymati o'rnatilsa, u kulrang tusga kiradi va faolligini yo'qotadi.
setText()	Tugmaga matnni o'rnatadi.
text()	Tugmada ko'rsatilgan matnni qaytaradi.
toggle()	Tugma holatini almashtiradi.

Ushbu jadvaldagi metodlar orqali QPushButton widgetini to'liq boshqarish mumkin bo'ladi.

Nazorat savollari:

1. Tugmacha vazifasini bajaruvchi vidjet qanday nomlanadi?
2. QPushButton vidjetining qanday mashxur metodlarini bilasiz?
3. Tugmachaning toggle() metodidan qanday maqsadda foydalaniladi?

8.4.3.§. QLineEdit vidjeti

QLineEdit - bu matnning bir qatorini kiritish va tahrirlash imkonini beruvchi vidjet. Ushbu vidjetda Cencel va Repeat, Cut va Past hamda "Sudrab olib tashlash" funksiyalari mavjud.

```
import sys
from PyQt5.QtWidgets import (QWidget, QLabel,
                               QLineEdit, QApplication)

class Examples(QWidget):
    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        self.lbl = QLabel(self)
```

```

qlineE = QLineEdit(self)

qlineE.move(100, 100)
self.lbl.move(100, 40)

qlineE.textChanged[str].connect(self.changed)
self.setGeometry(300, 300, 280, 170)
self.setWindowTitle('QlineEdit Example')
self.show()

def changed(self, text):
    self.lbl.setText(Label text)
    self.lbl.adjustSize()

if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Examples()
    sys.exit(app.exec_())

```

Ushbu misolda QLineEdit vidjeti va QLabel ko'rsatilgan. Tahrirlash paneliga kiritgan matnimiz darhol teg vidjetida ko'rsatiladi.

```
qlineE = QLineEdit(self)
```

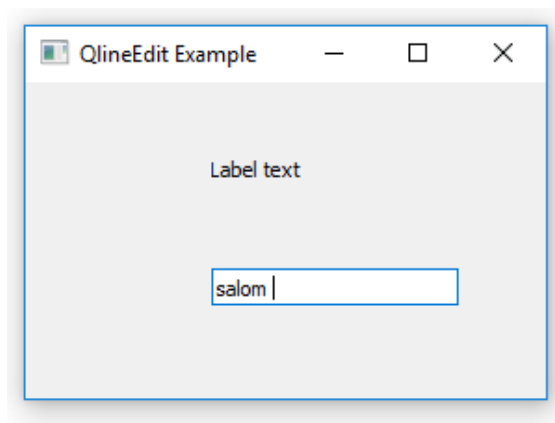
- QLineEdit vidjetining qline obyekti yaratildi.

```
qlineE.textChanged[str].connect(self.changed)
```

- Agar QLineEdit vidjetidagi matn o'zgarsa, changed() funksiyasi ishga tushadi.

```
def changed(self, text):
    self.lbl.setText(text Label)
    self.lbl.adjustSize()
```

- changed funksiyasining ichida kiritilgan matnni teg vidjetiga o'rnatish buyrug'i joylashtirilgan. Yorliq o'lchamini matn uzunligiga mos ravishda o'zgartirish uchun adjustSize() metodidan foydalanilgan .



47-rasm. Yuqoridagi dastur ko‘rinishi

Nazorat savollari:

1. PyQt da yorliq yaratish uchun qaysi vidgetdan foydalaniladi?
2. QlineEdit vidjetining vazifasi qanday?
3. QlineEdit vidjetining xususiyatlari qanday?

8.4.4.§. QComboBox vidjeti

QComboBox - bu foydalanuvchiga variantlar ro‘yxatidan (ochiladigan ro‘yxat) element tanlash imkonini beruvchi vidjet.

```
import sys

from PyQt5.QtWidgets import (QWidget, QLabel,
                              QComboBox, QApplication)

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.lbl = QLabel("Ubuntu", self)
        combo = QComboBox(self)
        combo.addItem(["Ubuntu", "Mandriva",
                      "Fedora", "Arch", "Gentoo"])

        combo.move(50, 50)
```

```

        self.lbl.move(50, 150)

        combo.activated[str].connect(self.onActivated)

        self.setGeometry(300, 300, 300, 200)
        self.setWindowTitle('QComboBox')
        self.show()

    def onActivated(self, text):
        self.lbl.setText(text)
        self.lbl.adjustSize()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Misolda QComboBox vidjetining tarkibidagi ro'yxatda beshta element mavjud. Bular Linux operatsion tizimi distributivlarining nomlari bo'lib, QLabel vidjeti tanlangan variantni ko'rsatadi.

```

combo = QComboBox(self)
combo.addItem(["Ubuntu", "Gentoo",
               "Arch", "Fedora", "Mandriva"])

```

- beshta variantga ega bo'lgan **QComboBox** vidjetini yaratildi.

```

combo.activated[str].connect(self.onActivated)

```

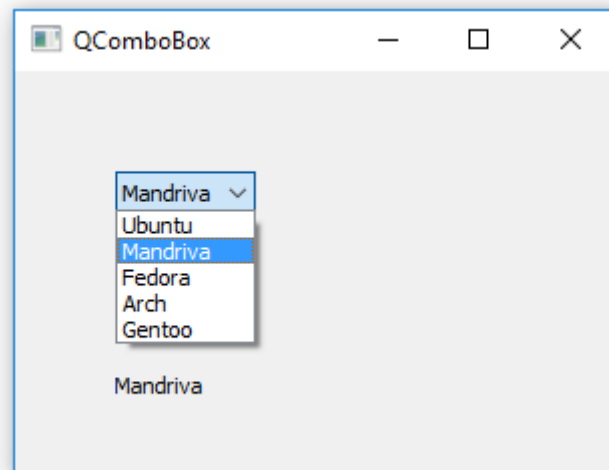
- elementni tanlagandan so'ng, **onActivated()** funksiyasi aktivlashadi.

```

def onActivated(self, text):
    self.lbl.setText(text)
    self.lbl.adjustSize()

```

- funksiya tarkibida tanlangan elementning matnini QLabel vidjetiga joylashtiriladi. Oldingi misoldagi kabi Label o'lchamini matn o'lchamiga moslashtirilgan.



48-rasm. QComboBox widgetining ko‘rinishi.

Nazorat savollari:

1. QComboBox vidjetining vazifasi qanday?
2. QComboBox vidjeti qanday xususiyatlarga ega?
3. QComboBox vidjeti qanday qiymatlar qabul qiladi?

8.4.5.§. QCheckBox vidjeti

QCheckBox - tasdiqlash maydoni (vidjet ikkita holatni qabul qiladi: yoqish va o‘chirish). Checkboxlar ilovada yoqish yoki o‘chirib qo‘yish mumkin bo‘lgan funktsiyalari uchun qo‘llaniladi.

Oynaning sarlavhasini almashtiradigan checkbox yaratish dasturini ko‘rib chiqaylik:

```
import sys

from PyQt5.QtWidgets import *
from PyQt5.QtCore import Qt

class Example(QWidget):

    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
```

```

chb = QCheckBox('Show title', self)
chb.move(20, 20)
chb.toggle()
chb.stateChanged.connect(self.changeTitle)

self.setGeometry(300, 300, 250, 150)
self.setWindowTitle('QCheckBox')
self.show()

def changeTitle(self, state):
    if state == Qt.Checked:
        self.setWindowTitle('QCheckBox')
    else:
        self.setWindowTitle('')

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Dastur tahlili:

```
chb = QCheckBox('Show title', self)
```

- bu **QCheckBox** sinfining konstruktori.

```
chb.toggle()
```

- oynaning sarlavhasini belgilab qo'yildi. Odatda oyna sarlavhasi o'rnatilmagan va checkbox o'chirilgan holatda bo'ladi.

```
chb.stateChanged.connect(self.changeTitle)
```

- **changeTitle()** funksiyasini **stateChanged** signaliga bog'landi. **changeTitle()** funksiyasi oyna sarlavhasini almashtiradi.

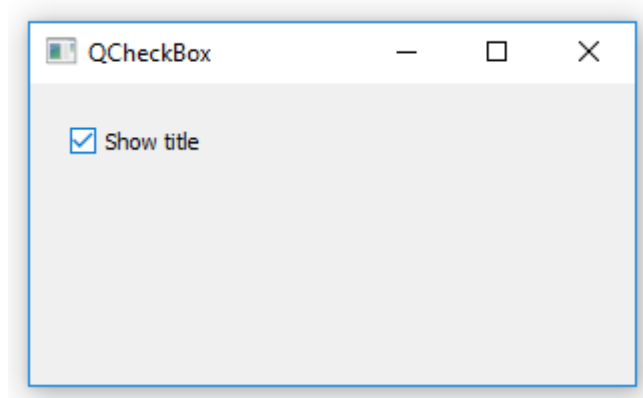
```
def changeTitle(self, state):
    if state == Qt.Checked:
```

```

        self.setWindowTitle('QCheckBox')
    else:
        self.setWindowTitle('')

```

- Agar vidjet belgilansa, oynaning sarlavhasini o‘rnatiladi. Aks holda, sarlavhani bo‘sh satrga almashtiriladi.



49-rasm. QCheckBox widgetining ko‘rinishi.

Nazorat savollari:

1. Belgilash uchun qo‘llaniladigan vidjet qanday nomlanadi?
2. QCheckBox vidjetining vazifasi qanday?
3. QCheckBox vidjeti qanday qiymatlar qabul qiladi?

8.4.6.§. QRadioButton vidjeti

Radio tugmalari odatda GUI-da ko‘rinadi, ushbu vidjet orqali foydalanuvchiga tanlash uchun turli variantlar ro‘yxati taqdim etiladi. Checkbox dan farqli o‘laroq, bir nechta variantdan faqat bitta tugmachasini tanlash mumkin. **QRadioButton** vidjeti GUI dasturi uchun Radio tugmalarini joylashtirish maqsadida foydalaniladi.

Quyida **QRadioButton** vidjetidan foydalanish bo‘yicha oddiy misol keltirilgan:

```

from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("Radio Buttons")

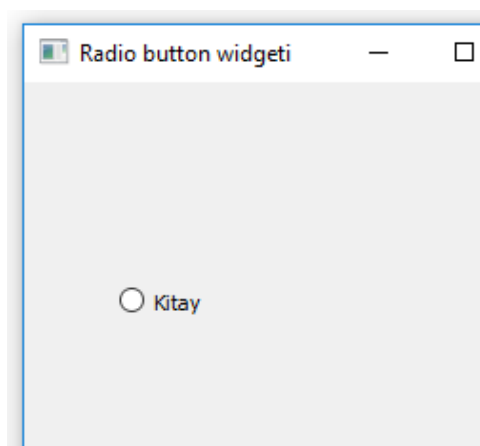
radio = QtWidgets.QRadioButton(win)

```

```
radio.setText("Kitay")
radio.move(50,100)

win.show()
sys.exit(app.exec_())
```

Yuqoridagi kodning chiqishi:



50-rasm. Radio button vidjetining ko‘rinishi

QRadioButtonning qiymatlariga murojaat qilish.

Siz QRadioButton qiymatlarini text() radio tugmasiga tayinlagan matn qiymatini qaytaradigan usul yordamida olish mumkin.

Foydalanish mumkin bo‘lgan yana boshqa metod **isChecked()** mavjud. Bu Radiobutton tanlangan yoki tanlanmaganligini bildiruvchi butun sonni qaytaradi.

Bu yerda funksiyani chaqirishning juda o‘ziga xos shaklidan **show()** metodidan foydalanmoqdamiz. Yuborish tugmachasini yaratish o‘rniga QRadioButton ning metodi - **toggled.connect()** dan foydalanildi. Bu radio tugmasi holati o‘zgartirilganda **show()** funksiyasini chaqiradi.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

def show():
    print(radio.text())
    print(radio.isChecked())
```



```

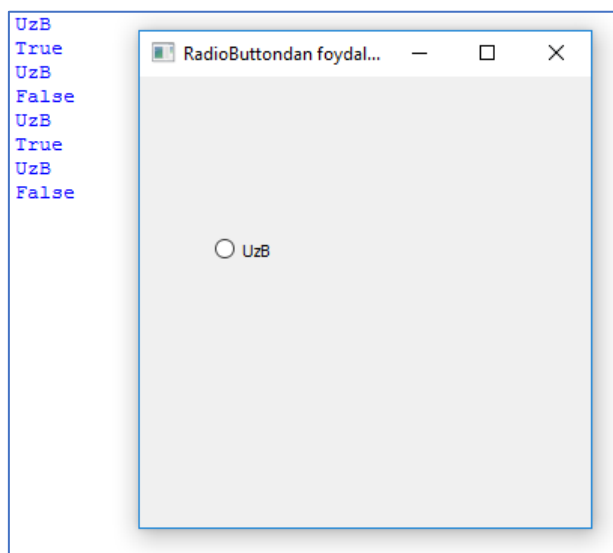
app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("RadioButtondan foydalanish")

radio = QtWidgets.QRadioButton(win)
radio.toggled.connect(show)
radio.setText("UzB")
radio.move(50,100)

win.show()
sys.exit(app.exec_())

```

Quyida yuqoridagi kodning GUI chiqishi, uning chiqishi bilan birga ekran tasviri berilgan. Ushbu natijaga erishish uchun RadioButton-ni ikki marta bosdik, uni tanladik va bekor qildik. Har safar bosganimizda, ekranda 2 ta satr chiqdi.



51-rasm. QRadioButton tugmachasini ishlatishga misol

RadioButton guruhlari

Radio tugmalari turli variantlarni tanlashga mo'ljallangan. Buning uchun **QGridLayout** kabi guruhlash vidjetidan foydalanish kerak .

Radio tugmalar guruhida bir vaqtning o'zida faqat bitta radio tugmachani tanlash imkoni beriladi. Agar foydalanuvchi boshqa tugmani tanlasa, avval tanlangan radio tugmasi o'chadi.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("CodersLegacy")

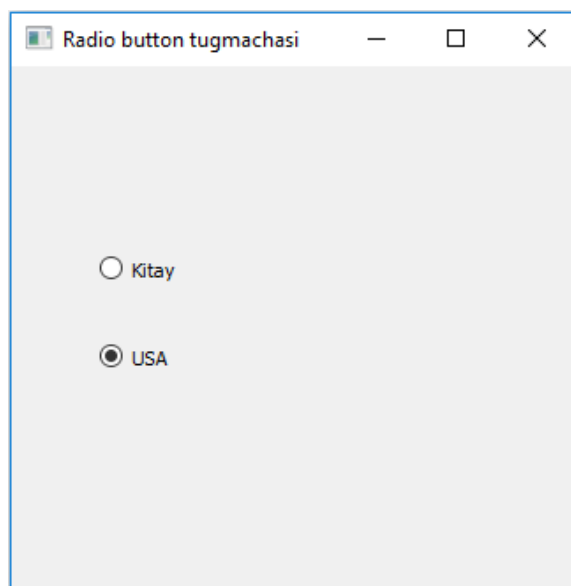
layout = QGridLayout()

radio = QtWidgets.QRadioButton(win)
radio.setText("Kitay")
radio.move(50,100)
layout.addWidget(radio, 0, 0)

radio2 = QtWidgets.QRadioButton(win)
radio2.setText("USA")
radio2.move(50,150)
layout.addWidget(radio2, 0, 1)

win.show()
sys.exit(app.exec_())
```

Quyida yuqoridagi kodning natijasi keltirilgan. Unda bir vaqtning o'zida faqat bitta tugmani tanlash mumkin.



52-rasm. QGridLayout vidjetidan gruhlashda boydalanishga namuna

Quyida QRadioButton vidjeti uchun mavjud usullarning kompilyatsiyasi keltirilgan:

10-jadval

Metodlar	Tavsif
setChecked()	Belgilangan katak holatini o‘zgartiradi.
setText()	Radio button bilan bog‘langan matnni o‘rnatadi.
text()	Radio button matn qiymatini oladi.
isChecked()	Radio button tanlangan yoki yo‘qligini tekshiradi. “True”ni qaytaradi, aks holda “False”.

Ushbu metodlar orqali radio buttonlarni turlicha boshqarish imkoniyati mavjud.

Nazorat savollari:

1. Radiobutton bilan checkbox ning farqi nimada?
2. Radiobuttonlarni qanday gruh qilib yaratish uchun qaysi vidjetdan foydalaniladi?
3. Radio butonlarning tanlanganligini bilish uchun qaysi metoddan foydalaniladi?

8.4.7.§. QTextEdit vidjeti

QLineEdit vidjeti singari, **QTextEdit** foydalanuvchidan matn shaklida ma’lumot olish uchun ishlatiladi. Biroq, faqat bitta satrda kiritiladigan **QLineEdit**dan

farqli o'laroq, **QTextEdit** foydalanuvchi bir necha satr matn yoki hatto bir nechta paragraflarni kiritishi mumkin bo'lgan katta maydonni taklif qiladi.

Asosan vidjetlardan foydalanishda **QMainWindow** dan foydalaniladi. Ammo ushbu vidjet uchun **QWidget** dan foydalanish tafsifiy qilinadi. Bular orasida juda ko'p farq yo'q, lekin agar **TextEdit** to'g'ri ishlashini istasangiz, **QWidget** dan foydalanish maqsadga muvofiq bo'ladi.

Bundan tashqari, oddiy QTextEdit vidjetini yaratish uchun obyekt yaratish uchun QTextEdit sinfidan foydalaniladi.

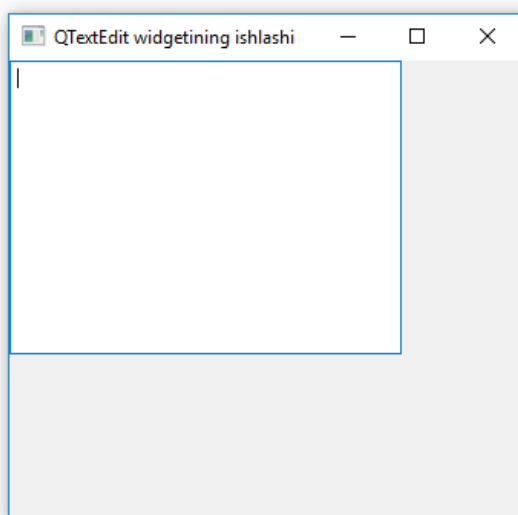
```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QWidget
import sys

app = QApplication(sys.argv)
wind = QWidget()
wind.setGeometry(300,300,300,300)
wind.setWindowTitle("QTextEdit widgetining ishlashi")

malumotlar = QtWidgets.QTextEdit(wind)

wind.show()
sys.exit(app.exec_())
```

Yuqoridagi kodning GUI chiqishi:



53-rasm. QLineEdit vidjeti uchun yozilgan kodning natijasi

QTextEdit ning matniga murojaat qilish.

QTextEditga kiritilgan matnni olish uchun toPlainText() metodi va QPushButton vidjetidan foydalangan holda olish mumkin.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def show_txt():
    print(txt_edit.toPlainText())

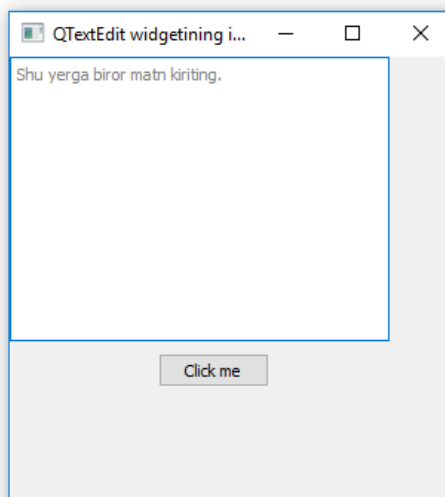
app = QApplication(sys.argv)
wind = QWidget()
wind.setGeometry(300,300,300,300)
wind.setWindowTitle("QTextEdit widgetining ishlashi")

txt_edit = QtWidgets.QTextEdit(wind)
txt_edit.setPlaceholderText("Shu yerga biror matn kiriting.")

button = QtWidgets.QPushButton(wind)
button.setText("Click me")
button.clicked.connect(show_txt)
button.move(100,200)

wind.show()
sys.exit(app.exec_())
```

Yuqoridagi dasturning natijasi quydagicha ko‘rinishda bo‘ladi:



54-rasm. QTextEdit vidjeti matniga murojaat qilish

QTextEdit vidjeti uchun barcha muhim metodlar:

11-jadval

Usul	Tavsif
toPlainText()	TextEdit vidjetida ko'rsatilgan matnni qaytaradi.
setPlainText()	Vidjetda ko'rsatilgan matnni o'rnatish uchun foydalaniladi.
setReadOnly(<i>bool</i>)	Vidjet faqat o'qish uchun mo'ljallangan (yozilishi mumkin emas) yoki yo'qligini belgilash uchun ishlatiladi.
isReadOnly()	TextEdit faqat o'qiladimi yoki yo'qligiga qarab mantiqiy qiymatni qaytaradi.
setPlaceholderText(<i>text</i>)	Vidjet bilan o'zaro aloqada bo'lganida yo'qolib ketadigan vidjetga bir nechta kulrang matnni o'rnatish uchun foydalaniladi.
setUndoRedoEnabled(<i>bool</i>)	Foydalanuvchi uchun Bekor qilish va Qayta tiklash funksiyalaridan foydalanish imkoniyatini o'chiradi/yoqadi.
isUndoRedoEnabled()	Bekor qilish/Qayta qilish funksiyasi yoqilganligiga qarab True yoki False qiymatini qaytaradi.

Nazorat savollari:

1. QTextEdit vidjetining xususiyatlari qanday?
2. QTextEdit vidjetining imkoniyatlari qanday?
3. QTextEdit vidjetini yaratish uchun qanday amallar bajariladi?

8.4.8.§. QInputDialog vidjeti

PyQt5 o'zining QInputDialog vidjetiga ega bo'lib, u bizga turli xil usullarda ma'lumot kiritish uchun turli xil kiritish dialoglarini hosil qilish imkonini beradi.

Bu mavzuda rasmlar va misollar bilan to'liq barcha variantlar va usullarning ro'yxatini topish mumkin.

QInputDialog - getText

Biz eng keng tarqalgan matn kiritish dialogidan boshlaymiz. U oddiy kirish maydonini va OK va Cancel tugmachalaridan iborat oynani taklif qiladi.



55-rasm. *QInputDialog* vidjetining ko‘rinishi

QInputDialog vidjetining **getText()** funksiyasi biroz murakkab bo‘lganligi uchun avval uni ko‘rib chiqaylik. Funksiya **getText()** ikkita qiymatni qaytaradi va bu qiymatlarni ikkita o‘zgaruvchiga yuklab olish kerak bo‘ladi. Birinchi o‘zgaruvchi “txt” foydalanuvchi ma’lumotlarini oladi, ikkinchi o‘zgaruvchi “bosildi” OK tugmasi bosilganmi yoki bosilmaganligini bildiruvchi “True” yoki “False” qiymatini oladi.

```
input, pressed = QInputDialog.getText(parent, Title, Label,
EchoMode)
```

Quyidagilar uchun parametrlar **getText()**:

- **parent**: Siz yaratgan asosiy oyna, unda ushbu dialog oynasi ko‘rsatilishi kerak.
- **Title**: Muloqot oynasining sarlavha satrida paydo bo‘ladigan matn
- **Label**: Kirish maydonining tepasida paydo bo‘ladigan matn.
- **EchoMode**: Kirish maydoni kiritishni qabul qilishda foydalanadigan aks-sado rejimi.

QInputDialog widjeti uchun to‘liq kod quyidagicha ko‘rinishga ega:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def display():
    txt, bosildi = QInputDialog.getText(wind, "Matn kiritish
maydoni",
                                     "Matn:",
                                     QLineEdit.Normal, "")
    if bosildi:
```

```

        label.setText(txt)
        label.adjustSize()

app = QApplication(sys.argv)
wind = QWidget()
wind.setGeometry(400,400,250,250)
wind.setWindowTitle("QInputDialog widgetida misol")

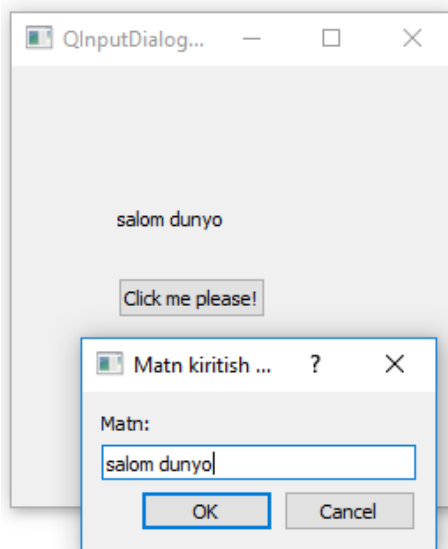
label = QLabel(wind)
label.setText("Bo'sh maydon")
label.move(60,80)

btn = QPushButton(wind)
btn.setText("Click me please!")
btn.clicked.connect(display)
btn.move(60,120)

wind.show()
sys.exit(app.exec_())

```

Oldingi darslarimizda qilganimizdan farqli o'laroq, biz chiqishni konsolda emas, balki to'g'ridan-to'g'ri GUI-ning o'zida ko'rsatmoqdamiz. Biz buni label matnini muloqot oynasidan qaytarilgan matn bilan yangilash orqali qilamiz.

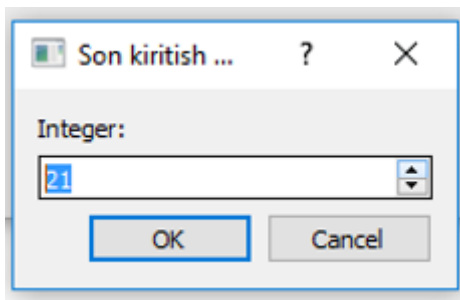


56-rasm. Yuqoridagi dastur natijasi

Keyingi misollarda joyini tejash va takrorlanishni oldini olish kodning faqat o'zgaruvchan qismini, ya'ni chaqirilayotgan funksiyani keltirib o'tiladi.

QInputDialog – getInt() funksiyasi

Butun son qiymatini o'zgartirish uchun strelkalardan -  foydalanish mumkin yoki qiymatni foydalanuvchi qo'lda kiritishi mumkin.



57-rasm. QInputDialog widgetining getInt funksiyasidan foydalanish

```
QInputDialog.getInt(parent, Title, Label, Default, Min, Max, Step)
```

Bu yerda ishlatilgan parametrlarning izohlari keltirilgan:

- **parent:** Siz yaratgan asosiy oyna, unda ushbu dialog oynasi ko'rsatilishi kerak;
- **Title:** Muloqot oynasining sarlavha satrida paydo bo'ladigan matn;
- **Label:** Kiritish maydonining tepasida paydo bo'ladigan matn;
- **Default:** Muloqot oynasi ishga tushirilganda mavjud bo'lgan standart qiymat;
- **Min:** Tanlanishi mumkin bo'lgan minimal qiymat;
- **Max:** Tanlash mumkin bo'lgan maksimal qiymat;
- **Step:** O'qlarni bosganingizda qiymatlar orasidagi o'sish.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def display():
    numerical , pressed = QInputDialog.getInt(wind, "Son
kiritish maydoni", "Integer: ", 10, 0, 100, 1)
    if pressed:
        label.setText(str(numerical))
        label.adjustSize()
```

```

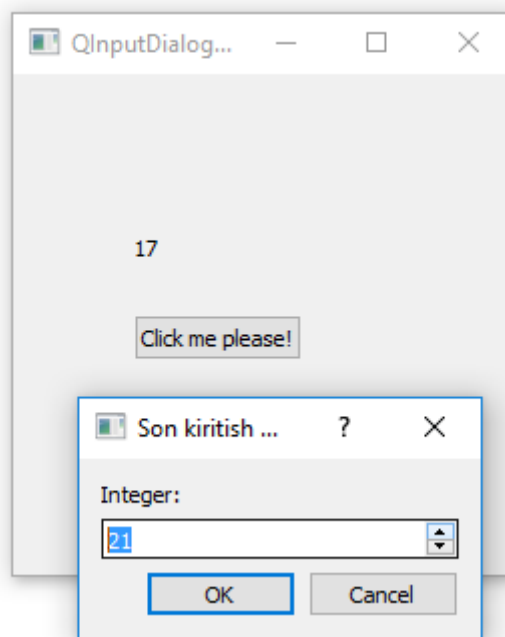
app = QApplication(sys.argv)
wind = QWidget()
wind.setGeometry(400,400,250,250)
wind.setWindowTitle("QInputDialog widgetida misol")

label = QLabel(wind)
label.setText("Bo'sh maydon")
label.move(60,80)

btn = QPushButton(wind)
btn.setText("Click me please!")
btn.clicked.connect(display)
btn.move(60,120)

wind.show()
sys.exit(app.exec_())

```



58-rasm. getInt funksiyasi natijasi

QInputDialog - getItem

```

option, pressed = QInputDialog.getItem(parent, Title, Label,
Options, current, editable)

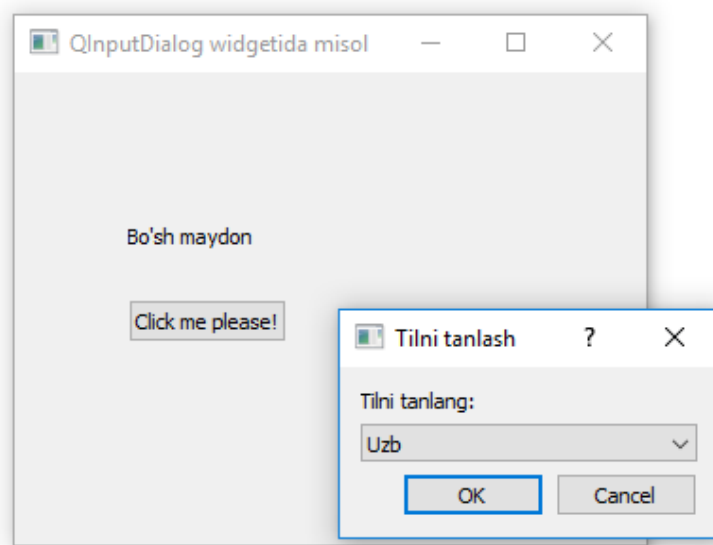
```

Bu yerda ishlatilgan parametrlarning izohlari keltirilgan:

- parent: yaratilgan asosiy oyna, unda ushbu dialog oynasi ko'rsatilishi kerak.

- Title: Muloqot oynasining sarlavha satrida paydo boʻladigan sarlavha
- Label: Kirish maydonining tepasida paydo boʻladigan matn.
- Options: Variantlar sifatida koʻrsatiladigan qatorlar roʻyxati/toʻplami.
- current: Hozirda tanlangan element. Indeksni ifodalash uchun butun son qiymatlarini oladi.
- editable: Oʻzingizning shaxsiy opsiyangizni qoʻshish uchun parametrlarni tahrirlashingiz mumkinligini aniqlash uchun True yoki False qiymatini oladi.

```
def display():
    options = ("Uzb", "Ru", "EN")
    option , pressed = QInputDialog.getItem(wind, "Tilni
tanlash","Tilni tanlang:", options, 0, False)
    if pressed:
        label.setText(option)
        label.adjustSize()
```



59-rasm. Yuqoridagi getItem uchun keltirilgan kodning natijasi

QInputDialog - getDouble

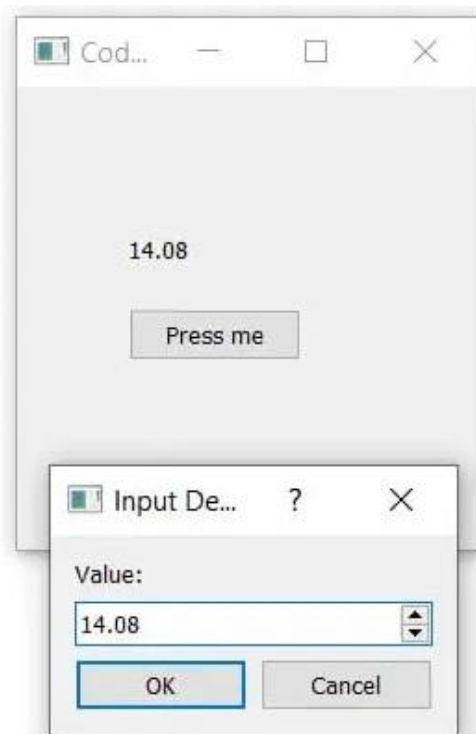
```
decimal, pressed = QInputDialog.getDouble(parent, Title, Label,
Default, Min, Max, Step)
```

Bu yerda ishlatilgan parametrlarning izohlari keltirilgan:

- parent: Siz yaratgan asosiy oyna, unda ushbu dialog oynasi koʻrsatilishi kerak.

- Title:Muloqot oynasining sarlavha satrida paydo bo‘ladigan sarlavha
- Label:Kirish maydonining tepasida paydo bo‘ladigan matn.
- Default:Muloqot oynasi ishga tushirilganda mavjud bo‘lgan standart qiymat.
- Min:Tanlanishi mumkin bo‘lgan minimal qiymat.
- Max:Tanlash mumkin bo‘lgan maksimal qiymat.
- Step:O‘qlarni bosganingizda qiymatlar orasidagi o‘shish.

```
def display():
    options = ("Options1", "Options2", "Options3")
    decimals , pressedOK = QInputDialog.getDouble(window,
    "Input Decimal","Value:", 11.00, 0, 100, 2)
    if pressedOK:
        label.setText(str(decimals))
        label.adjustSize()
```



60-rasm. QInputDialog ning getDouble funksiyasiga keltirilgan kodning natijasi

Nazorat savollari:

1. `QInputDialog` vidjetini yaratish uchun qanday amallar bajariladi?
2. `QInputDialog` vidjetining vazifasi qanday?
3. `QInputDialog` vidjetining qanday xususiyatlari mavjud?

8.4.9.§. `QMessageBox` vidjeti

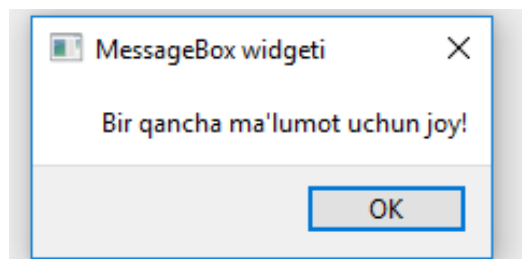
`QMessageBox` - bu dialoglar yaratish uchun ishlatiladigan foydali PyQt5 vidjeti. Ushbu dialog oynalari turli maqsadlarda ishlatilishi va turli xil xabarlar va tugmalarni ko'rsatish uchun ko'p jihatdan moslashtirilgan bo'lishi mumkin.

`QMessageBox` oynasini yaratish

Quyida oddiy `QMessageBox` vidjetini yaratmoqda. Ammo bu hech qanday qo'shimcha funksiyalar yoki sozlashlarsiz faqat `MessageBox` vidjetining o'zining tinchlik xolatidagi ko'rinishi bo'ladi.

```
def display():  
    msg = QMessageBox(wind)  
    msg.setWindowTitle("MessageBox widgeti")  
    msg.setText("Bir qancha ma'lumot uchun joy!")  
  
    x = msg.exec_()
```

Va quyida chiqish. Agar siz ekranda oddiy xabarni ko'rsatmoqchi bo'lsangiz, buni qilishning yo'li.



61-rasm. `QmessageBox` vidjetining o'zining ko'rinishi

`QMessageBox` piktogrammalari

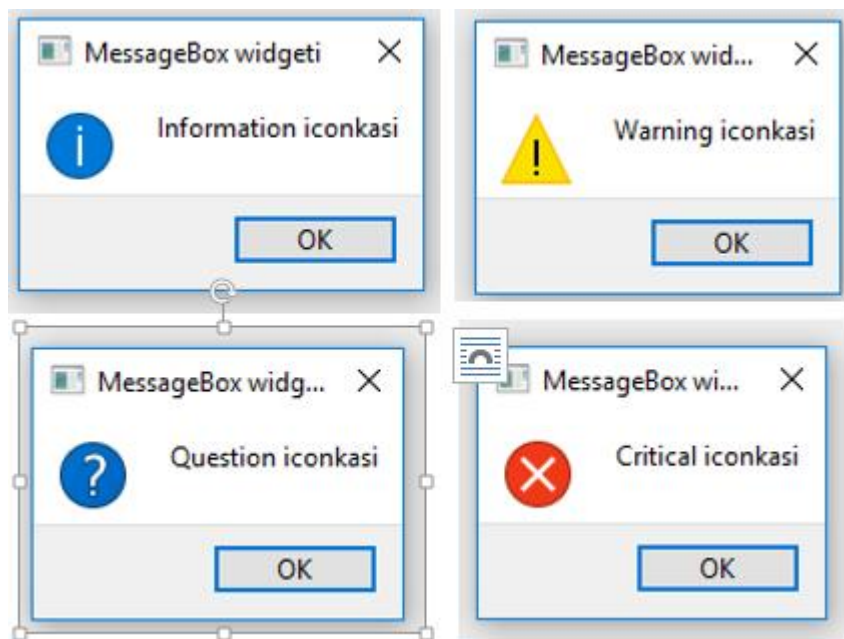
`QMessageBox` - da `.setIcon()` metodini chaqirish orqali foydalanish mumkin bo'lgan 4 xil turdagi piktogramma mavjud bo'lib, ko'rsatmoqchi bo'lgan xabar turiga qarab qaysi birini ishlatishni hal qilinadi.

- QMessageBox.Question
- QMessageBox.Information
- QMessageBox.Warning
- QMessageBox.Critical

Yuqoridagilardan birini quyida ko‘rsatilganidek **setIcon()** funksiyasiga o‘tkazish kifoya .

```
msg.setIcon(QMessageBox.Question)
```

Quyida ko‘rsatilishi mumkin bo‘lgan to‘rt xil xabarlar mavjud.



62-rasm. QMessageBox pictogrammalarining turlari

QMessageBox tugmalari

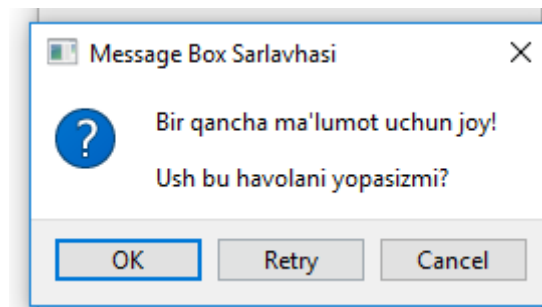
Hozirgacha biz QMessageBox taqdim etgan standart “OK” tugmasidan foydalanib kelmoqdamiz. Biroq, QMessageBox foydalanish uchun o‘ndan ortiq turli xil tugmalar taklif qiladi. Quyida ba’zi tugmalardan foydalanish misoli keltirilgan.

setStandardButtons() funktsiyadan foydalanib xohlagan tugma turini o‘ratish mumkin.

```
def display():
    msg = QMessageBox(wind)
    msg.setWindowTitle("Message Box Sarlavhasi")
    msg.setText("Bir qancha ma'lumot uchun joy!")
```

```
msg.setIcon(QMessageBox.Question)
msg.setStandardButtons(QMessageBox.Cancel
|QMessageBox.Ok
|QMessageBox.Retry)
msg.setInformativeText("Ush bu havolani yopasizmi?")
x = msg.exec_()
```

Ushbu dasturning natijasi quyida keltirilgan:



63-rasm. QMessageBox ga doir oddiy dastur natijasi

Buni allaqachon payqagan bo'lishingiz mumkin, lekin tugmalar tartibi ularning xabarlar oynasida qanday ko'rinishiga ta'sir qilmaydi.

Default Button

Yuqoridagi rasmga qarab "OK" tugmasi atrofida ko'k kontur borligini payqash mumkin. Bu ushbu tugma **Default Button** ekanligini bildiradi.

setDefaultButton() funktsiyasidan foydalanib, standart tugmani o'zgartirish mumkin. Kodga quyidagi qatorni qo'shish ko'k rangni "**Cancel**" tugmasi atrofida bo'lishiga olib keladi.

```
msg.setDefaultButton(QMessageBox.Cancel)
```

Bu yerda **QmessageBox** vidjetida foydalanish mumkin bo'lgan turli xil tugma turlarining to'liq ro'yxati keltirilgan:

- QMessageBox.Ok
- QMessageBox.No
- QMessageBox.Yes
- QMessageBox.Cancel

- QMessageBox.Close
- QMessageBox.Abort
- QMessageBox.open
- QMessageBox.Retry
- QMessageBox.Ignore
- QMessageBox.Save
- QMessageBox.Retry
- QMessageBox.Apply
- QMessageBox.Help
- QMessageBox.Reset
- QMessageBox.SaveAll
- QMessageBox.YesToAll
- QMessageBox.NoToAll

Detailed Text va **InformativeText**

Odatda QMessageBox-da Matnni ko'rsatadigan faqat bitta maydon mavjud. Aslida esa, qo'shimcha matn qo'shish uchun yana ikkita qo'shimcha maydon mavjud.

Birinchisi, QMessageBox oynasining o'zida qo'shimcha matn bo'limi. Bu qo'shish mumkin bo'lgan qo'shimcha matn qatoriga o'xshaydi. Ushbu yangi matn maydonini qo'shish uchun faqat setInformativeText() funksiyasiga murojaat qilish kerak.

Ikkinchisi QMessageBox-dan kengaytirilgan maydonda ko'rsatiladi. Bu **Detailed Text** – ya'ni "Batafsil matn" deb ataladi. Ushbu bo'limni o'rnatish avtomatik ravishda ushbu maydonni ko'rsatish uchun ishlatiladigan tugmani qo'shadi. Ushbu batafsil matn yoziladigan maydonni qo'shish uchun faqat **setDetailedText()** funksiya kerak bo'ladi.

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def show_popup():
```



```

msg = QMessageBox(win)
msg.setWindowTitle("Message Box sarlavhasi")
msg.setText("Bu qandaydir matn")
msg.setIcon(QMessageBox.Question)

msg.setStandardButtons(QMessageBox.Cancel | QMessageBox.Ok)
msg.setDefaultButton(QMessageBox.Ok)

msg.setDetailedText("Ma'lumot haqida batafsil.....")
msg.setInformativeText("InformativeText - bu xatolik
haqidagi batafsil ma;lumot berish")
x = msg.exec_()

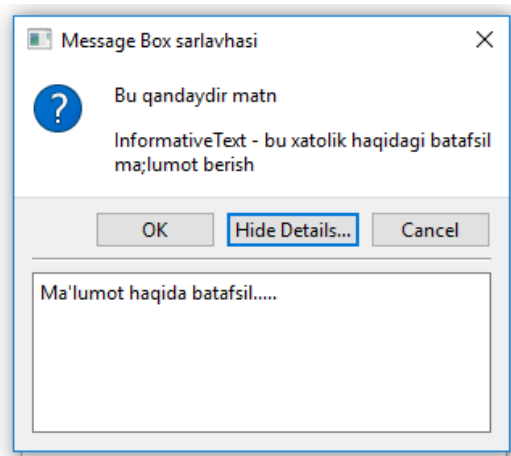
app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("QMessageBox widgeti haqida")

button = QtWidgets.QPushButton(win)
button.setText("A Button")
button.clicked.connect(show_popup)
button.move(100,100)

win.show()
sys.exit(app.exec_())

```

Yuqoridagi dasturning ishga tushirilganidan keyin quyidagi messagebox oynasi paydo bo'ladi:



64-rasm. QMessageBox widgetining to'liq ko'rinishi

Hide Details/Show Details tugmasini bosish mos ravishda xabarlar qutisi ostidagi qo'shimcha maydonni yashirish/ko'rsatish mumkin.

QMessageBox qiymatlarini olish

Yuqorida juda ko'p turli xil sozlashlar va xususiyatlarni muhokama qilindi, ammo aslida bu turli xususiyat va tugmalarning hech birini kodga bog'lanmadi. Misol uchun, agar MessageBoxda 3 xil tugma bo'lsa, qaysi biri bosilganligini qanday bilish mumkin? Buning uchun quyida ko'rsatilgan kodyaqqol javob bo'la oladi:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
import sys

def show_popup():
    msg = QMessageBox(win)
    msg.setWindowTitle("Message Box sarlavhasi")
    msg.setText("Bu qandaydir matn")
    msg.setIcon(QMessageBox.Question)

    msg.setStandardButtons(QMessageBox.Cancel|QMessageBox.Ok)
    msg.setDefaultButton(QMessageBox.Ok)

    msg.setDetailedText("Ma'lumot haqida batafsil.....")
    msg.setInformativeText("InformativeText - bu xatolik
haqidagi batafsil ma;lumot berish")
    msg.buttonClicked.connect(popup)
    x = msg.exec_()

def popup(i):
    print(i.text())
```

```

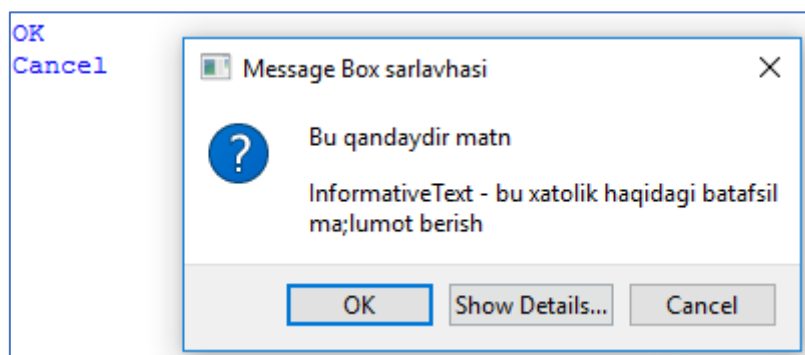
app = QApplication(sys.argv)
win = QMainWindow()
win.setGeometry(400,400,300,300)
win.setWindowTitle("QMessageBox widgeti haqida")

button = QtWidgets.QPushButton(win)
button.setText("A Button")
button.clicked.connect(show_popup)
button.move(100,100)

win.show()
sys.exit(app.exec_())

```

buttonClicked.connect() usulidan foydalanib MessageBoxdagi birorta tugma bosilganda funktsiyani ishga tushirish mumkin. Bunday holda, messageboxni **popup** deb nomlangan funktsiyaga bog‘landi. popup() funsiyas bitta parametr qabul qiladi.



65-rasm. QMessageBox vidjetining to‘liq imkoniyatlari bilan ko‘rinishi

MessageBoxning tugmasi bosilganda, u avtomatik ravishda bosilgan tugmani **buttonClicked.connect()** da e‘lon qilingan funktsiyaga uzatadi. Ushbu tugmadagi text() funktsiyasini chaqirish esa tugmaning matnini qaytaradi.

Nazorat savollari:

1. QMessageBox vidjeti qanday vazifani bajaradi?
2. QMessageBox qanday ishga tushiriladi?
3. QMessageBox ning qanday xususiyatlari mavjud?
4. QMessageBox vidjetida qanday pictogrammalar mavjud?

5. QMessageBoxga o'rnatish mumkin bo'lgan standart buttonlarni keltiring?

8.4.10.§. QProgressBar vidjeti

QProgressBar - jarayonning qancha qismi bajarilganligini yaqqol tasvirllovchi paneli (uzoq davom etgan vazifalarni qayta ishlashda foydalaniladigan vidjet). Bu foydalanuvchilarga vazifa bajarilayotgani haqida xabar berish uchun jarayonni jonlantiradi. QProgressBar vidjeti uchun minimal va maksimal qiymatni o'rnatish mumkin. Ushbu vidjet standart minimum, maximum qiymat sifatida: 0 va 99 qiymatlarini qabul qiladi.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtCore import QTimer

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.pbar = QProgressBar(self)
        self.pbar.setGeometry(30, 40, 200, 25)

        self.btn = QPushButton('Start', self)
        self.btn.move(40, 80)
        self.btn.clicked.connect(self.doAction)

        self.timer = QTimer()
        self.step = 0

        self.setGeometry(300, 300, 280, 170)
        self.setWindowTitle('QProgressBar')
        self.show()

    def timerEvent(self, e):

        if self.step >= 100:
            self.timer.stop()
            self.btn.setText('Finished')
            return
```

```

        self.step = self.step + 1
        self.pbar.setValue(self.step)

    def doAction(self):
        if self.timer.isActive():
            self.timer.stop()
            self.btn.setText('Start')
        else:
            self.timer.start(100, self)
            self.btn.setText('Stop')

if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Ushbu misolda gorizontal harakatlanish paneli va tugma mavjud. Tugma harakatlanish satrini ishga tushiradi va to'xtatadi.

Dastur tahlili:

```
self.pbar = QProgressBar(self)
```

- bu yerda **QProgressBar** vidjetining `pbar` nomli konstruktori yaratildi.

```
self.timer = QtCore.QBasicTimer()
```

- jarayonlarni faollashtirish uchun taymer ob'ektidan foydalanildi.

```
self.timer.start(100, self)
```

- taymer hodisasini boshlash uchun **start()** metodini ishga tushirish kerak. Ushbu metod ikkita parametrga ega: timeout va hodisalar ob'ekti.

```

def timerEvent(self, e):
    if self.step >= 100:
        self.timer.stop()
        self.btn.setText('Finished')
        return

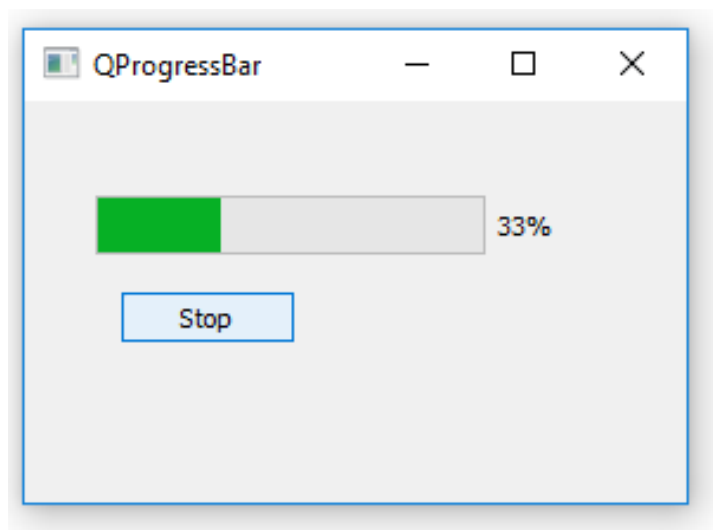
    self.step = self.step + 1
    self.pbar.setValue(self.step)

```

- har bir QObject va uning avlodlarida **timerEvent()** hodisani qayta ishlovchi funksiyasi mavjud.

```
def doAction(self):  
  
    if self.timer.isActive():  
        self.timer.stop()  
        self.btn.setText('Start')  
  
    else:  
        self.timer.start(100, self)  
        self.btn.setText('Stop')
```

- **doAction()** funksiyasi ichida taymerni ishga tushiriladi va to'xtatiladi.



66-rasm. QProgressBar widgetining ilovada ko'rinishi.

Nazorat savollari:

1. QProgressBar vidjetining imkoniyatlari qanday?
2. QProgressBar vidjetini yaratish qanday amalga oshiriladi?
3. QProgressBar vidjetining xususiyatlari qanday?

8.4.11.§. QSlider (slayder) vidjeti

QSlider - *slayder* (oddiy slayderga ega vidjet). Ushbu sozlagich oldinga va orqaga tortilishi mumkin. Shu tarzda, ma'lum bir vazifa uchun qiymat tanlanadi. Ba'zan, slayderdan foydalanish raqamni kiritish yoki spinnerdan foydalanishdan ko'ra tabiiyroqdir. Quyidagi misolda bitta **QSlider** va bitta **QLabel** vidjetlaridan foydalanilgan. Yorliq tasvirni ko'rsatadi. Slayder yorliqni boshqaradi.

```

import sys
from PyQt5.QtWidgets import (QWidget, QSlider,
                              QLabel, QApplication)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QPixmap

class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):

        sld = QSlider(Qt.Horizontal, self)
        sld.setFocusPolicy(Qt.NoFocus)
        sld.setGeometry(30, 40, 100, 30)
        sld.valueChanged[int].connect(self.changeValue)

        self.label = QLabel(self)
        self.label.setPixmap(QPixmap('mute.png'))
        self.label.setGeometry(160, 40, 80, 30)

        self.setGeometry(300, 300, 280, 170)
        self.setWindowTitle('QSlider')
        self.show()

    def changeValue(self, value):

        if value == 0:
            self.label.setPixmap(QPixmap('mute.png'))
        elif value > 0 and value <= 30:
            self.label.setPixmap(QPixmap('min.png'))
        elif value > 30 and value < 80:
            self.label.setPixmap(QPixmap('med.png'))
        else:
            self.label.setPixmap(QPixmap('max.png'))

if __name__ == '__main__':
    app = QApplication(sys.argv)

```

```
ex = Example()  
sys.exit(app.exec_())
```

Yuqoridagi misolda ovoz balandligini boshqarishni simulyatsiya qilish mumkin. Slayder boshqaruvini bilan yorliqdagi tasvirini o'zgartiriladi.

Dastur tahlili:

```
sld = QSlider(Qt.Horizontal, self)
```

- Bu yerda gorizontal slayderni yaratildi.

```
self.label = QLabel(self)  
self.label.setPixmap(QPixmap('mute.png'))
```

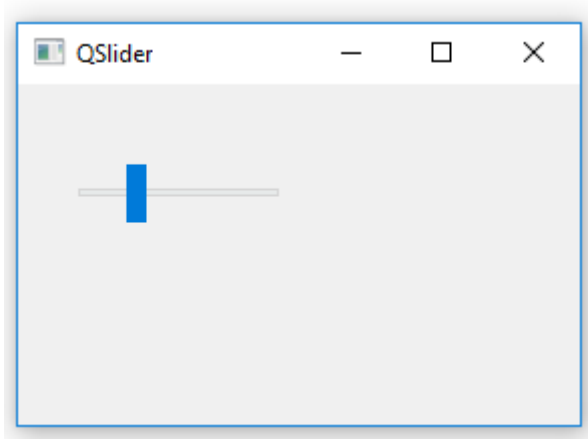
- QLabel vidjetini yaratildi va unga dastlabki "Mute" tasvirini o'rnatildi.

```
sld.valueChanged[int].connect(self.changeValue)
```

- **valueChanged** signalini belgilagan **changeValue()** funksiyasiga bog'landi.

```
if value == 0:  
    self.label.setPixmap(QPixmap('mute.png'))  
...
```

- slayderning qiymatiga asoslanib, tasvirni tegga o'rnatiladi. Yuqoridagi kodda, agar slayder nolga o'rnatilgan bo'lsa, **mute.png** tasvirini tegga o'rnatiladi.



67-rasm. QSlider vidjetining ilovada ko'rinishi.

Nazorat savollari:

1. QSlider vidjetining asosiy xususiyatlari qanday?
2. QSlider vidjetining ishlatilish o'rni qanday?
3. QSlider ning qiymatlarini qanday olish mumkin?

8.4.12.§. QFileDialog vidjeti

Fayl dialoglari GUI-ni o'z ichiga olgan har qanday dasturiy ta'minotning muhim qismidir. Ishlab chiqilgan GUI dasturida ushbu widget juda ko'p foydalaniladi. Eng muhimi, foydalanuvchi o'zi tanlagan Fayl yo'lida Faylni tanlash/saqlash imkoniyatiga ega bo'ladi.

PyQt5 QFileDialog vidjeti - **.getOpenFileName**, **.getOpenFileNames()** va **.getSaveFileName()** kabi turli xil metodlarini taqdim etadi.

.getOpenFileName() dialog oynasi

Birinchi parametr sifatida **None** qiymatini, ikkinchi parametr sifatida fayl dialog oynasiga bermoqchi bo'lgan Sarlavha matnini kiritiladi. Oxirgi parametr uchun ochiladigan fayl turlarini kiritiladi. Fayl turlari ikkita nuqtali vergul (";;") bilan ajratib, bir nechta variantni kiritish mumkin. Quyidagi kodga 3 ta mumkin bo'lgan tanlovni kiritilgan.

Yana bir muhim narsani ta'kidlab o'tish kerakki, File Dialog funksiyasi ikkita parametrni qaytaradi. Birinchisi, tanlangan faylning Fayl yo'li, ikkinchisi esa fayl tanlangan yoki tanlanmaganligiga qarab mantiqiy qiymatdir. Ushbu QFileDialog widgetiga doir misolni ko'rib chiqamiz:

```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import *
from PyQt5 import QtCore
import sys

def dialog():
    file , check = QFileDialog.getOpenFileName(None,
"QFileDialog.getOpenFileName()", "", "Barcha fayllar
(*);;Python fayl (*.py);;Matn fayl (*.txt)")
    if check:
        print(file)
```

```

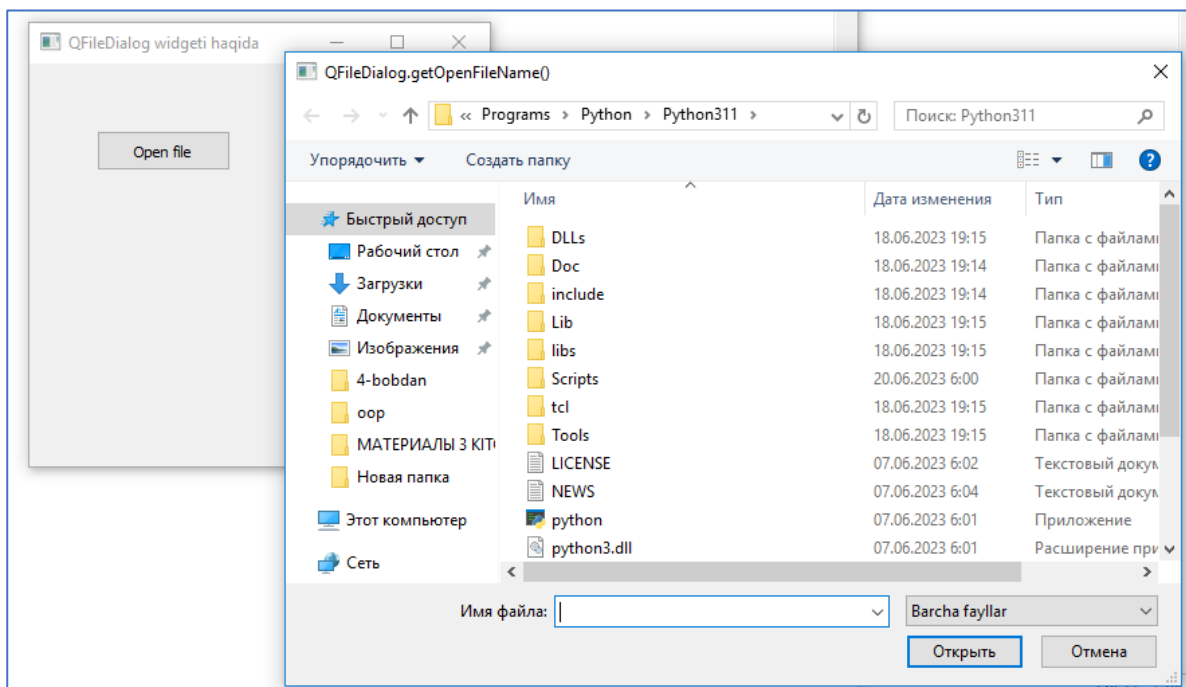
app = QApplication(sys.argv)
wind = QMainWindow()
wind.setGeometry(400,400,300,300)
wind.setWindowTitle("QFileDialog widgeti haqida")

button = QPushButton(wind)
button.setText("Open file")
button.clicked.connect(dialog)
button.move(50,50)

wind.show()
sys.exit(app.exec_())

```

Yuqoridagi kodning natijasi quyidagicha bo'ladi:



68-rasm. `getOpenFileName` metodiga misolning natijasi

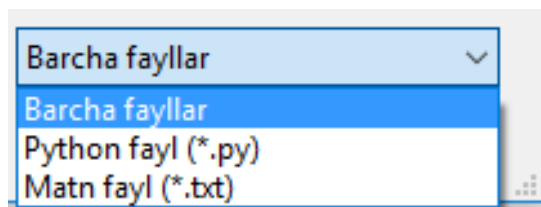
Yuqoridagi oyna paydo bo'lgach, Ish stoliga o'tish va u yerdan Python faylini tanlash quyidagi qatorni qaytardi.

```
C:/Users/CodersLegacy/Desktop/Game.py
```

Pastki o'ng burchakdagi ochiladigan fayl turlari joylashgan dropdown tugmasi orqali fayl turi tanlanib o'zgartirish mumkin. Agar bu yerga yana qo'shimcha fayl

turini qo‘shishga zarurat paydo bo‘lsa, unda “FileDialog” funksiyasining oxiridagi parametrga qo‘shish kerak bo‘ladi.

Ushbu dasturning fayl turlariga quyidagilar kiritilgan va shulardan birini tanlash mumkin:



69-rasm. Fayl turlarini dastur orqali tanlash qismi

getOpenFileNames dialog oynasi

Bunga juda o‘xshash yana bir FileDialog mavjud *getOpenFileNames()*. Bu metod bilan oldingisining yagona farqi qo‘shimcha "s" harfidadir. Ushbu fayl dialog faylni ochilayotgan paytda CTRL tugmachasini bosib ushlab turib, sichqoncha bilan bir nechta fayllarni ustiga bosish orqali bir vaqtning o‘zida bir nechta fayllarni tanlash imkonini beradi.

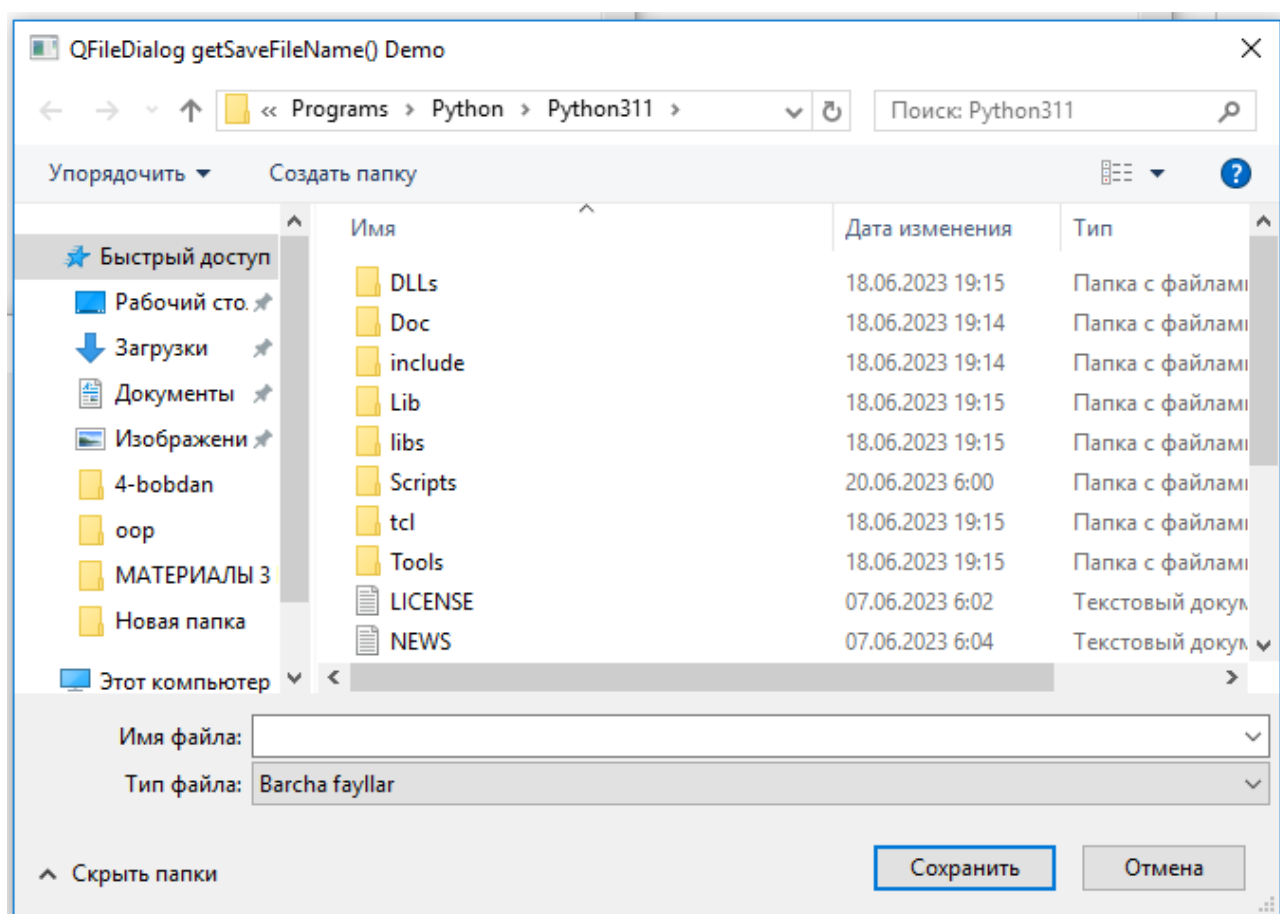
Agar bir nechta faylni tanlasansa, ularning fayl yo‘llari qatorlarni o‘z ichiga olgan ro‘yxatda qaytariladi. Muloqot oynasining ko‘rinishi bir xil, shuning uchun rasmga ehtiyoj qolmaydi.

getSaveFileName

Barcha fayl dialog oynalari 90% o‘xshashlikka ega. Ularning barchasi fayl yo‘llari bilan bog‘liq satrlarni u yoki bu tarzda qaytarish uchun mo‘ljallangan. Faqatgina farq Dialogning ko‘rinishida.

```
def dialog():
    fayl , check = QFileDialog.getSaveFileName(None,
        "QFileDialog getSaveFileName() Demo","", "Barcha
        fayllar (*);;Python fayllar(*.py);;Matn fayllar
        (*.txt)")
    if check:
        print(fayl)
```

Combobox joylashuvi (ochiladigan tanlov) o'zgardi va tugmalardan biridagi matn Ochishdan Saqlashga o'zgartirildi.



70-rasm. *getSaveFileName* matodiga keltirilgan kodning natijasi

Nazorat savollari:

1. QFileDialog vidjetining imkoniyatlari qanday?
2. QFileDialog vidjetidan foydalanish qanday amalga oshiriladi?
3. QFileDialog vidjetining qanday metodlari mavjud?
4. Faylni saqlash uchun QFileDialog vidjetining qaysi metodidan foydalaniladi?
5. Faylni ochish uchun QFileDialog vidjetining qaysi metodidan foydalaniladi?

8.4.13.§. QPixmap vidjeti

QPixmap – bu tasvirlarni ekranga chiqarish uchun ishlatiladigan vidjetlardan biri. Quyidagi misolda tasvirni ekranda ko'rsatish uchun **QPixmap** metodidan foydalanilgan.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtGui import QPixmap
```

```

class Example(QWidget):
    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        h_box = QHBoxLayout(self)
        pixmap = QPixmap("blue_black.png")

        lbl = QLabel(self)
        lbl.setPixmap(pixmap)

        h_box.addWidget(lbl)
        self.setLayout(h_box)

        self.move(300, 200)
        self.setWindowTitle('Blue and Black')
        self.show()

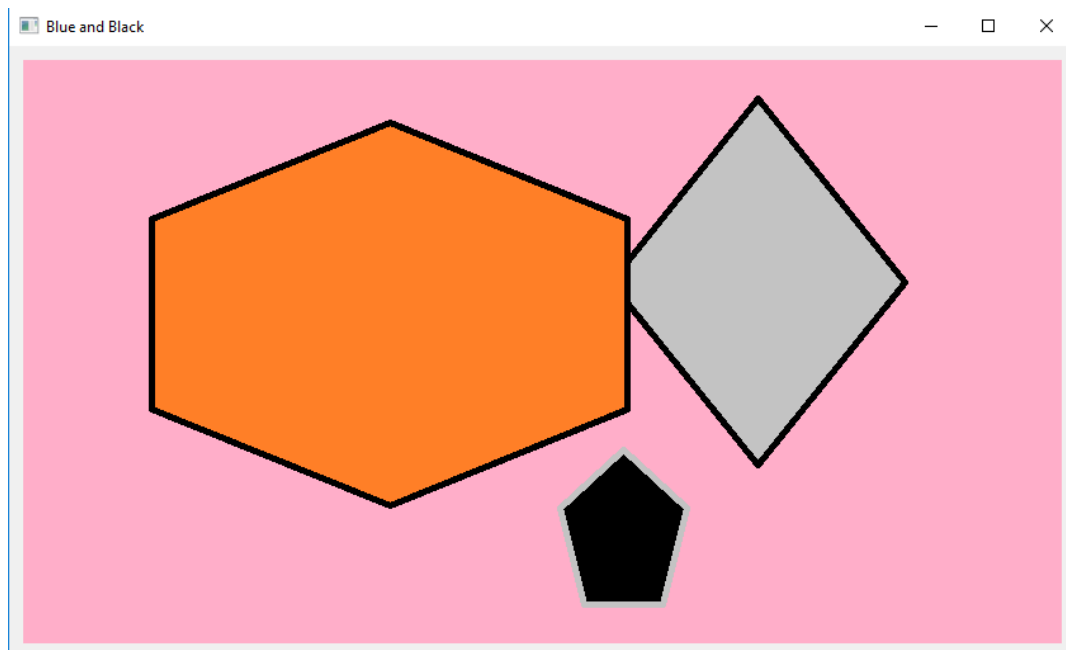
if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Ushbu dasturda `blue_black.png` nomli rasmni oynada ko'rsatish uchun quyidagi amallarni bajarildi:

- 1) `pixmap = QPixmap("blue_black.png")` - **QPixmap** sinfi ob'yekti yaratildi.
- 2) `lbl = QLabel(self)`
`lbl.setPixmap(pixmap)` - rasmni `QLabel` vidjetidan foydalanib ekranga joylashtirildi.



71-rasm. QPixmap vidjetiga rasm joylashtirilgandagi ko‘rinishi
Nazorat savollari:

1. QPixmap vidjetining imkoniyatlaridan qanday foydalaniladi?
2. QPixmap vidjetining qanday xususiyatlari mavjud?
3. QPixmap vidjeti yordamida rasm joylashtirish qanday amalga oshiriladi?

8.4.14.§. QSplitter vidjeti

QSplitter foydalanuvchiga vidjetlar hajmini ular orasidagi chegarani sudrab borish orqali boshqarish imkonini beradi. Ushbu misolda ikkita ajratuvchi bilan tashkil etilgan uchta QFrame vidjetini ko‘rsatilgan.

```
import sys
from PyQt5.QtWidgets import *
from PyQt5.QtCore import Qt

class Example(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        h_box = QHBoxLayout(self)
```

```

top_left = QFrame(self)
top_left.setFrameShape(QFrame.StyledPanel)

top_right = QFrame(self)
top_right.setFrameShape(QFrame.StyledPanel)

bottom = QFrame(self)
bottom.setFrameShape(QFrame.StyledPanel)

splitter1 = QSplitter(Qt.Horizontal)
splitter1.addWidget(topleft)
splitter1.addWidget(topright)

splitter2 = QSplitter(Qt.Vertical)
splitter2.addWidget(splitter1)
splitter2.addWidget(bottom)

h_box.addWidget(splitter2)
self.setLayout(h_box)

self.setGeometry(300, 300, 300, 200)
self.setWindowTitle('QSplitter')
self.show()

if __name__ == '__main__':

    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Ushbu misolda uchta QFrame vidjeti va ikkita ajratuvchi mavjud.

```
top_left = QFrame(self)
top_left.setFrameShape(QFrame.StyledPanel)
```

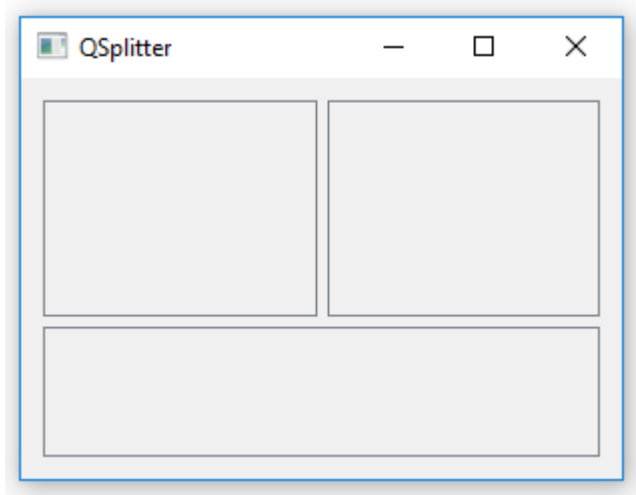
QFrame vidjetlari orasidagi chegaralarni ko‘rish uchun bezatilgan freymdan foydalanish mumkin.

```
splitter1 = QSplitter(Qt.Horizontal)
splitter1.addWidget(top_left)
splitter1.addWidget(top_right)
```

QSplitter vidjetini yaratib va unga ikkita vidjet qo‘shaylik:

```
splitter2 = QSplitter(Qt.Vertical)
splitter2.addWidget(splitter1)
```

Bundan tashqari, ajratuvchini boshqa ajratuvchi vidjetga qo‘shish mumkin.



72-rasm. QSplitter vidjetining ko‘rinishi.

Nazorat savollari:

1. QSplitter vidjeti qo‘llanilish usullari qanday?
2. QSplitter vidjetining asosiy metodlari qanday?
3. QSplitter vidjetinig qanday xususiyatlari mavjud?

8.4.15.§. QCalendar vidjeti

QCalendarWidget oylik kalendar vidjetini taqdim etadi. Bu foydalanuvchiga sanani sodda va intuitiv tarzda tanlash imkonini beradi.


```

import sys
from PyQt5.QtWidgets import *
from PyQt5.QtCore import QDate

class Example(QWidget):
    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):

        calc = QCalendarWidget(self)
        calc.setGridVisible(True)
        calc.move(20, 20)
        calc.clicked[QDate].connect(self.showDate)

        self.lbl = QLabel(self)
        date = calc.selectedDate()
        self.lbl.setText(date.toString())
        self.lbl.move(130, 260)

        self.setGeometry(300, 300, 350, 300)
        self.setWindowTitle('Calendar')
        self.show()

    def showDate(self, date):
        self.lbl.setText(date.toString())

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())

```

Yuqoridagi misolda kalendar vidjeti va label vidjetidan foydalanilgan bo‘lib, joriy tanlangan sana teg vidjetida ko‘rsatiladi.

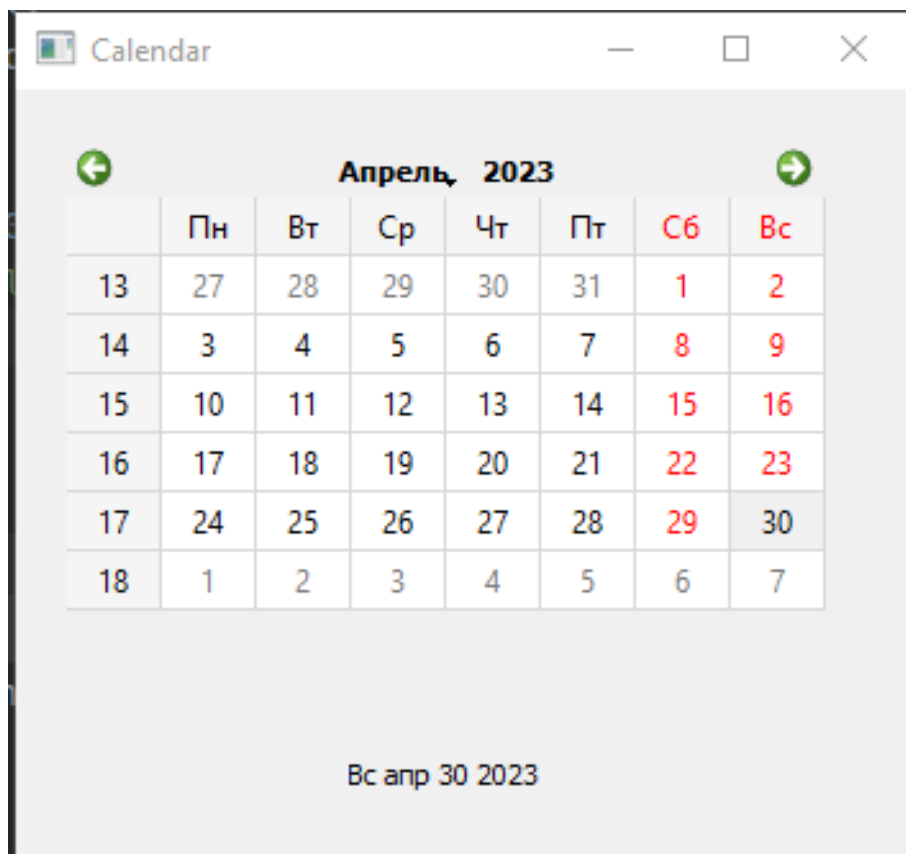
Dastur tahlili:

```
calc.clicked[QDate].connect(self.showDate)
```

- vidjetdan sanani tanlansa, bosilgan [QDate] signali yonadi. Ushbu signalni maxsus **showDate()** funksiyasiga biriktirildi.
-

```
def showDate(self, date):
    self.lbl.setText(date.toString())
```

- **selectDate()** metodini chaqirish orqali tanlangan sanani qaytariladi. Keyin sana ob'ektini satrga aylantirib va uni label vidjetiga o'rnatiladi.



73-rasm. QCalendar vidjetining ishlari va ko'rinishi.

Nazorat savollari:

1. QCalendar vidjeti qanday imkoniyatlarga ega?
2. QCalendar vidjetining qanday metodlari mavjud?
3. QCalendar vidjetining asosiy xususiyatlari qanday?

8.5.8. PyQt da CSS ssenariylarini qo'llanilishi

PyQt bilan CSS uslublar jadvallaridan foydalanish mumkin. **setStyleSheet()** metod chaqiruvi ilovadagi turli komponentlarni stylelash erkinligini beradi. Ushbu metod argumenti sifatida oddiy CSS kodlarni kiritish mumkin, masalan:

```
lbl_red.setStyleSheet("color: red;"
                      "border-style: dashed;")
```

```
"border-width: 3px;"  
"border-color: #FB8072;"  
"border-radius: 3px")
```

PyQt CSS stylelar jadvali

```
import sys  
from PyQt5.QtWidgets import QApplication, QWidget, QLabel,  
QVBoxLayout  
  
class MyApp(QWidget):  
  
    def __init__(self):  
        super().__init__()  
        self.initUI()  
  
    def initUI(self):  
  
        lbl_red = QLabel('Red')  
        lbl_green = QLabel('Green')  
        lbl_blue = QLabel('Blue')  
  
        lbl_red.setStyleSheet("color: red;"  
                               "border-style: solid;"  
                               "border-width: 2px;"  
                               "border-color: #FA8072;"  
                               "border-radius: 3px")  
        lbl_green.setStyleSheet("color: green;"  
                                 "background-color: #7FFFD4")  
        lbl_blue.setStyleSheet("color: blue;"  
                                "background-color: #87CEFA;"  
                                "border-style: dashed;"  
                                "border-width: 3px;"  
                                "border-color: #1E90FF")  
  
        vbox = QVBoxLayout()  
        vbox.addWidget(lbl_red)  
        vbox.addWidget(lbl_green)  
        vbox.addWidget(lbl_blue)  
  
        self.setLayout(vbox)
```

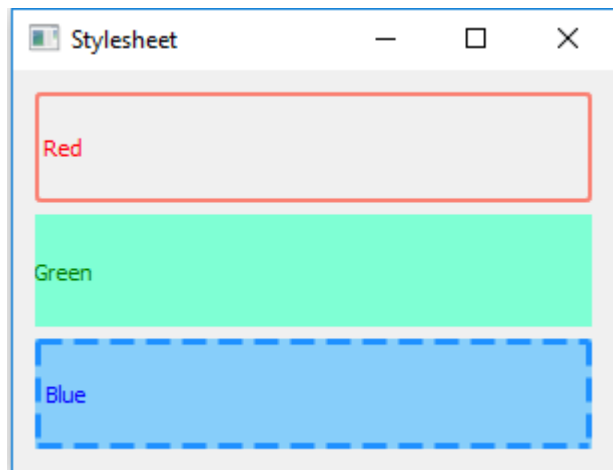
```

        self.setWindowTitle('Stylesheet')
        self.setGeometry(300, 300, 300, 200)
        self.show()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = MyApp()
    sys.exit(app.exec_())

```

Turli uslublarda uchta yorliq vidjetini bezaldi va u quyidagi ko‘rinishga ega bo‘ldi:



74-rasm. Yuqoridagi css stylelangan labellar ko‘rinishi

Tahlillash:

```

lbl_red = QLabel('Red')
lbl_green = QLabel('Green')
lbl_blue = QLabel('Blue')

```

QLabel sinfidan uchta yorliqli vidjet yaratish uchun foydalanildi. Yorliq matni mos ravishda "Qizil", "Yashil" va "Moviy" ranglar o‘rnatildi.

```

lbl_red.setStyleSheet("color: red;"
                       "border-style: solid;"
                       "border-width: 2px;"
                       "border-color: #FA8072;"
                       "border-radius: 3px")

```

lbl_red QLabel vidjeti uchun **SetStyleSheet()** metodidan foydalanib, harf rangini qizil rangga, chegara chizig‘ini “solid”, chegara qalinligini 2px, chegara rangini #FA8072 va chegara chizig‘ining cheti qalinligi 3px qilib belgilandi.

```
lbl_green.setStyleSheet("color: green;"  
                        "background-color: #7FFFD4")
```

Xuddi shunday, **lbl_green** yorliqning harf rangini yashil va #7FFFD4 rangiga bo'yaldi.

```
lbl_blue.setStyleSheet("color: blue;"  
                      "background-color: #87CEFA;"  
                      "border-style: dashed;"  
                      "border-width: 3px;"  
                      "border-color: #1E90FF")
```

lbl_blue yorliqning harf rangini ko'k, fonni esa #87CEFA, chegarani chiziqcha uslubi, chegara chizig'i qalinligini 3px va #1E90FF chegara rangini o'rnatdi.

```
vbox = QVBoxLayout()  
vbox.addWidget(lbl_red)  
vbox.addWidget(lbl_green)  
vbox.addWidget(lbl_blue)  
self.setLayout(vbox)
```

Uchchala QLabel vidjetini vertikal joylashtirish uchun vertikal quti tartibidan (QVBoxLayout()) foydalanildi.

Nazorat savollari:

1. CSS style larni PyQt vidjetlari bilan qo'llash uchun qanday amallarni bajarish kerak?
2. Vidjetlarni vertikal holatda joylashtirish uchun qanday vidjetdan foydalanish mumkin?
3. setStyleSheet metodi qanday vazifani bajaradi?

8.6.8.PyQt5 da oddiy bloknnot dasturi

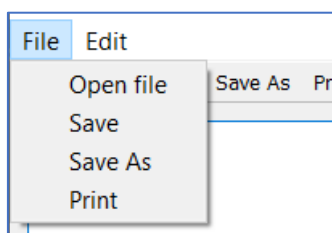
Ushbu mavzuda PyQt5 paketi yordamida bloknnot dasturini yaratishni ko'rib chiqiladi.

Bloknnot oddiy matn muharriri bo'lib, ochiq matnli fayllarni yaratish, ochish, tahrirlash va o'qish imkonini beradi.

PyQt5 – bu crossplatformali GUI asboblari to‘plami bo‘lib, ushbu kutubxona taqdim etgan vositalar va soddaligi tufayli interaktiv desktop ilovalarni juda oson ishlab chiqish mumkin.

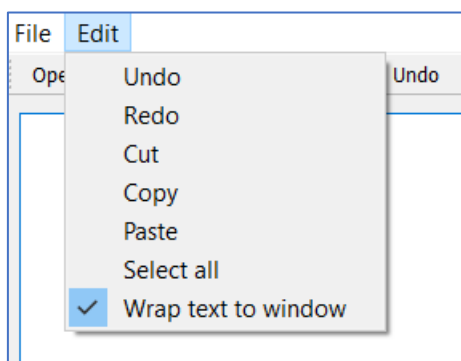
Ushbu “bloknout” deb nomlanuvchi dasturning ko‘rinishini tuzishning bosqichlari quyidagicha:

1. Vertikal tartibni yarating
2. QPlainTextEdit ob’yektini dasturga qo‘shish
3. Konteyner yaratish, ya’ni QWidget ob’yektini
4. Ushbu vertikal tartibni konteynerga o‘rnatish
5. Holat bo‘yicha statuslarni ko‘rsatish uchun status satrini yarating
- 6 Fayl menyusi satrini yaratish va qo‘shish, quyida fayl menyusi paneli qanday ko‘rinishga ega bo‘lishi keltirilgan:



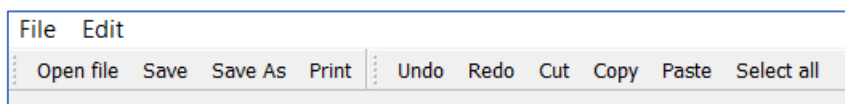
75-rasm. Fayl menyusi tarkibi

7. **Edit** menyusi satrini yaratish va unga turli xil tahrirlash amallarini qo‘shish, quyida edit menyusi tarkibi qanday ko‘rinishga ega bo‘lishi keltirilgan:



76-rasm. Edit menyusi tarkibi.

8. Fayl va Edit amallari uchun ikkita asboblari paneli yaratiladi. Quyida asboblari paneli qanday ko‘rinishda bo‘lishi keltirilgan:



Bloknot dasturining dasturiy qismini (backend) amalga oshirish bosqichlari:

1. Path o‘zgaruvchisi yaratiladi va unga “None” boshlang‘ich qiymat beriladi;
2. Fayl menyusi amallarining har biriga amallarni qo‘shiladi, bular asboblar panelidagi amallar bilan bir xil, chunki ikkalasi ham bir xil amallarni bajaradi.
3. O‘tkazilgan qiymatni ko‘rsatuvchi muhim usul yaratiladi. Popup oyna, u faylni saqlash yoki ochishda xatolik yuz berganligini ko‘rsatish uchun ishlatiladi;
4. Fayl nomiga ko‘ra oyna sarlavhasini o‘zgartiruvchi **update_title** metodini yaratish;
5. **file_open** amali ichida try-except blokini yaratiladi. Faylni ochish mobaynida, **window_title** - sarlavha va **path** - yo‘lni qiymatlarini yangilanadi
6. **Save** metodi ichida **path** –ya’ni saqlash yo‘li bo‘lmasa, **path_to_save** usulini chaqiriladi va boshqa URL ga faylni saqlanadi
7. **Save-as** metodi ichida faylni foydalanuvchi tomonidan tanlangan yo‘lda saqlanadi;
8. **print_file** amali ichida **QPrintDialog** ob’yekti yordamida faylni chop etiladi;
9. **Edit** tugmasi amali ichida bloknot satrlarini wrap rejimini belgilangan holatga ko‘ra o‘rnatiladi;
10. Xuddi shunday “Undo” – bekor qilish, “Redo” - takrorlash, “Cut” - kesish, “Copy” - nusxalash, “Paste” - joylashtirish va “Select all” - Barchasini tanlash amallarini kerakli funksiyalar bilan bog‘lash.

Dastur kodi quyidagicha bo‘ladi:

```
# zarur kutubxonalarni import qilish
from PyQt5.QtGui import *
from PyQt5.QtWidgets import *
from PyQt5.QtCore import *
from PyQt5.QtPrintSupport import *
import os
import sys

class MainWindow(QMainWindow):

    def __init__(self, *args, **kwargs):
        super(MainWindow, self).__init__(*args, **kwargs)
```

```

self.setGeometry(100, 100, 600, 400)
layout = QVBoxLayout()
self.editor = QPlainTextEdit()
fixedfont = QFontDatabase.systemFont (QFontDatabase.
FixedFont)
fixedfont.setPointSize(12)
self.editor.setFont(fixedfont)
self.path = None
layout.addWidget(self.editor)
container = QWidget()

container.setLayout(layout)
self.setCentralWidget(container)
self.status = QStatusBar()
self.setStatusBar(self.status)

# fayl asboblar panelini yaratish
file_toolbar = QToolBar("File")

# fayl asboblar panelini oynaga qo'shish
self.addToolBar(file_toolbar)

# fayl menyusini yaratish
file_menu = self.menuBar().addMenu("&File")

# fayl menyusiga qo'shish uchun amallar yaratish
# ochiq fayl amalini yaratish
open_file_action = QAction("Open file", self)

# sozlash holati bo'yicha maslahat
open_file_action.setStatusTip("Open file")

# ochiq faylga amal qo'shish
open_file_action.triggered.connect(self.file_open)

# buni fayl menyusiga qo'shish
file_menu.addAction(open_file_action)

```



```

# buni asboblarni paneliga qo'shish
file_toolbar.addAction(open_file_action)

# xuddi shunday saqlash amalini yaratish
save_file_action = QAction("Save", self)
save_file_action.setStatusTip("Save current page")
save_file_action.triggered.connect(self.file_save)
file_menu.addAction(save_file_action)
file_toolbar.addAction(save_file_action)

# xuddi shunday save actionni yaratish
saveas_file_action = QAction("Save As", self)
saveas_file_action.setStatusTip("Save current page to
specified file")

saveas_file_action.triggered.connect(self.file_saveas)
file_menu.addAction(saveas_file_action)
file_toolbar.addAction(saveas_file_action)

# print uchun
print_action = QAction("Print", self)
print_action.setStatusTip("Print current page")
print_action.triggered.connect(self.file_print)
file_menu.addAction(print_action)
file_toolbar.addAction(print_action)

# matnni tahrirlash uchun boshqa asboblarni paneli
yaratish
edit_toolbar = QToolBar("Edit")

# ushbu asboblarni panelini asosiy oynaga qo'shish
self.addToolBar(edit_toolbar)

# tahrirlash menyusini yaratish
edit_menu = self.menuBar().addMenu("&Edit")

# asboblarni paneli va menyu paneliga amallar qo'shish

```

```
# bekor qilish amali uchun
undo_action = QAction("Undo", self)

# holat bo'yicha StatusTip qo'shish
undo_action.setStatusTip("Undo last change")

# bekor qilish amaliga funksiyasini bog'lash
undo_action.triggered.connect(self.editor.undo)

# buni asboblari va menyu paneliga qo'shish
edit_toolbar.addAction(undo_action)
edit_menu.addAction(undo_action)

# takrorlash amali
redo_action = QAction("Redo", self)
redo_action.setStatusTip("Redo last change")

# qaytadan bajarish amaliga funksiyasini bog'lash
redo_action.triggered.connect(self.editor.redo)

# buni menyu va asboblari paneliga qo'shish
edit_toolbar.addAction(redo_action)
edit_menu.addAction(redo_action)

# cut amali
cut_action = QAction("Cut", self)
cut_action.setStatusTip("Cut selected text")

# cut amaliga funksiyasini bog'lash
cut_action.triggered.connect(self.editor.cut)

# buni menyu va asboblari paneliga qo'shish
edit_toolbar.addAction(cut_action)
edit_menu.addAction(cut_action)

# nusxa ko'chirish harakati
copy_action = QAction("Copy", self)
copy_action.setStatusTip("Copy selected text")
```

```

# cut amaliga funksiyasini bog'lash
copy_action.triggered.connect(self.editor.copy)

# buni menyu va asboblari paneliga qo'shish
edit_toolbar.addAction(copy_action)
edit_menu.addAction(copy_action)

# joylashtirish amali
paste_action = QAction("Paste", self)
paste_action.setStatusTip("Paste from clipboard")

# ishga tushirilganda nusxa olingan matnni
joylashtiring
paste_action.triggered.connect(self.editor.paste)

# buni menyu va asboblari paneliga qo'shish
edit_toolbar.addAction(paste_action)
edit_menu.addAction(paste_action)

# barchasini tanlash amali
select_action = QAction("Select all", self)
select_action.setStatusTip("Select all text")

# bu ishga tushganda butun matnni tanlang
select_action.triggered.connect(self.editor.selectAll
)

# buni menyu va asboblari paneliga qo'shish
edit_toolbar.addAction(select_action)
edit_menu.addAction(select_action)

# wrap amali
wrap_action = QAction("Wrap text to window", self)
wrap_action.setStatusTip("Check to wrap text to
window")

```

```

# uni tekshiriladigan qilish
wrap_action.setCheckable(True)

# buni tekshirish
wrap_action.setChecked(True)

# harakat qo'shish
wrap_action.triggered.connect(self.edit_toggle_wrap)

# uni asboblari paneliga emas, balki tahrirlash
menyusiga qo'shish
edit_menu.addAction(wrap_action)

# qo'ng'iroqni yangilash sarlavhasi usuli
self.update_title()

# barcha komponentlarni ko'rsatish
self.show()

# dialogning muhim usulini yaratish
# xatolarni ko'rsatish uchun
def dialog_critical(self, s):

    # QMessageBox ob'ektini yaratish
    dlg = QMessageBox(self)

    # dlg-ga matnni o'rnatish
    dlg.setText(s)

    # unga belgini o'rnatish
    dlg.setIcon(QMessageBox.Critical)

    # dlg-ni ko'rsatish
    dlg.show()

# file open action tomonidan chaqirilgan harakat
def file_open(self):

```

```

        # yo'l va bool qiymatini olish
        path, _ = QFileDialog.getOpenFileName(self, "Faylni
ochish", "", "Text documents (*.txt);Barcha tiplar (*.*)")

        # agar path to'g'ri bo'lsa
        if path:
            # path bo'yicha faylni ochishga harakat qil
            try:
                with open(path, 'r+') as f:
                    # faylni o'qing
                    text = f.read()

            # agar xato bo'lsa
            except Exception as e:

                # kritik usul yordamida xatoni ko'rsatish
                self.dialog_critical(str(e))
            # aks holda
            else:
                # path qiymatini yangilang
                self.path = path

                # matnni yangilang
                self.editor.setPlainText(text)

                # sarlavhani yangilang
                self.update_title()

# faylni saqlash harakati deb nomlangan harakat
def file_save(self):

    # agar saqlash yo'li bo'lmasa
    if self.path is None:

        # save_as metodini chawirish
        return self.file_saveas()

    # aks holda save_to_path metodini chaqirilsin

```

```

        self._save_to_path(self.path)

# file_saveas metodi
def file_saveas(self):
    path, _ = QFileDialog.getSaveFileName(self, "Faylni
saqlash", "", "Text documents (*.txt);Barcha turlar (*.*)")

    # agar dialog bekor qilinsa, ya'ni yo'l tanlanmaydi
    if not path:
        return

    self._save_to_path(path)

# save_to_path metodi
def _save_to_path(self, path):
    text = self.editor.toPlainText()

    try:
        with open(path, 'w') as f:
            f.write(text)

    except Exception as e:
        self.dialog_critical(str(e))

    else:
        self.path = path
        self.update_title()

# file_print metodi
def file_print(self):

    # QprintDialog ob'yektini yaratish
    dlg = QPrintDialog()

    if dlg.exec_():
        self.editor.print_(dlg.printer())

# update_title metodi

```

```

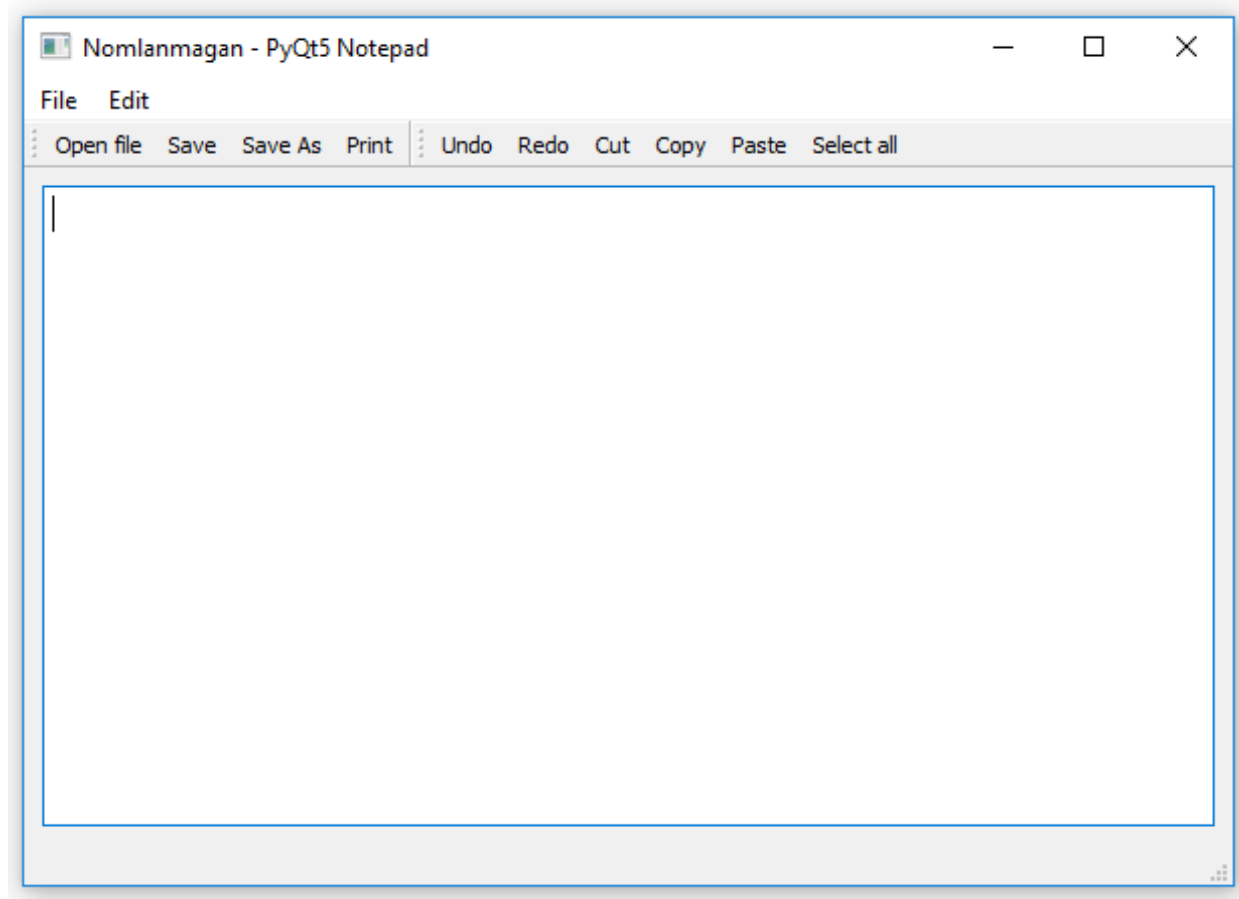
def update_title(self):
    self.setWindowTitle("%s - PyQt5 Notepad" %(os.path.
basename (self.path)
    if self.path else "Nomlanmagan"))
def edit_toggle_wrap(self):
    self.editor.setLineWrapMode(1 if self.editor.
lineWrapMode() == 0 else 0 )

if __name__ == '__main__':
    app = QApplication(sys.argv)
    app.setApplicationName("PyQt5- Notepad")
    window = MainWindow()

    app.exec_()

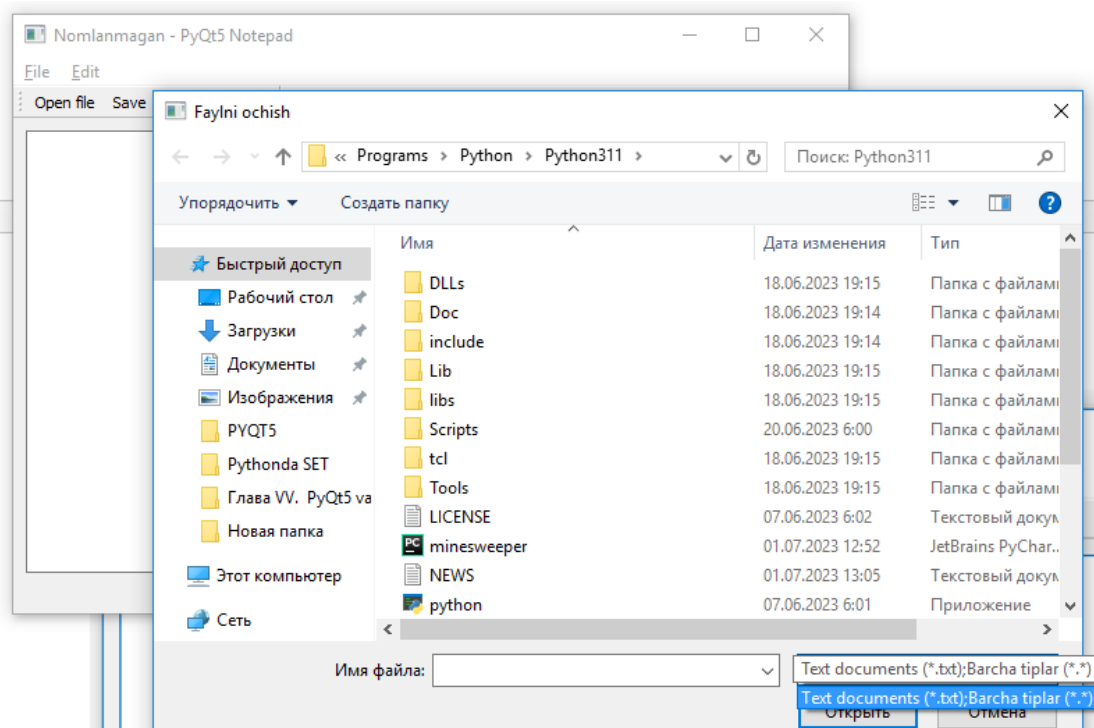
```

Dastur ishga tushirilgandan so‘ng quyidagi oyna paydo bo‘ladi:



78-rasm. Bloknnot dasturining ko‘rinishi

Ushbu dasturdan faylni saqlash, ochish va print amallari File menyusida jamlangan:



79-rasm. Faylni ochish amalini ko‘rinishi

Nazorat savollari:

1. Bloknot dasturining ko‘rinishi qaysi uskunalardan foydalanib tuzildi?
2. QFileDialog sinfining qanday metodlaridan foydalanildi?
3. Faylni yozish uchun oshish amali qanday bajariladi?
4. Try-except bloki nima maqsadda ishlatiladi?
5. Menudagi buyruqlarga funksiyalarni bog‘lash qanday amalga oshiriladi?

IX-BOB. PYTHONDA TARMOQ DASTURLASH

9.1.§. Pythonda Tarmoq dasturlashiga kirish

Soket - bu jarayonlar o'rtasida ma'lumot almashinuvini ta'minlash uchun dasturlash interfeysi.

Socket turlari:

- Server - xabarlarni qabul qiluvchi socket.
- Mijoz - xabarlarni yuboradigan socket.

Soketlar protokollarning transport qatlamida ishlaydi va shunga mos ravishda 2 tur mavjud:

- **Oqim** (TCP asosida, kodda ko'rsatilgan `SOCK_STREAM`) - TCP protokoli asosida o'rnatilgan ulanishga ega soketlar, ikki tomonlama bo'lishi mumkin bo'lgan baytlar oqimini uzatadi - ya'ni Ilova ma'lumotlarni qabul qilishi va yuborishi mumkin.
- **Datagram** (UDP asosida, kodda ko'rsatilgan `SOCK_DGRAM`) - ular o'rtasida aniq ulanishni talab qilmaydigan socketlar. Xabar belgilangan socketga yuboriladi va shunga mos ravishda belgilangan socketdan qabul qilinishi mumkin.

Soket tarkibiasllida IP manzil va portdan iborat bo'ladi.

IP manzil - IP protokoli yordamida qurilgan kompyuter tarmog'idagi tugunning yagona tarmoq manzili. IPv4 protokoli versiyasida IP manzili uzunligi 4 bayt (masalan, 192.168.0.3), IPv6 protokoli versiyasida esa IP manzili 16 bayt (masalan, 2001:0db8:85a3:0000:0000: 8a2e: 0370: 7334). IP-manzil noyob bo'lishi kerak.

Port - transport qatlami protokollari (TCP, UDP va boshqalar) sarlavhalarida yozilgan natural son. Port bir xil xost ichidagi paketni qabul qilish jarayonini aniqlash uchun ishlatiladi.

Python socketlar bilan ishlash uchun o'rnatilgan `socket` kutubxonasidan foydalanadi. Modulning asosiy funktsiyalaridan biri bu `socket()` ulanish bilan ishlash uchun tegishli funktsiyalarga ega bo'lgan socket tipidagi ob'ektni qaytaradigan funktsiyadir.:

```
class socket.socket
sock = socket.socket()
```

socket ob'yektining funksiyalari

- **socket.bind(address)** - Socketsni manzilga bog'laydi (IP manzil va portni ishga tushiradi). Bundan oldin socket ulanmagan bo'lishi kerak.
- **socket.listen([backlog])** - Ulanishlarni qabul qilish uchun serverni almashtiradi. "backlog (int)" parametri server qabul qiladigan ulanishlar sonidir.
- **socket.accept()** - Ulanishni qabul qiladi va mijozdan xabar kutayotganda dasturni bloklaydi. Natijada kortejni qaytaradi:
 - **conn**: ma'lumotlarni yuborish/qabul qilish uchun ishlatilishi mumkin bo'lgan ulanish ob'ekti (socket);
 - **address**: mijozning manzili.
- **socket.recv(bufsize[, flags])** - socketdan ma'lumotlarni ikkilik formatda (baytlar to'plami) o'qiydi va qaytaradi. Parametr - bitta xabardagi baytlarning maksimal soni. **bufsize (int)**
- **socket.send(bytes[, flags])** - Mijozga ma'lumotlarni yuboradi va yuborilgan baytlar sonini qaytaradi. Parametr ikkilik ma'lumotlardir. **bytes (bytes)**
- **socket.close()** - Socketni yopadi.

Socket bilan ishlash ko'p jihatdan fayl ob'yekti bilan ishlashga o'xshaydi. Printsip - ochiq ulanish - ma'lumotlar - yopiq ulanish deb hisoblanadi.

Nazorat savollari:

1. port deb nimaga aytiladi?
2. Socket nima ?
3. Socketning qanday turlari mavjud?
4. Socket ob'yektining qanday funksiyalari mavjud?

9.2.8. Mijoz-server ilovalari

Python-da o'rnatilgan socket moduli tarmoq orqali muloqot qilish uchun funktsionallikni ta'minlaydi. Ushbu modul socket sinfi shaklida so'rovlarni yuborish va qabul qilish uchun past darajadagi interfeysda aniqlanadi. Va yaratayotgan narsadan qat'iy nazar - server yoki mijoz dasturi, so'rovlarni yuborish uchun socket ob'yektini yaratish kerak bo'ladi:

```
import socket  
sock = socket.socket()
```

socket bilan ishlashni tugatgandan so'ng, uni close() metodi yordamida yopish kerak.

```
import socket  
client = socket.socket()  
client.close()
```

Socket bilan keyingi harakatlar qanday rozetka yaratilayotganiga bog'liq bo'lib, server yoki mijoz uchun yaratiladi. Bunday holda, Python socketlarida eng oddiy mijozni yaratishni ko'rib chiqiladi.

Serverga bog'lanish. **connect()** metodi serverga ulanish uchun ishlatiladi.

```
socket.connect(address)
```

Ushbu **connect** metodi parametr sifatida server manzilini qabul qiladi. Manzil odatda kortej sifatida ifodalanadi

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki lokal xost nomi bo'lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo'lgan 2 baytlik qiymatdir. Masalan:

```
import socket  
client = socket.socket()  
client.connect(("www.kun.uz", 80))  
print("Connected...")  
client.close()
```

Bu yerda birinchi parametr sifatida: "www.kun.uz" manziliga va ikkinchi parametr sifatida 80-portga ulanishga harakat qilmoqda. Ya'ni, www.kun.uz80-portda

ishlaydigan oddiy veb-saytga ulanishga harakat qilinmoqda (odatda veb-server 80-portda ishlaydi). Ulangandan so'ng, shunchaki konsolga satr chiqariladi va socketni yopiladi.

Server bilan o'zaro ulanish.

Ulanish o'rnatilgandan so'ng serverga ma'lumotlarni yuborish va undan qabul qilish mumkin. Ma'lumotlarni yuborish uchun **socket.send()** usuli qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Soketdan ma'lumotlarni qabul qilish uchun **socket.recv()** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize[, flags])
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Bufer hajmini 2 ga karrali qilish yaxshidir, masalan, 512, 1024 va boshqalar. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Masalan, "www.kun.uz" serveriga ma'lumotlarni yuboriladi va undan qaytgan javobni qabul qilinadi:

```
import socket
client = socket.socket()
client.connect(("www.kun.uz", 80))
message = "GET / HTTP/1.1\r\nHost:www.kun.uz\r\n
Connection: close\r\n\r\n"
print("Connecting...")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bunday holda, satr veb-sayt tushunadigan standart HTTP protokoli sarlavhalari bilan yuboriladi. "GET/HTTP/1.1\r\nHost: www.kun.uz HTTP so'rov formati birinchi navbatda so'rov turini, so'ralgan manbaga yo'lni va maxsus protokol versiyasidan iborat so'rov qatorini () o'z ichiga oladi . Ya'ni, bu yerda xabarda GET so'rovi "/" yo'li bo'ylab yuborilganligini bildiriladi (ya'ni www.kun.uz saytining ildiziga). Bu holda

HTTP / 1.1 protokoli qo'llaniladi. So'rov qatori ikki karet to'plami va qaytariladigan belgilar qatori \r\n bilan tugashi kerak.

Bundan tashqari, HTTP so'rovida sarlavhalar bo'lishi mumkin. Shunday qilib, bu holda "Xost" sarlavhasini yuboriladi, bu esa xost manziliga ishora qiladi. Bu holda bu "www.kun.uz:80". Va shuningdek, bu holatda, "close" qiymatiga ega bo'lgan "connection" sarlavhasini yuboriladi - bu qiymat serverga ulanishni yopishni buyuradi.

Bu yerda satrlarni emas, faqat baytlarni yuborish mumkinligi sababli, uni yuborilganida satrni baytlar to'plamiga aylantiriladi. Buning uchun **bytes** sinf ob'yektini qaytaradigan **encode()** usuli qo'llaniladi.

```
client.send(message.encode())
```

Serverga xabar yuborilganidan so'ng, `recv()` funksiyasidan foydalanib ma'lumotlarni olishga harakat qilinadi. Bunday holda, olingan ma'lumotlar uchun bufer 1024 bayt hajmiga ega bo'ladi. Usulning natijasi olingan ma'lumotlardir. Biroq, socket ma'lumotlarni baytlar to'plami shaklida oladi (aniqrog'i, ob'yekt bytes). Ularni satrga aylantirish uchun `decode()` usulidan foydalaning:

```
data = client.recv(1024) # serverdan ma'lumotlarni olish
print("Server sent: ", data.decode())
```

Va agar ushbu dasturni Python-da ishga tushirilsa, quyidagi kabi javob olish mumkin:

```
Connected...
Server sent: HTTP/1.1 301 Moved Permanently
Connection: close
Content-Length: 0
Server: Varnish
Retry-After: 0
Location: https://www.python.org/
Accept-Ranges: bytes
Date: Tue, 02 May 2023 17:52:32 GMT
Via: 1.1 varnish
X-Served-By: cache-bma1653-BMA
```

```
X-Cache: HIT
```

```
X-Cache-Hits: 0
```

```
X-Timer: S1683049953.637569,VS0,VE0
```

```
Strict-Transport-Security: max-age=63072000; includeSubDomains;  
preload
```

Shuningdek, server bizga http sarlavhalarida veb-sayt <https://www.kun.uz/> manziliga ko‘chirilganligini ko‘rsatadigan qatorni yuboradi.

Nazorat savollari:

1. Socket nima?
2. decode() funksiyasi nima maqsadda foydalaniladi?
3. Serverga ma’lumot jo‘natish qanday amalga oshiriladi?
4. Serverdan ma’lumotni qanday qabul qilin olinadi?
5. Connect() metodi qanday parametrlarni qabul qiladi?

9.3.8. Server dasturini yaratish

Python-da serverni, shuningdek mijozni yaratish uchun socket sinfi ham ishlatiladi, ammo bu mijoz soketi bilan ishlashdan biroz farq qiladi.

Serverni ulanish

Server kiruvchi ulanishlarni kutib turadi (listening), ularni qandaydir tarzda qayta ishlaydi va javob yuboradi. Server o‘z ishini boshlashi uchun, avvalo, u ulanishlarni tinglaydigan manzilni aniqlab olishi kerak. Buning uchun **bind()** metodi qo‘llaniladi.

```
socket.bind(address)
```

Bu usul server ishga tushadigan manzilni qabul qiladi. Odatiy bo‘lib, manzil ikki elementdan iborat to‘plamdir:

```
(host, port)
```

Birinchi element string sifatida xost hisoblanadi. Bu, masalan, "127.0.0.1" shaklidagi IP-manzil yoki mahalliy xost nomi bo‘lishi mumkin. Ikkinchi parametr raqamli port raqamidir. Port 0 dan 65535 gacha bo‘lgan 2 baytlik qiymatni ifodalaydi. Bir xil manzilda (bir xil mashinada) bir nechta turli tarmoq ilovalari ishga tushirilishi mumkinligi sababli, port ushbu ilovalarni farqlash imkonini beradi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
```

Bu yerda server 12345-portda ishlaydi. E'tibor bering, barcha portlar ham bepul bo'lmisligi mumkin. Ammo, qoida tariqasida, band bo'lgan portlar unchalik ko'p emas.

Manzil sifatida joriy xost nomidan foydalanamiz. Uni olish uchun funktsiyadan foydalaniladi **socket.gethostname()** (odatda bu joriy kompyuterning nomi).

Ulanishlarni tinglash

Serverni bog'lagandan so'ng ulanishlarni tinglash uchun uni ishga tushirish kerak. Buning uchun **listen()** metodi qo'llaniladi.

```
socket.listen([backlog])
```

Bu backlog parametrini qabul qiladi. backlog – bu socket uchun ruxsat etilgan navbatdagi kiruvchi ulanishlarning maksimal soni. Ya'ni, mijozlar ulanganda, ular navbatda turishadi va server joriy mijozni qayta ishlaguncha kutishadi. Agar navbatda server tomonidan qayta ishlanishini kutayotgan mijozlar soni allaqachon belgilangan bo'lsa, barcha yangi mijozlar rad etiladi. Masalan:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)
```

Mijozni qabul qilish va qayta ishlash

accept() metodi kiruvchi ulanishlarni qabul qilish uchun ishlatiladi. Bu usul 2 ta tuple tipli qiymatni qaytaradi

```
(conn, address)
```

Birinchi element `conn` - server mijoz bilan aloqa qiladigan boshqa socket ob'ektini ifodalaydi. Ikkinchi element `address` - ulangan mijozning manzili hisoblanadi. Shuni ta'kidlash kerakki, mijoz bilan o'zaro aloqalar tugagandan so'ng, `conn` socketini `close()` usuli bilan yopish kerak bo'ladi.

Tuplening birinchi elementi - `conn` yordamida mijozga xabar yuborish yoki aksincha ma'lumotlarni qabul qilish mumkin. Socketdan ma'lumotlarni qabul qilish uchun **`socket.recv()`** usuli qo'llaniladi.

```
bytes = socket.recv(bufsize)
```

Bu zarur parametr sifatida bir vaqtning o'zida boshqa socketdan olinishi mumkin bo'lgan baytlardagi maksimal bufer hajmini oladi. Qaytish qiymati boshqa socketdan olingan baytlar to'plamidir.

Ma'lumotlarni yuborish uchun **`socket.send()`** metodi qo'llaniladi, u parametr sifatida yuboriladigan ma'lumotlar to'plamini oladi.

```
socket.send(bytes)
```

Endi bir misolni ko'rib chiqaylik. `Server.py` faylida serverni quyidagi kod bilan aniqlaylik:

```
import socket

server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server starts")

con, addr = server.accept()
print("connection: ", conn)
print("client address: ", addr)
```



```
message = "Hello Client!"
con.send(message.encode())
con.close()
print("Server ends")
server.close()
```

Bu yerda server mijozni qabul qiladi, uning ulanishi haqidagi ma'lumotlarni chiqaradi va mijozga "Hello Client!" qatorini qaytarib yuboradi.

Va client.py faylida quyidagi mijoz kodini aniqlanadi:

```
import socket
client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Bizning serverimiz va mijozimiz bitta kompyuterda ishlaganligi sababli, **socket.gethostname()** server kodidagi kabi ulanish uchun server manzilini aniqlash uchun funksiya va port 12345 ishlatiladi. Serverga ulangandan so'ng undan ma'lumotlarni olinadi va uni konsolda ko'rsatadi.

Avval server kodini ishga tushiraylik. Va keyin mijoz kodini ishga tushiriladi. Natijada, server ulanishni qabul qiladi, u haqidagi ma'lumotlarni konsolda ko'rsatadi va mijozga xabar yuboradi:

```
c:\python>python server.py
Server starts
connection: <socket.socket fd=432, family=2, type=1, proto=0,
laddr=('192.168.0.102', 12345), raddr=('192.168.0.102', 61824)>
client address: ('192.168.0.102', 61824)
Server ends
```

Xususan, bu yerda server va mijoz 192.168.0.102 da ishlayotganini va mijoz 61824 portidan foydalanayotganini ko‘ramiz.

Va mijoz serverdan xabar oladi va uni konsolga chop etadi:

```
c:\python>python client.py  
Server sent: Hello Client!
```

Bir nechta mijozlar bilan ishlash

Bunday holda, server bitta mijozga xizmat qiladi va ishlashni to‘xtatadi. Agar biz server ko‘p mijozlar bilan ishlashni istasak, unda cheksiz tsikldan foydalanishga to‘g‘ri keladi:

```
import socket  
from datetime import datetime  
  
server = socket.socket()  
hostname = socket.gethostname()  
port = 12345  
server.bind((hostname, port))  
server.listen(5)  
  
print("Server running")  
while True:  
    con, addr = server.accept()  
    print("client address: ", addr)  
    message = datetime.now().strftime("%H:%M:%S")  
    con.send(message.encode())  
    con.close()
```

Bunday holda, masalan, o‘rnatilgan modul datetime va datetime.now().strftime() funksiyasidan foydalanib, joriy vaqtni satr sifatida olish, keyinchalik uni mijozga yuborish mumkin. Natijada, mijoz so‘rov bilan joriy vaqthaqidagi ma’lumotlarni ham oladi.

Ikki tomonlama aloqa

Yuqoridagi misollarda aloqa bir tomonlama edi - server ma'lumotlarni yuboradi va mijoz uni qabul qiladi. Keling, mijoz ham, server ham ma'lumotlarni jo'natgan va qabul qilganda eng oddiy ikki tomonlama aloqani ko'rib chiqaylik. Serverga mijozdan bir nechta satr olishiga ruxsat beraylik, uni o'zgartirib so'ngra mijozga qaytarib yuboraylik:

```
import socket
server = socket.socket()
hostname = socket.gethostname()
port = 12345
server.bind((hostname, port))
server.listen(5)

print("Server running")
while True:
    con, _ = server.accept()
    data = con.recv(1024)
    message = data.decode()
    print(f"Client sent: {message}")
    message = message[::-1]
    con.send(message.encode())
    con.close()
```

Mijoz konsoldan satr kiritish va uni serverga yuborish uchun kodni aniqlasin:

```
import socket

client = socket.socket()
hostname = socket.gethostname()
port = 12345
client.connect((hostname, port))
```

```
message = input("Input a text: ")
client.send(message.encode())
data = client.recv(1024)
print("Server sent: ", data.decode())
client.close()
```

Yuqoridagi ishning natijasi:

Client:

```
c:\python>python client.py
Input a text: hello
Server sent: olleh
c:\python>
```

Server:

```
c:\python>python server.py
Server running
Client sent: hello
```

Nazorat savollari:

1. Server bilan aloqani qanday amalga oshiriladi?
2. socket.bind() metodi nima amalni bajaradi?
3. socket.gethostname() metodi qanday vazifani bajaradi?
4. listen() metodining ishlash tamoyili qanday?
5. socket.recv() metodidan nima maqsadda foydalaniladi?

X-BOB. PYINSTALLER va AUTO-PY-TO-EXE paketlari bilan ishlash

Ushbu bo‘limda pythonda yaratilgan loyihalarning yuklanuvchi faylini hosil qilish bo‘yicha bilim va ko‘nikmalarga berib o‘tiladi.

Dasturchi yaratgan dasturini kimgadir taqdim etishi uchun albatda butun bir loyihasini emas abalki butun loyihani bitta yuklanuvchi faylga birlashtirib, keyin taqdim etadi. Bunday bitta faylga birlashtirishning turli usullari mavjud.

10.1.§. Pythonning PYINSTALLER paketi va uning imkoniyatlari

PyInstaller paketi yordamida yuklanuvchi faylni hosil qilishni bir nechta qadamlarga bo‘lish mumkin:

1-qadam: PyInstaller paketini kompyuterga o‘rnatish.

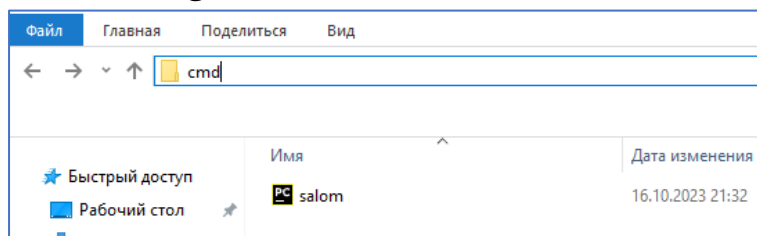
PyInstaller paketini kompyuterga o‘rnatish uchun quyidagi buyruqni buyruklar satriga kiritiladi:

```
pip install pyinstaller
```

Shundan so‘ng ushbu paketdan fodalanish mumkin bo‘ladi.

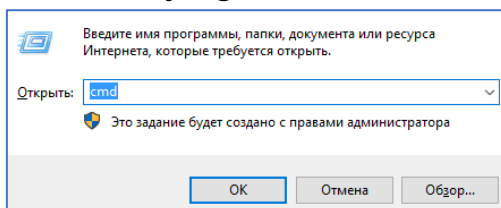
2-qadam: PyInstaller yordamida bajariladigan fayl hosil qilish.

Endi *PyInstaller* paketi yordamida Python tilida yozilgan loyihani bitta faylga jamlash kerak. Buning uchun project joylashgan papkaga kirish va buyruq qatoriga **CMD** buyrug‘ini kiritib, enter tugmasini bosiladi:



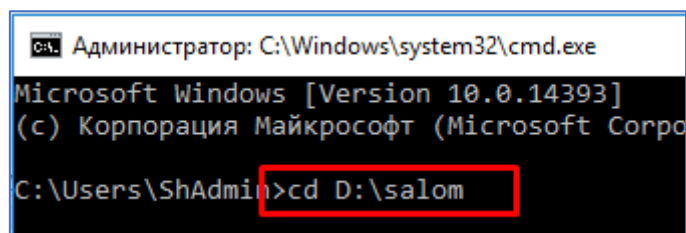
80-rasm. Buyruq qatoriga CMD buyrug‘ini kiritish

Yoki ikkinchi usuli kompyuteringizdan PUSK+R tugmasi bosiladi va hosil bo‘lgan «**Выполнить**» oynasiga “CMD” buyrug‘i kiritiladi.



81-rasm. «Выполнить» oynasi

Va hosil bo‘lgan konsol oynasiga **cd** buyrug‘i orqali kerakli papkaga kirish mumkin. Buning uchun **cd** va undan keyin Python skriptingiz saqlanadigan joy quyidagi ko‘rinishda kiritiladi va enter tugmasi bosiladi va ko‘rsatilgan kotalokga kiriladi:



```
Администратор: C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.14393]
(с) Корпорация Майкрософт (Microsoft Corporation)
C:\Users\ShAdmin>cd D:\salom
```

82-rasm. **cd** buyrug‘i orqali kerakli papkaga kirish

Keyin bajariladigan amal faylni yaratish uchun quyidagi shablondan to‘g‘ri foydalanish qilishdan iborat:

```
pyinstaller --onefile pythonFileName.py
```

Ushbu shablonda *pythonFileName* "**salom**" (va fayl kengaytmasi albatta .py) bo‘lgani uchun bajariladigan faylni yaratish buyrug‘i quyidagicha bo‘ladi:

```
pyinstaller --onefile salom.py
```

ENTER tugmasini bosiladi va pyinstaller paketi avtomatik ravishda ushbu papka ichida dist papkasini hosil qiladi. Yuklanuvchi exe fayl esa ushbu papkaning ichida joylashgan bo‘ladi. Endi ushbu exe faylni python dasturi o‘rnatilmagan kompyuterlarda ham ishga tushirish mumkin.

Nazorat savollari:

1. Pyinstaller paketining vazifasi qanday?
2. Pyinstaller paketini o‘rnatish qanday amalga oshiriladi?
3. Pyinstaller paketidan qanday foydalaniladi?

10.2.§. Pythonning **AUTO-PY-TO-EXE** paketi va uning imkoniyatlari

Pythonda yaratilgan loyihalarni yagona .exe faylga yig‘ishning yana bir usuli – bu auto-py-to-exe paketidan foydalangan holda judayam funksiyanal va visual tarzda amalga oshirish mumkin. Buning uchun ushbu paketni kompyuterga o‘rnatish kerak bo‘ladi.

auto-py-to-exe paketini kompyuterga o‘rnatish. Kompyuterga python paketlarini o‘rnatishning bir nechta usullari mavjud. Bullardan **pip**, **github** hamda **offline** holda o‘rnatish mumkin. Odatda pip orqali o‘rnatiladi. Chunki bu usul qulay hisoblanadi.

Pip orqali o‘rnatish. Auto-py-to-exe paketini kompyuterga Pip orqali o‘rnatish quyidagicha amalga oshiriladi: Birinchi navbatda **“Команда строка”** oynasi ochib olinadi va u joyga pip install auto-py-to-exe buyrug‘i kiritiladi va enter tugmasi bosiladi. Shundan so‘ng avtomatik ravishda kompyuterga ushbu paket o‘rnatiladi:

```
$ pip install auto-py-to-exe
```

GitHubdan o‘rnatish. Ushbu paketni to‘g‘ridan-to‘g‘ri GitHubdan o‘rnatish mumkin. GitHubdan auto-py-to-exe paketini o‘rnatish uchun avval GitHub omborini klonlash kerak bo‘ladi. U quyidagi kod orqali amalga oshiriladi:

```
git clone https://github.com/brentvollebrecht/auto-py-to-exe.git
```

Keyin auto-py-to-exe papkasiga cd buyrug‘i orqali o‘tish kerak.

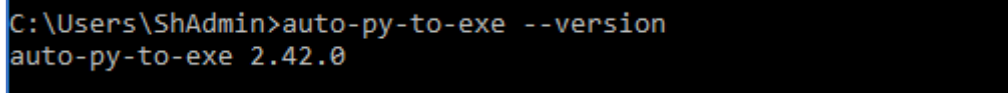
```
$ cd auto-py-to-exe
```

Endi python buyrug‘i orqali setup.py faylini ishga tushirish kerak.

```
$ python setup.py install
```

Quyidagi buyruq yordamida versiyani ham tekshirish mumkin:

```
auto-py-to-exe --version
```



```
C:\Users\ShAdmin>auto-py-to-exe --version  
auto-py-to-exe 2.42.0
```

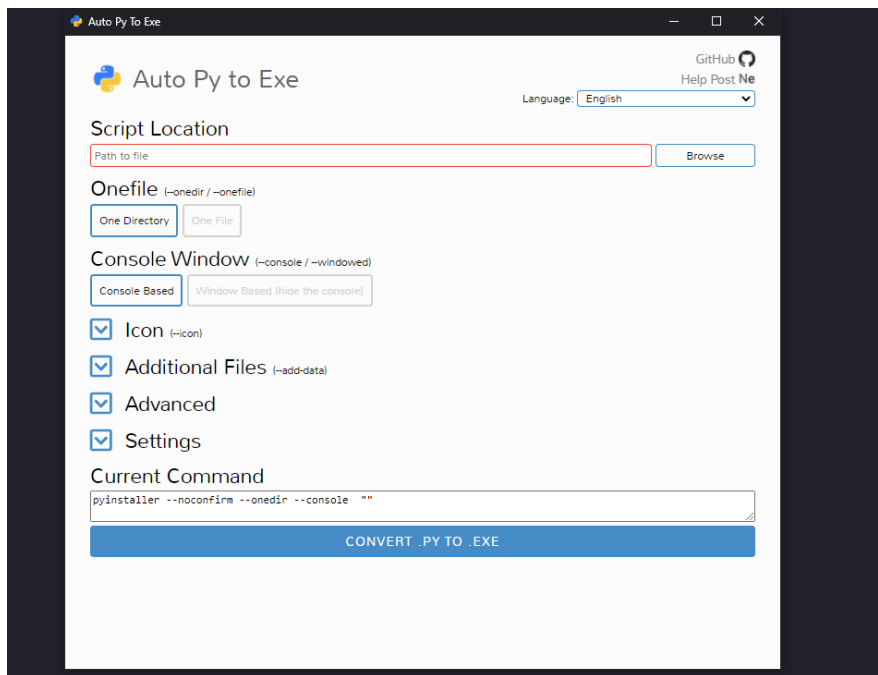
83-rasm. Paketning versiyasini bilish

Auto py to exe ning joriy versiyasi 2.42.0 va u endi kompyuterga o‘rnatilgan.

Ilovani ishga tushirish. auto-py-to-exe ilivasini ishga tushirish uchun terminalda quyidagi buyruqni bajarish kerak bo‘ladi:

```
$ auto-py-to-exe
```

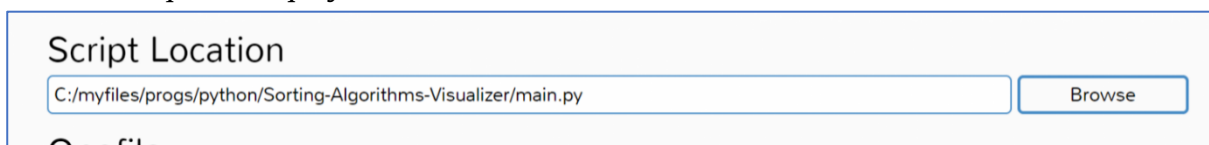
Shundan so‘ng quyidagi “Auto py to exe” dasturi ishga tushadi:



84-rasm. avto-py-to-exe GUI interfeysi

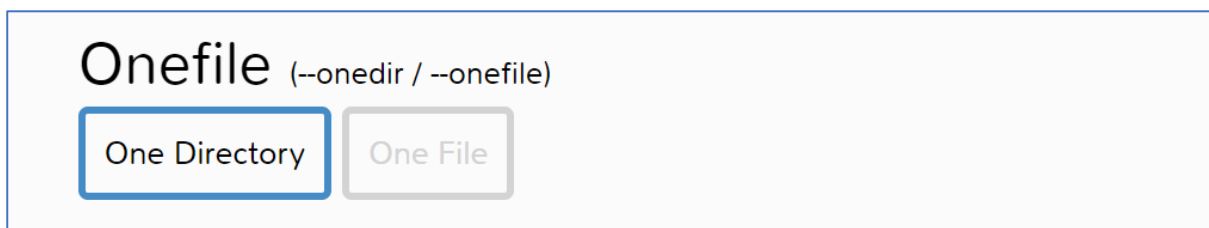
Endi ushbu interfeysdan .py faylni yuklanuvchi .exe fayliga aylantirish uchun foydalaniladi.

Konvertatsiya jarayoni. 1-qadam. Fayl joylashuvini qo‘shish. Python faylini .exe fayliga o‘zgartirish uchun avval uning yo‘lini ko‘rsatish kerak bo‘ladi. Buning uchun konvertatsiya qilinadigan faylning manziliga o‘tiladi va keyin yo‘lni ko‘rsatib, qo‘shib qo‘yiladi:



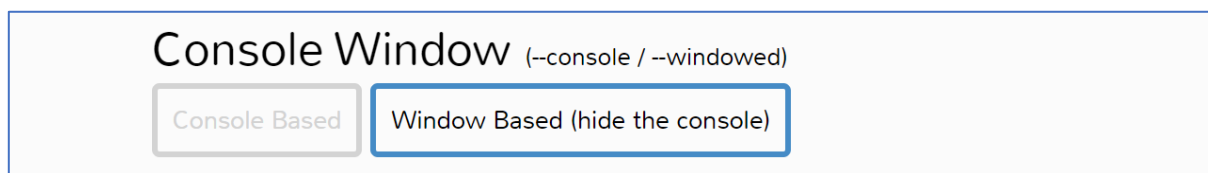
85-rasm. Faylni yo‘lini ko‘rsatish

2-qadam: "One Directory" yoki "One File" ni tanlash. Interfeysda "One Directory" yoki "One File" ni tanlash imkoniyati mavjud. Odatda ko‘pchilik katta Python loyihalar bir nechta fayllardan tashkil topgan bo‘ladi, shuning uchun "One Directory" ni tanlash afzalroq. Ushbu parametr barcha kerakli fayllar bilan papkani yaratadi, shuningdek .exe faylini ham yaratib qo‘yadi.



86-rasm. "One Directory" yoki "One File" ni tanlash

3-qadam. “Console Based” yoki “Windows Based” ni tanlash. Yuqoridagi amallarni bajarilganidan so‘ng dastur turini tanlash kerak: **Console** yoki **Windows** asoslangan. Agar "Windows Based" ni tanlansa, bu dasturning barcha konsol natijalarini yashiradi. Agar yaratilgan dastur konsol chiqishiga asoslangan holda ishlaydigan bo‘lsa, unda "Console Based" ni tanlash maqsadga muvofiqq bo‘ladi. Agar dasturda GUI ilovasi bo‘lsa yoki foydalanuvchiga konsol chiqishini ko‘rsatish shart bo‘lmasa, unda "Windows Based" ni tanlash tavsiya qilinadi. Bu yerda ikkinchi ikkinchi variantni tanlangan, chunki ushbu dasturda GUI mavjud.



87-rasm. Konsolga asoslangan yoki oynaga asoslangan

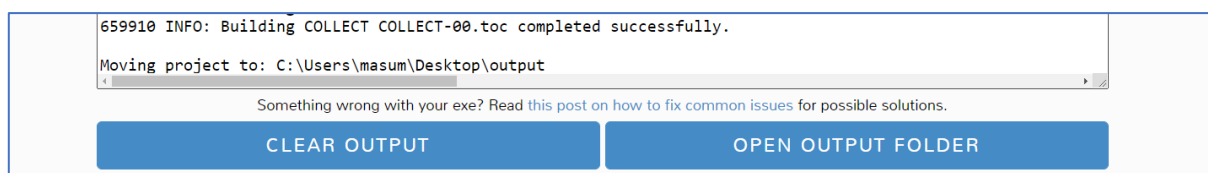
4-qadam. Konvertatsiya qilish. Yuqoridagi barcha amallarni bajarilganidan keyin qolgan sozlamalari masalan, piktogramma, qo‘shimcha fayllar va hk. ni o‘z standart holatida qoldirib, konvertatsiya qilish mumkin. Buning uchun faqat **CONVERT .PY TO .EXE** tugmasini bosish kifoya.



88-rasm. Konvertatsiya qilish uchun tugma.

Jarayonni yakunlash uchun biroz kutishingiz kerak bo‘ladi.

Open output folder. Konvertatsiya jarayoni tugaganidan so‘ng, “Open output folder” tugmasini bosish orqali konvertatsiya qilingan fayl joylashgan papkani ochish mumkin.



89-rasm. Open output folder tugmasi

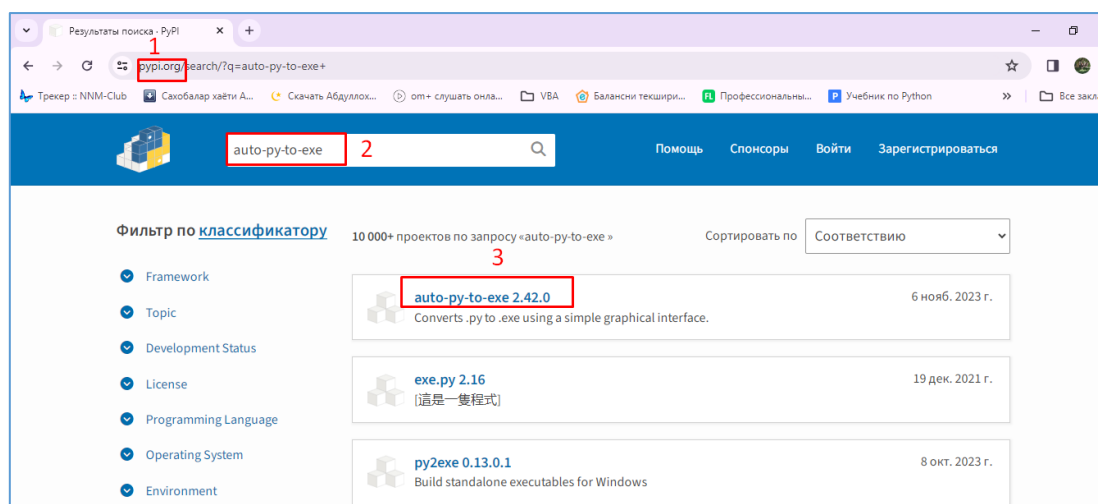
Shundan so‘ng .exe fayl joylashgan papka ochiladi. Endi uni Python-ni o‘rnatmasdan boshqa kompyuterlarda ham bemalol ishga tushirib foydalanish mumkin.

Nazorat savollari:

1. Auto-py-to-exe paketi kompyuterga qanday o'rnatiladi?
2. Auto-py-to-exe paketining qanday sozlamalari mavjud?
3. Auto-py-to-exe paketining PyInstaller paketidan qanday farqi mavjud?
4. Auto-py-to-exe paketi yordamida yaratilgan .exe fayl qaysi papkada yaratiladi?

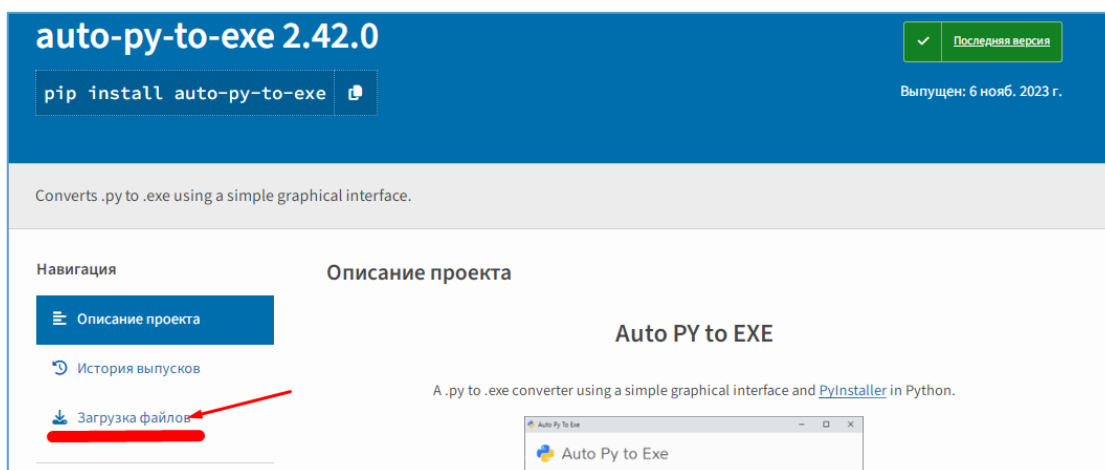
10.3.§. Python paketlarini offline holatda kompyuterga o'rnatish

Ba'zi hollarda internet tarmog'iga ulanmagan kompyuterlarga ham python paketlarini o'rnatishga zarurat paydo bo'lib qoladi. Bunday vaziyatlarda kompyuterga pythonning kerakli paketini olib kelib, o'rnatiladi. Buning uchun boshqa bir internetga ulangan kompyuterdan **pypi.org** saytiga kirib, kerakli paketni qidirib topiladi va yuklab olinadi. Masalan:



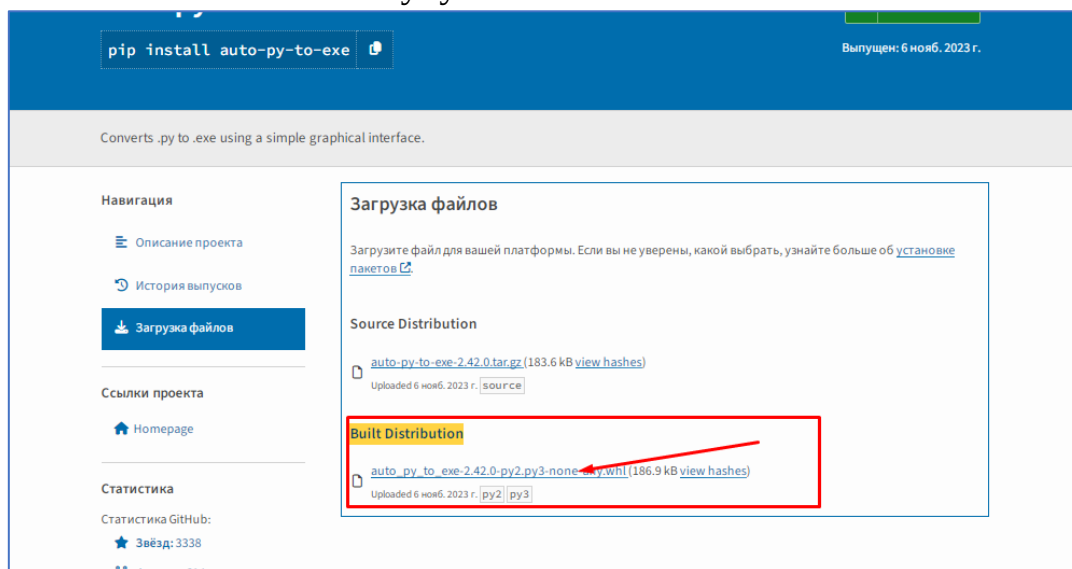
90-rasm. Pypi.org sayti bilan ishlash

Ushbu kerakli paketni topib, uni tanlanganda paket haqida ko'plab kerakli ma'lumotlar beruvchi oyna ochiladi. U oynada paketni yuklab olish tugmasi ham mavjud va yuklab olish "Загрузка файлов" bo'limini tanlanlash orqali amalga oshiriladi.



91-rasm. Pypi.org saytidan kerakli paketni yuklab olish

Shundan so‘ng paketni yuklab olish uchun “**Загрузка файлов**” oynasi ochiladi. U yerdan “**Built Distribution**” bo‘limidagi faylni yuklab olish uchun maxsus ssilkani bosiladi va avtomatik ravishda fayl yuklab olinadi.



92-rasm. Kerakli paketni yuklab olish

Ushbu yuklab olingan faylni boshqa bir kompyuterga ko‘chirib o‘tkaziladi. Endi ushbu kerakli yukanuvchi faylni kompyuterga o‘rnatish kerak. Buning uchun fayl oddiy .exe fayllar kabi emas balki maxsus usulda o‘rnatilishi kerak bo‘ladi. Ushbu amal “**py -m pip install file name**” shabloni yordamida amalga oshiriladi. Bizning misolimizda file Downloads papkasida joylashganligi uchun terminaldan osha papkaga kirib borib, undan so‘ng o‘rnatish kodini teriladi va enter tugmasi bosish orqali o‘rnatish amalga oshiriladi.

```
Microsoft Windows [Version 10.0.14393]
(c) Корпорация Майкрософт (Microsoft Corporation), 2016. Все права защищены.

C:\Users\ShAdmin\Downloads>py -m pip install auto-py-to-exe-2.42.0-py2.py3-none-any.whl
```

93-rasm. Kerakli paketni lokal holatda o‘rnatish

Shundan so‘ng, agarda hech qanday xatolik ro‘y bermasa, paket kompyuterga muvoffaqiyatli o‘rnatiladi.

Barcha paketlar kompyuterga lokal holatda o‘rnatilishi mumkin.

Nazorat savollari:

1. Python paketlarini kompyuterga qanday usullardan foydalanib, o‘rnatish mumkin?
2. Kerakli paketlarni lokal holda o‘rnatish qanday amalga oshiriladi?
3. Kompyuterga paketlarni lokal holada o‘rnatishning qanday afzalliklari bor?

FOYDALANILGAN ADABIYOTLAR RO‘YXATI

1. Sh.A. Mengliyev, O.A. Abdug‘aniyev. “Python dasturlash tili”. Termiz - 2021.
2. Франсуа Шолле. “Глубокое обучение на Python”. — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»)..
3. Пол Бэрри. “Изучаем программирование на Python”; [пер. с англ. М. Райтман]. — Москва: Эксмо, 2017 год.
4. Н.А.Прохоренко, В.А.Дронов. “Python3 и PyQt5. Разработка приложений”. СПб.: БХВ-Петербург, 2016. — 832 с.: ил.
5. Чан, Уэсли. “Python: создание приложений. Библиотека профессионала”, 3-е изд. [Пер. с англ. - М. : ООО "И.Д. Вильямс"], Москва: Санкт-Петербург • Киев 2015.
6. Sh.R.Sapayev, A.A.Abidov, “Dasturlash texnologiyalari” o‘quv qo‘llanma. Toshkent 2022. 123b
7. Марк Саммерфилд. “Программирование на Python3. Подробное руководство” [Пер. с англ. — СПб]. - Москва: Санкт-Петербург—2009 год.